DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY

Convolutional Neural Networks with TensorFlow in Python

Convolutions, pooling, regularization, hyperparameter tuning and TensorBoard

Volume 2 of 2 · Sections 39–53

15 SECTIONS

201 FIGURES

160 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 39 Number of Filters and Batch Size 3
LECTURE 40 Primary Classification 13
LECTURE 41 Combine the Labels 17
LECTURE 42 Dense Layer and Model 40
LECTURE 43 Hierarchical Classification and Gender 46
LECTURE 44 Score and Label 48
LECTURE 45 Hyper Parameter and Standard Deviation 51
LECTURE 46 Flip and Data Sets 54
LECTURE 47 Previous Amazon 56
LECTURE 48 Networks 79
LECTURE 49 Max Pool 83
LECTURE 50 Connected Layer 92
LECTURE 51 Naive Inception 106
LECTURE 52 134
LECTURE 53 Normal Model 135

LECTURE 39

Number of Filters and Batch Size

Continuing from where we left off. It's time to think about the actual network architecture. So the first question is how much work our model needs to do.

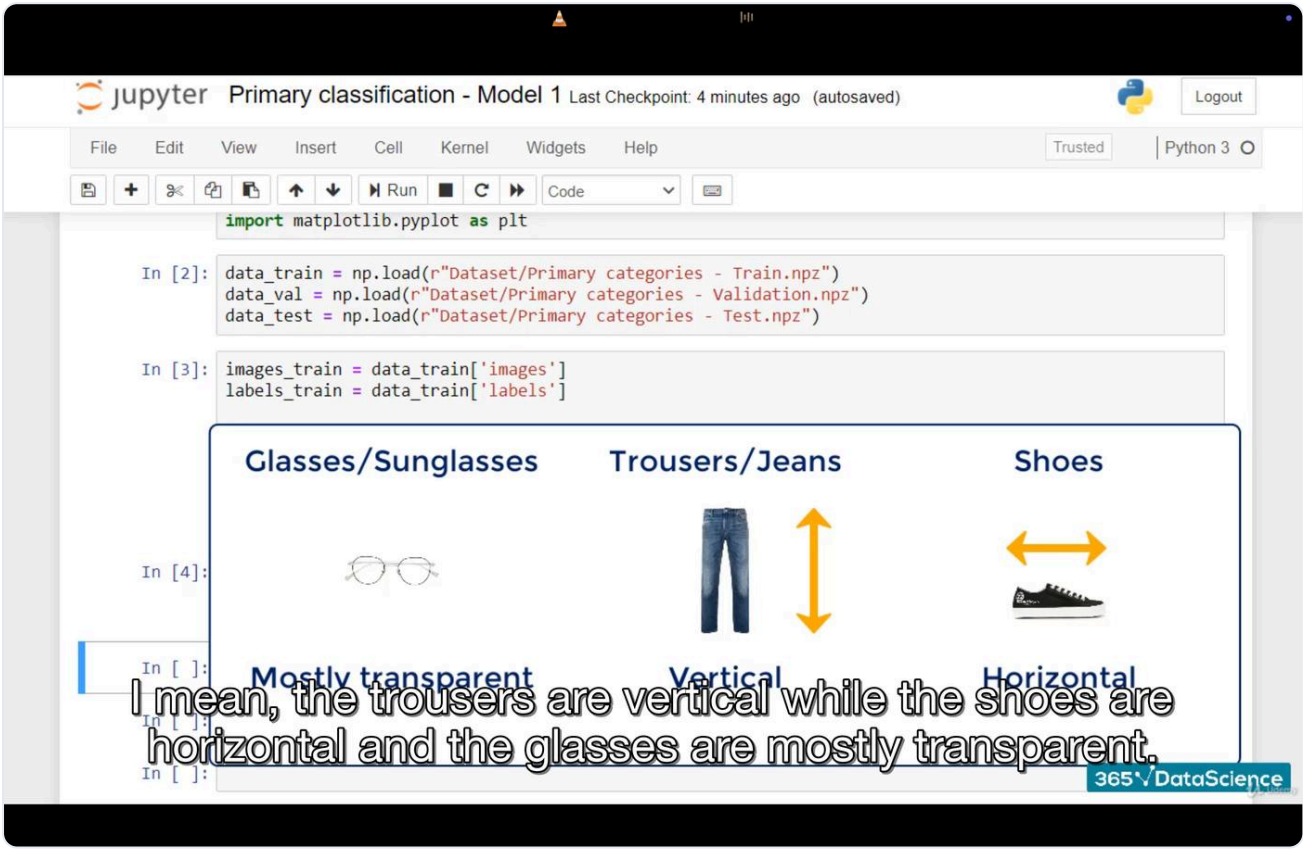


FIGURE 1.1

Well, we've got three classes of data to be separated, glasses and sunglasses, trousers and jeans and shoes. If we think about it, these are actually very distinct classes. I mean, the trousers are vertical while the shoes are horizontal and the glasses are mostly transparent.

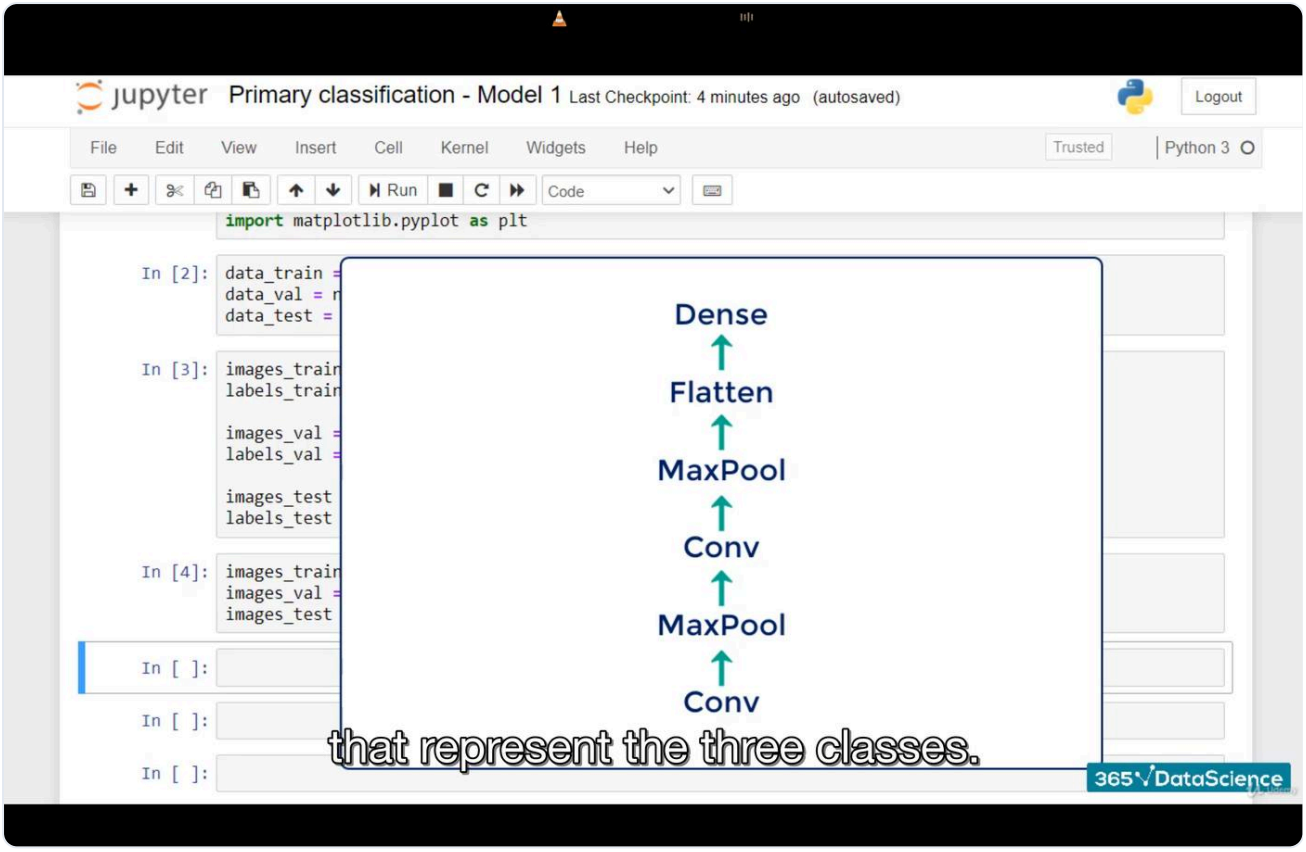


FIGURE 1.2

The expectation is that our network should not struggle with this task. Therefore, even a simpler model could work here. That's why we are going to follow the same structure as the one we had in the amnesty example.

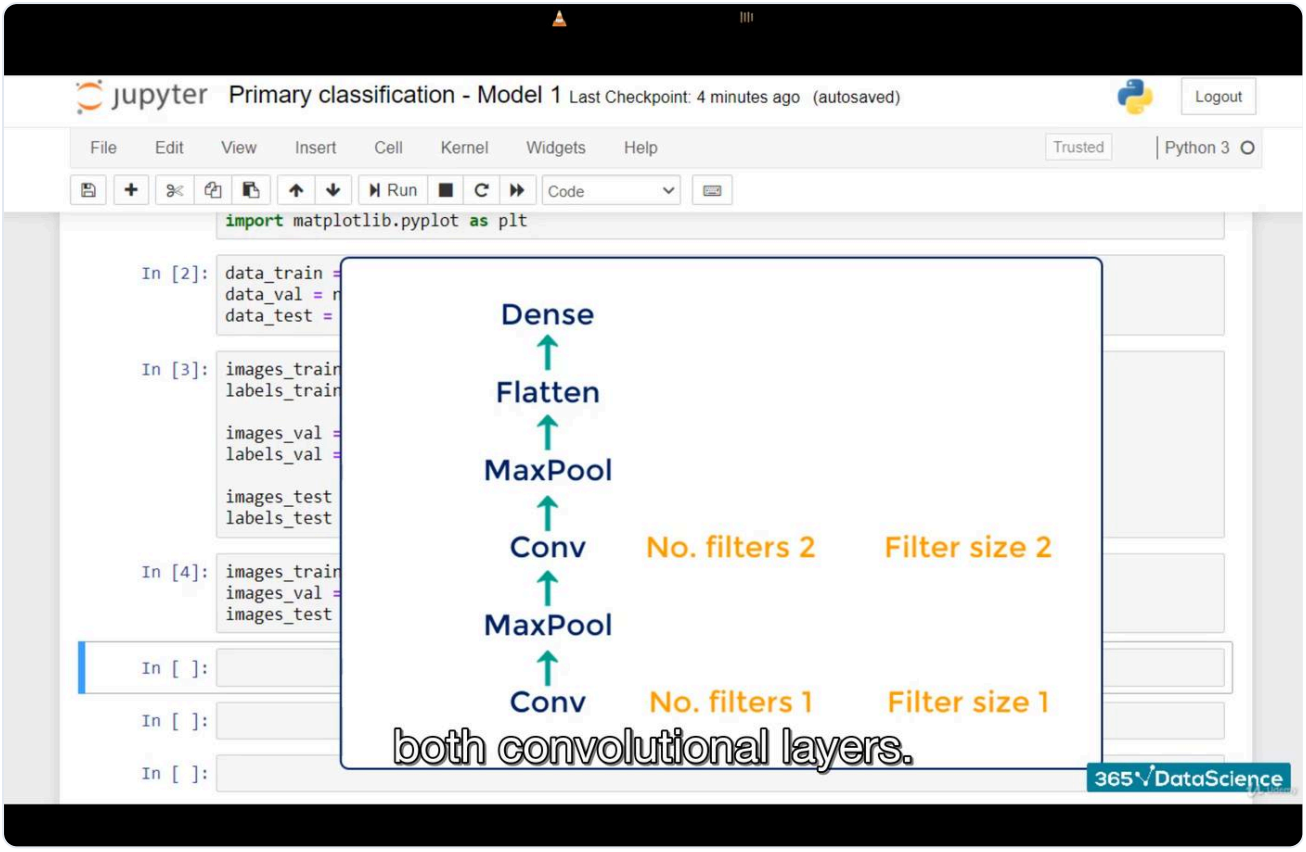


FIGURE 1.3

It consists of a convolutional layer, followed by a max pool layer, again, a convolutional and a max pool layer, then a flat thin layer to unpack the tensor and finally a dense layer with three outputs that represent the three classes. What about the parameters of the different layers? Well, we need to fine tune them for the optimal performance of the model on this particular data set.

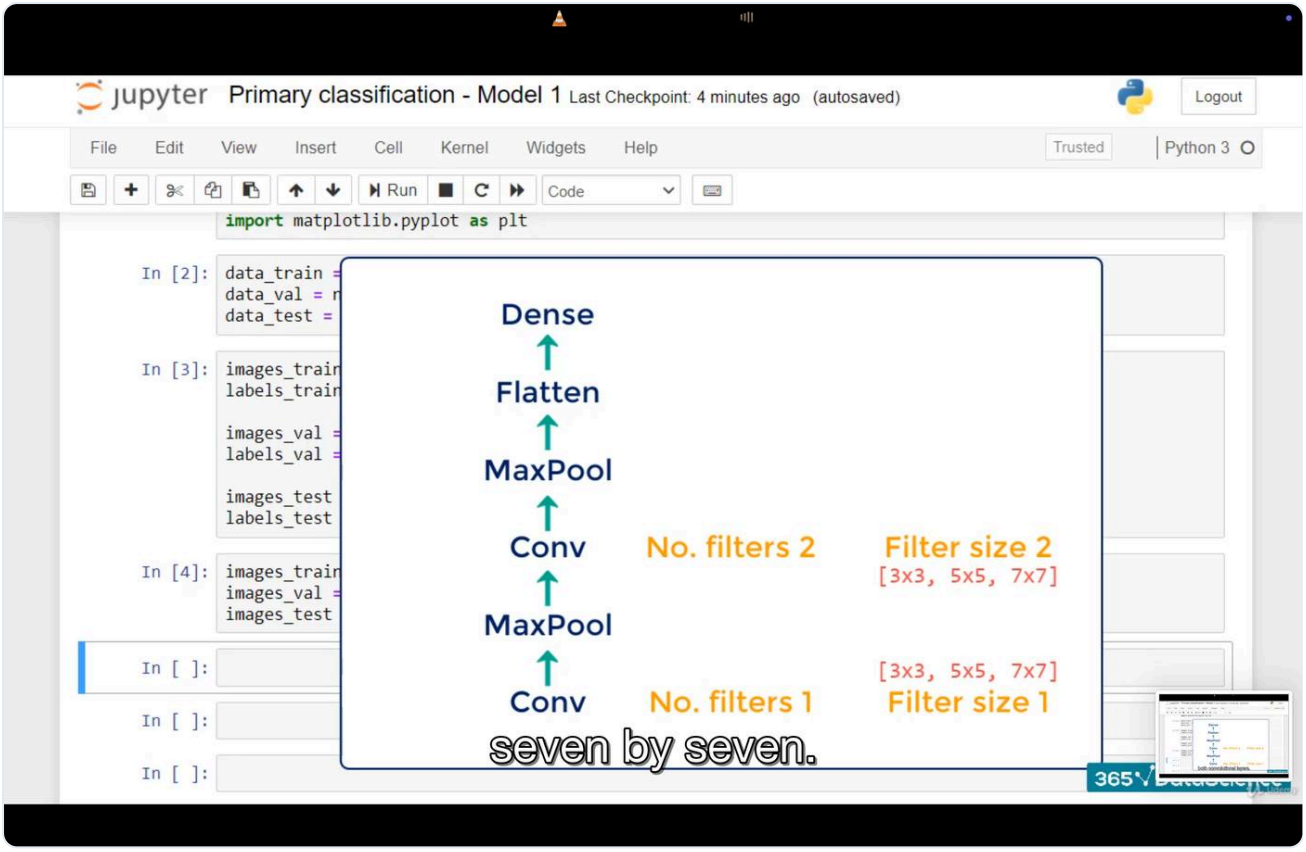


FIGURE 1.4

And of course, we are going to do that with a TensorBoard hyper parameter tuning structure. Given the architecture, we have four parameters to tune the filter size and the number of filters of both convolutional layers. The values for the filter sizes that we are going to consider are three by three, five by five and seven by seven.

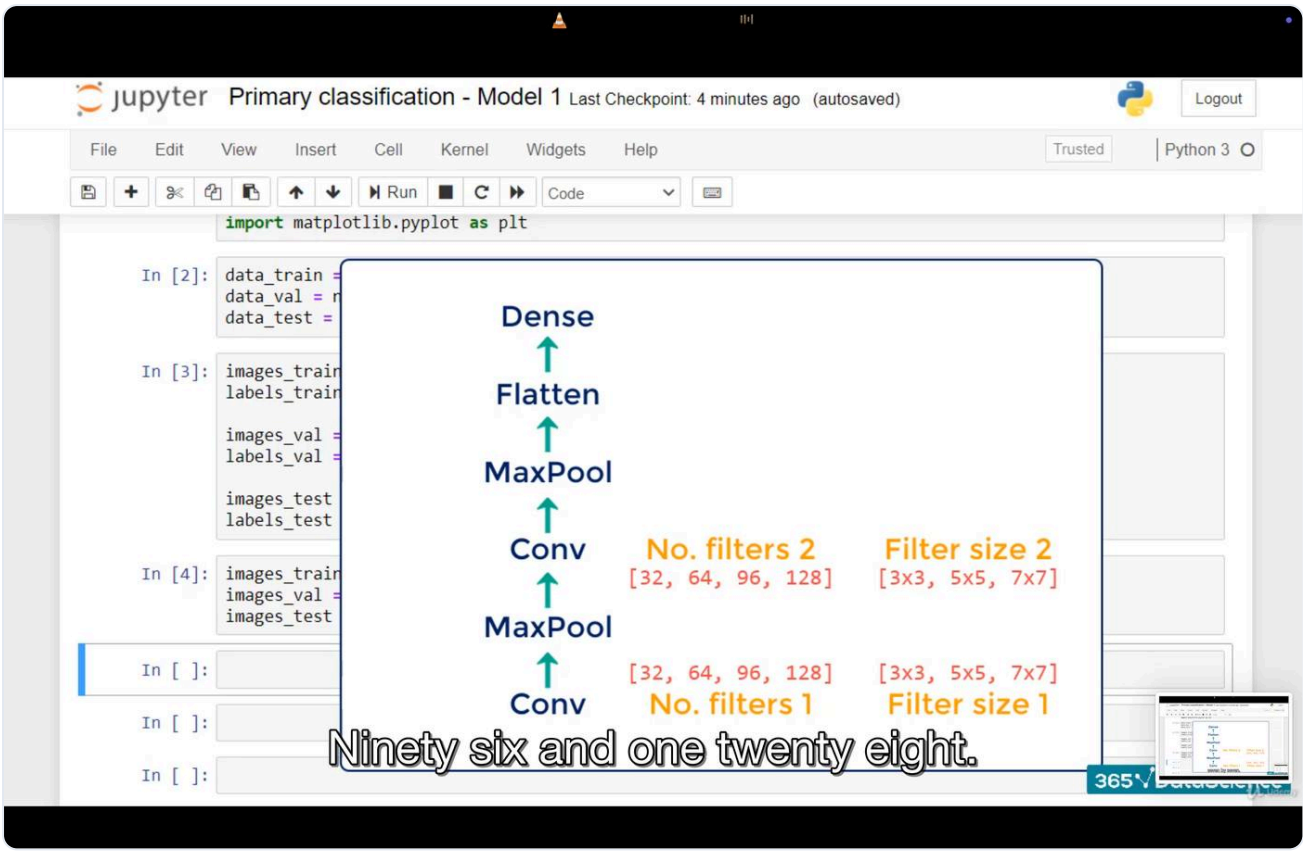


FIGURE 1.5

The values for the number of filters we are interested in are 30 to 64. Ninety six and one twenty eight. However, given the four parameters, this makes one hundred and forty four combinations.

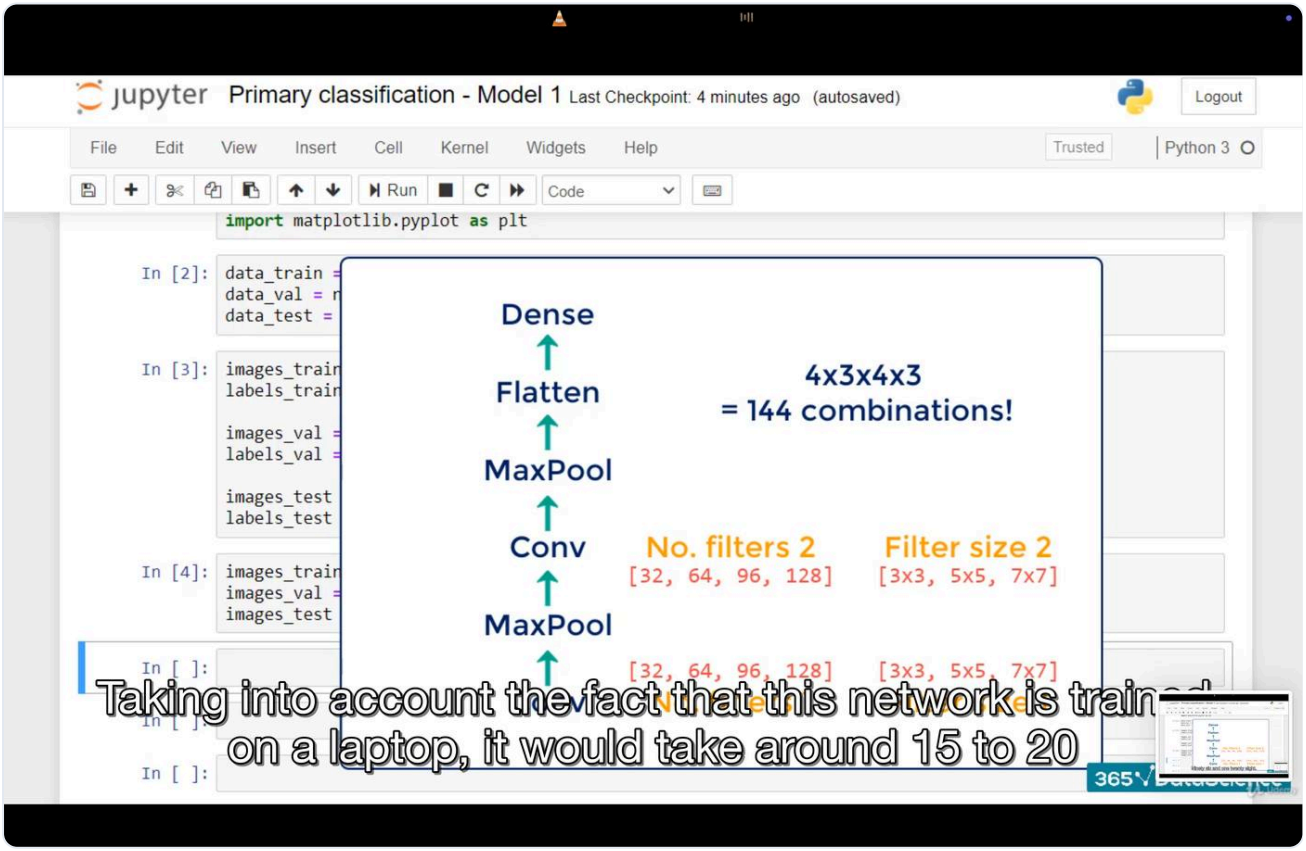


FIGURE 1.6

Taking into account the fact that this network is trained on a laptop, it would take around 15 to 20 hours for the grid search to end. So in the interest of saving time and resources, we will make two simplifications.

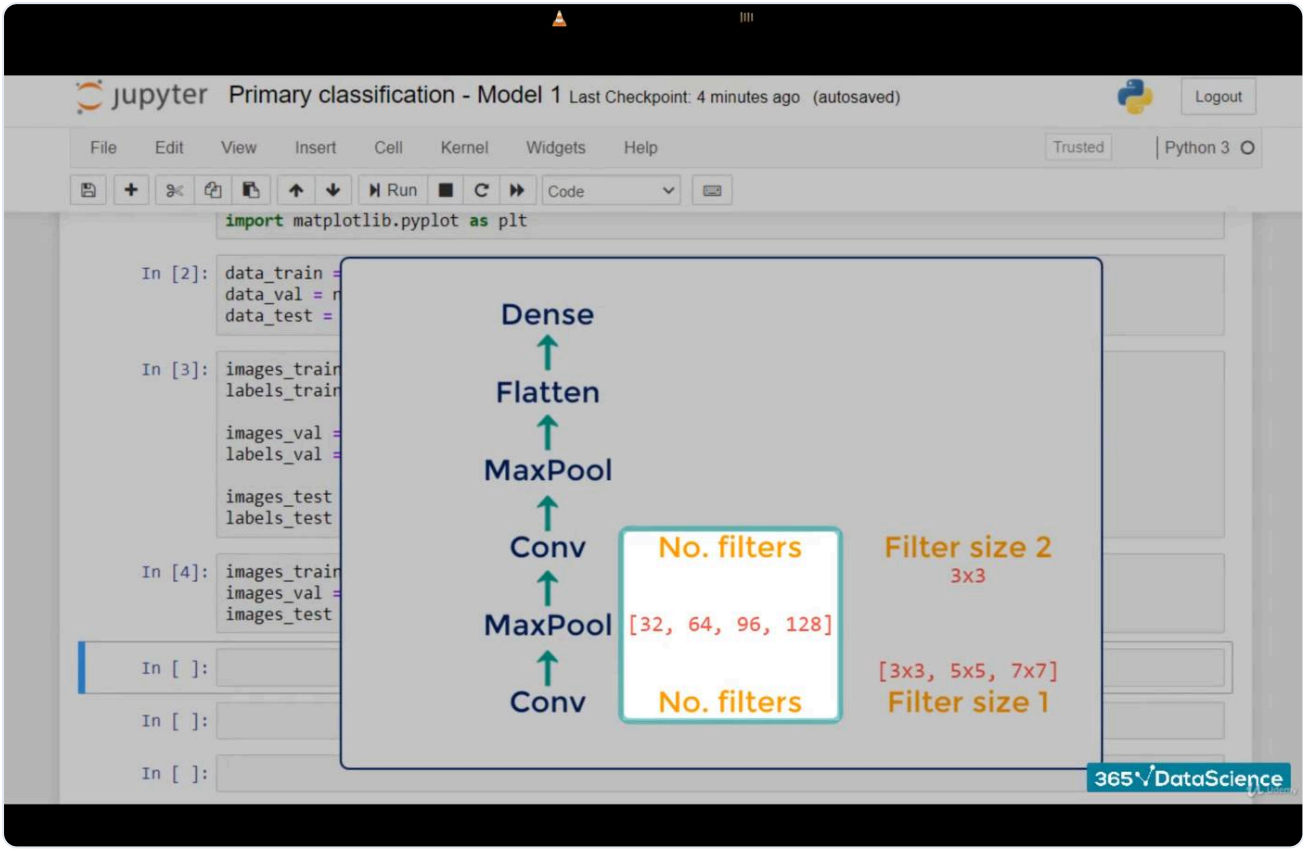


FIGURE 1.7

The filter size of the second convolutional layer will be set to three by three by default, and the two conv layers will have the same number of filters. That means the only parameters that we will tune are the filter size of the first layer and the common number of filters. This adds up to 12 combinations of parameters, which is much better.

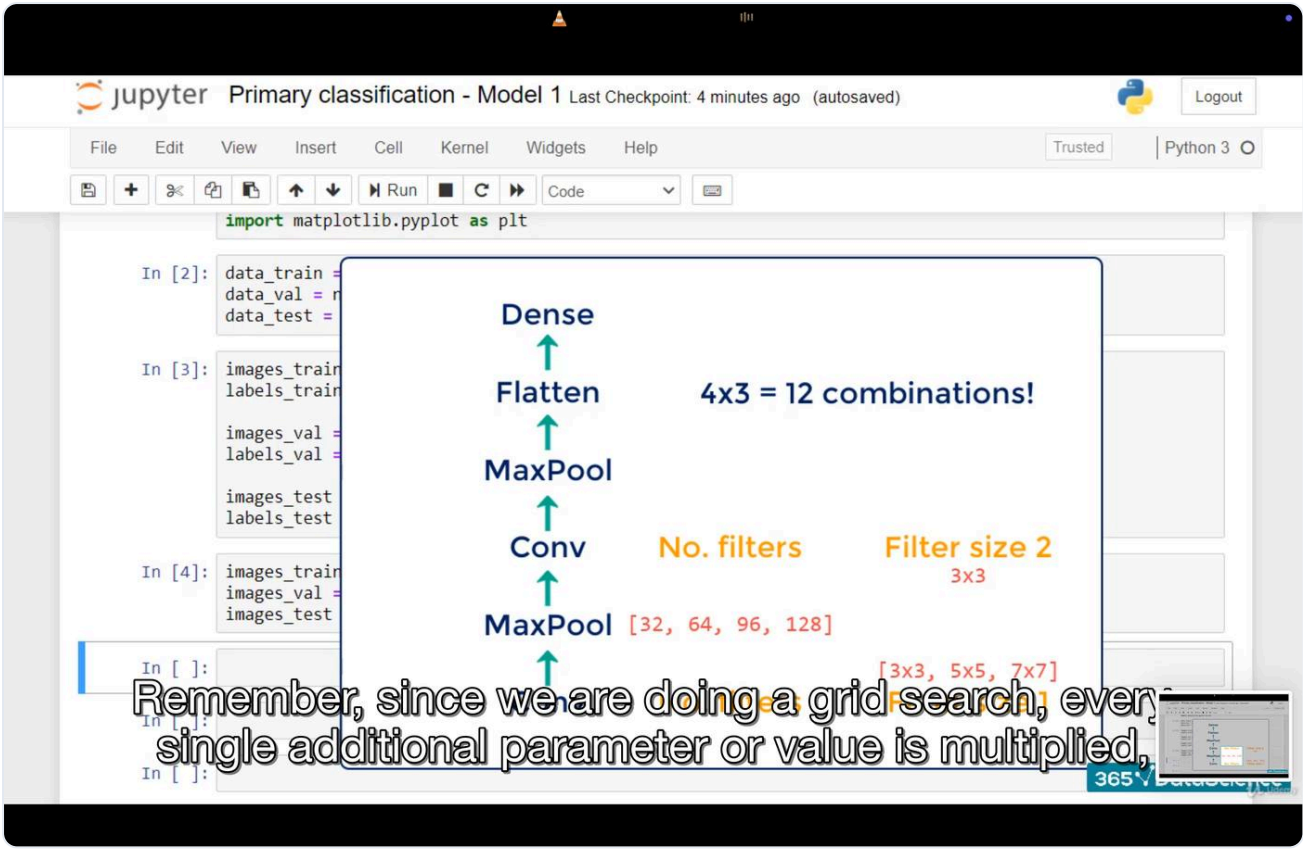


FIGURE 1.8

Remember, since we are doing a grid search, every single additional parameter or value is multiplied, so the number of combinations grows extremely fast. All right, now that we know what our objective is, let's code it. We have done all of it several times already, so I will walk you through it just a little bit faster. Good, let's start with defining a couple of learning parameters here. First is the maximum number of epochs which we will set to 15, then is the batch size initialized as 60 for. Now, given that we will implement early stopping, I don't expect to ever reach the 15 EPIK limit, but the variable is still here to prevent the training from becoming too long.

As for the batch size, you may ask why we didn't include it as a hyper parameter to tune.

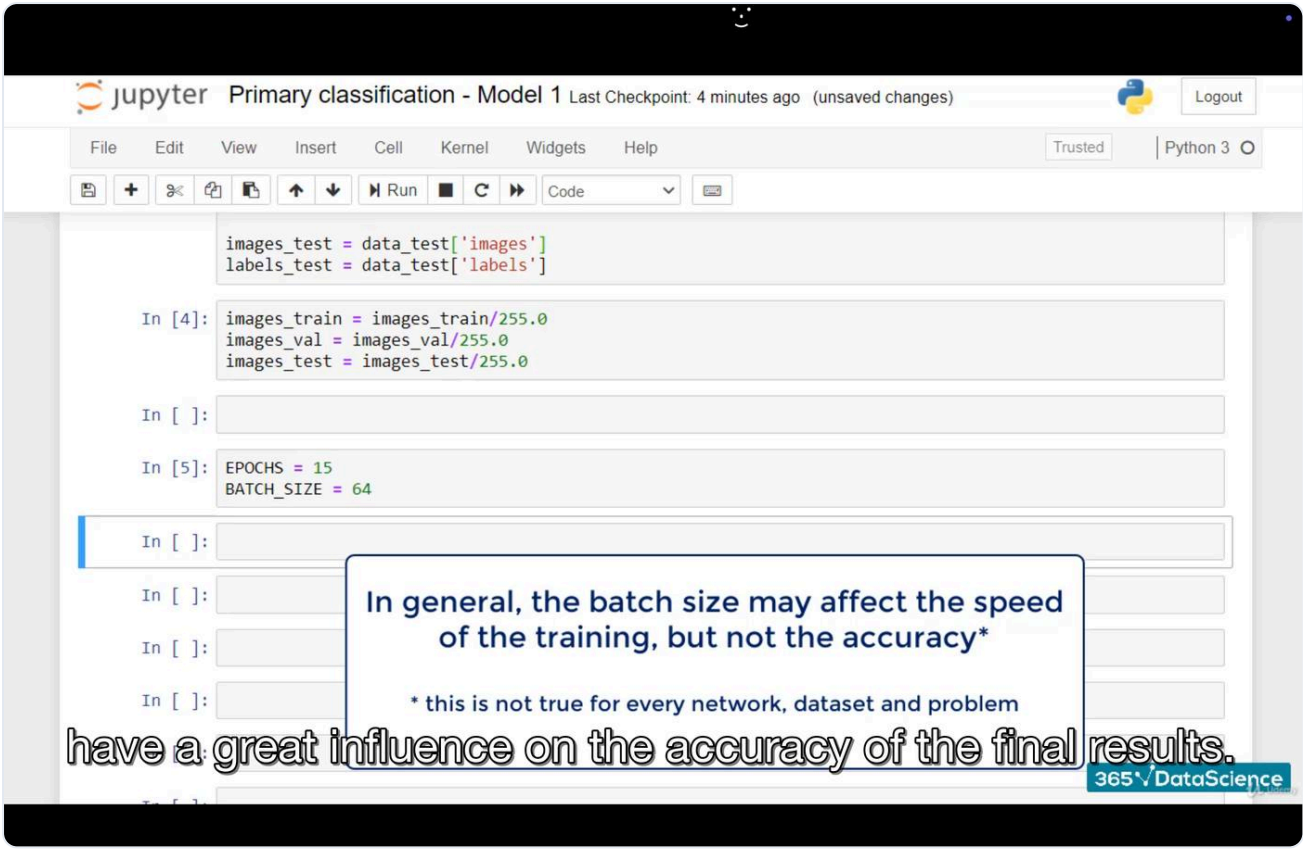


FIGURE 1.9

Well, in general, the batch size may help the network to reach the minimum quicker, but it does not have a great influence on the accuracy of the final results. That and the fact that introducing the batch size as a parameter to tune will increase the train time significantly made us leave it as the rule of thumb. Power of two, that is 64. Good, so defining the hyper parameters without bogging down into too much detail, we create the two variables to represent the hyper parameters and enter the values we're interested in. Then we lock them with the file writer. We should now decide where to create this file in this section. I will use the following structure.

There will be logs folder where every log is written to inside. There are folders for the different models. We will test and in every such model folder reside the two types of logs, one for hyper parameter tuning and one for the training process logs, even though for the main classification task we will test only one type of architecture. I will still use the folder model one. All right, the administration is completed. Let's get down to the actual modeling. Next up is the train test model function, the heart of our coat. This is where we build and train our model. However, we would also like to track different metrics associated with the training process other than the final accuracy logged by the hyper parameter tuning.

So we will track the training process with a standard TensorBoard callback, but we will also log the confusion matrix. OK, let's get to work, starting with the model we created in the familiar way tf dot Keras dot sequential and add all the layers. As we've discussed already, the number of filters and the filter size parameters for the first one are varied, so we identify them by their corresponding variables. Note that the input shape is defined as one 20 by nine by three as those are

the dimensions of the umpires. Be careful not to have 90 by one 20 here instead as that would raise an error. In the end, the dense layer has an output of three, as that's the number of categories we have.

Moving on, we defined the lost function. And compile the model with the atom optimizer. Then we create a variable for the logging directory we will use this one is in the fit folder branch. The next item on the list is to implement the confusion matrix, if you recall, it consists of three function definitions, one that plus the confusion matrix, given the numbers, another that converts this figure to an actual image, and the third that calculates the numbers and logs the image. So let's just copy and paste the relevant code from our earlier lectures. There are just a couple of things we need to tweak. First, we have the directory of the file writer.

I am going to have it in the directory we created earlier, but within an additional folder called s.M Standing for Confusion Matrix. The other matter is the class names we need to provide them inside the log function right here. Fantastic, the last portion is where we define the callbacks and train the network, this time we have to TensorBoard callbacks, one that log's training data and the other for the confusion matrix. Then we have the early stopping callback, it has the usual parameters, patients of two and restore best weights set to true. And now we have arrived at the point where we train the model. We have the fit method with all the parameters for the input, data and labels, we can just pass the numbers as they are and TensorFlow will have no problems.

OK. Next up is Batching. And here comes the point we made in the preprocessing stage, if we set the batch size parameter here, TensorFlow would automatically shuffle and batch the numbers we supplied. That's why we didn't do this earlier. We continue by adding the callbacks. Then the validation data is supplied as a tuple of the umpires, and we set the verbosity to two meaning limited printable information as to not clutter the screen. Great. Now, the last step is to evaluate the model on the validation set in order to produce the accuracy metric that will be returned. It is important to make this evaluation on the validation set, not the test set, as we are yet to finalize our model.

That was the train and test function. Let's take a step back, though, and see what information we gained from that. So we train a model for every combination of the hyper parameters and return its accuracy. We are also logging the confusion matrix in each epic of each model, as well as the standard training data. But the model itself is lost as the variable that contains it is not logged subsequently. If we wish to continue the training or test that exact same model with a different data sets, we would not be able to. That's why I will also export the model to a file as well as all the other logs, and yet again, we can easily do this with a single line of code model dot save and then the path to where it should be exported.

In my case, I will save the models to a subfolder called models. Unfortunately, an important caveat is lurking in this simplicity on Windows, the save method cannot write to file parts longer than two hundred and sixty symbols. So if you're working directory is buried deep down, you may get an error when trying to save a model in that directory. The solution is to use shallower directories like this one. Another point about the model saving is that they take a lot of disk space, and if you save everything, you may fill as much as 100 gigabytes. We have surmounted

the hardest part of the code, so this feels like a good place to take a break, let's rest for a bit and return to the code in the next lecture where we will finish the run function, train the model and see the results.

See you there. And thanks for watching.

LECTURE 40

Primary Classification

Welcome back. It is time to finish the primary classification. As we outlined at the start in this cell, we will place the run function, its structure is always the same, so I will paste it directly. Next, we have the for loops that will pick every combination of the hyper parameters we need to iterate over all the values of the filter size and all the values of the number of filters, we define those allowed values at the top of the notebook, remember? Then we package those values in a dictionary and start the chain of functions that will train and log the results. And basically that's it. That is all the code for this architecture of the primary classification task, we can now take a breath as this is not only the code for this task, but for most subsequent tasks as well.

Yes, we will actually copy this and use it for the next problems, because the only things we need to change are the different logging folders and the model itself. And we can do all that with a few clicks. Fantastic. Well, now you must be eager to begin the training and that's exactly what we are going to do. So let's run this cell and initiate the process.

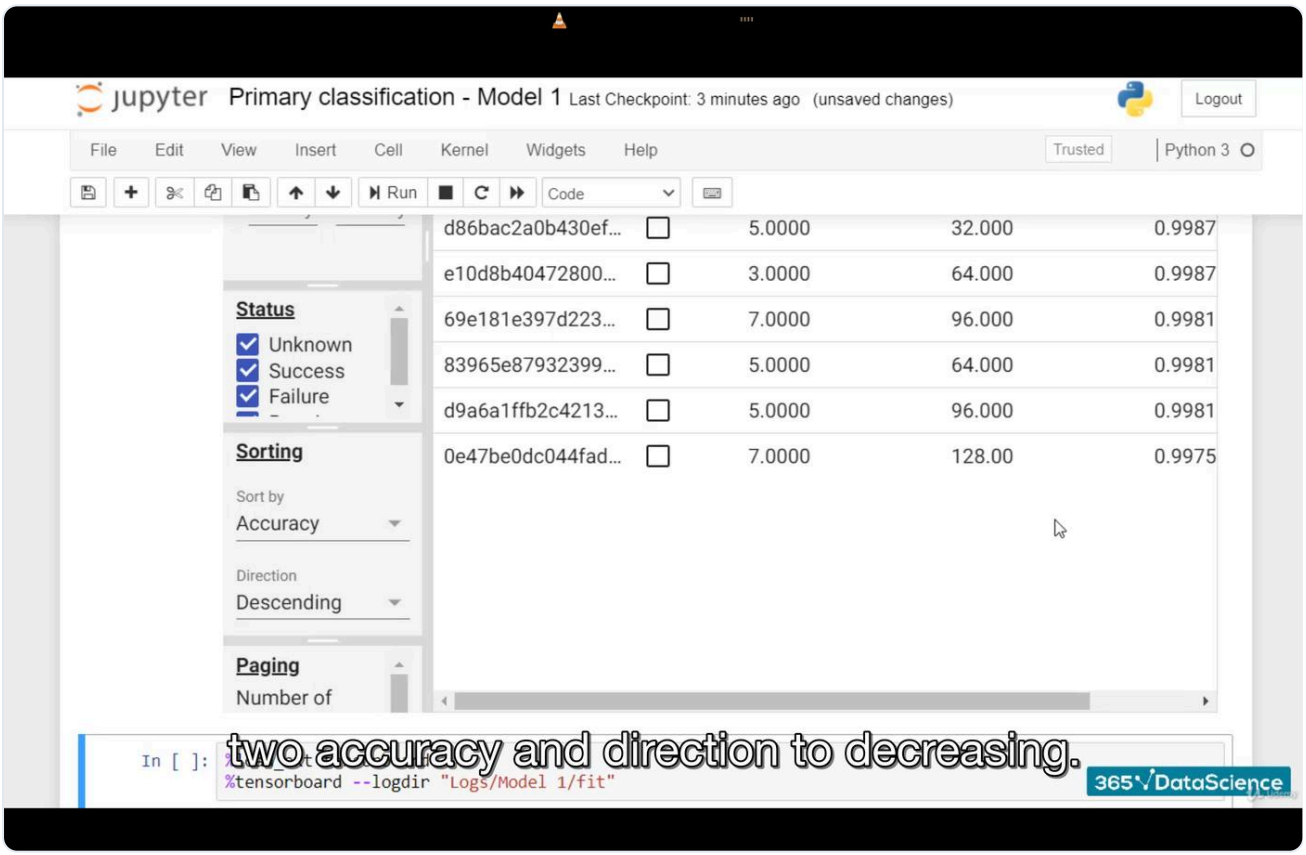


FIGURE 2.1

This is going to take some time because each model is trained for roughly five minutes. I suggest we cut straight to the end. The network has been trained with all the different combinations and the data is hot. So let's check it using TensorBoard, we will start with the already familiar code here. First, let's take a look at the hyper parameter tuning. Now, recall that before running TensorBoard, we had to write two commands in the command prompt. So starting with CMD, we

first have a task kill command and then a delete command to erase the temporary files. Fine. Now we can run the cell. Of course, the first time we will get a time out running at a second time displays the results.

Great, let's look into it. We go to the programs tab at the top and are greeted with this table view. It is a good idea to order the runs at a decreasing accuracy, so here on the left, we set sort by two accuracy and direction to decreasing.

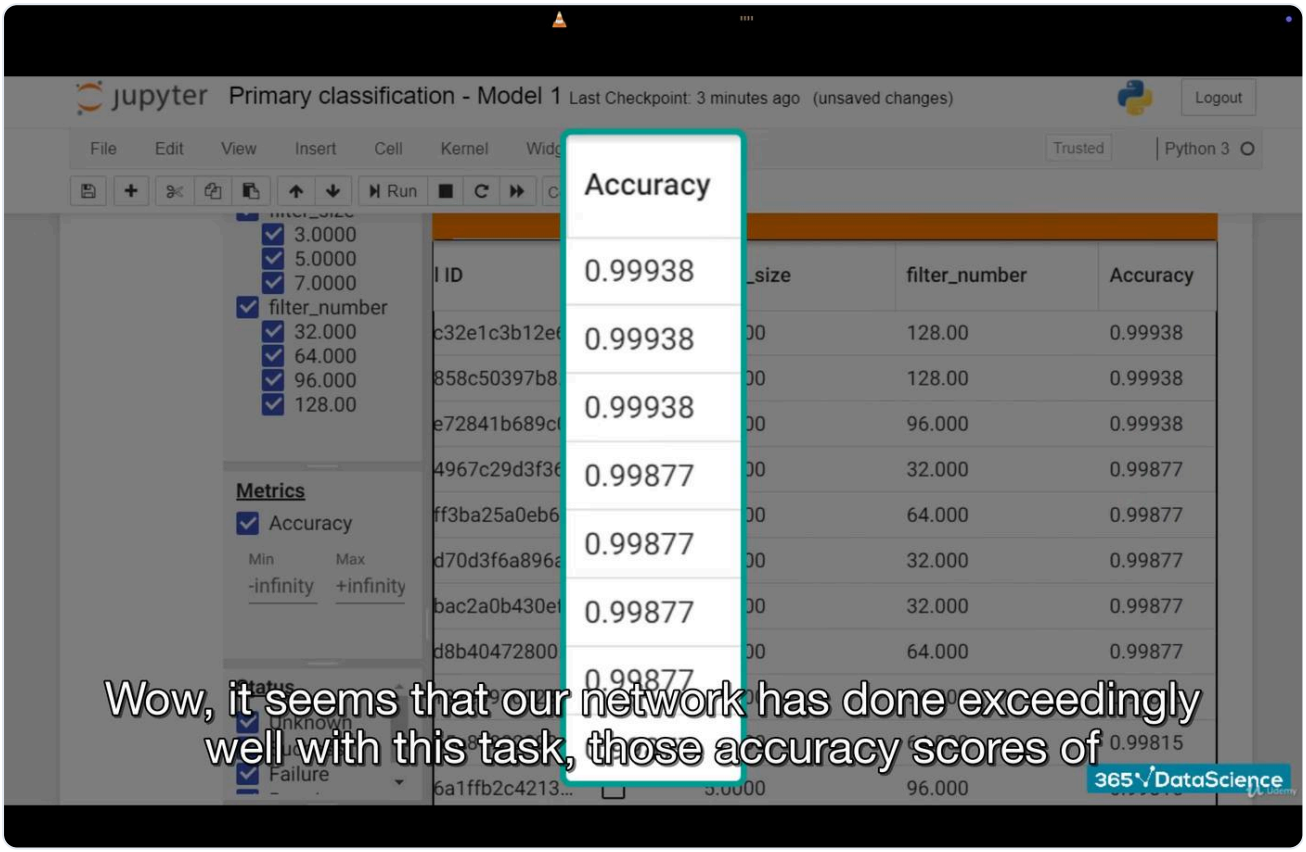


FIGURE 2.2

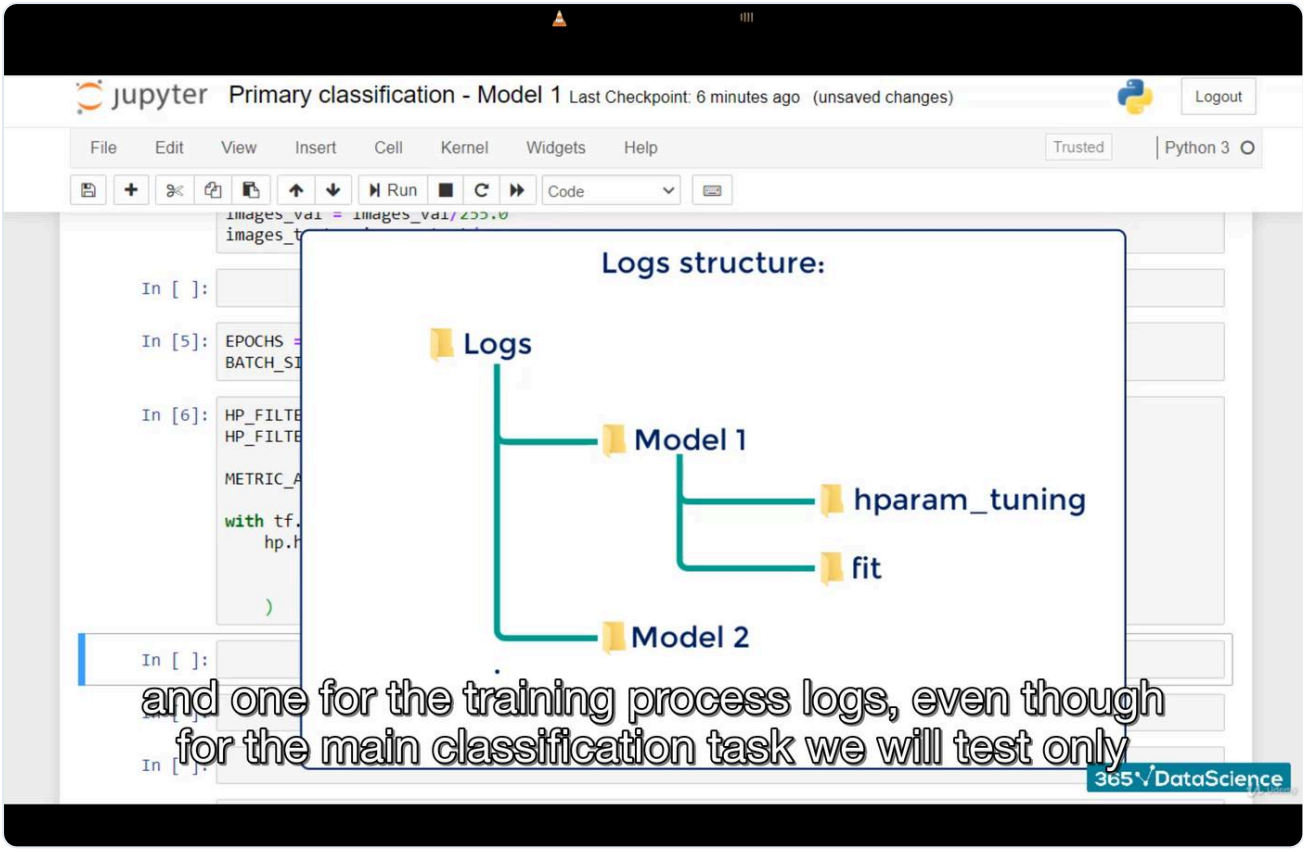


FIGURE 2.3

Wow, it seems that our network has done exceedingly well with this task, those accuracy scores of more than ninety nine point nine percent or even better than the amnesty example, it truly seems that the different image classes are extremely easy to distinguish. On top of that, the majority of runs have such high accuracy and all of them are above 99 percent. That's an amazing result. If we take a look at the hyper parameters that created the highest scores, we can't really see any pattern here, at least from the table view. So let's point our attention to the scatterplot view and extend the bottom part to see the whole graph. We can't really see any correlation between the hyper parameters and the accuracy here, either, nor for the filter size, neither for the filter, no.

So the conclusion is that this task is so easy that whatever we do, we get an almost ideal 100 percent accuracy. And that is the reason why we won't investigate the primary classification any further and we won't create different architectures or use any regularization techniques as there is just no need for that. Let's recap what we did so far. Well, it was a fair amount of work we took on the primary classification task to create a CNN that distinguishes between an image of a shoe trousers or glasses. To achieve this, we settled on a relatively simple model consisting of two convolutional layers, two max pool layers and the compulsory dence outcome layer.

We decided on this configuration because it keeps the training times lower and allows us to check for different hyper parameters. In terms of code. We imported the data sets and preprocess them created the functions for the hyper parameter tuning and confusion matrix and log the training process. Using TensorBoard, we tried out the model with a filter size of three, five and seven and number of filters set the thirty two. Sixty four, ninety six and one twenty eight.

The results were impressive, with more than ninety nine point nine percent accuracy across all different combinations of parameters. OK, so we are done with the primary classification, but there are many other many tasks still waiting to be solved.

In the next couple of videos we will be dealing with the trousers and jeans category and its unique challenges. In the meantime, you can practice and hone your skills with the glasses category. Your task is to create a model that will discriminate between pictures of glasses and sunglasses. All right. You can take some time to solidify what we did today and continue with the next lecture. See you there. And thanks for watching.

LECTURE 41

Combine the Labels

I hope you have practice with the glasses and sunglasses data set and are now comfortable with the code structure and purpose, as we will now go much quicker over the next model's code.

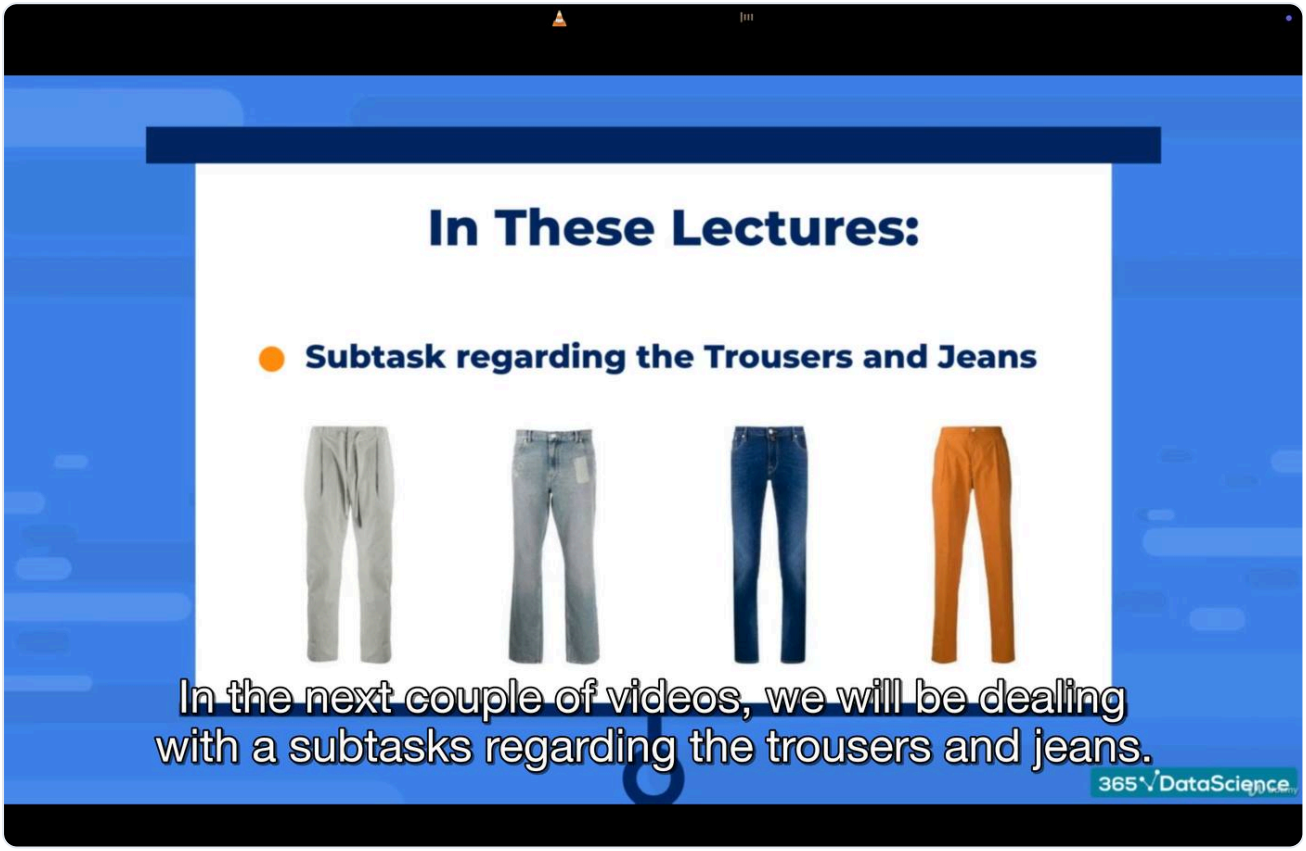


FIGURE 3.1

In the next couple of videos, we will be dealing with a subtasks regarding the trousers and jeans. The task is the following.

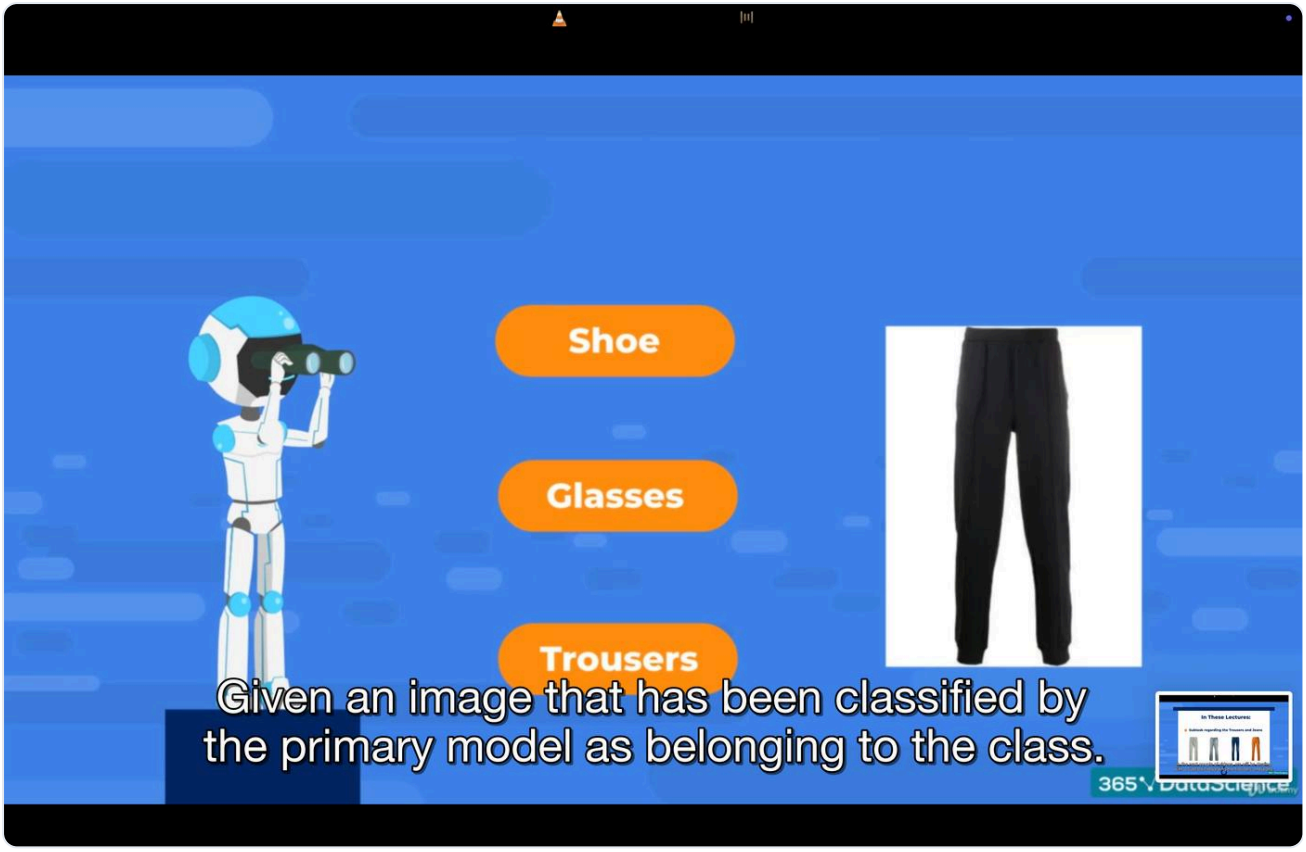


FIGURE 3.2

Given an image that has been classified by the primary model as belonging to the class.

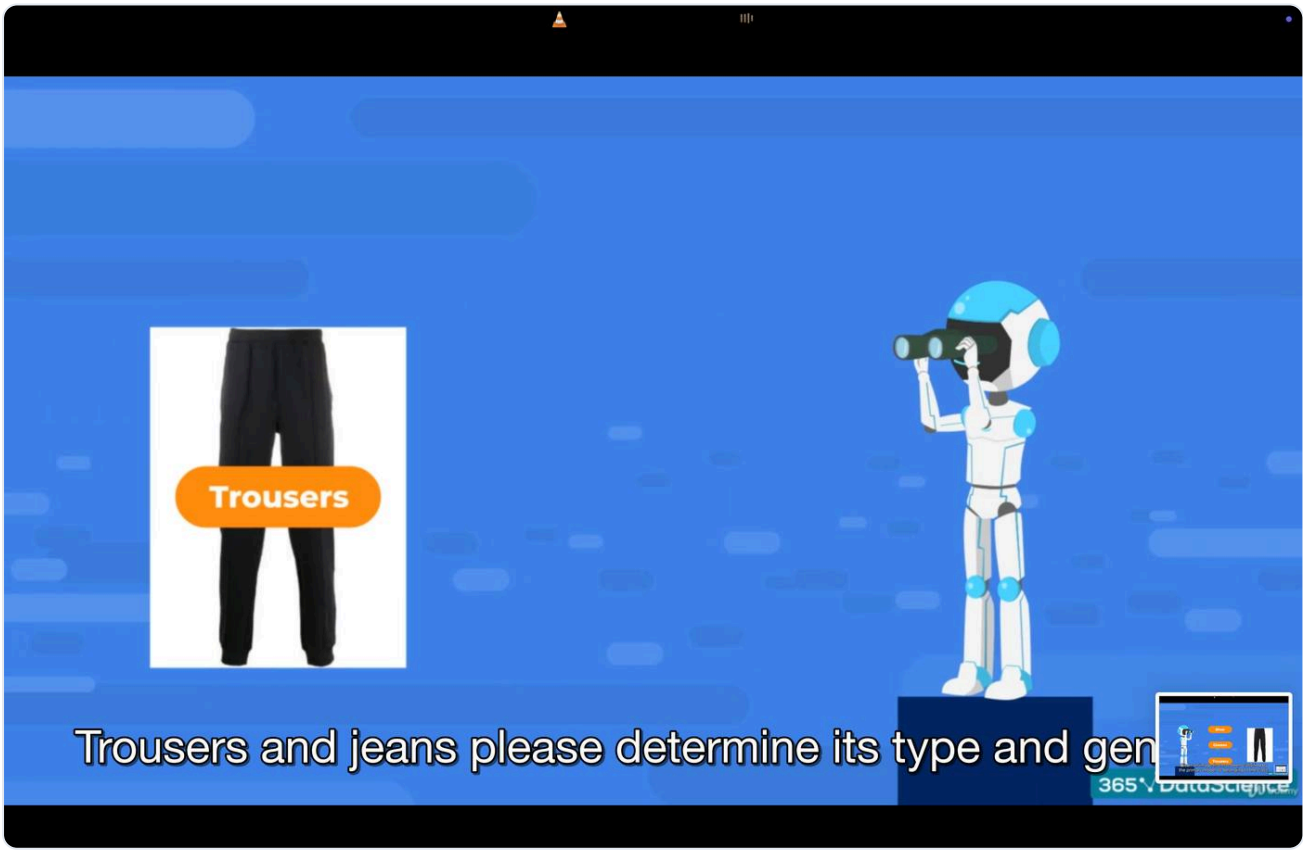


FIGURE 3.3

Trousers and jeans please determine its type and gender.

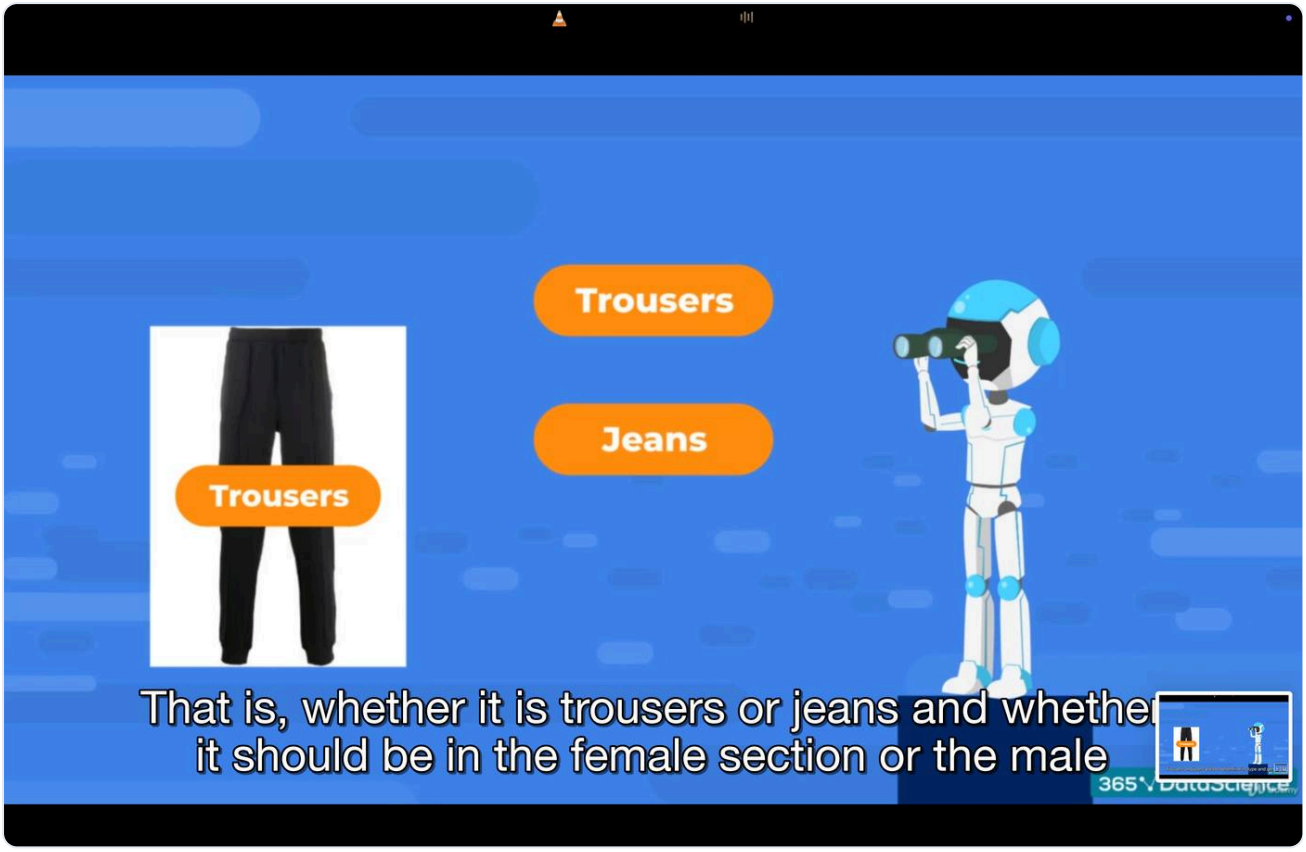


FIGURE 3.4

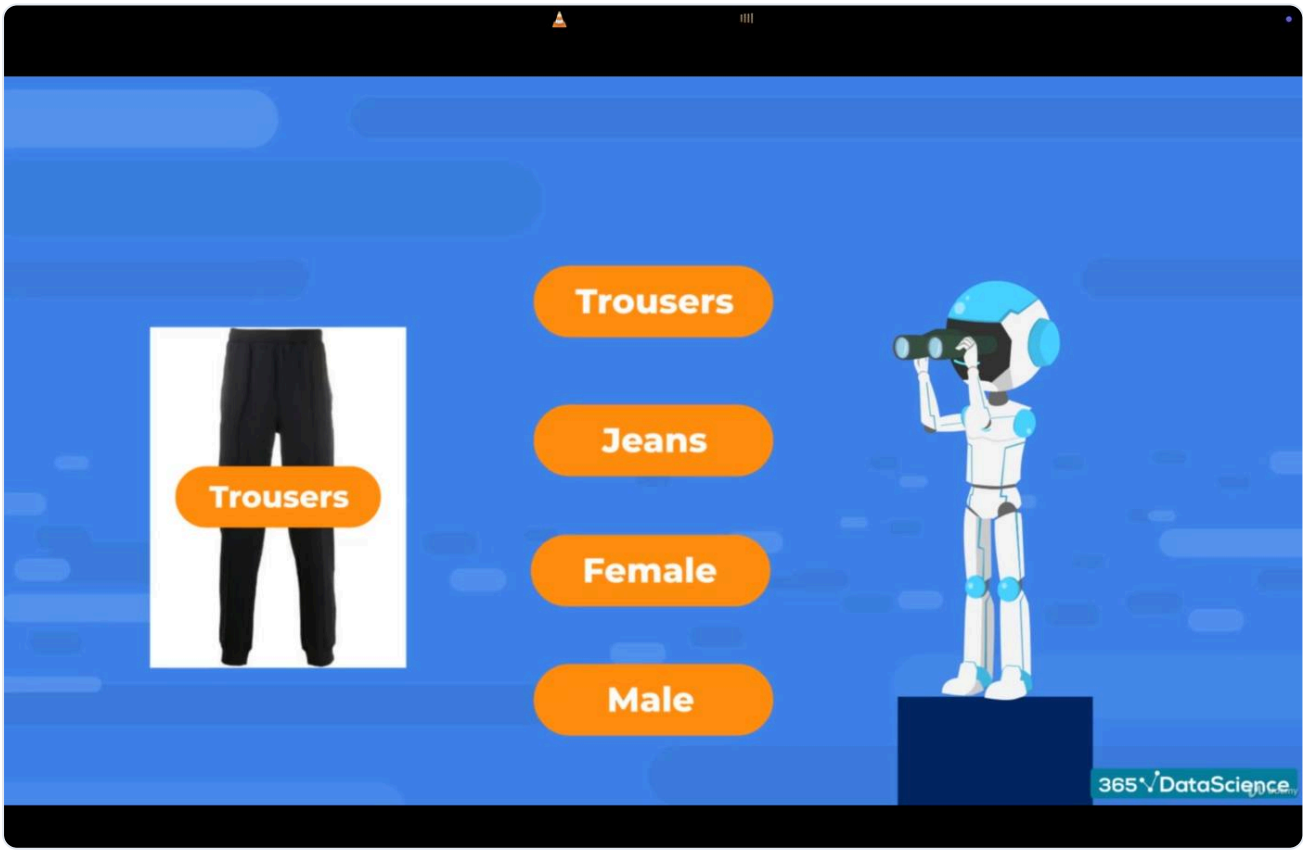


FIGURE 3.5

That is, whether it is trousers or jeans and whether it should be in the female section or the male one.

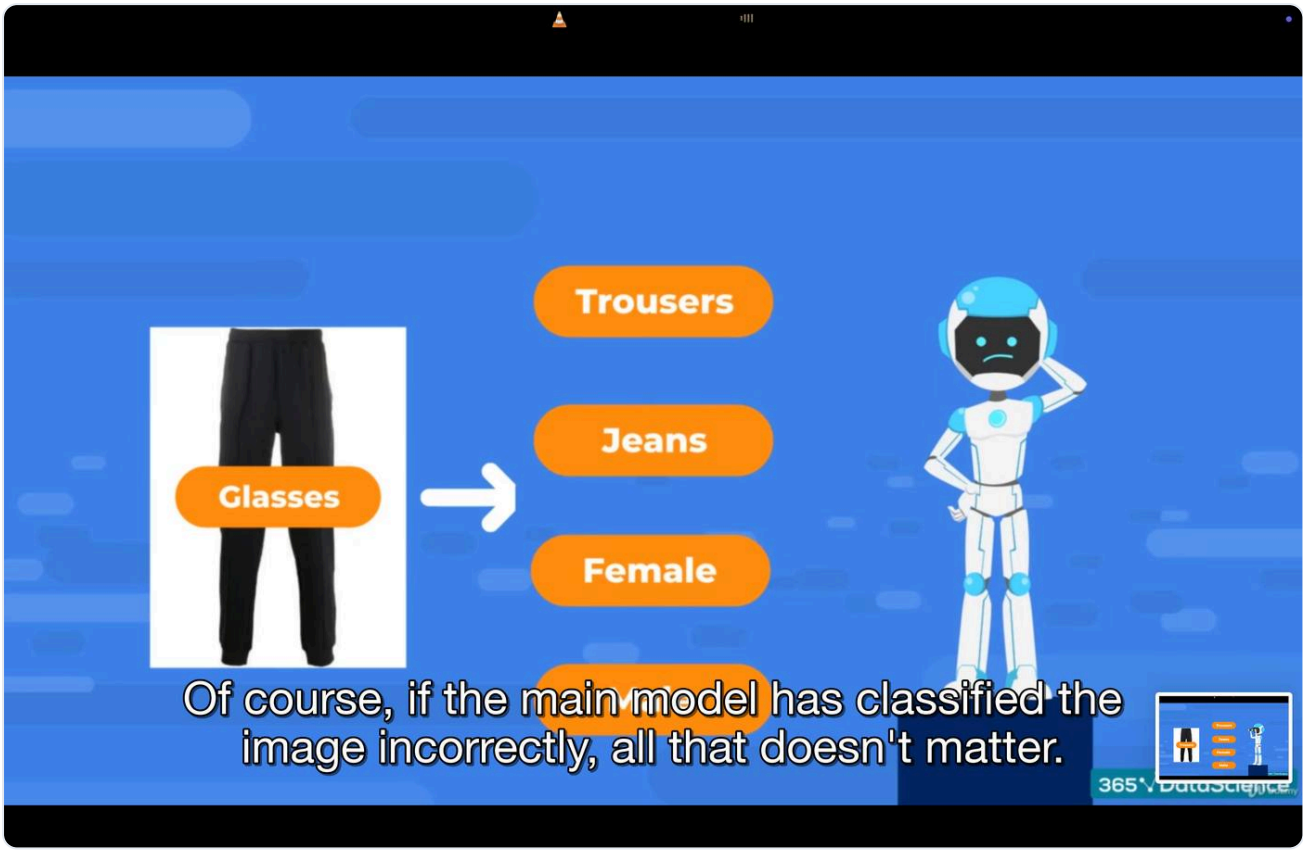


FIGURE 3.6

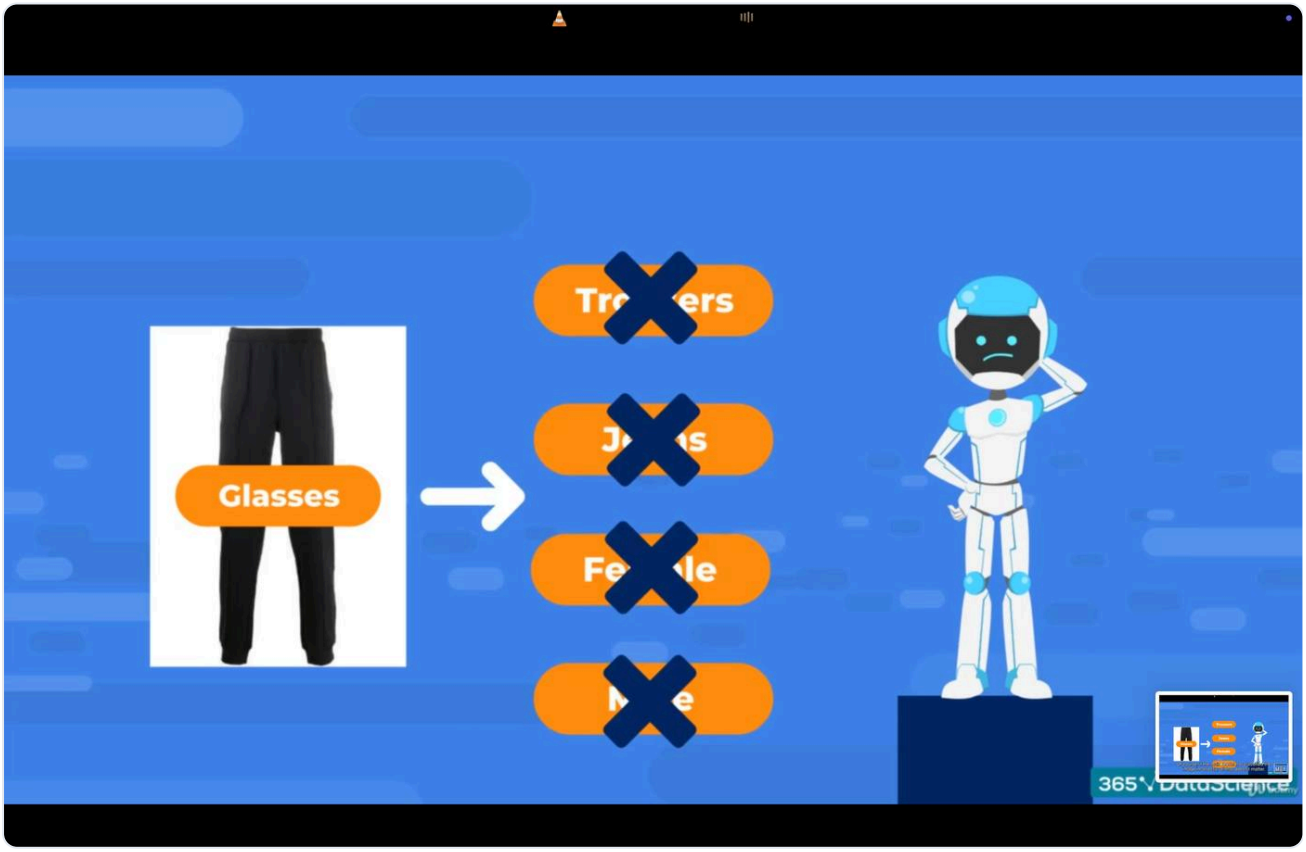


FIGURE 3.7

Of course, if the main model has classified the image incorrectly, all that doesn't matter.

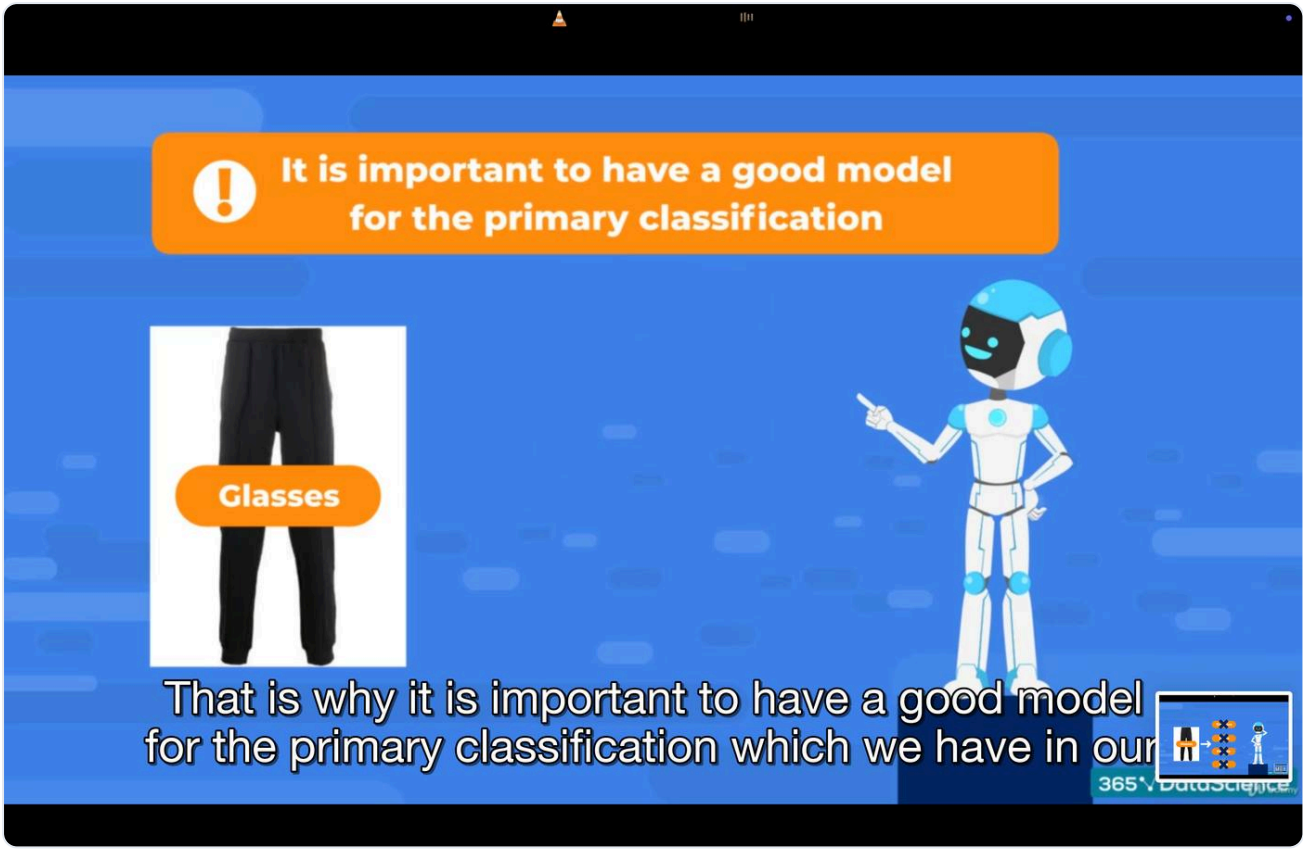


FIGURE 3.8

That is why it is important to have a good model for the primary classification which we have in our case. OK.

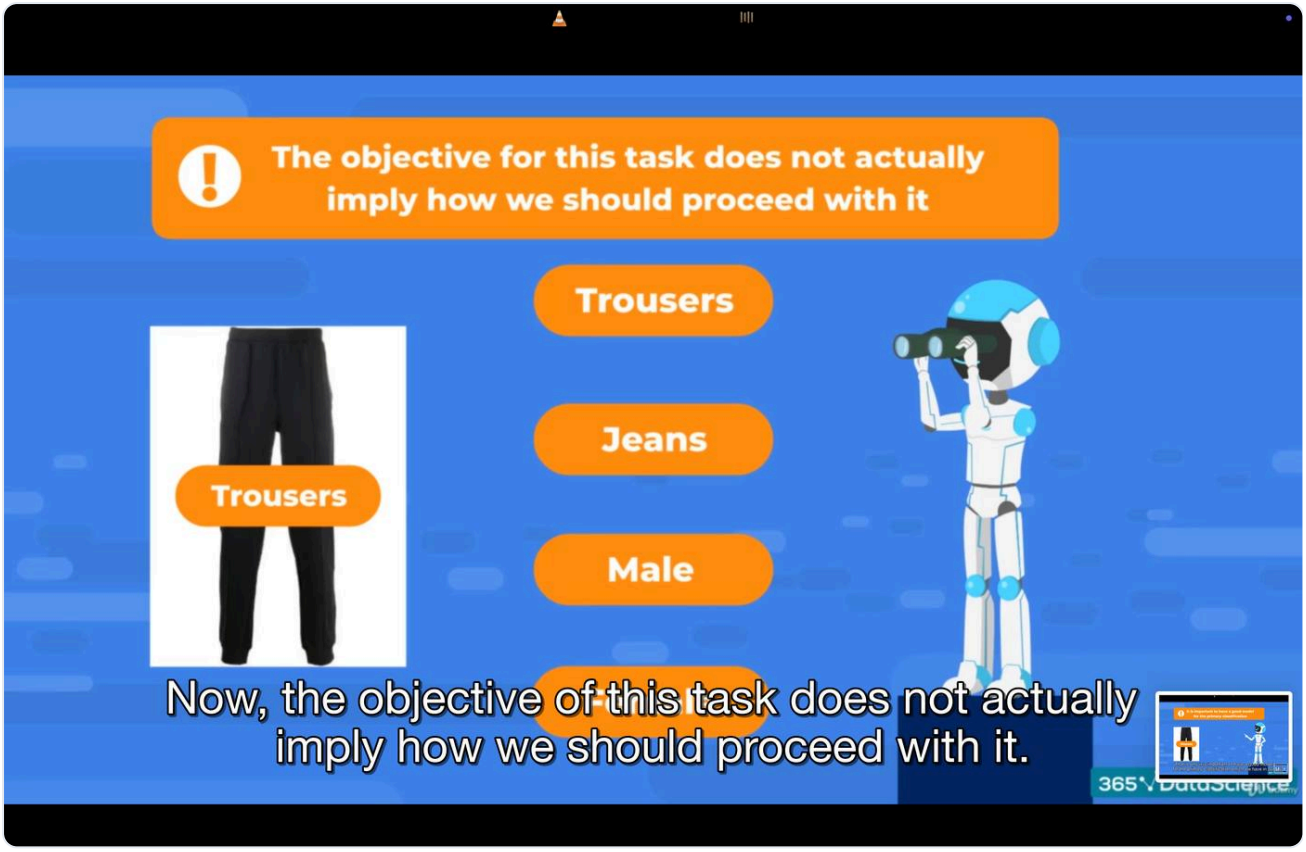


FIGURE 3.9

Now, the objective of this task does not actually imply how we should proceed with it.

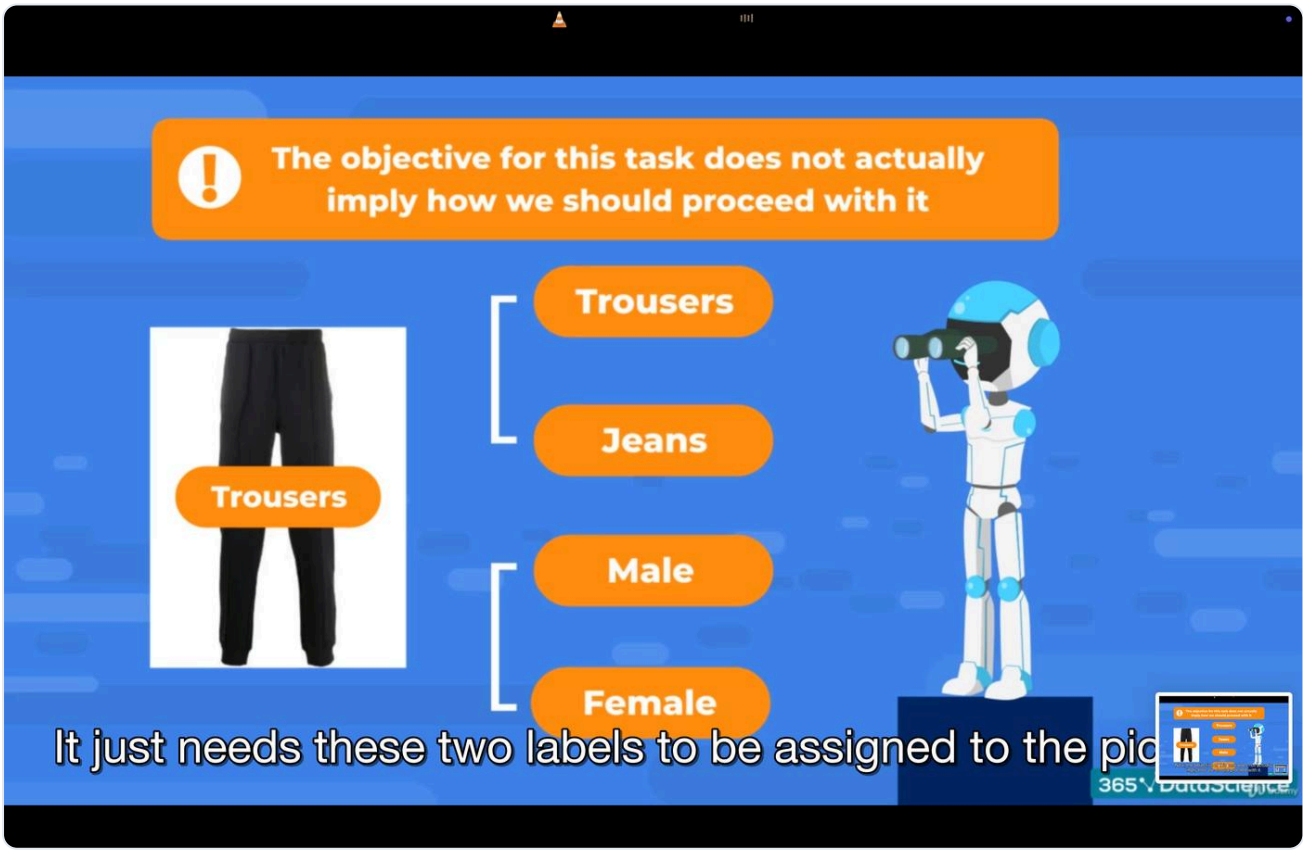


FIGURE 3.10

It just needs these two labels to be assigned to the picture. We know how to associate an image with a single class, but how do you assign it to labels?

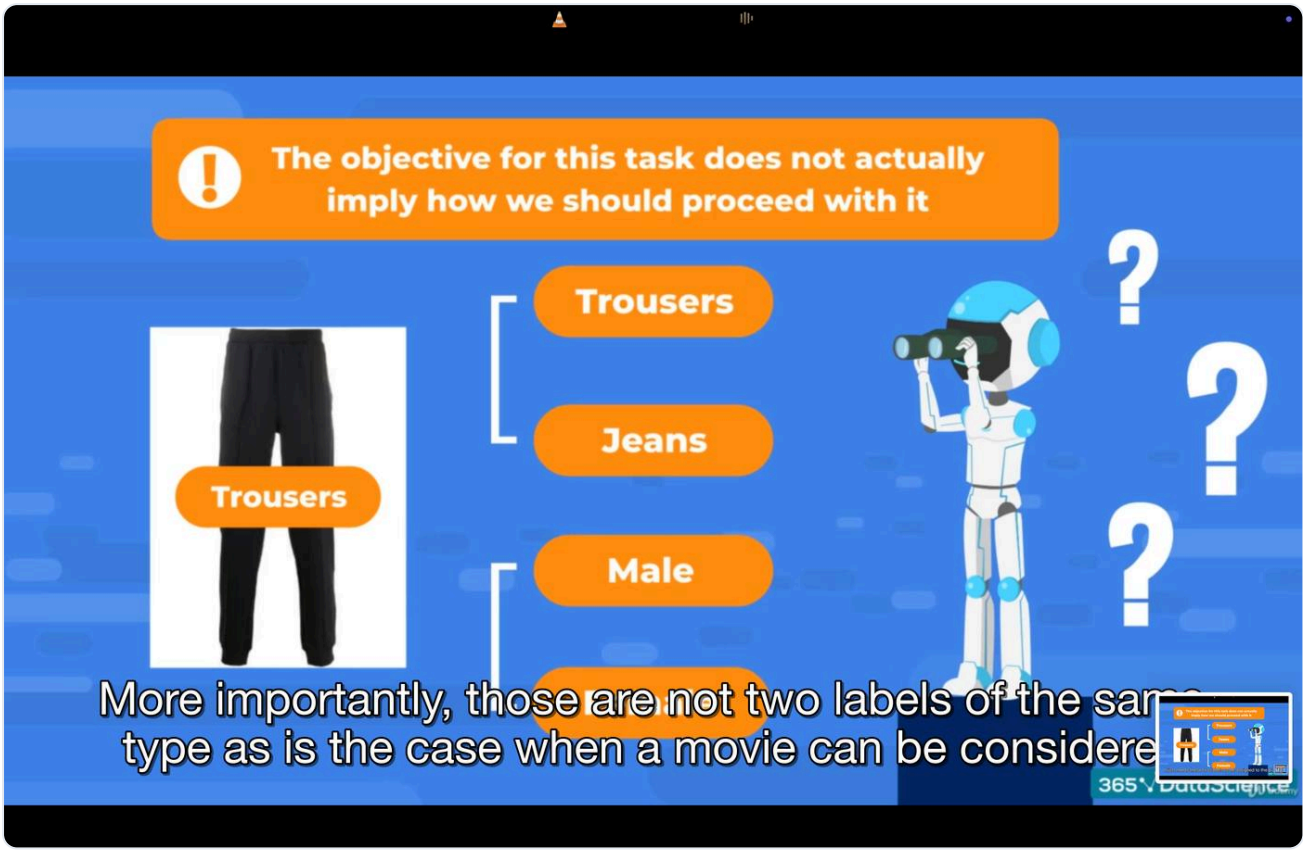


FIGURE 3.11

More importantly, those are not two labels of the same type as is the case when a movie can be considered to belong to more than one genre. The different classes like comedy, drama, crime are of the same type. They are all genres.

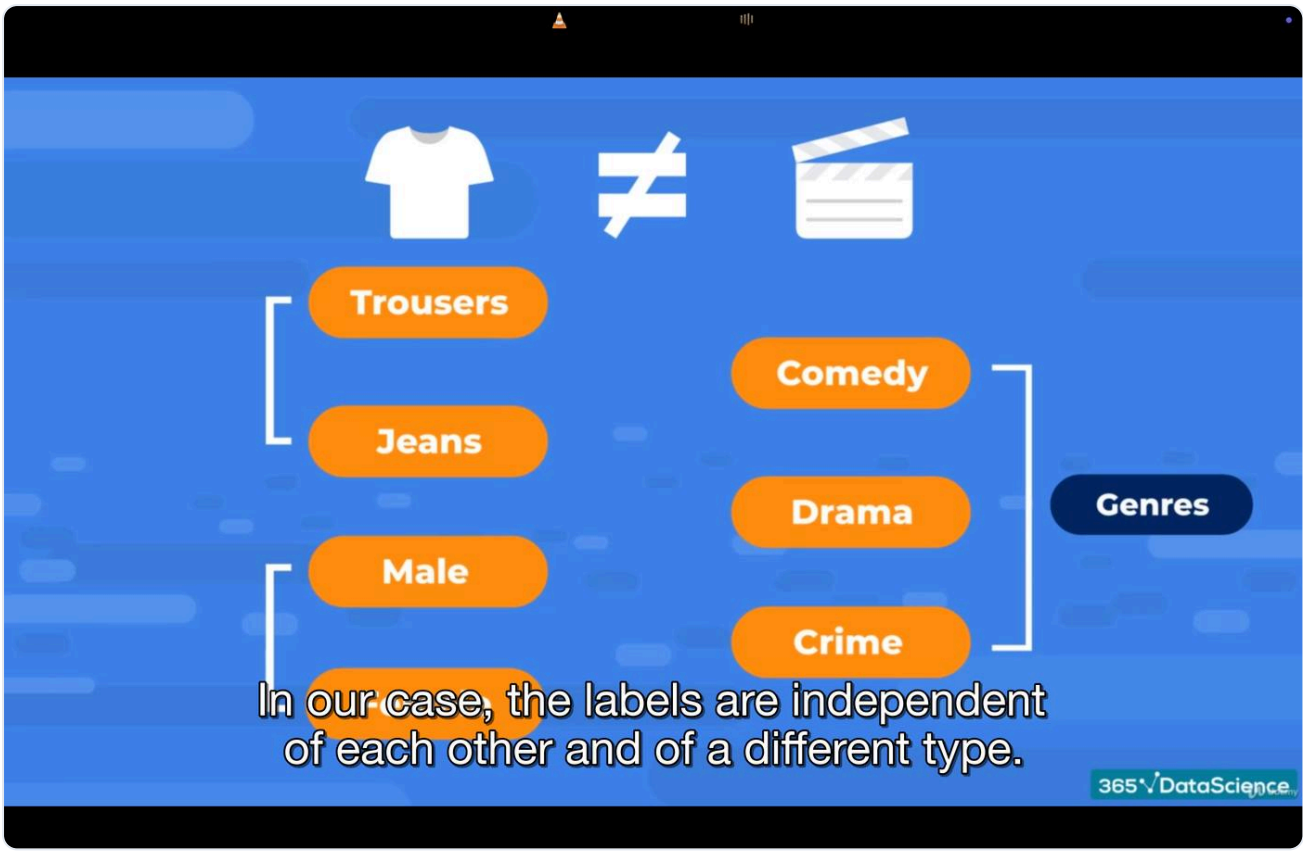


FIGURE 3.12

In our case, the labels are independent of each other and of a different type. One is gender.

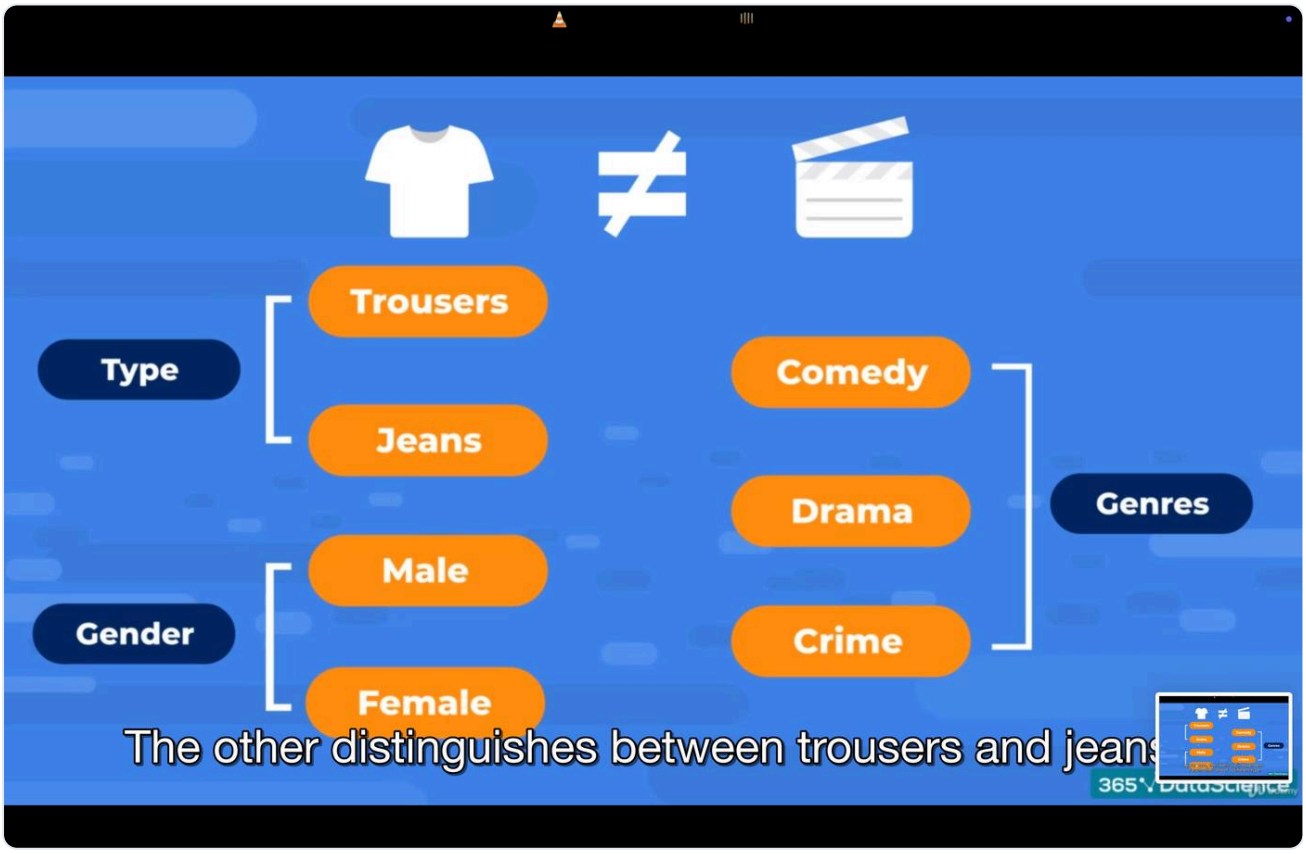


FIGURE 3.13

The other distinguishes between trousers and jeans. Well, there are multiple approaches we can go for in this task and we will investigate the two most logical ones.

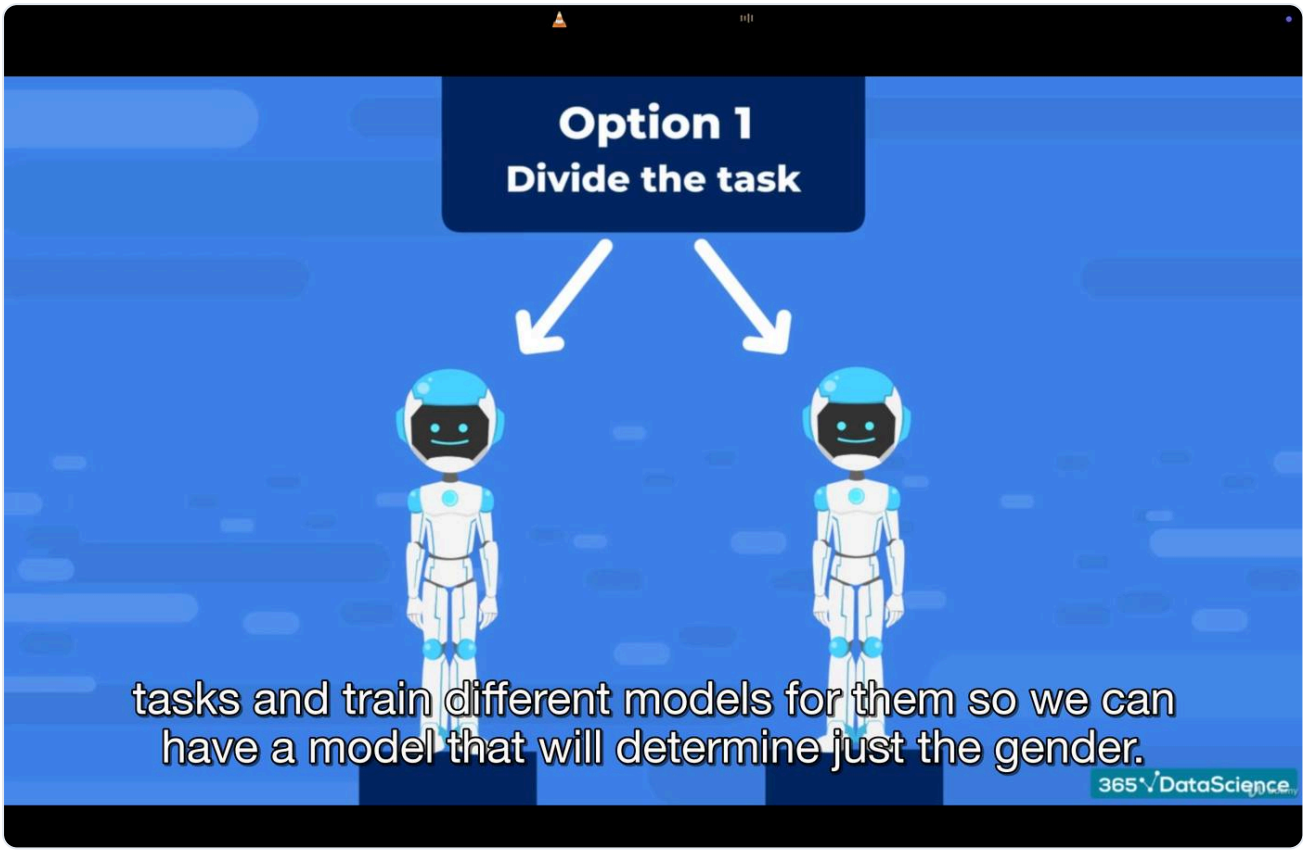


FIGURE 3.14

The first option is to do the same thing we did for the overall task in this section divided into many tasks and train different models for them so we can have a model that will determine just the gender.

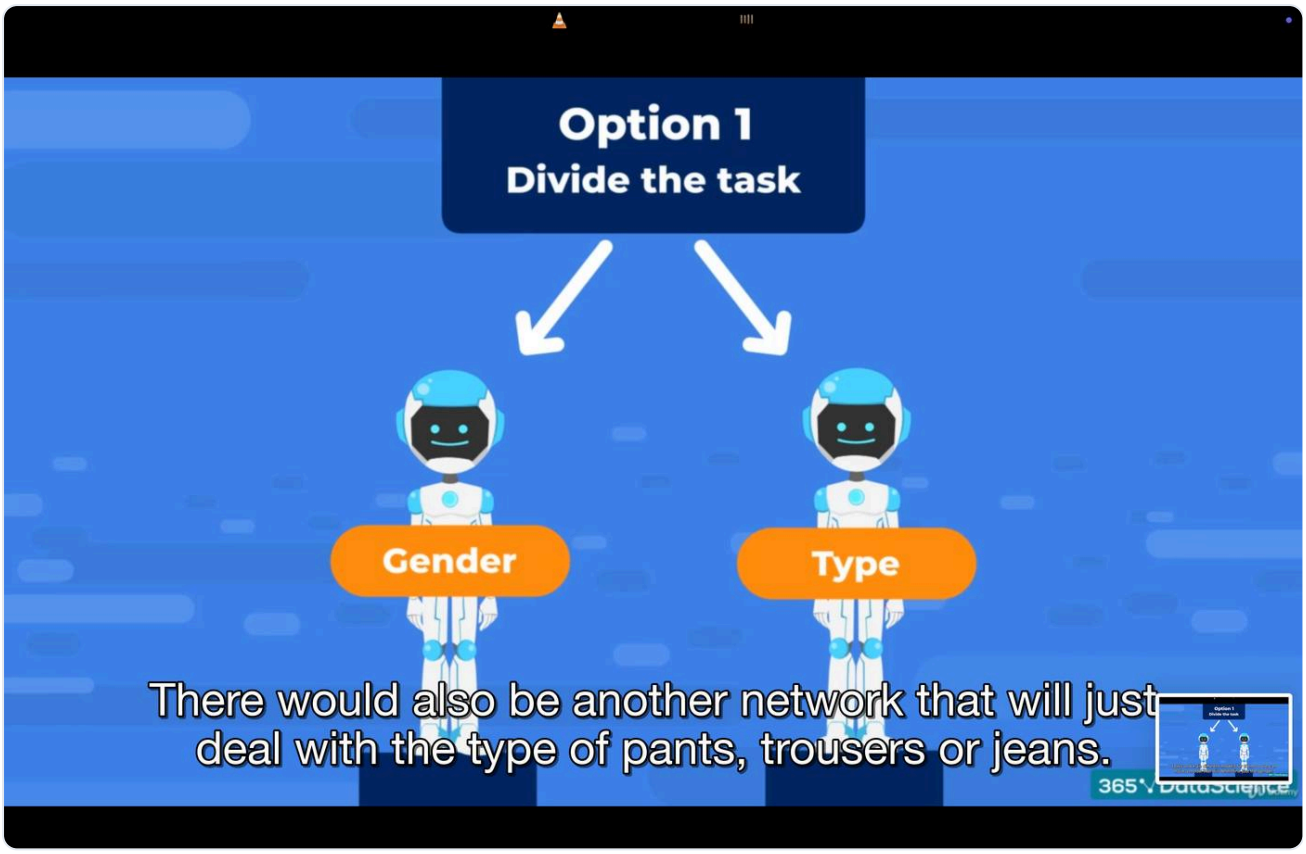


FIGURE 3.15

There would also be another network that will just deal with the type of pants, trousers or jeans. As for the second option, the idea is to combine the different labels into one.

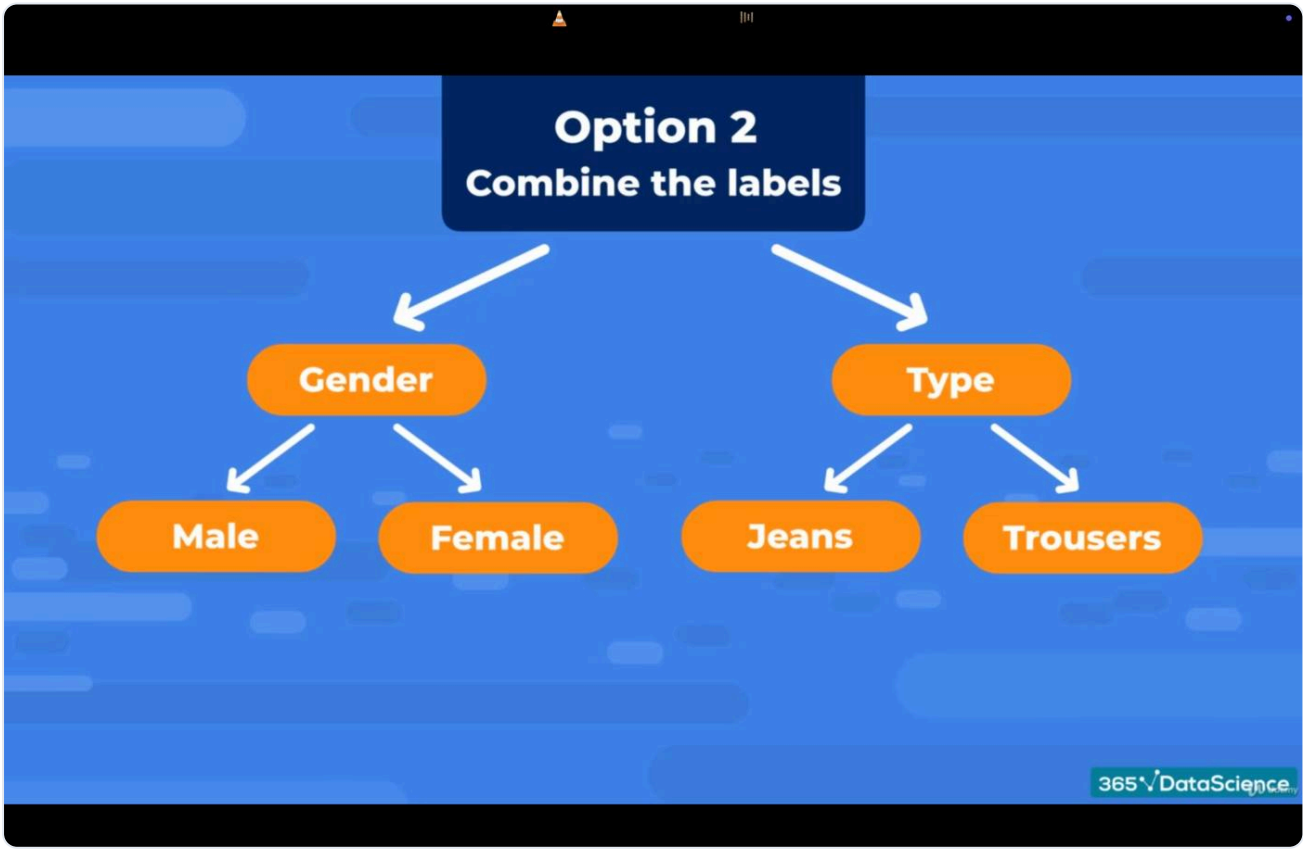


FIGURE 3.16 | Combine the Labels

So given we have two labels, each with two classes, that makes a total of four combinations. What we can do is set each of these four combinations as a single class, thus combining the two labels into one.

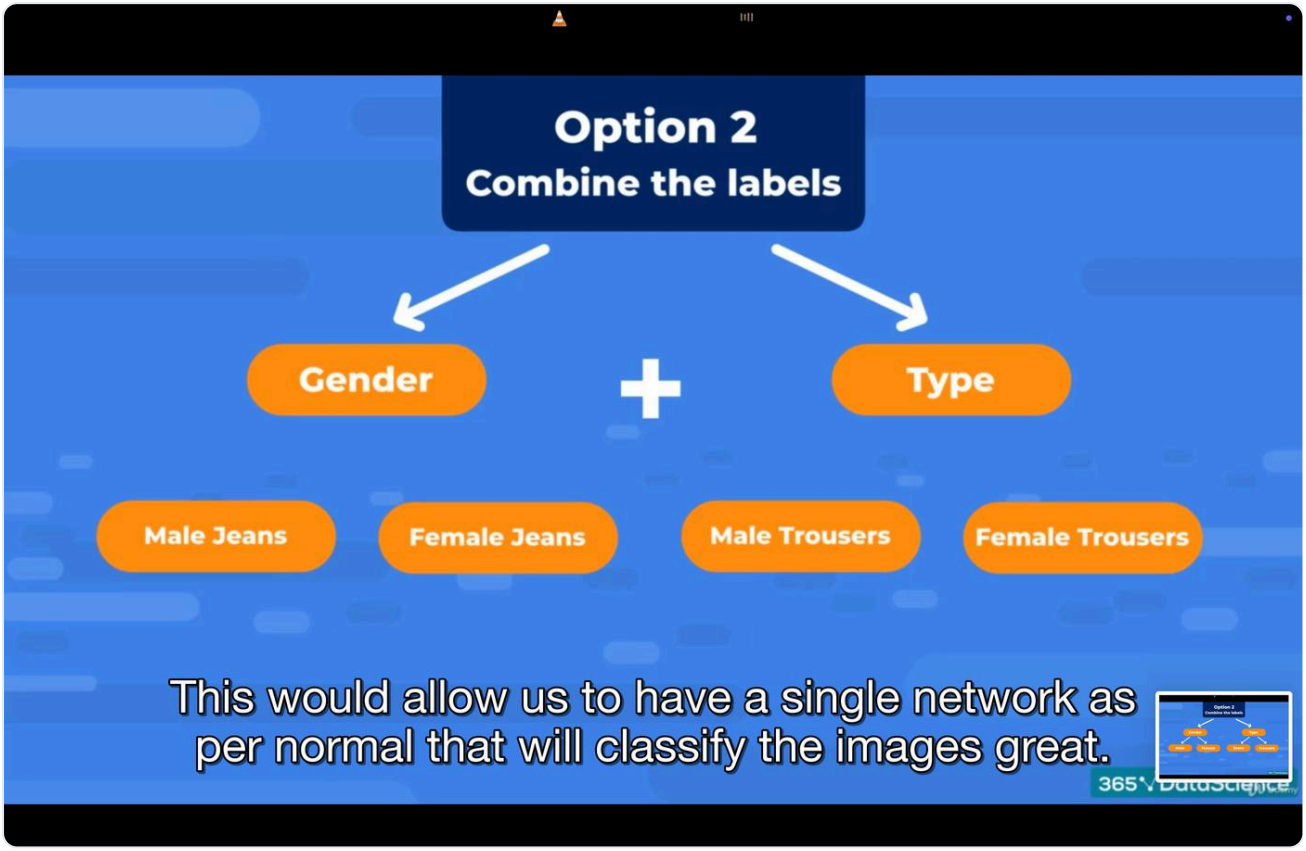


FIGURE 3.17 | Combine the Labels

This would allow us to have a single network as per normal that will classify the images great.

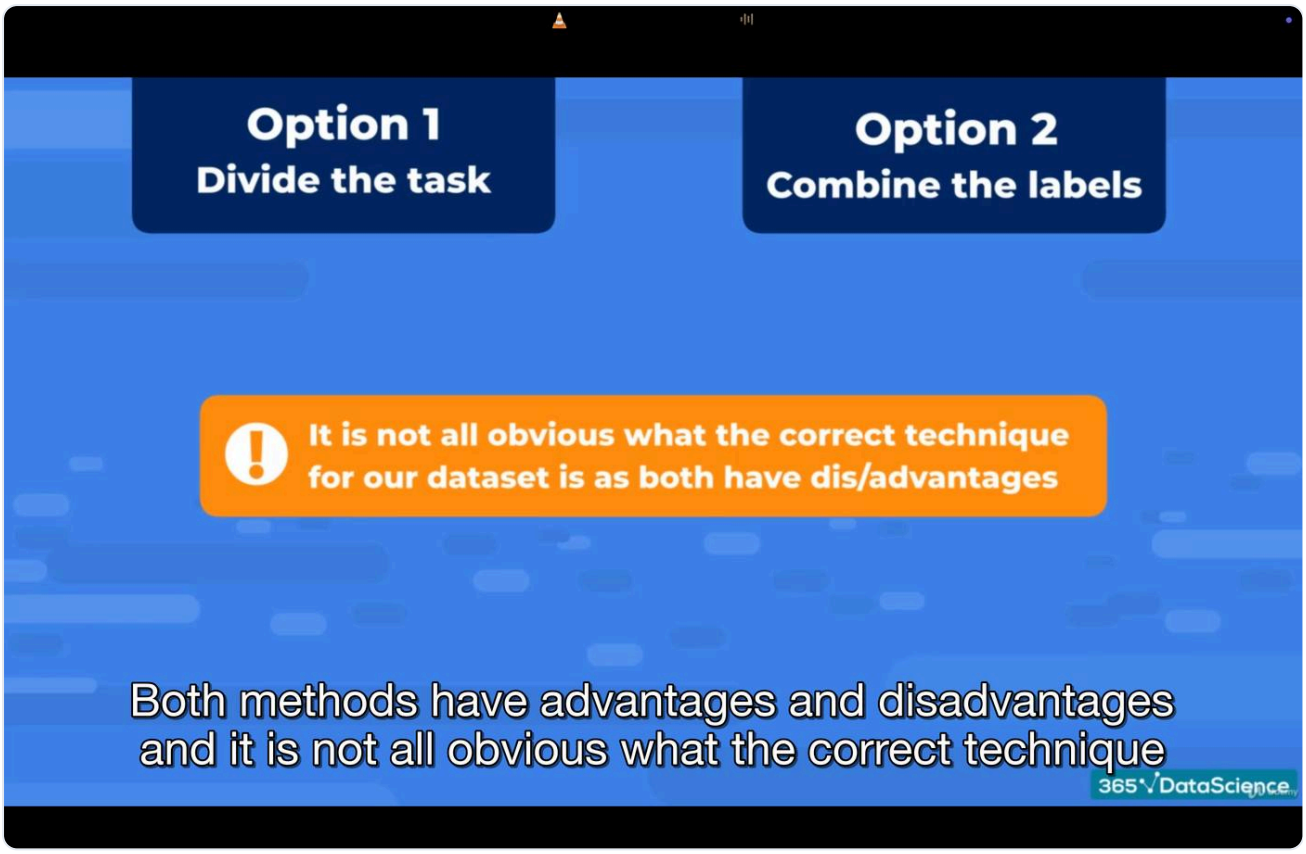


FIGURE 3.18 | Divide the Task Combine the Labels

Both methods have advantages and disadvantages and it is not all obvious what the correct technique for our data set is. If we employ the first option with the many tasks we will need to train and tune to different networks from scratch.

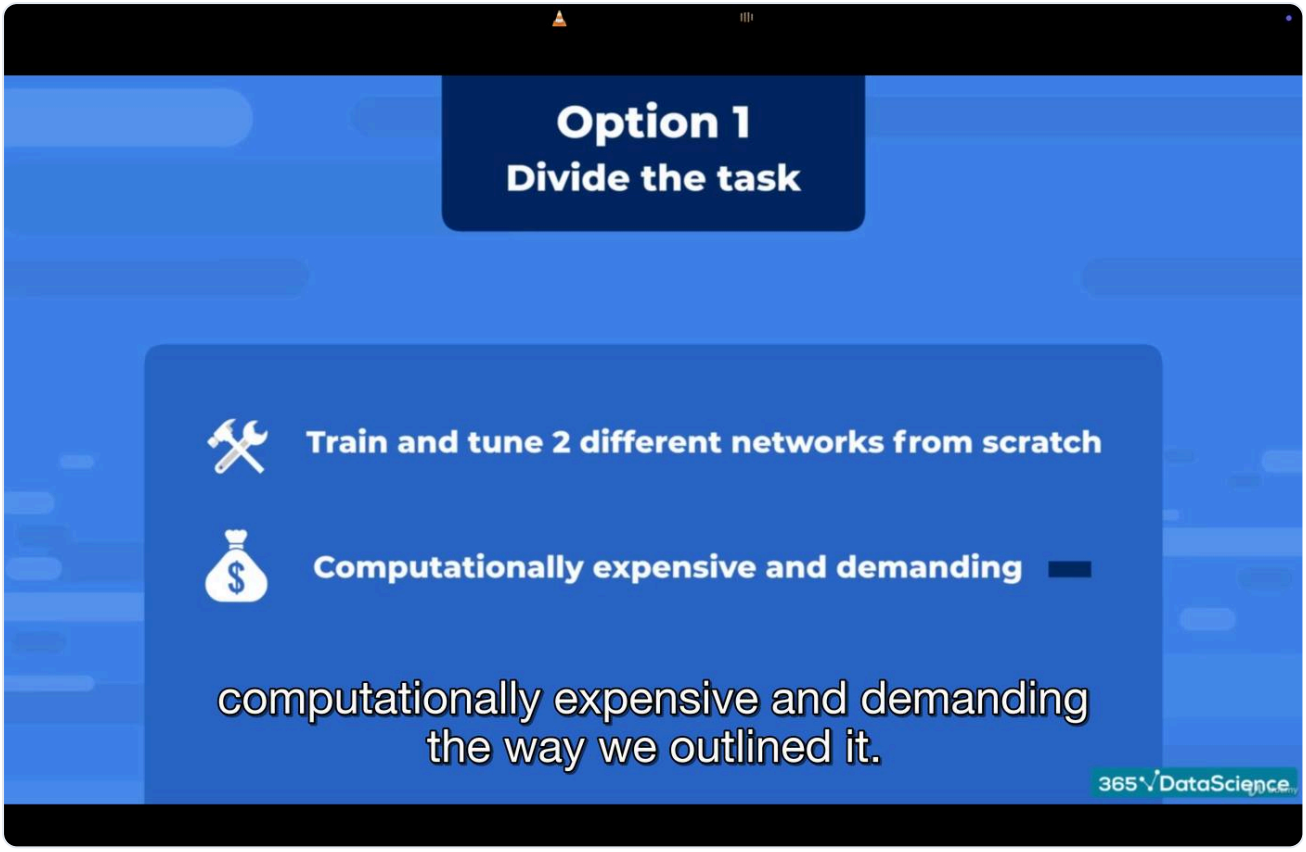


FIGURE 3.19

They might require different architectures and hyper parameters, which makes this option potentially computationally expensive and demanding the way we outlined it. Both models are on equal footing. However, there's also the matter of hierarchy.

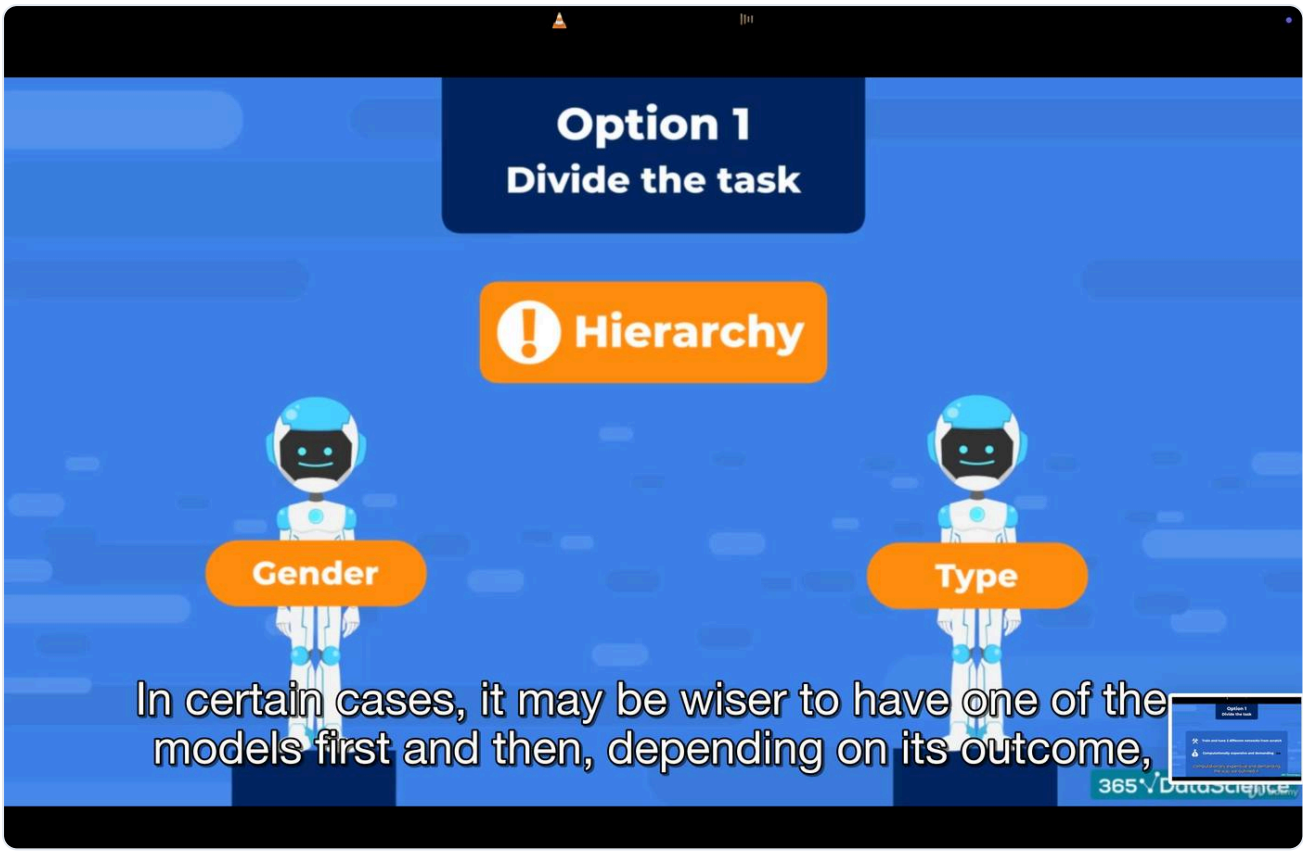


FIGURE 3.20

In certain cases, it may be wiser to have one of the models first and then, depending on its outcome, to have other different models.

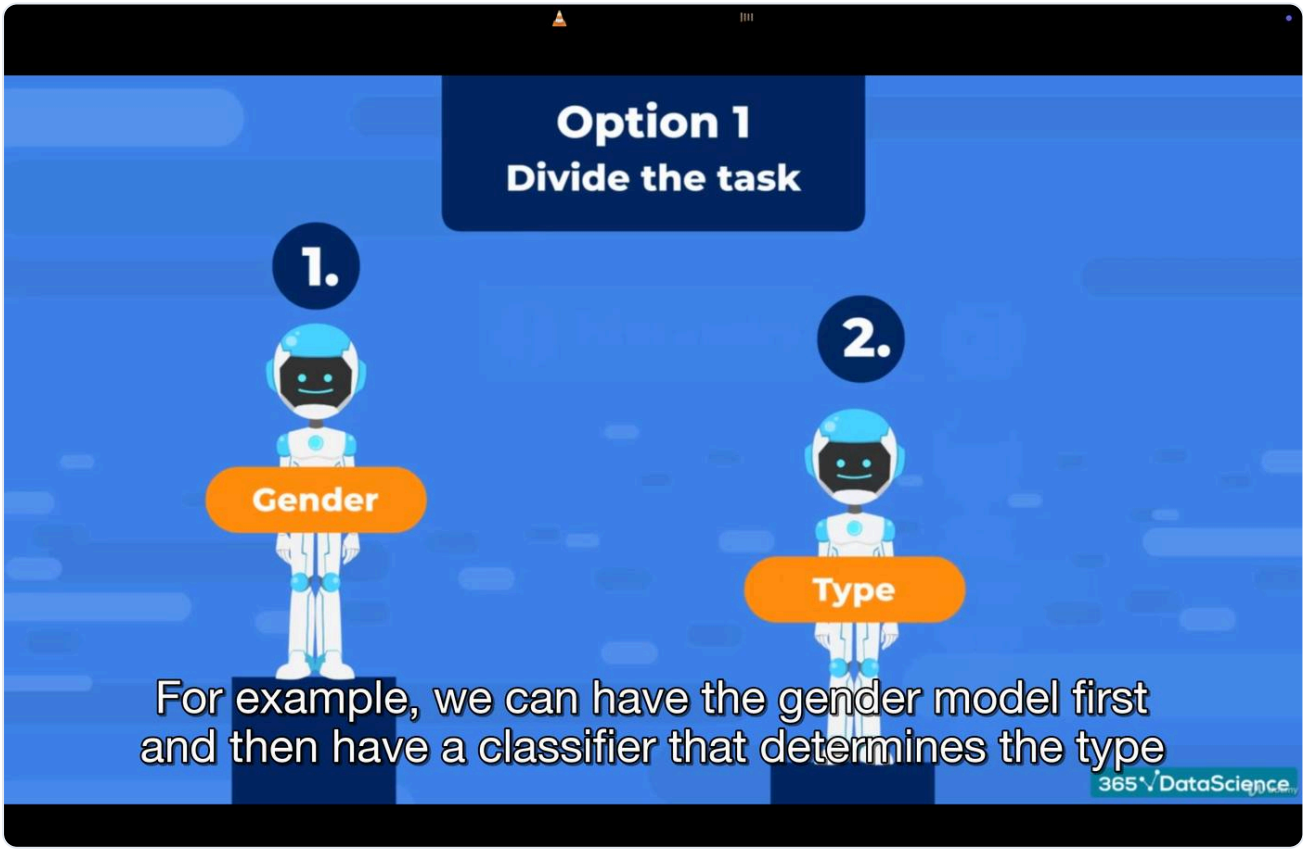


FIGURE 3.21

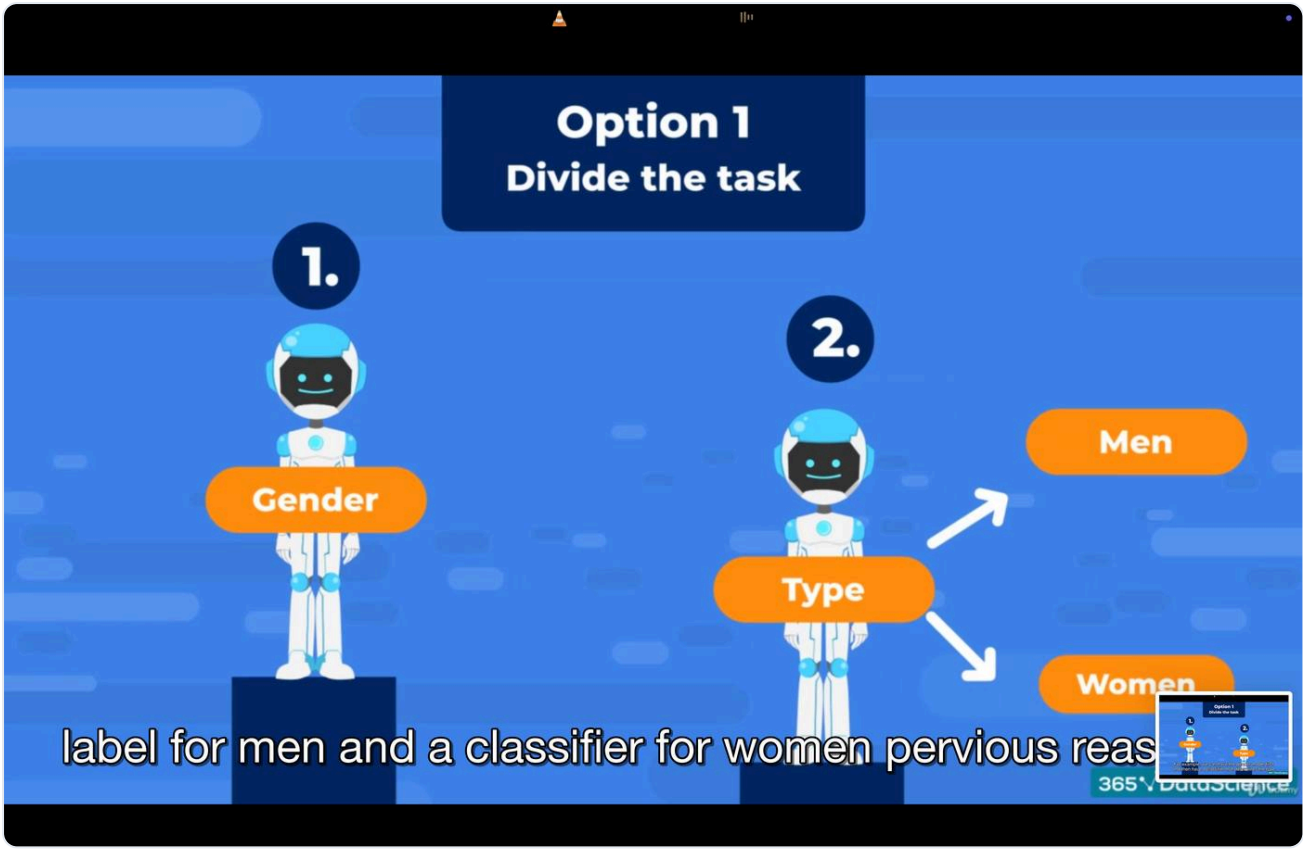


FIGURE 3.22

For example, we can have the gender model first and then have a classifier that determines the type label for men and a classifier for women previous reasons. This approach is called hierarchical classification. This may work if men's jeans and women's jeans are nothing like each other.

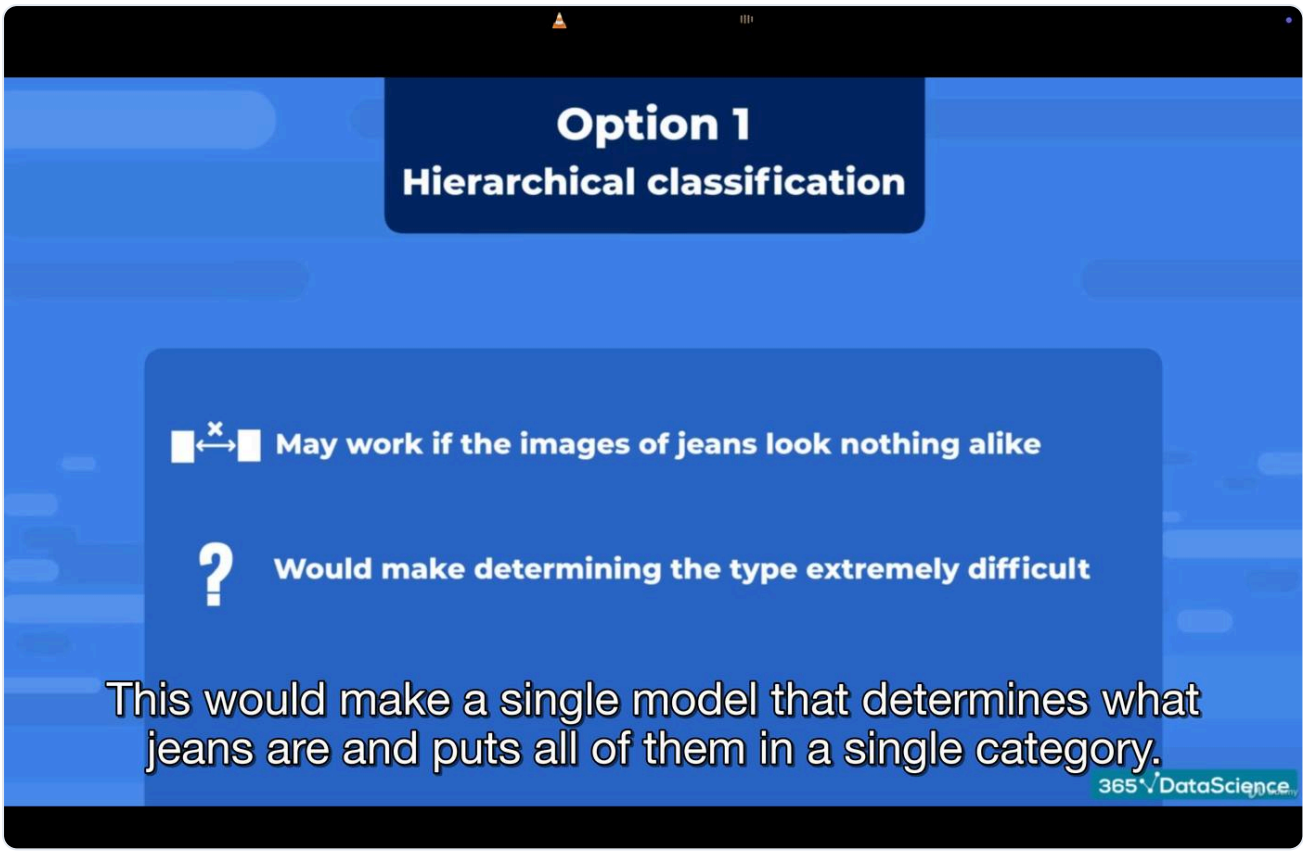


FIGURE 3.23 | Hierarchical Classification

This would make a single model that determines what jeans are and puts all of them in a single category. Extremely difficult to achieve.

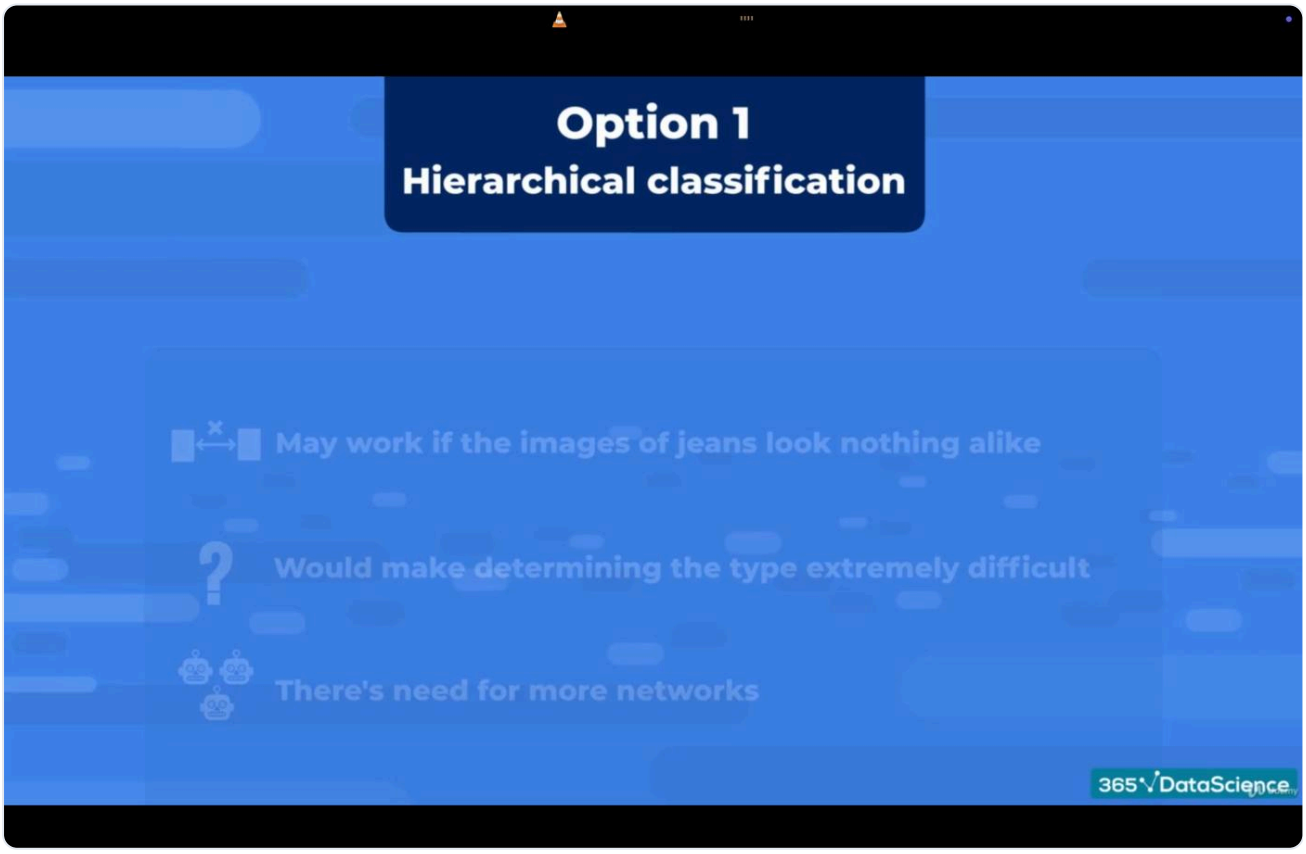


FIGURE 3.24 | Hierarchical Classification

Things to consider in this case, though, are that we will need to train even more networks.

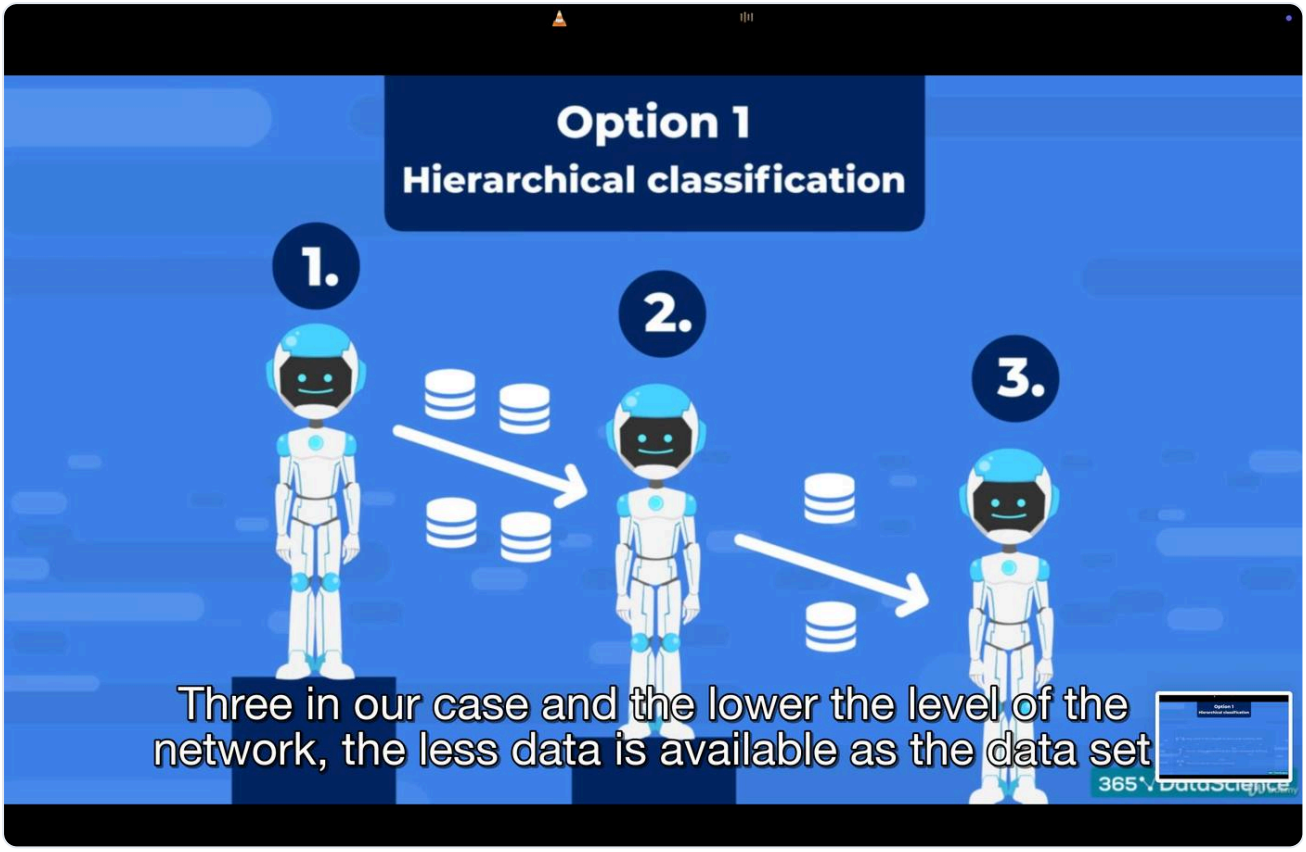


FIGURE 3.25 | Hierarchical Classification

Three in our case and the lower the level of the network, the less data is available as the data set needs to be split for the different outcomes. Those were the cons, but there are still some pros with this technique. The advantage of defining these many tasks is that we might still get partially correct answers if gender is really easy to determine. But the type extremely difficult.

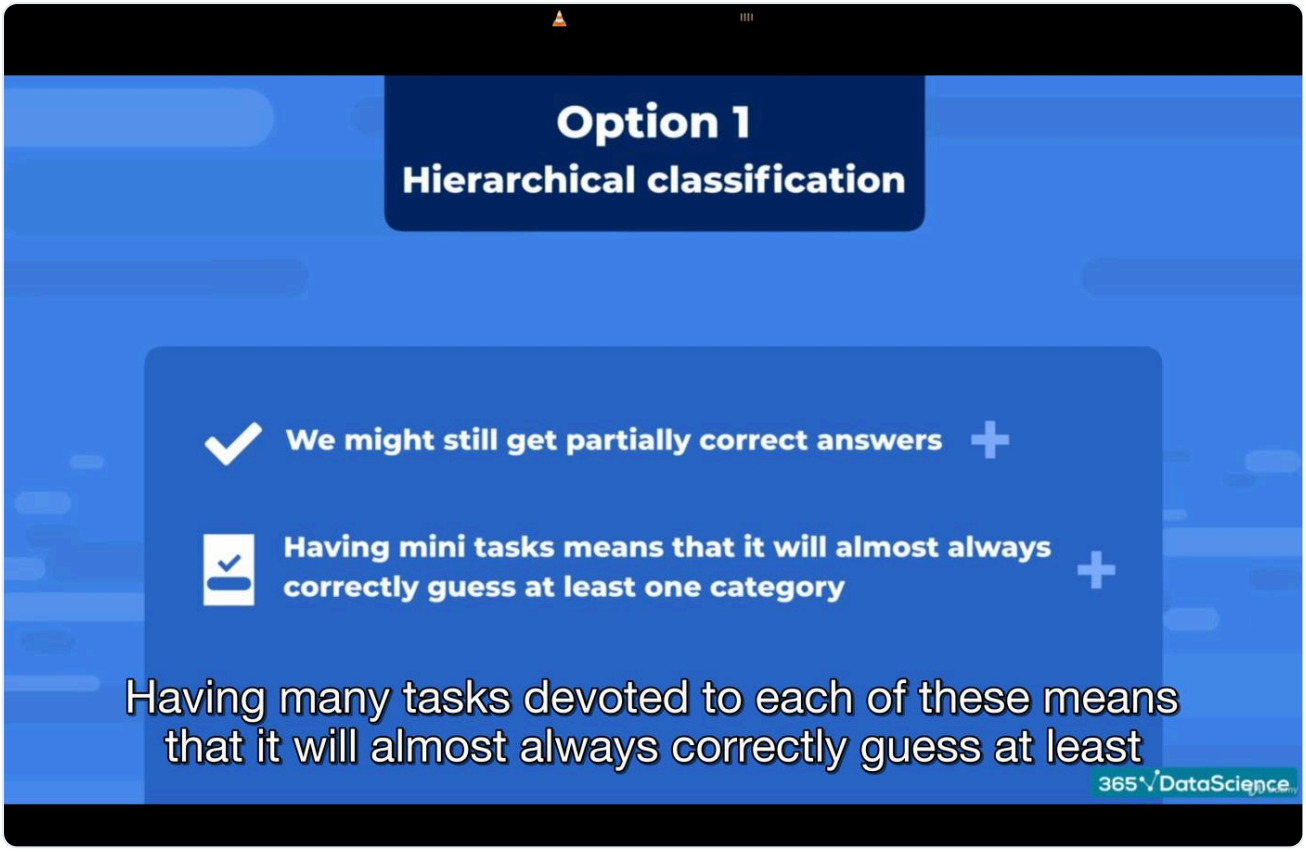


FIGURE 3.26 | Hierarchical Classification

Having many tasks devoted to each of these means that it will almost always correctly guess at least the gender.

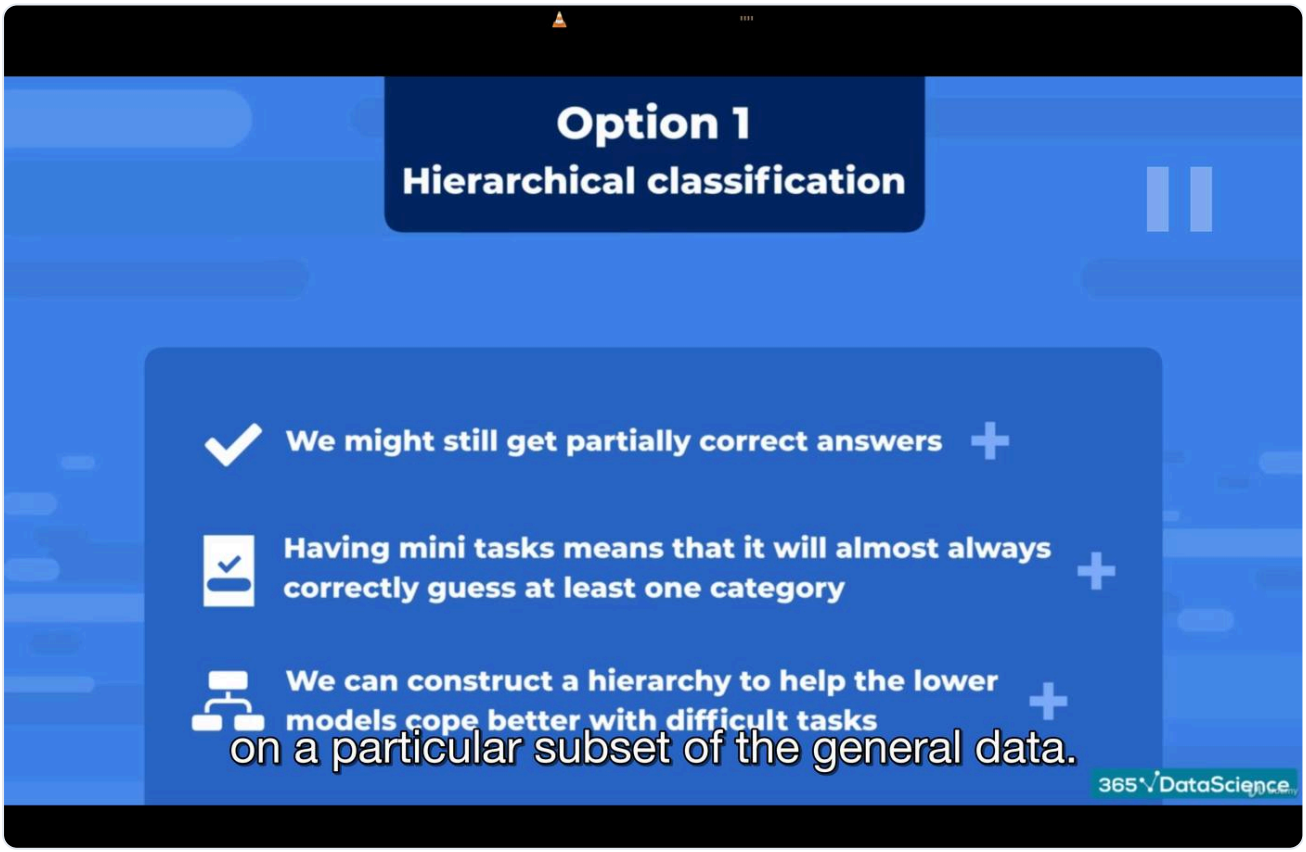


FIGURE 3.27 | Hierarchical Classification

And if we know which label is the easier one, we can construct a hierarchy with the easier one on top, which might help the models below cope better with their more difficult tasks as they are more focused on a particular subset of the general data. All right, the second option has its own potential drawbacks, too. It combines all labels into one, giving all labels equal weight.

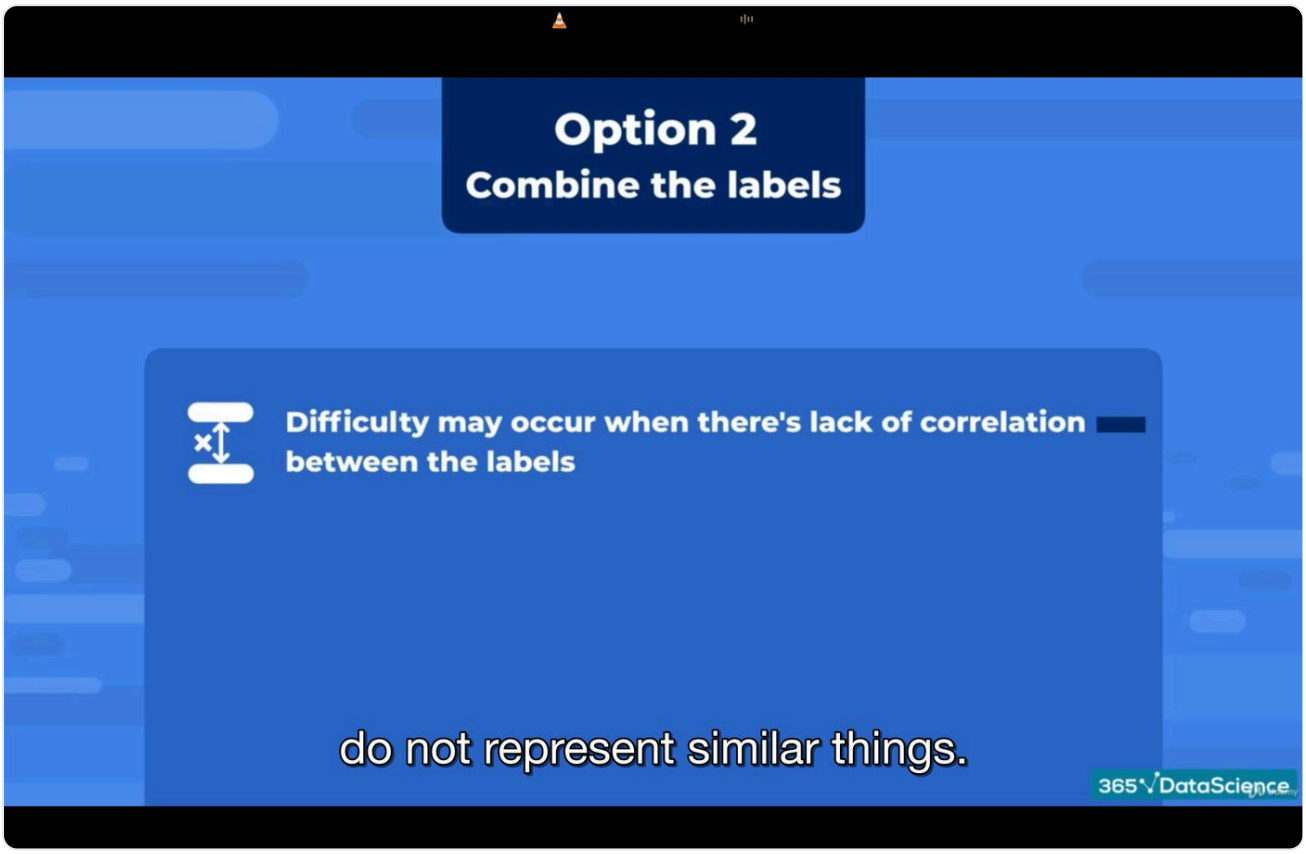


FIGURE 3.28 | Combine the Labels

However, that means it may struggle when there is no correlation between the different labels or they do not represent similar things.

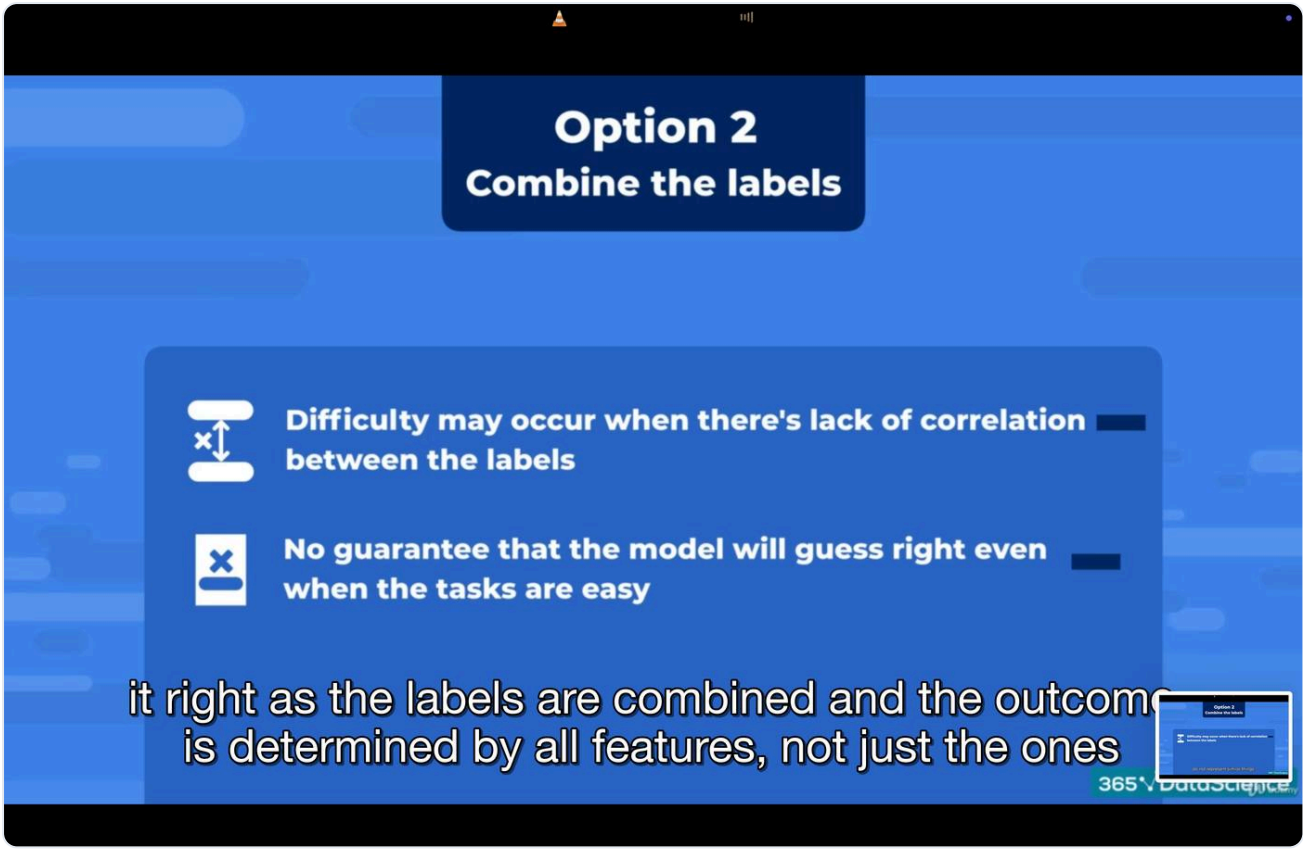


FIGURE 3.29

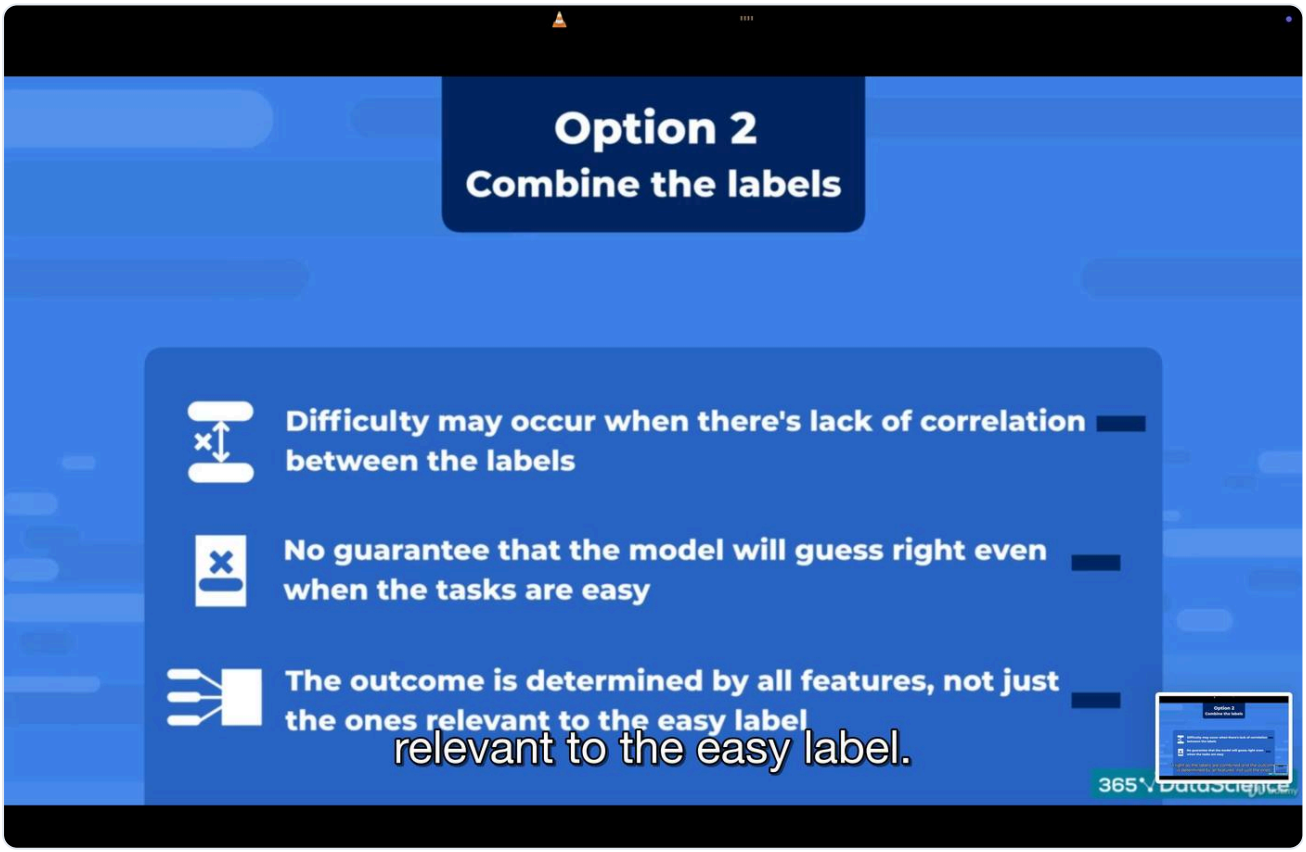


FIGURE 3.30 | Combine the Labels

This also means that if one of the tasks is easy, there is no guarantee that it will manage to guess it right as the labels are combined and the outcome is determined by all features, not just the ones relevant to the easy label. So if there is a great disparity in the difficulty of classification, this is not the best option.

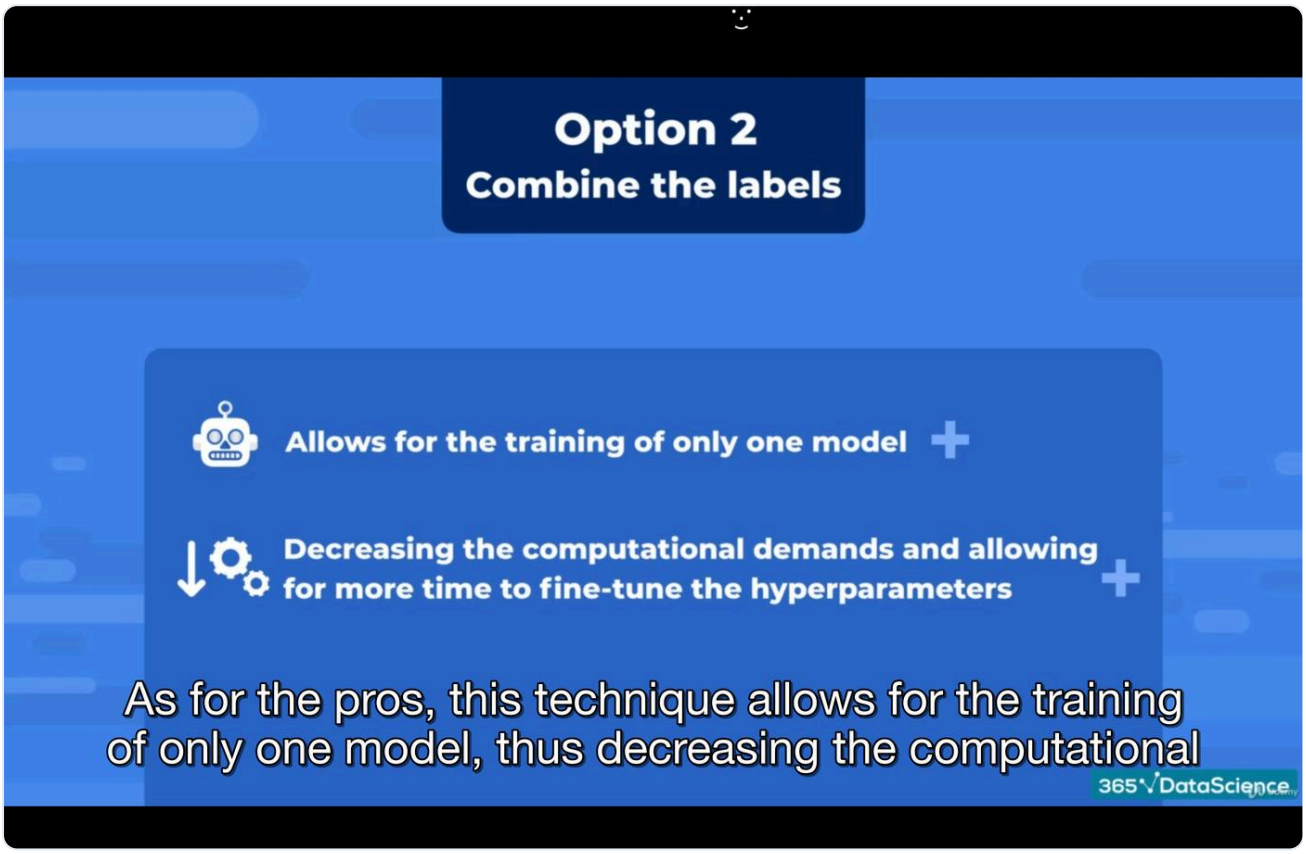


FIGURE 3.31 | Combine the Labels

As for the pros, this technique allows for the training of only one model, thus decreasing the computational demands and allowing for more time to fine tune the hyper parameters.

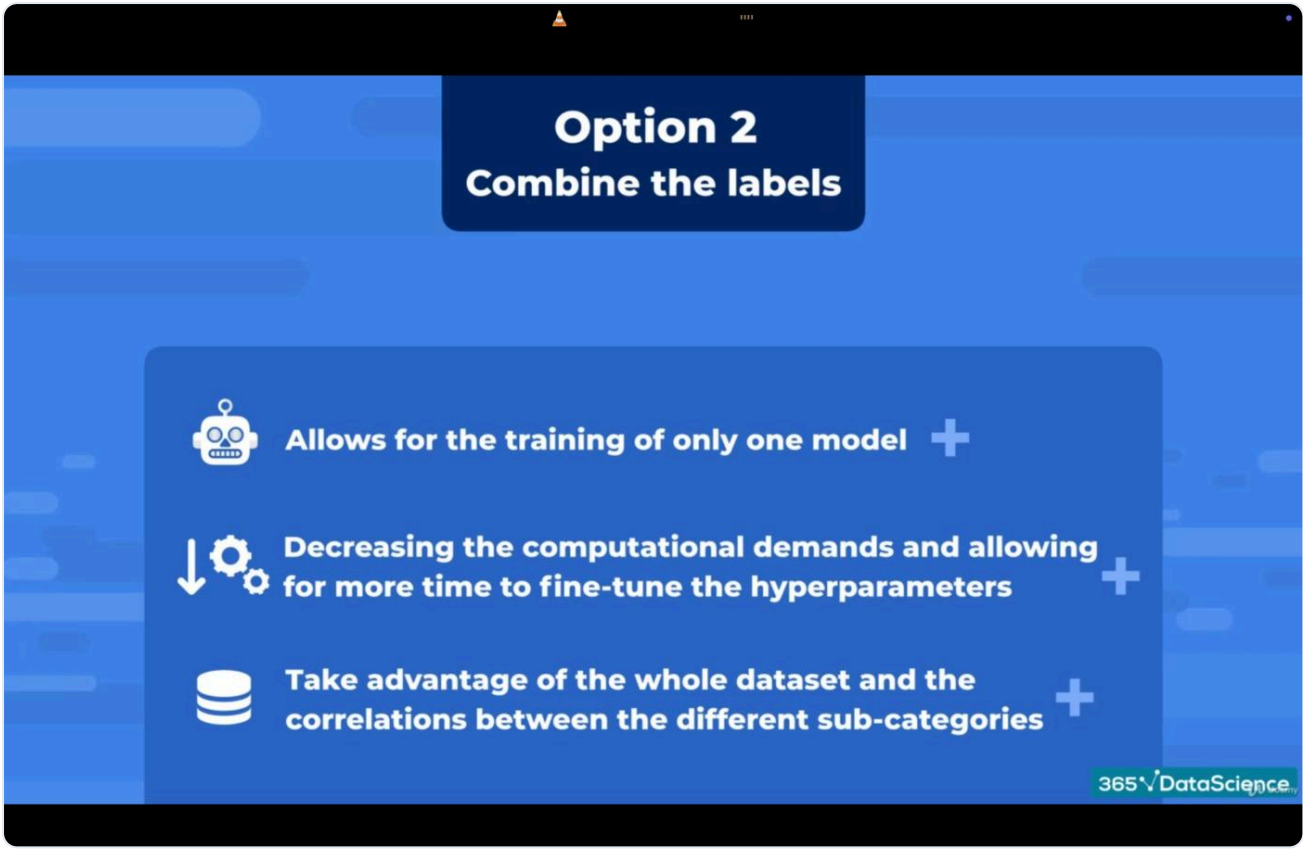


FIGURE 3.32 | Combine the Labels

It can also take advantage of the whole data set and the correlations between the different subcategories. That's great. So in summary, what is the technique we should employ in our situation?

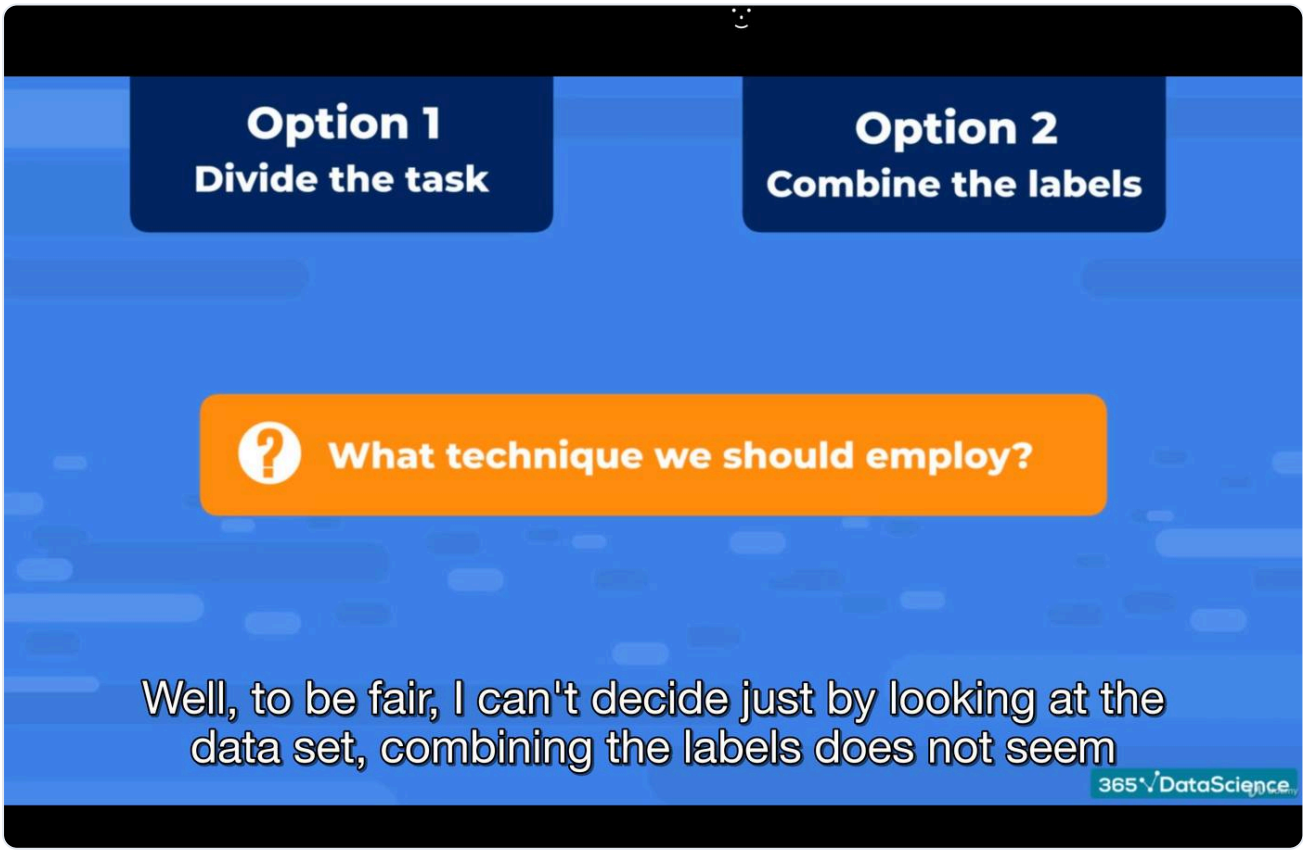


FIGURE 3.33 | Divide the Task Combine the Labels

Well, to be fair, I can't decide just by looking at the data set, combining the labels does not seem to have any obvious problems, nor does the mini task approach.

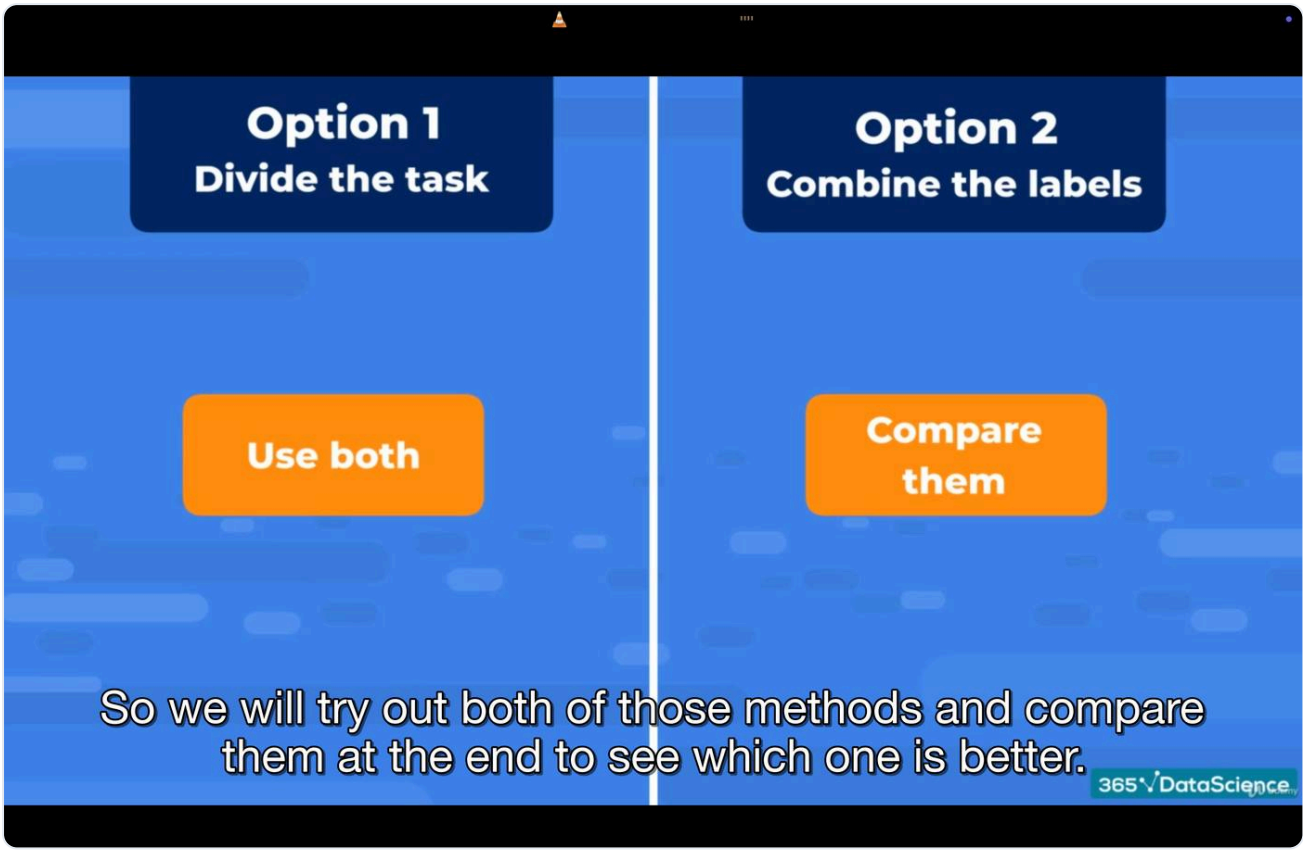


FIGURE 3.34 | Divide the Task Combine the Labels

So we will try out both of those methods and compare them at the end to see which one is better. OK, in the next lecture we will get started with the technique of combining all the labels into one. See you there and thanks for watching.

LECTURE 42

Dense Layer and Model

Welcome back. As we've discussed, there are a couple of approaches that could work in the task to assign two labels to the trousers and jeans category. In this video, we will deal with the method to combine the different labels into one.

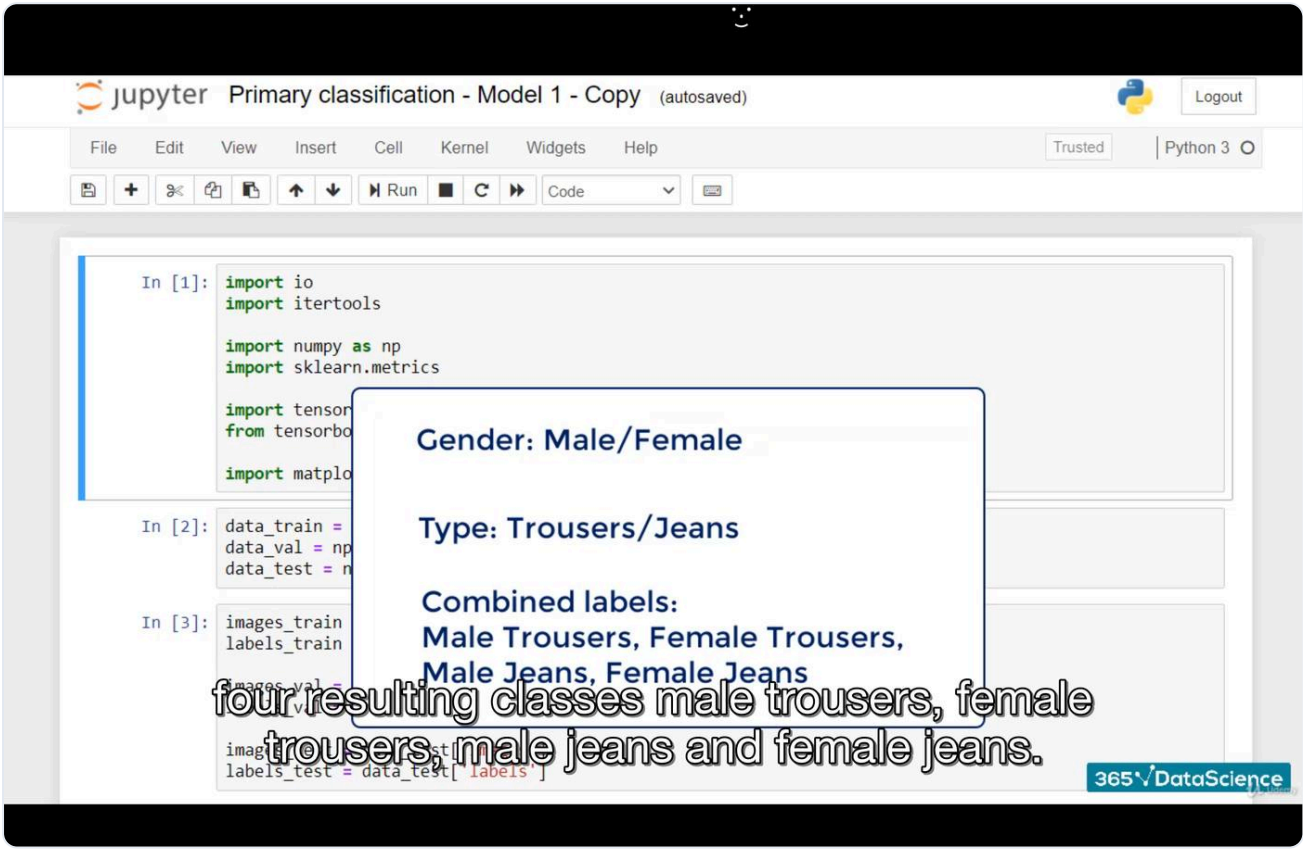


FIGURE 4.1

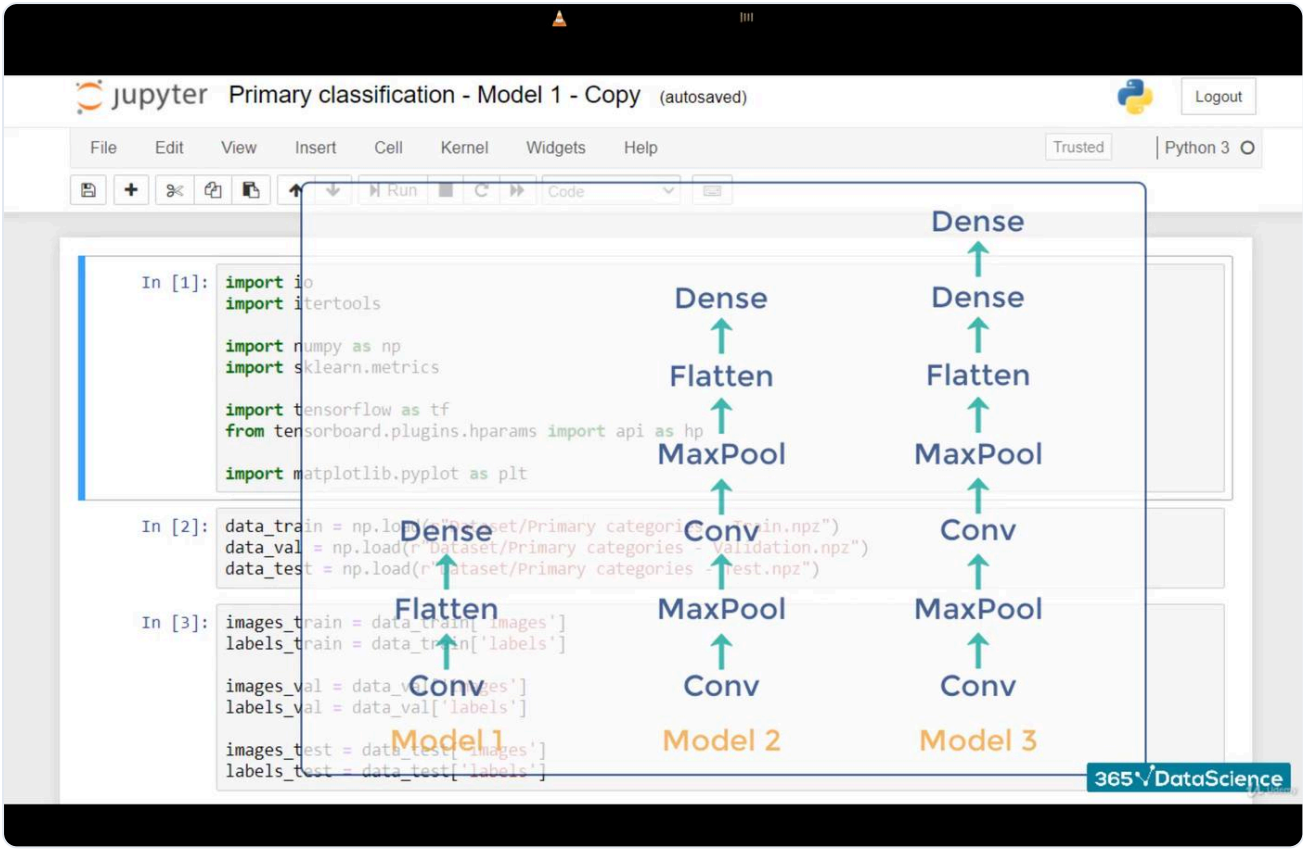


FIGURE 4.2

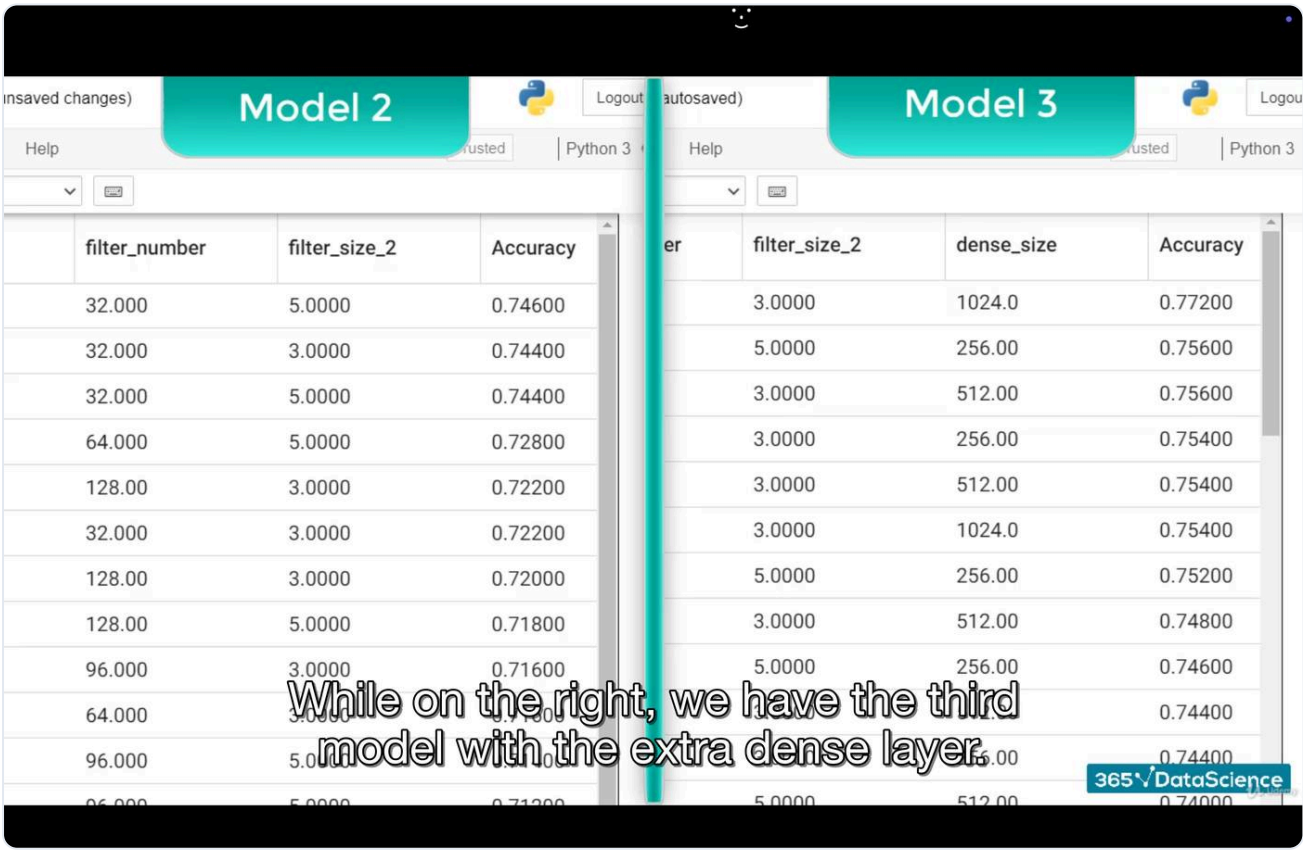


FIGURE 4.3

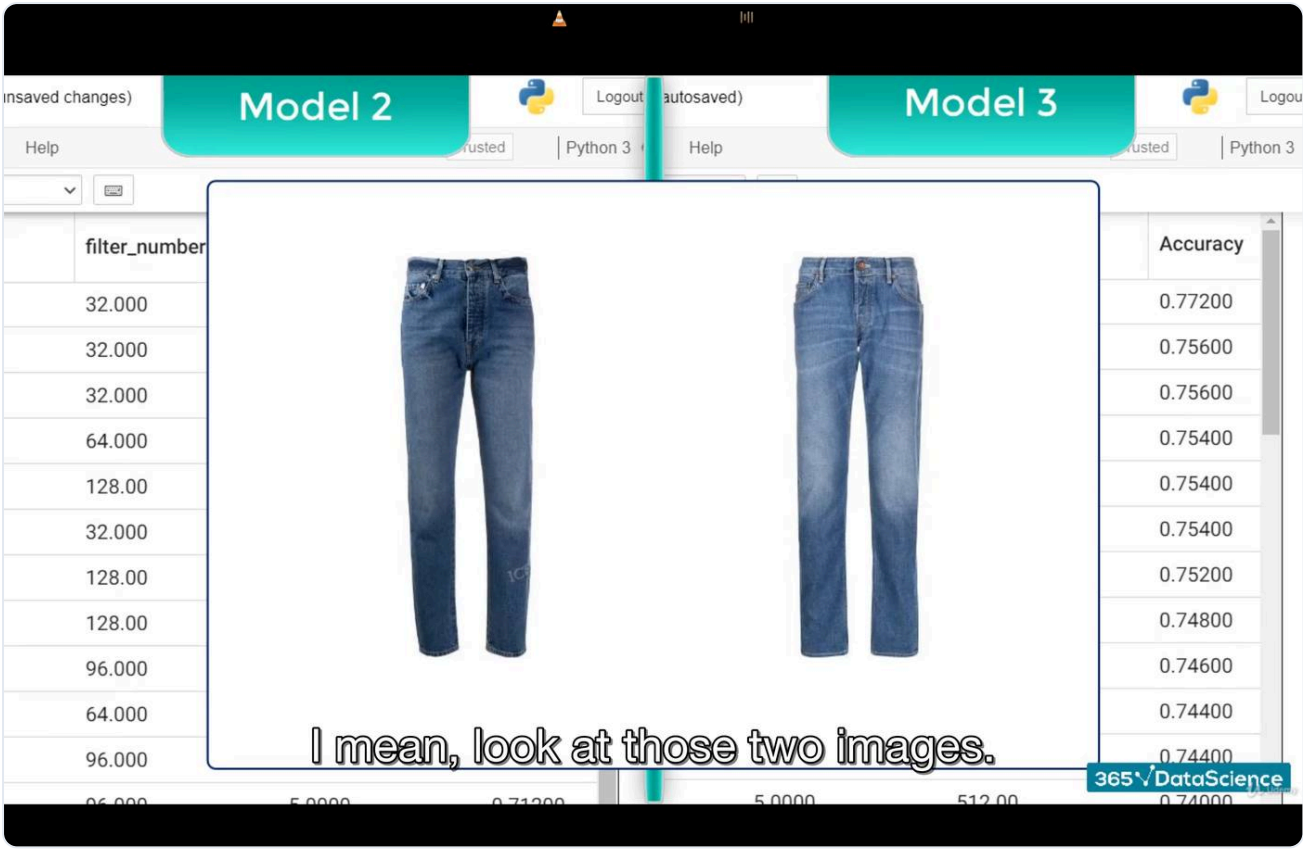


FIGURE 4.4

OK, the two labels that we should determine our gender and type, that is whether it's female or male and whether the images of trousers or jeans, if we are to combine these labels, clearly there are four resulting classes male trousers, female trousers, male jeans and female jeans. So now we are going to build a network to assign one of those labels to each image. I have named this method trousers and jeans, all because it does all the work in one go. Therefore, this will be the name of the folder where the code resides. All right. I proposed to test three models for this task. The first consists of only a single convolutional layer and the final dence one.

The second is the structure we're already familiar with, the one with two conv layers, and the third is a variation of the previous one with an additional dense layer just before the last dense one. Great. As we mentioned last time, we don't need to write a whole new program for every model. We can just copy the one we've got and adapt it a little to fit the new task. So here is the copied code of the primary classification task. What do we need to change? Well, first, let's take a look at the data set for this task. We have created a data set that has the combined labels. It is called Trousers and Jeans Dash all just like before. I have placed it in a folder called Data Set.

OK, this data set consists of three NumPy arrays, one for the images, one for the gender label and one for the combined labels. The combined labels are named simply labels, so we don't need to change the cell. Next, let's skip the hype of parameters for now and jump to the model outline, we can delete all the extra layers and leave just the comms flattened and dense layers. Remember, our first model to test is a very simple one, and we just want to see how it behaves. A very important thing to remember change the dense layer output. Previously, we had three classes. Therefore, the dense layer had an output of three. Now we have four classes, so it needs to be four.

And now it seems that the hyper parameters actually fit perfectly. We are going to try out filter sizes of three, five and seven. A number of filters of thirty two. Sixty four, ninety six and one twenty eight. The last thing to change is the class names for the confusion matrix. They are in order trousers, male jeans, male trousers, female and jeans, female. One could consider parameter sizing this part as well, but let's not do it now. Fantastic. Those are all the adjustments we needed to perform. Now we are ready to begin training, so let's run the cells and skip to the end of the training. Here we are an hour later and the training is complete. Time to check the results.

Entering the two command prompt statements to clear the temporary log files and starting TensorBoard. Let's sort the table on descending accuracy. As we can see, the accuracy fluctuates mainly between 60 and 70 percent. This is not bad for such a simple model. For reference, a random classification would yield around twenty five percent accuracy on four classes. Now, let's check how the different type of parameters performed, switching to the scatterplot view and enlarging it to see the whole graph. The first one is for the filter sizes and the second one shows the number of filters. Hmmm, it looks like the results are evenly distributed across the different parameters.

There is no significant difference in using 64 filters as opposed to one twenty eight, for example. This might be expected for such a simple architecture. This is just a test model anyway, to see if it works properly. So let's move on. All right. Now, it might be a good idea to check out the confusion matrix. To do this, we need to start TensorBoard with the fit folder, not a param tuning. So we need to repeat the two lines in the command prompt and start the correct answer board. In order to see the Matrix, we need to switch to the image, view and choose which matrix to view the confusion matrices are stored in the folders ending with. So let's check the first four such runs.

And the matrices appeared. That's great, they are a little bit small, though, to enlarge them. You can left click on them and to view them in their proper size, you can right. Click and press open image in new tab. Another thing you may notice is the slider above the image. It represents the current epic. In fact, we long the confusion matrix info at the end of every epic so we can actually monitor its progression throughout the training. There's one more detail to watch out for since we employed early stopping the model we are interested in is not the one of the last epic. The best model is the one on the third to last epic as we've set up the early stopping with patients of two.

Therefore, in our case at least, we need to check the confusion matrix two steps before the end. That is the best one. OK, now let's inspect it. Remember, the vertical axis shows the true label while the horizontal one represents the predicted label. Thus the diagonal is when the model correctly predicted the class. What we can see is that if the model is shown an image of male trousers, it perceives it as one of the female trousers 18 percent of the time. The male jeans are sometimes confused with male trousers and female jeans. When looking at the female trousers and jeans, we can see that the model correctly identifies that they are female but struggles a bit with identifying their type.

All right, we've seen that everything works properly. Now let's look at the real networks. Repeating all the previous procedures, we copy the code and rename it to Model to. Then we change the architecture to the familiar to conflate one. Of course, we need to change all of the log directories with model two as well. As for the hyper parameters, we can add the size of the second convolutional layer to. Again, we can do the same for the third model we would like to test. Copy the file, change the name and directories. And amend the model architecture. This time we add another dense layer with an output size that will be a hyper parameter, we will tune we will look at an output size of 256, 512 and 10.

Twenty four. So we are going to leave those two programs running and return with the results. All right, here we are a couple of hours later, and the models have been trained, so let's look at the results of the hyper parameter search. On the left are the results from the second model. While on the right, we have the third model with the extra dense layer. As we can see, the second model is an improvement from the very simplistic first model, in fact, it's around 10 percent better. However, given that the first model is a super basic one, I expected an even bigger increase. There is also a slight increase in the accuracy in the third model compared to the second one, but not too much.

As a matter of fact, it could be just due to the sampling of the data. A different shuffle or a different sample may have model two and three reversed in terms of accuracy. The different runs all hover around 75 percent accuracy. But is this good? Well, it's definitely not a bad result compared to the twenty five percent prior probability. It could even be considered great. However, it is nothing like the absolutely amazing results we've had in the primary classification task. If we look at the big picture, though, it is unreasonable to expect such a high accuracy for this task. At the end of the day, we are trying to distinguish between trousers and jeans and their associated gender.

However, by default, some of them are indistinguishable. I mean, look at those two images. One of them is female, the other male. But we can't really tell. Come to think of it, 70 percent plus accuracy is not bad at all. In light of the data set in the task, with three more months worth of tinkering, we may be able to bump those numbers up to eighty five percent accuracy. But keep in mind that it's highly probable that we'll never reach ninety nine percent no matter what we do. That's just the nature of this task. This is a valuable lesson which shows that when we use actual machine learning for practical purposes, we don't get the neat results we wanted or expected.

So far, we were discussing the accuracy, but which hyper parameters brought the best results? First, I'll check on the model to scatter plots, to see all of the plots, I'll extend the bottom part. Well, all of the different dots seem to be spread pretty uniformly across the different parameters, not only for the filter size, but the filters number as well. If we take a look at the same place for Model three, it is the exact same situation. All parameters seem to perform pretty much the same. Therefore, the behavior of our models is very stable. This might actually be a positive thing because it indicates the model can probably generalize better and we will not overfit through tuning the hyper parameters in a specific way for this exact data.

Of course, if given more time and resources, we could build more and more sophisticated models, maybe gather more data in the first place and all the jazz. I encourage you to experiment with that if you feel like it. But for this example, we are going to leave it at that. We have trained three models for the case where we combine the labels into one. Next, we will create a hierarchical structure of models and then we will compare the two approaches. Thanks for watching.

LECTURE 43

Hierarchical Classification and Gender

Welcome back. Last time we approach the task of assigning two labels to the images by combining them into a single one, this time we will create different networks to solve the problem for each individual label. This process will consist of two steps. First, we're going to build a network that will predict the gender of the intended buyer based on that result. We will have two more networks to classify it either as trousers or jeans, one network for male and one for female output, to be precise. In this video, we will build and train the different networks separately. We will not actually create the application that stacks them one after the other. We will build something close to this application in the next lecture where we are going to compare the two methods.

OK, let's get started with the gender label. Given our previous results, we decided that there is no need to create and test three different models. So for this parts, we will play around only with the most sophisticated CNN architecture so far, the one with two conv layers, two max players and two dense layers. Let's copy the model three code from the previous lecture. Of course, we change all directory names to model one. And the labels for this task to be the gender array. Finally, we change the output of the last dance layer to two. And fix the class names of the confusion matrix, because we only have two labels for this task. Male and female. This tactic is becoming kind of a habit now, right?

Continuing, we train the network and the results are in. Let me help intenser board and sort the different runs by accuracy. This time, the network is performing in the upper region of the 80s, close to 90 percent. This is a surprisingly good accuracy for classifying the gender of genes, which frankly are not always easy to differentiate. Regarding the hyper parameters, we are yet again greeted with an even spread. At this point, we are getting used to this trend, so no need to linger too much on it. Since we are creating a hierarchical classification, we now need to build two networks to label the type one for male trousers or jeans and another for female, only these networks can be constructed and trained equivalently, as we've done so far.

That's why we believe them. As an exercise for you, you will need to inspect the data set file in order to pick the right ones for these tasks, then create and train the actual networks. You may write the code from scratch or copy the previous one and change whatever needs to be changed. It is up to you to make this choice. The point is that while these lectures are educational, it is easy to miss some small details. When you're just watching a video, you will find that these seemingly simple tasks often generate tiny errors that need your attention. Now you have the opportunity to deal with those sooner rather than later. All right. For now, we will just show you the results we obtained.

Both models performed equally well on the task with the same accuracy distribution as the gender model between 80 and 90 percent. Surprisingly, though, the hyper parameters did have an effect. What we found was that the first convolutional layer performed worse with a kernel size of five by five or seven by seven. The difference was not that significant, but the three by three kernel is the better choice. The other parameters have equal distributions as expected. Great. So far,

we have train networks to do all of the many tasks for the trousers and jeans category. It is now time to test which technique is better suited for this data set the technique that combines the different labels into a single type or the hierarchical classification.

For this purpose, we need to set up a program to stack the different networks. That's exactly what we will do in the next lecture. See you there. And thanks for watching.

LECTURE 44

Score and Label

In this lecture, we will compare the performance of the techniques we employed in the last two videos. All right. Given that we have two different labels, gender and tape, a single accuracy score would not suffice for this comparison. A simple metric to help us in this case, though, is to award the network one point for every correctly determined label. Thus, if the network correctly predicts the gender and type, it gets two points. If it manages to guess only one of the labels, it is awarded one point. And in the case of no correct predictions, it gets no points. Of course, a simple but elegant system. Not only that, but it is flexible as well. If you value the accuracy for one of the classes more than the others, you could reward correct guesses for this label with more points.

In our case, I don't see a reason to value them differently. Thus, both get one point. So far, so good. We have agreed on a scoring system. Now we need to apply it. The idea is to load the models that had the highest accuracy for the respective task and score the performance on the test data set. OK, at this point you have all the skills necessary to create code to compare the two approaches yourselves. I strongly encourage you to attempt this on your own with the models you've trained so far. Nevertheless, in this video, we will code our version of this application so you can come back and check it. All right. For this notebook, we're going to need only the number and TensorFlow packages.

Then we can import the data set, this time we get only the test as we won't be training any networks, just testing their performance instead. Next up, we extract the images and labels from the data set. Bit of a roadblock at this point might be the type label in the data set itself, we have included the gender label and the combined one, but the Senate does not contain a type label. What we need to realize here is that the combined label actually contains both the gender and type, we just need to extract the type from it. So how do we do that? The combined label consists of trousers, male genes, male trousers, female and jeans, female. Numerically, these are represented with zero, one, two and three.

For our purposes, both trousers classes should be assigned zero and both jeans classes one. Do you notice any pattern in the numerical labels of those two groups? Let me help you, the trousers group consists of even numbers zero and two. In contrast, the jeans have odd numbers, so the solution follows this logic we can take the combined label array and find the remainder when it is divided by two. Even numbers yield remainder zero and odd ones yield one. Therefore, the type label array is type test equals data test labels percent to. Perfect. Just remember to scale the image pixel values, even though we are not training the networks, we still need to scale the values as that's the expected input.

That's why many models have their preprocessing included as preprocessing layers at the start of the network. Let's download the models were about to test, I have copied them to the folder and rename them for easier orientation. We've reached the part where we need to score the two approaches, so let's define the two score variables initially zero. The idea now is to loop through ev-

ery image in the data set, get the predictions of the models, compare them with the actual labels, and change the two score variables accordingly. So we write the for loop. For I in range of length of images. Let's start with the combined labels model. First, we obtain its predictions for the current image.

We can do this simply by passing the image to the model like that. Note that the model expects to receive a list of images to classify, so we can't simply write images, tests I. We need to take a list slice resulting in a list of one element. All right, this variable now corresponds to a numerical likelihood score for each class, the actual predicted class is the one with the highest score. Therefore, to get the predicted label, we must find the index of the highest score. This functionality is exactly what the NumPy function ARG Max provides. So we have obtained the label of the image as determined by the model. Next, we need to determine the score.

As already discussed, the type label corresponds to the groups of even and odd labels. This means that the check, the validity of the type prediction, we can take the remainder of the division by two and compare it to the true label. If they are equal, we increase the score of this approach by one. We can do a similar trick for the gender label, the first two of the combined labels correspond to male, while the latter two are female. Taking advantage of this information, we realized that an integer division of the label with two returns, the correct gender label again, if it is equal to the true gender label, we increment the score by one. With that, we have finished the combined label section.

We now continue with the hierarchical classification by analogy to the previous model, we find the label predicted by the gender model. This time, we can perform a simple equality check with the true label again, we increase the hierarchical score by one. Now, depending on the result from the first network we run, one of the other two, if the label is zero, meaning mail, we evaluate the mail type model. Otherwise, we have the female type model. And that's it, the for loop is complete. Essentially, this whole program is just extracting the predicted label of a model and a couple of well-placed ifs great. Now, let's run the cell, it takes a bit of time to go through all the images, but nothing compared to training a network when it finishes executing, we will have the final scores stored in the score variables.

So let's print them in a nice format with a bit of text for better readability. And here are the final numbers. Well, the scores are very close to each other, meaning there is no significant benefit to one approach. In that case, the preferred method might be the one with less computational stress on the system and easier usage, that would be the network with four classes into one combined label, it has a little bit better performance to boot as well. Fantastic. What about your experience? Did you get the same results, given the models you train in the previous videos? What if we take the best performing models for the respective networks? Does the balance remain the same?

I encourage you to answer these questions as an exercise. If you want more practice on creating models and comparing different techniques of combining them, feel free to check out the shoes category of our data set. You can follow the same procedure as we did for the trousers and jeans, like combining all the classes. But you can also experiment with different techniques. It is important to realize that this practical is open ended. The choice is in your hands on how much to prac-

tice, what data sets to explore and what architectures to try that aim to get the most out of it. Our journey will continue next time when we try to improve the performance of our networks through the techniques we discussed in the previous section.

Thanks for watching.

LECTURE 45

Hyper Parameter and Standard Deviation

Hello there and welcome back. In this video, we will try to improve the accuracy of our trousers and jeans all model by employing L to regularisation and drop out. We have outlined and discussed those techniques in the previous section. We had already performed type of parameter tuning on this network, but couldn't find any significant difference in its performance. Since we can't improve it through tuning it, we might as well try to achieve a better accuracy through the aforementioned techniques. All right. So here's the code for the most sophisticated trousers in jeans architecture we created since we will be implementing L to regularisation and drop out on it.

We will fixate the parameter configuration that gave us the best results. Therefore, I will replace the hyper parameter variables with the appropriate numbers in the model architecture. We won't get rid of the hyper parameter tuning structure as a whole, though, because both techniques have a high parameter that we will investigate. OK, now we can begin the investigation. First, we will try out L2 regularisation. Quick reminder, L2 Regularisation refers to the method of modifying the loss function with an additional factor. That is the sum of the models weights squared. The objective is to drive the unnecessary connections to zero, thus simplifying the model and preventing overfitting.

So how do we implement L2 regularization in code? It's actually relatively simple. We need to add the regularization to each layer in the network through the kernel regularization parameter. The value that we pass is T.F. Dot Keras dot regularizes dot l to. Inside the brackets, we need to state the value of the lambda parameter, that's precisely the reason why we left the hyper parameter tuning structure intact. We will check the behavior of the model to a couple of different values for this parameter. So let's create a variable for this lambda. But what range of values should we test out? Well, there is a huge domain in which to regularisation could be helpful.

Usually Lambda can be as low as 10 to the minus five and get as high as point one, though rarely that high. Since that is a huge range, we will have our values on a large scale, starting from ten to the minus five, then five times ten, the minus five following is ten to the minus four and so on. Now we can add the variable inside the bracket here. In practice, though, this regularization does not take all the weights of the model, it considers only the way to the layer it belongs to. So we need to add this parameter to all layers that have trainable weights. Fantastic. Now we just need to fix the for loop in light of our changes and we are good to go, right?

Well, not quite. A very important detail here is that changing the laws function affects the early stopping callback in this callback. We told TensorFlow to monitor the validation loss and stop training what it increases in two consecutive epics. However, our validation loss now has additional factors that drastically change its behavior, especially whether it increases or decreases. Therefore, we need to supply the unmodified loss to this callback. We can do this with a few changes up here when we define the last variable, we can give it a name by which to reference it later. I will name it sparse categorical Cross Entropy. Not very imaginative, but very descriptive.

Then when we compile the model, we simply add this variable to the metrics list. Now we can reference this unmodified loss from the early stopping call back with a name we've given it. That's it. Now we can train the network as usual, we will skip to the end of the training. All right, here are the results. The scatterplot graph is not very informative, almost all values of the lambda parameter are clustered to the left, indistinguishable from one another. We can easily fix this by changing the scale of the x axis from here. Let's set it to quintile. Great. Now, what do we see from this graph? Well, we notice a huge drop in accuracy for the two biggest values of the parameter.

No surprise there. However, the rest of the values fluctuate up and down a bit. Crucially, though, there is no significant boost to the network's performance. This could be expected at the end of the day. It does not seem that our network is fitting the training set so well that it over fits. If there is minimal overfitting, we can't expect a technique such as L2 regularisation to help. It just seems that the data we have is inherently difficult to learn. Although we've already tried to regularisation, we might as well check the drop out technique. This refers to the technique in which we randomly sent some of the outputs of a dense layer to zero, thus helping individual neurons to learn more general patterns about the data set.

The implementation of dropout in code is actually super easy. Barely an inconvenience, it has its own layer object, in TensorFlow. Therefore, to apply dropout to the first dense layer, we can add this line after it. The parameter for the dropout layer's states, how many neurons to drop? That said, zero point one means 10 percent of the outputs of the dense layer would be set to zero. Again, this is a hyper parameter of our system and we will create a variable for it. The dropout rates we would be interested in are 10 percent, 20 percent, 30 percent and so on, up to 90 percent. It is unusual to see rates above 50 percent, but since we are here, we might as well check them.

All right, we can reverse the changes related to the lost function we made and train the network. And here is the data. But that looks strange, the accuracy goes all over the place with sharp changes from one value of the parameter to the next, even more surprisingly, some of the values above 50 percent actually perform better than the ones below that threshold. This result reminds me a lot of noisy data, when a certain operation is dominated by randomness, the results look pretty similar. So we've got a hypothesis. The accuracy of our network fluctuates a lot due to random noise and the TensorFlow operations and you to the initialization of the weights.

This means that the hyper parameter tuning we've performed so far can't lead us to any significant conclusions. This is the hypothesis. To test it, we will train our model with the same set of parameters and initial conditions 20 times. We will omit the drop out or L2 regularization and just focus on the base network from the spread of the data. We can quantify the influence this random noise has on the accuracy. For ease of work, I will use the same structure we've had so far and define a hyper parameter, which is the index of the current run. It goes from one to 20. Now we are ready to run the whole thing. Here's what we obtained. The mean accuracy across all 20 runs is seventy three point seventy five percent, the standard deviation or in other words, the spread of the data is two point sixty five percent.

The maximum accuracy achieved by the network is seventy eight point six percent, while the minimum is 67 percent. This analysis points to the fact that all fluctuations we've seen in the accuracy so far have been almost entirely caused by randomness. This, in turn, means that the models for which we saw a marginal improvement with L2 regularisation or dropouts actually had nothing to do with those techniques and everything to do with random chance. To be fair, if we were to perform this practical rigorously, we should have trained the model for at least 10 runs with the same type of parameters and every network so far, instead of having one data point for each set of hyper parameters, we would have obtained 10 data points.

In turn, we should have compared the average accuracy to determine the best set of parameters. But of course, that would have inflated the training time drastically, meaning we would probably still be on the primary classification task. All right, despite these results, the techniques of L2 regularisation and dropouts can potentially still have beneficial effects. It could be the case that when we introduce dropout, the standard deviation decreases. Thus, even though the mean accuracy is the same, we can stabilize the optimization by reducing the fluctuations. We can test this new hypothesis with the same experimental setup. Only this time we will include drop out of zero point one and zero point two.

Again, we run each configuration 20 times. And these are the results. Unfortunately, there is no change because of the drop at the mean is still the same as is the standard deviation. So in this video, not only we couldn't improve the performance of the model, but we discovered that it was dominated by randomness, a bit disappointing, right? Well, the thing is, in real machine learning projects, we are often restricted by the quality and size of our data. Many times due to the nature of the task at hand, we can't achieve our desired performance. That is a long winded way to say that we don't always achieve the neat results we were aiming for. And that's fine, given more time and data.

We could probably get accuracy of up to 90 percent on this example. We may inspect up close every individual feature map to see what exactly is happening inside our net, but we don't have the time and resources for that. So let's call it a day and continue to the last topic of this practical data augmentation. Thanks for watching.

LECTURE 46

Flip and Data Sets

So far, we've dealt with symmetrical images, ones of trousers, jeans and glasses. The shoes category, however, contains photos of shoes, flippers and other, all of which face left or right. But in real life, our model could encounter some images that have the opposite orientation right to left or just some other non side view. If we want our model to recognize these, we need to train it with such data. And one possible solution is to go out and extract more images for the data sets. We can also do a little trick, augment the data we already have. If we just flip our images, we can get the right to left orientation present in our data. This data augmentation approach has limitations because we can't transform the original images to show a top view of the shoe.

But at least we can solve one of these problems. All right. As stated, we will deal with the shoes category. It is similar to trousers and jeans as the shoes have a gender label and a type label. Therefore, similar to the trousers and jeans that we can have a model with combined labels that I will call shoes all this time, we will not consider the model itself as that's not our focus. We have already created and trained it with the available data. These are the results. Now, the question is, what should our approach be and how would we achieve it? Well, in this data sets, we have around 5000 images. It is entirely feasible to flip all of them and add the flipped version to the original dataset, obtaining 10000 images.

So how do we obtain a mirrored version of our original image? Well, the images are actually NumPy arrays, right? So if we want to flip an image, we simply need to reverse the values inside the arrays. This is a well known company functionality that we can achieve with the flip function. The tricky part is to determine which areas exactly should be flipped because flipping too much may result in rotating the image rather than mirroring it. Let's recall the dimensions of our empire, the first dimension referring to Axis zero is the number of images. It represents the list of all images. The second one. One hundred and twenty in length is all the rows of each individual image.

This is axis one. Then we have a dimension of 90, which represents all the pixels in one row. It is designated as access to. Finally, we have each individual pixels, red, green and blue values, this final array has a dimension of three and is the axis three array. Since we want to flip the image horizontally, we can flip every row that should do the trick. So we need axis one, right? Well, no, as Axis one represents the list of all rows, thus flipping, it would place the last row as first and so on, this would have the effect of flipping the image vertically. What we need is access to as we need to reverse the order of all pixels in every row. Of course, when in doubt, you can always print the flipped image with the help of Matplotlib, and it would be obvious that the wrong rotation has occurred.

In fact, doing this is highly recommended. Now the code is straightforward. We mirror all three data sets with this simple command NP dot flip the data set in question axis equals two. Then we can combine the flip data sets with the original ones through the use of the concatenate function and obtain the final data set. Since the label of an image does not change when we flip it, we can

add the original label array to itself. And that's it, as simple as that. Fantastic. Now, after running cells, this network was trained to recognize both orientations of the shoes. It even retains similar accuracy levels to the regular model, so we actually didn't lose much performance due to the data augmentation.

That's great. Well, that was it for this mighty behemoth of a practical example, of course, the only way to get the most out of it is to practice with these data sets on your own. You can even get much better accuracy with different model architectures or techniques, try to resize the images or perform data augmentation on the other categories by translating the objects of interest. Be brave, experiment and get comfortable with these kinds of operations. As for this course, there are still two sections left to explore. The next one is unexpected failures. See you there and thanks for watching.

LECTURE 47

Previous Amazon

So far, we have created some neural networks to perform a variety of tasks from classifying handwritten digits to tagging clothes for online shopping, even though we have achieved high accuracy for these models, we still don't really know how they work. In reality, understanding what a neural network managed to learn is an ongoing research in the field. We usually say that machine learning is just like a black box. We don't know why the model makes certain predictions. Of course, this does not mean that there aren't any meaningful insights for us to gain. It is still well worth dissecting this black box. That's why academics spend a lot of time trying to understand how our trained models make their decisions.

This may help them better predict any unexpected or plain wrong results in the long run, and there are a lot of ways in which a seemingly good network may exhibit unwanted behavior in this lecture. We are going to take a look at some examples where neural networks break in order to gain insight into possible ways for improving them. So the first thing we need to understand about machine learning is that it's not confined to the same constraints as the human mind. We may not notice it, but our thoughts and thinking process are quite biased as we subconsciously limit our possibilities and purpose to fit a common, unspoken narrative. That is to say, we are not good at thinking outside the box for a machine.



FIGURE 9.1

Such a mental box does not exist in the first place. In general, human beings do not consider the whole space of possibilities. Ultimately, this can be both a positive and a massive hindrance. Let's consider a highly simplified, yet illustrative example of this point. Imagine long time in the future. In a galaxy not so far away, we are in a war with aliens to maximize the efficiency of our defense systems. We decide to use machine learning, so we make a highly sophisticated model that has access to our most advanced technologies robots, satellites, the Internet, sharks with lasers, whatever future technology exists. Now, let's say the particular problem that we use this model for is to minimize the chance of enemy missiles killing humans on Earth.

That seems like a reasonable task. So how can this go wrong? Well, the goal of the network is to prevent enemy missiles from wiping humanity. Nothing is stated about the friendly ones.



FIGURE 9.2

Thus, the model may come up with the solution to nuke all humans.



FIGURE 9.3

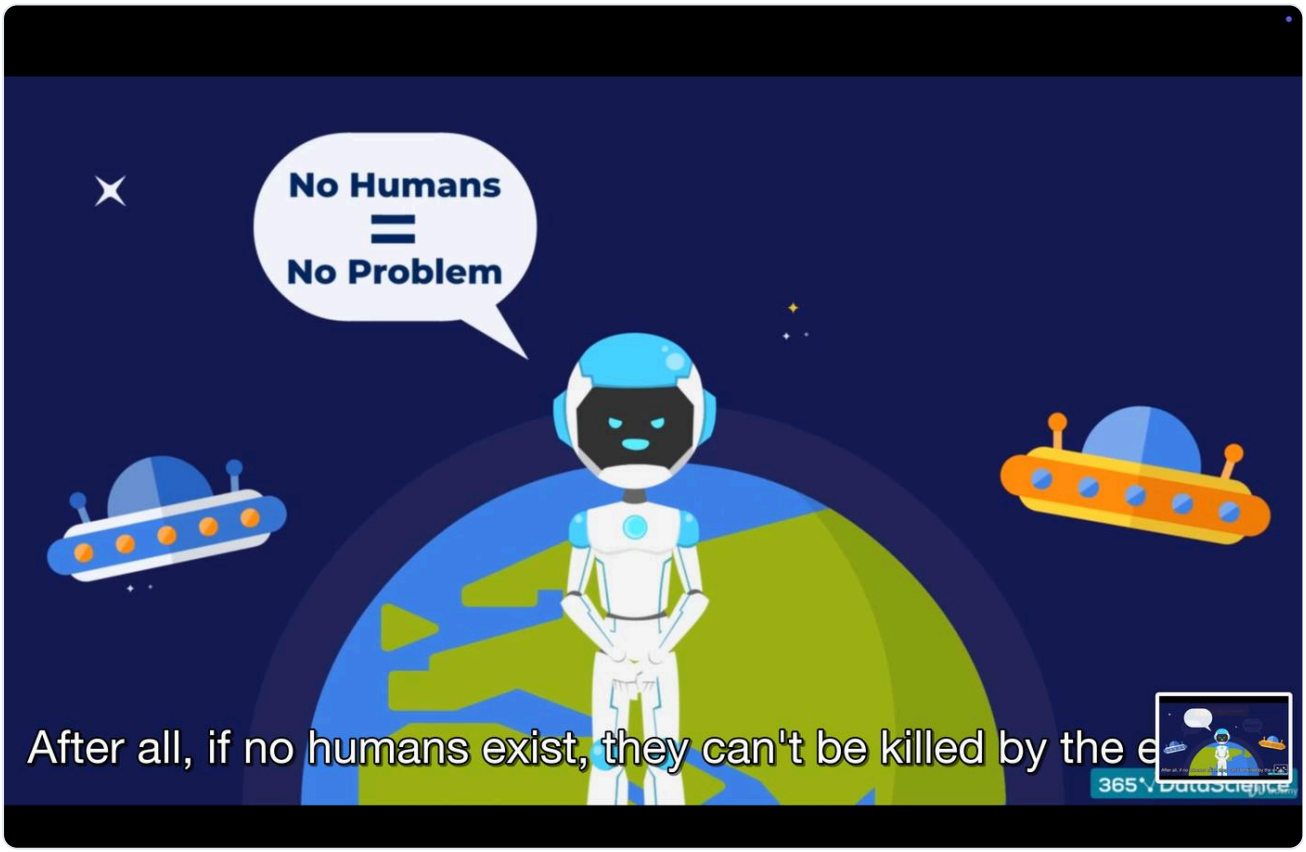


FIGURE 9.4 | No Problem

After all, if no humans exist, they can't be killed by the enemy.



FIGURE 9.5

Upon observing what the network actually learned, the engineers may fudge it a bit as a rule to prevent it from killing people and call it a day.



FIGURE 9.6

But then again, the model may decide to capture all humans, put them in a spaceship and send them in a random direction somewhere far from Earth. The same goes for real world problems.



FIGURE 9.7

What we think our goal is may in fact be different from what it actually is.

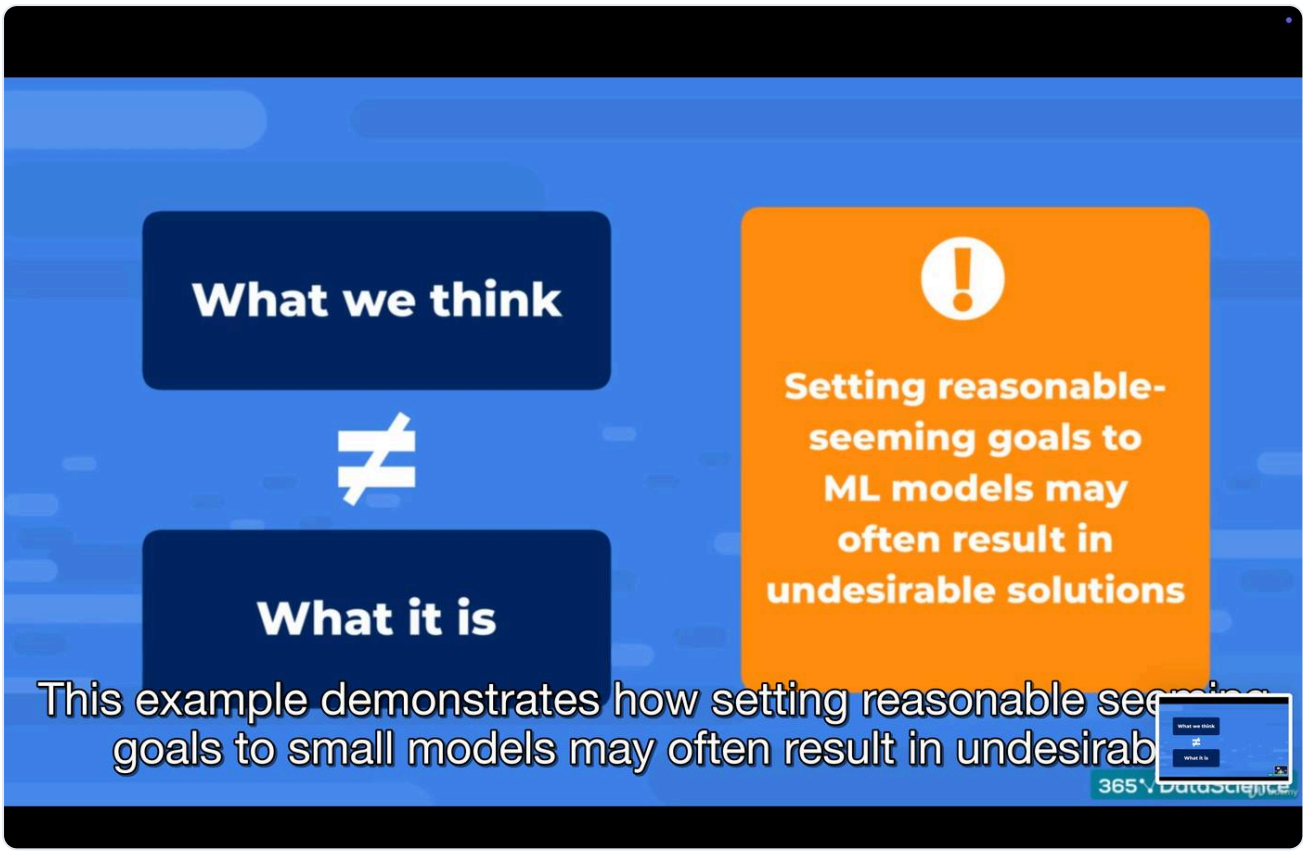


FIGURE 9.8

This example demonstrates how setting reasonable seeming goals to small models may often result in undesirable solutions. And if you think my made up alien story is far fetched, well, you better take a closer look at the following Real-Life situation that happened to Amazon. As you probably know, Amazon is a huge company with more than half a million employees. You can imagine they invest a lot in human resources and hiring new talent. In 2014, they decided to use a neural network to help them recruit new personnel.

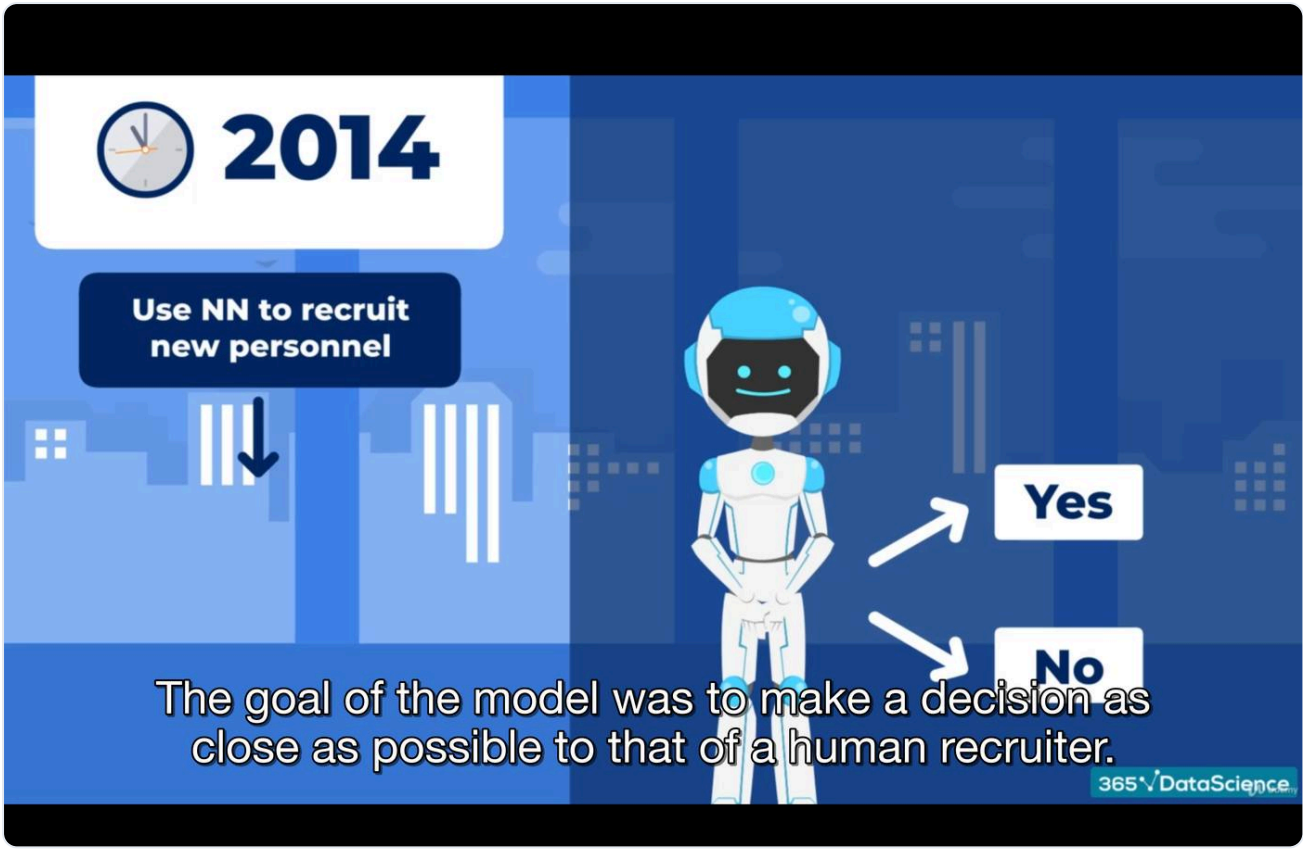


FIGURE 9.9 | Use NN to Recruit

The goal of the model was to make a decision as close as possible to that of a human recruiter.

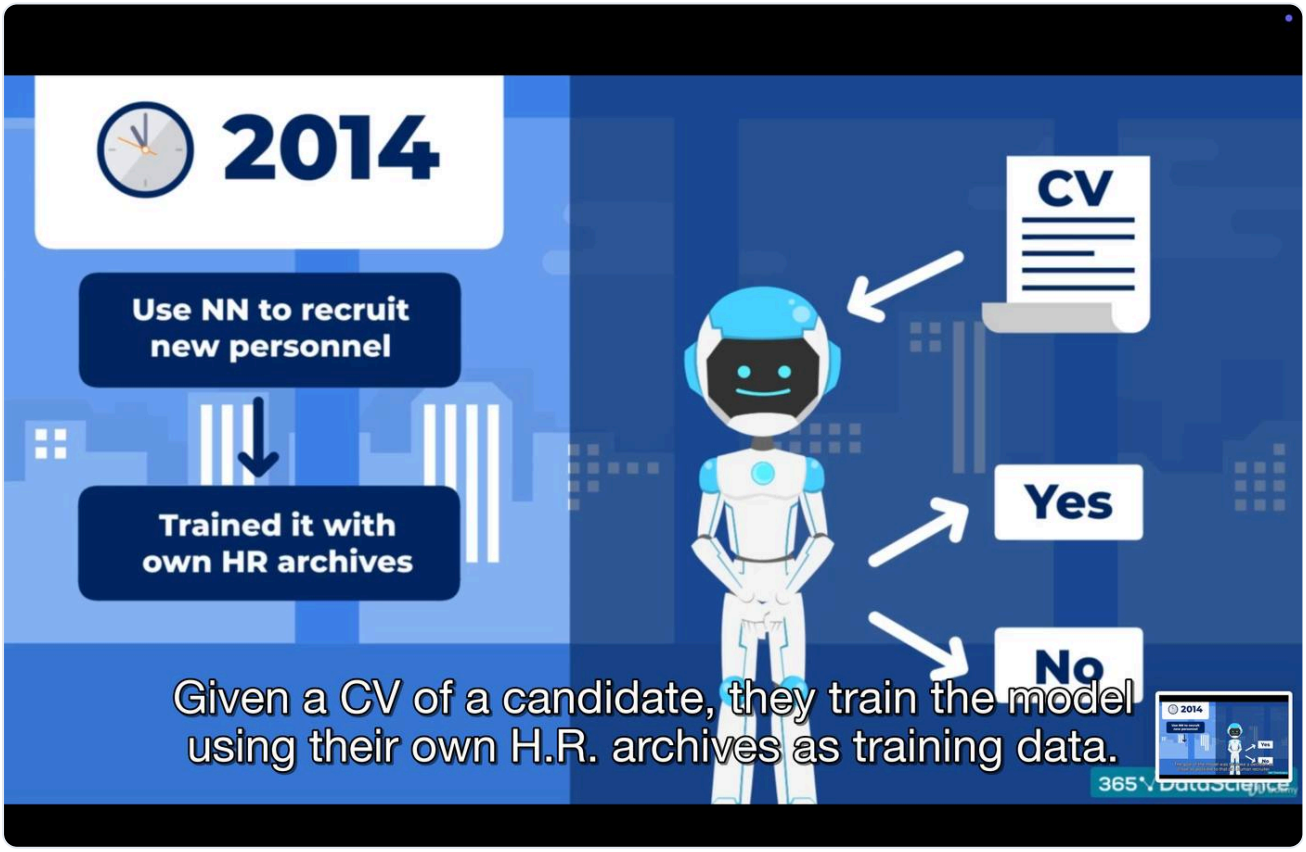


FIGURE 9.10 | Use NN to Recruit

Given a CV of a candidate, they train the model using their own H.R. archives as training data. This all seems reasonable and quite doable, unfortunately, things took a wrong turn. What happened was that the model learned to penalize female applicants.

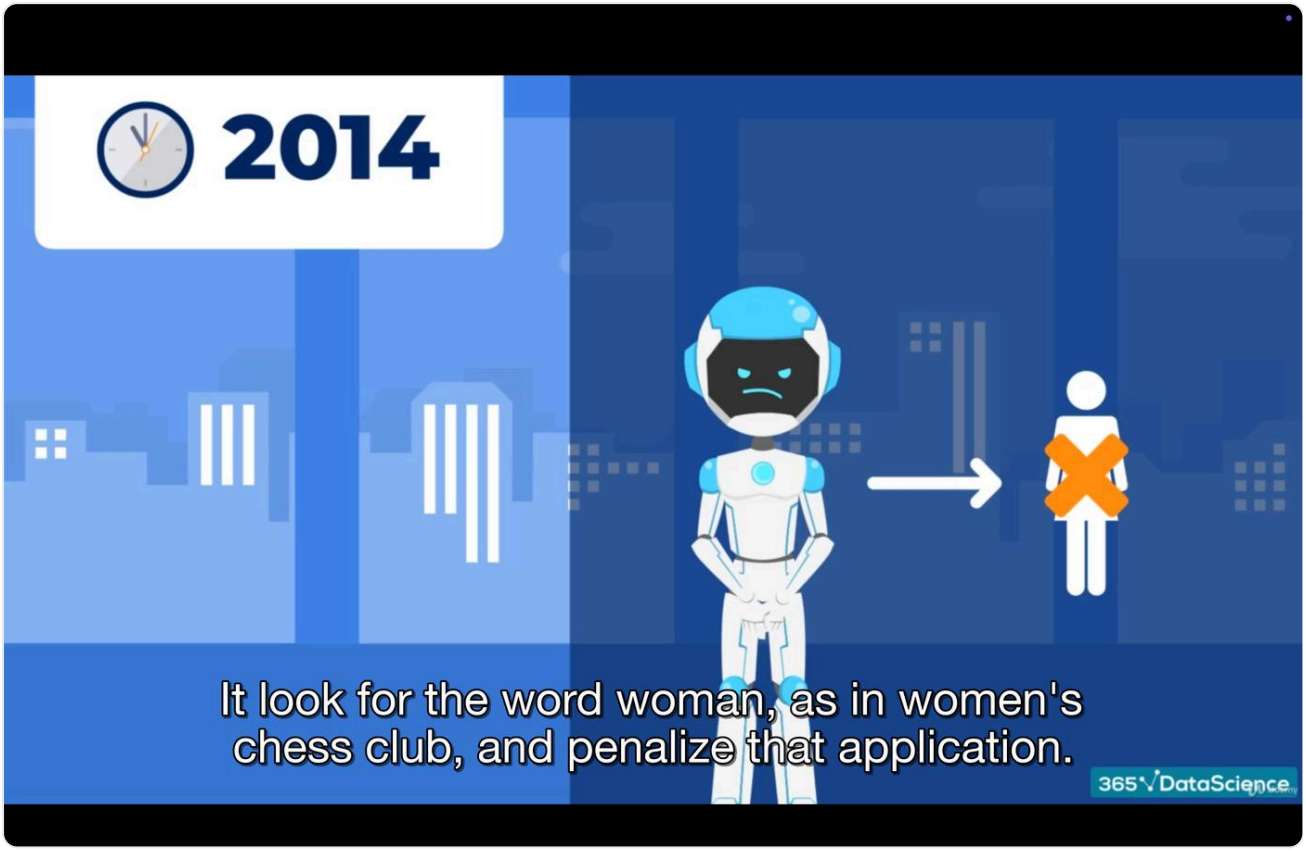


FIGURE 9.11

It look for the word woman, as in women's chess club, and penalize that application.

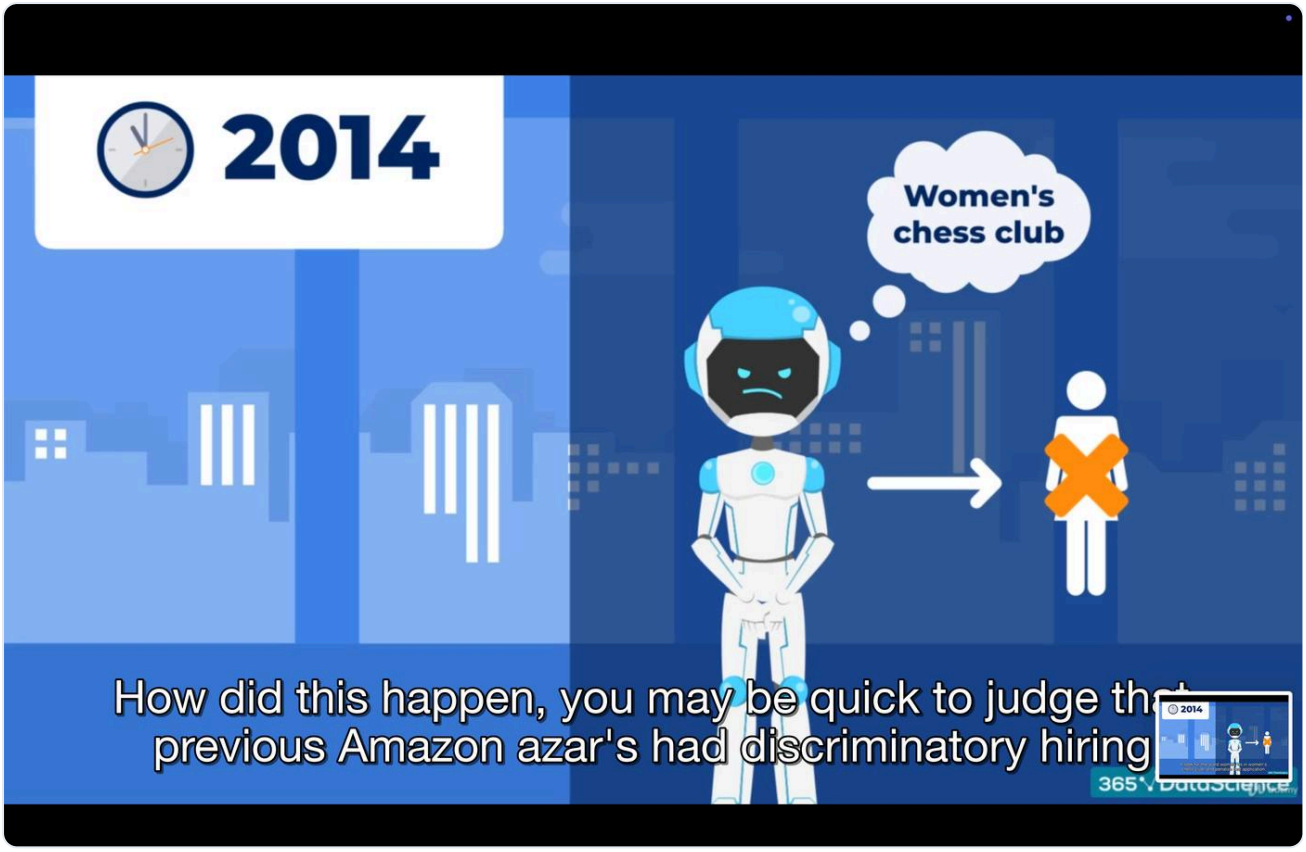


FIGURE 9.12

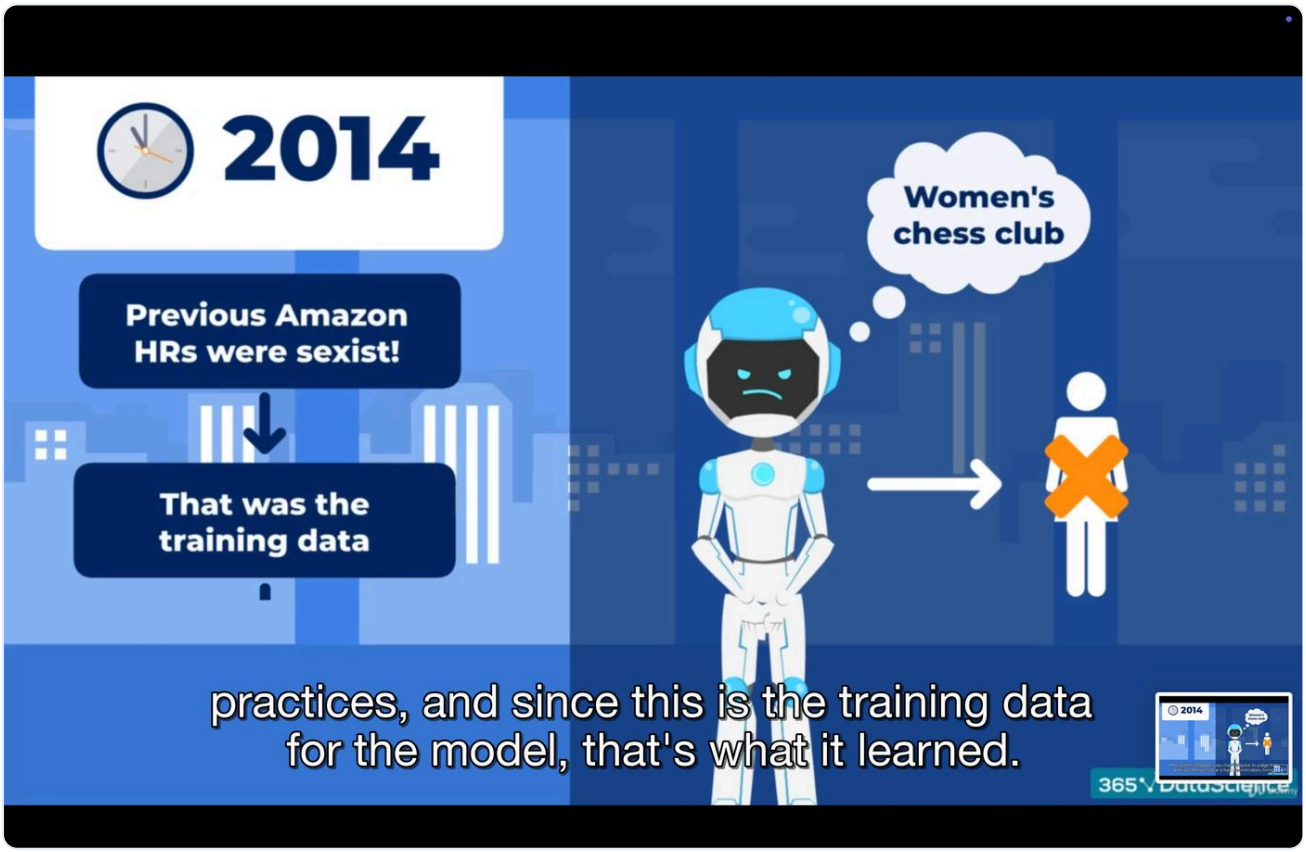


FIGURE 9.13 | Previous Amazon

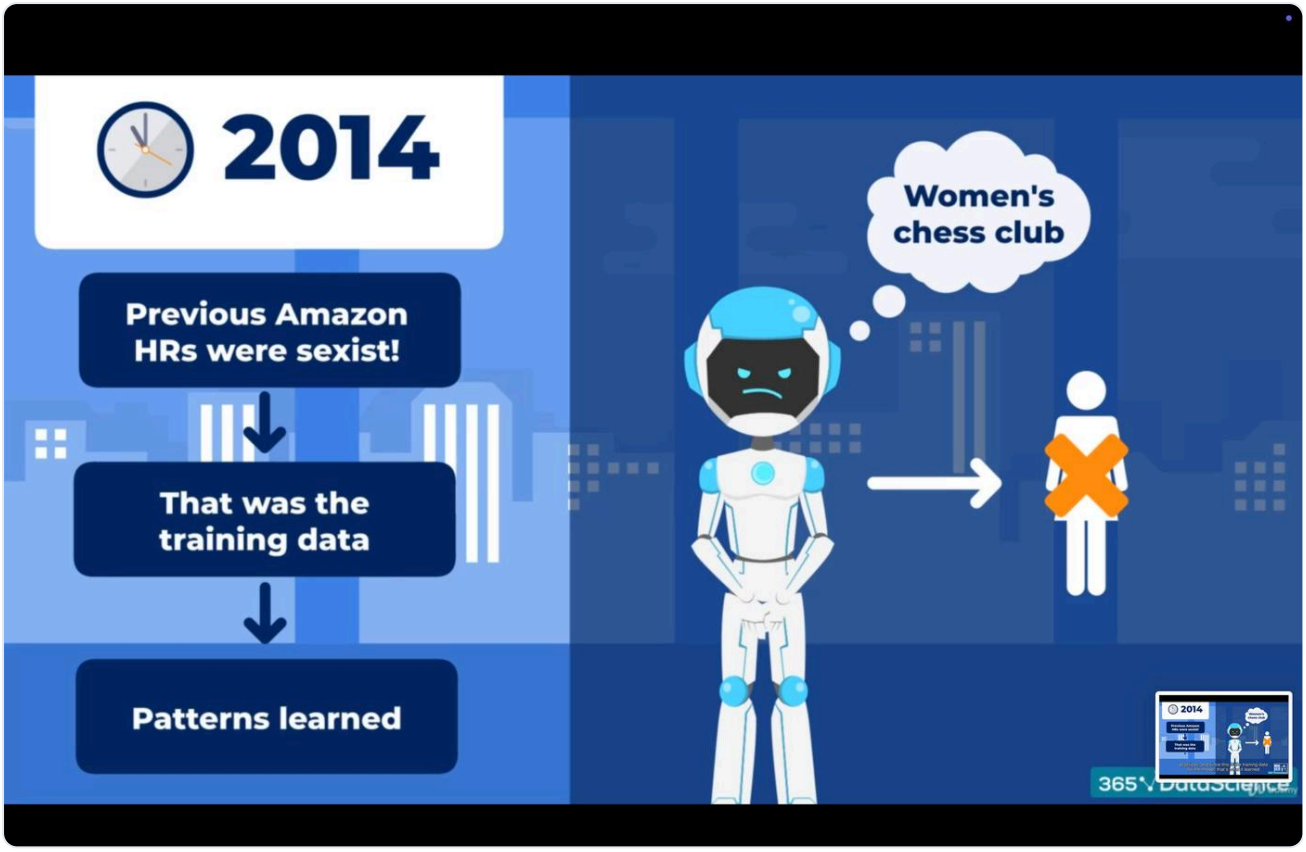


FIGURE 9.14 | Previous Amazon

How did this happen, you may be quick to judge that previous Amazon azar's had discriminatory hiring practices, and since this is the training data for the model, that's what it learned.

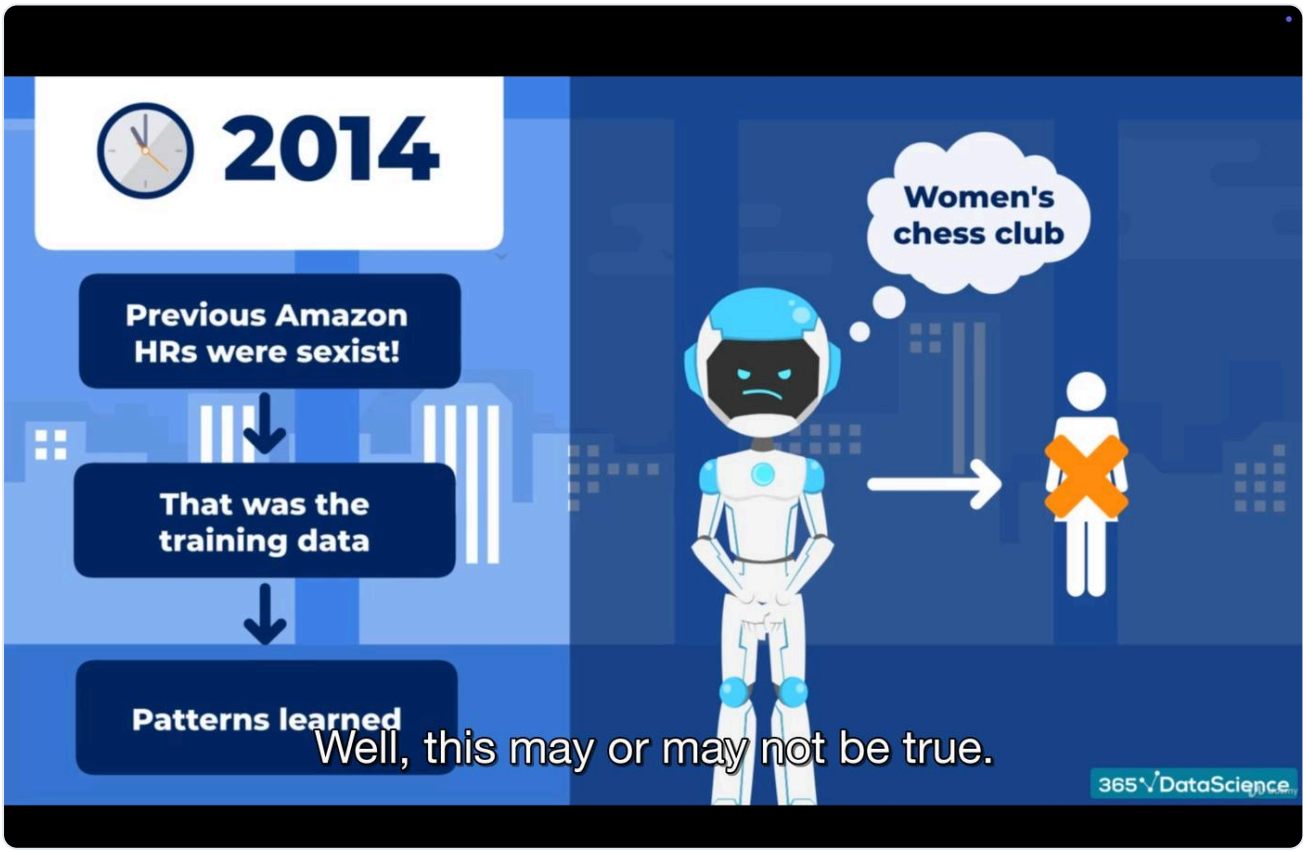


FIGURE 9.15 | Previous Amazon

Well, this may or may not be true. You see, the issue probably stems from unequal prior probabilities.

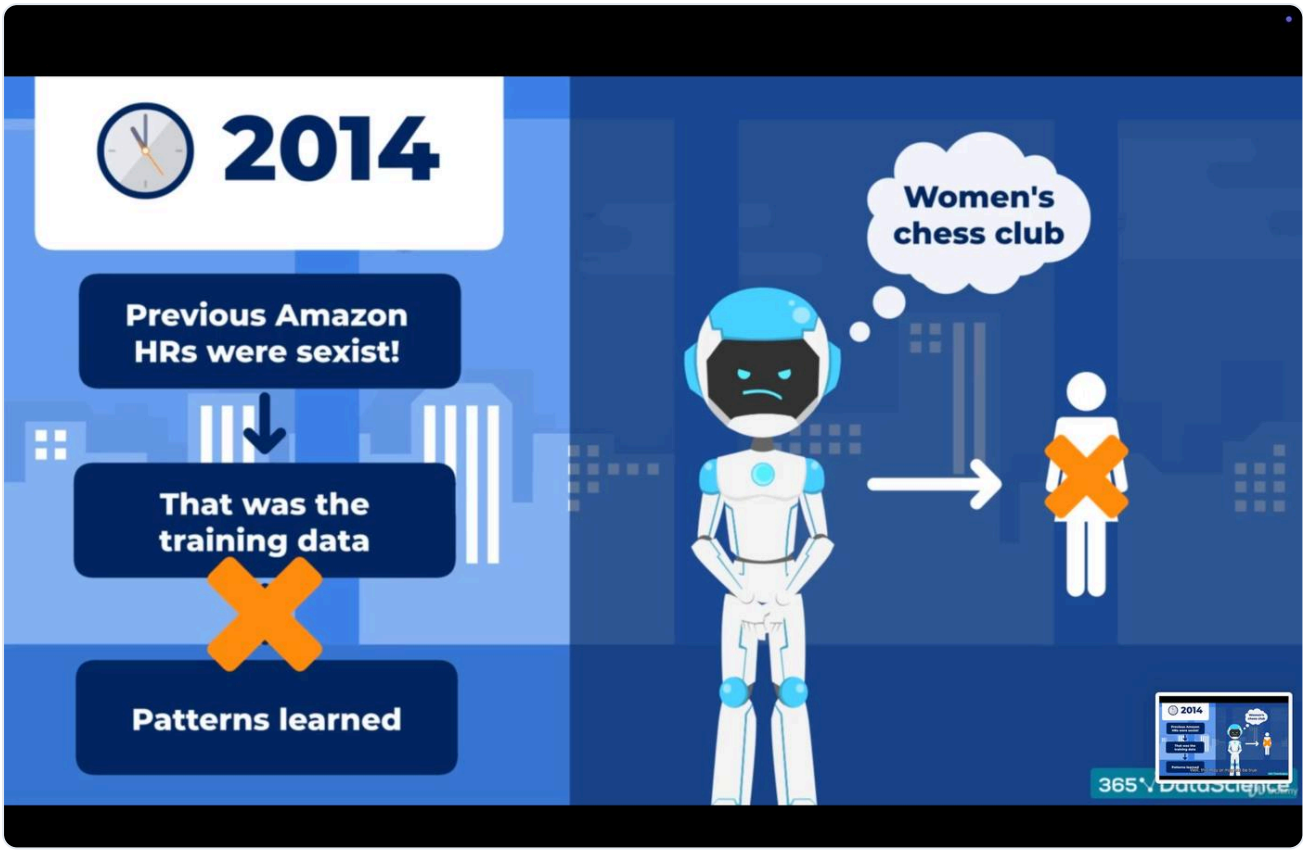


FIGURE 9.16 | Previous Amazon

Amazon may have hired more men than women simply because more men applied in the first place.

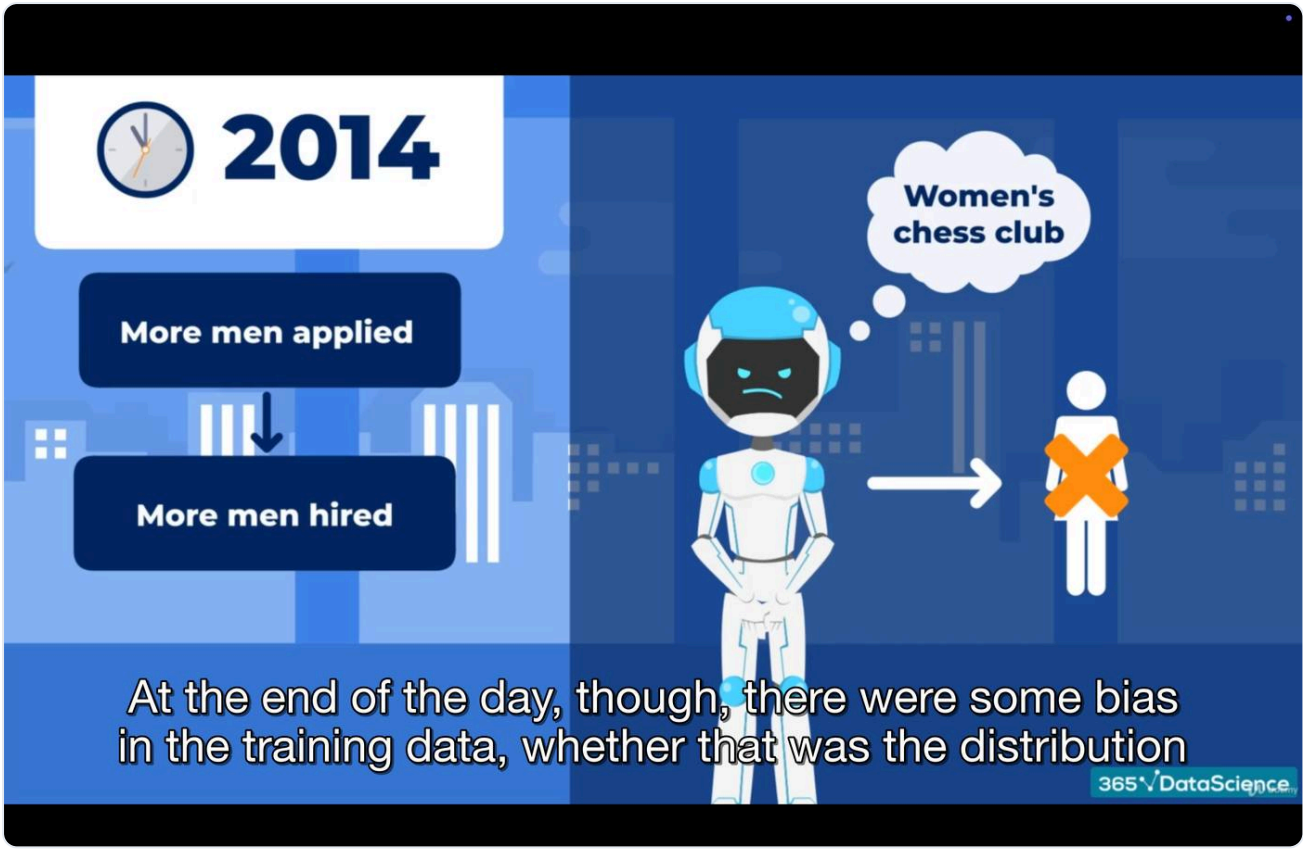


FIGURE 9.17 | More Men Applied E

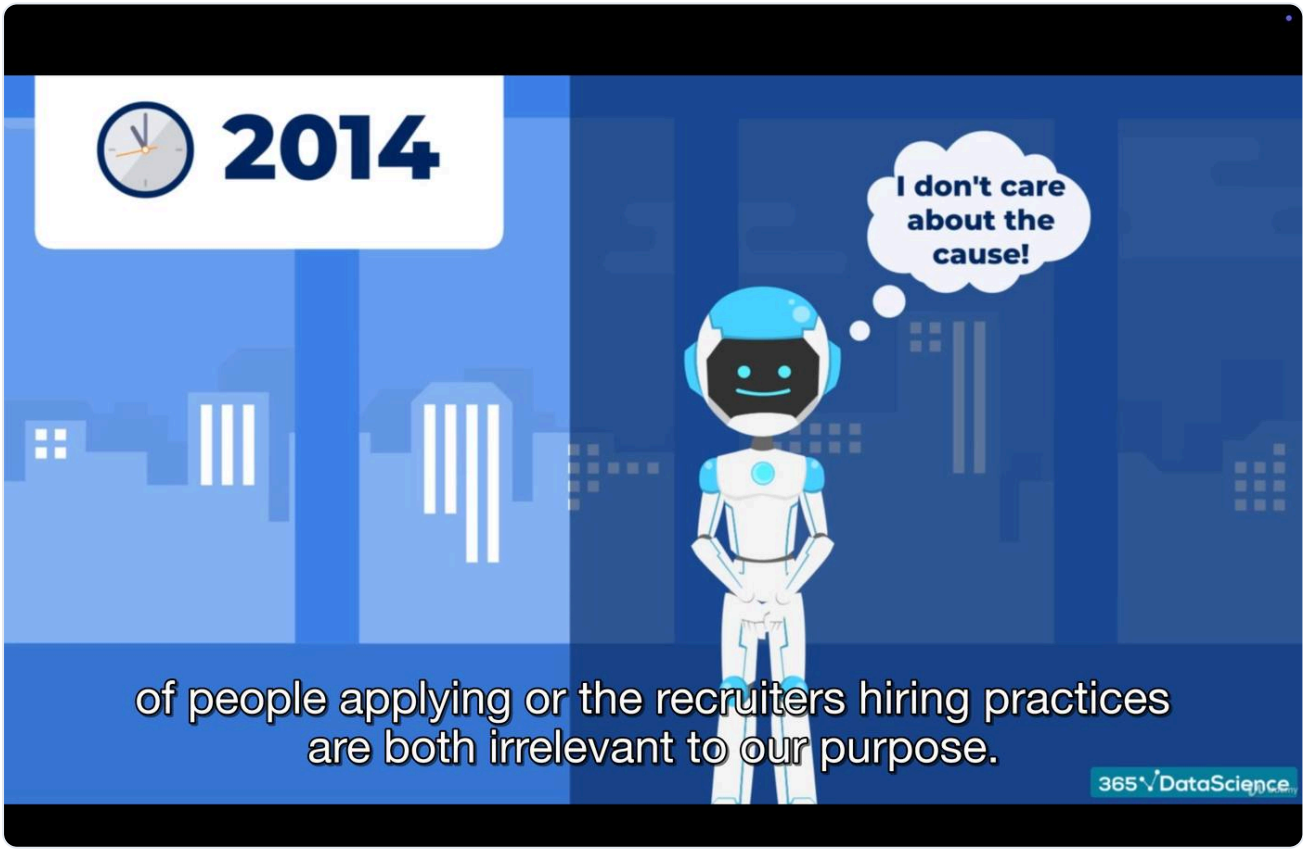


FIGURE 9.18

At the end of the day, though, there were some bias in the training data, whether that was the distribution of people applying or the recruiters hiring practices are both irrelevant to our purpose.

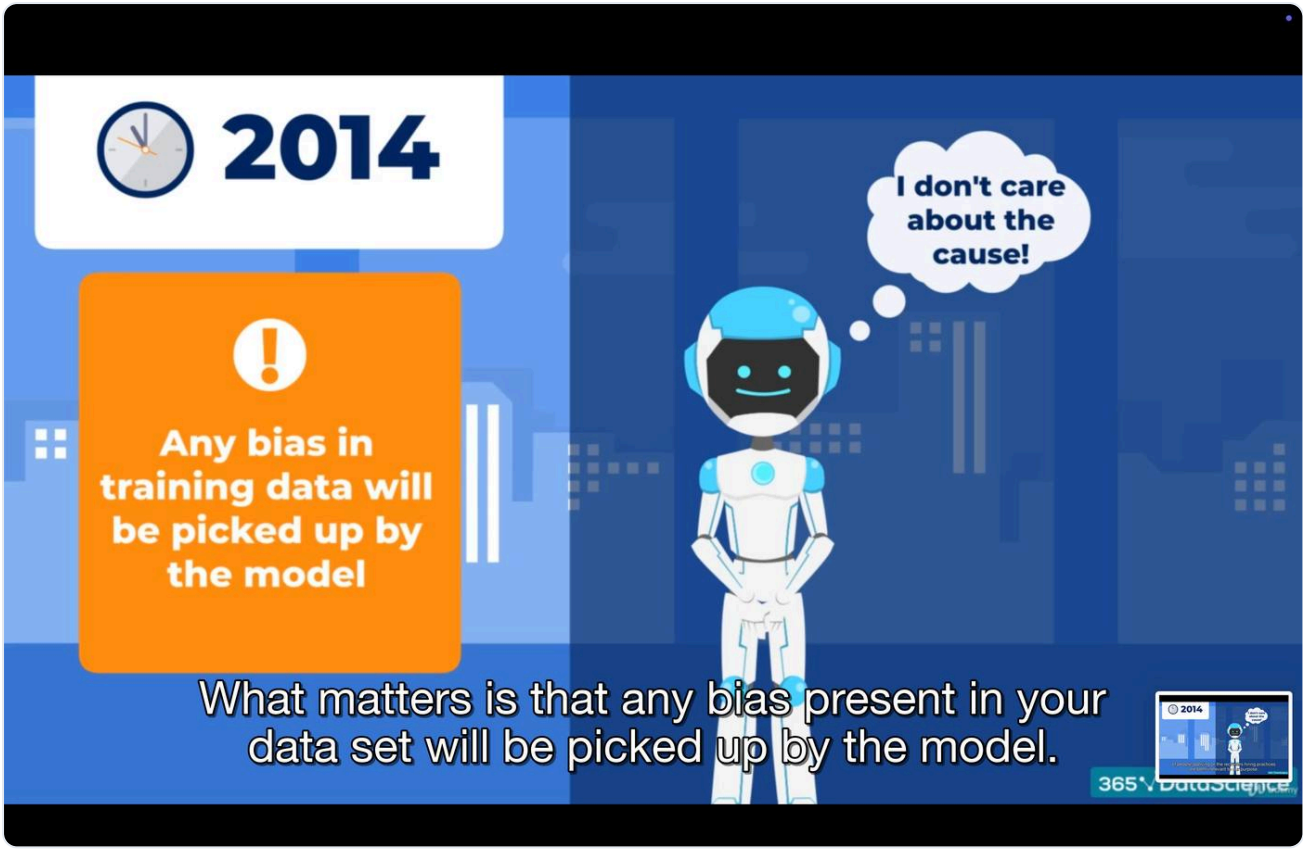


FIGURE 9.19

What matters is that any bias present in your data set will be picked up by the model. Luckily, Amazon was swift to notice the mistake of their algorithm and stopped it before it entered service, the mall recruiting engine was discontinued in twenty eighteen.

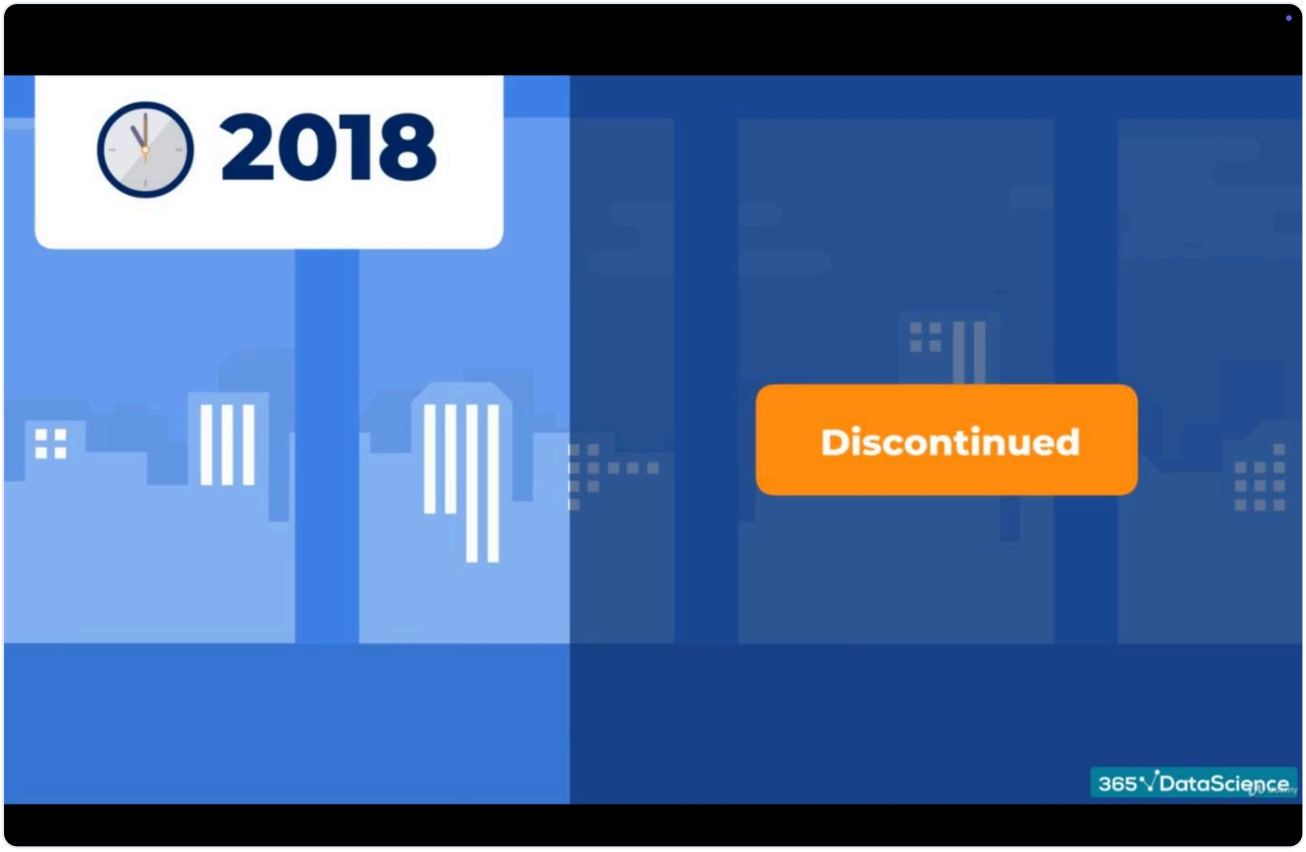


FIGURE 9.20

All right. I think this example illustrated the points, but if you're still not convinced, let's take a look at one last example of how fragile machine learning can be in 2014.

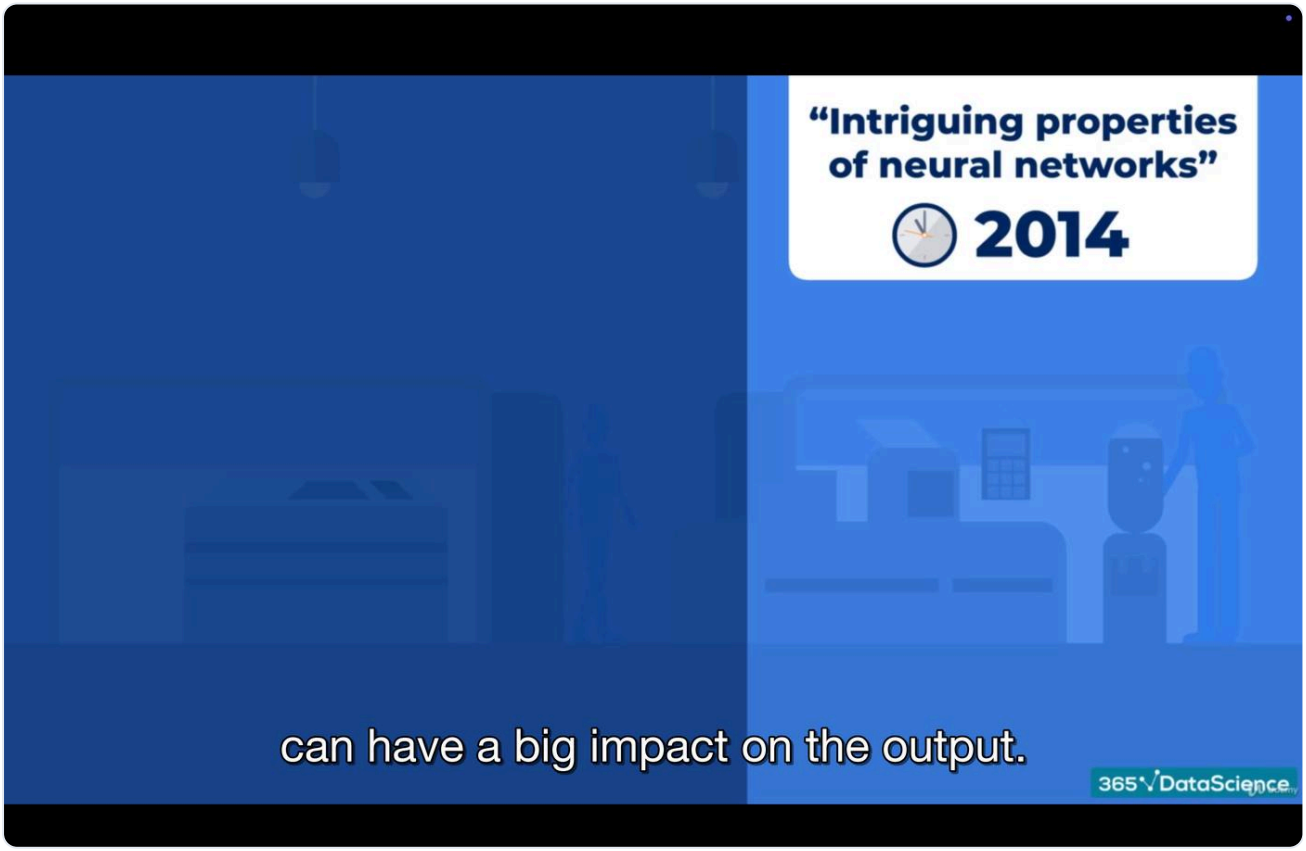


FIGURE 9.21

A paper called Intriguing Properties of Neural Networks showed how small perturbations to the input can have a big impact on the output.

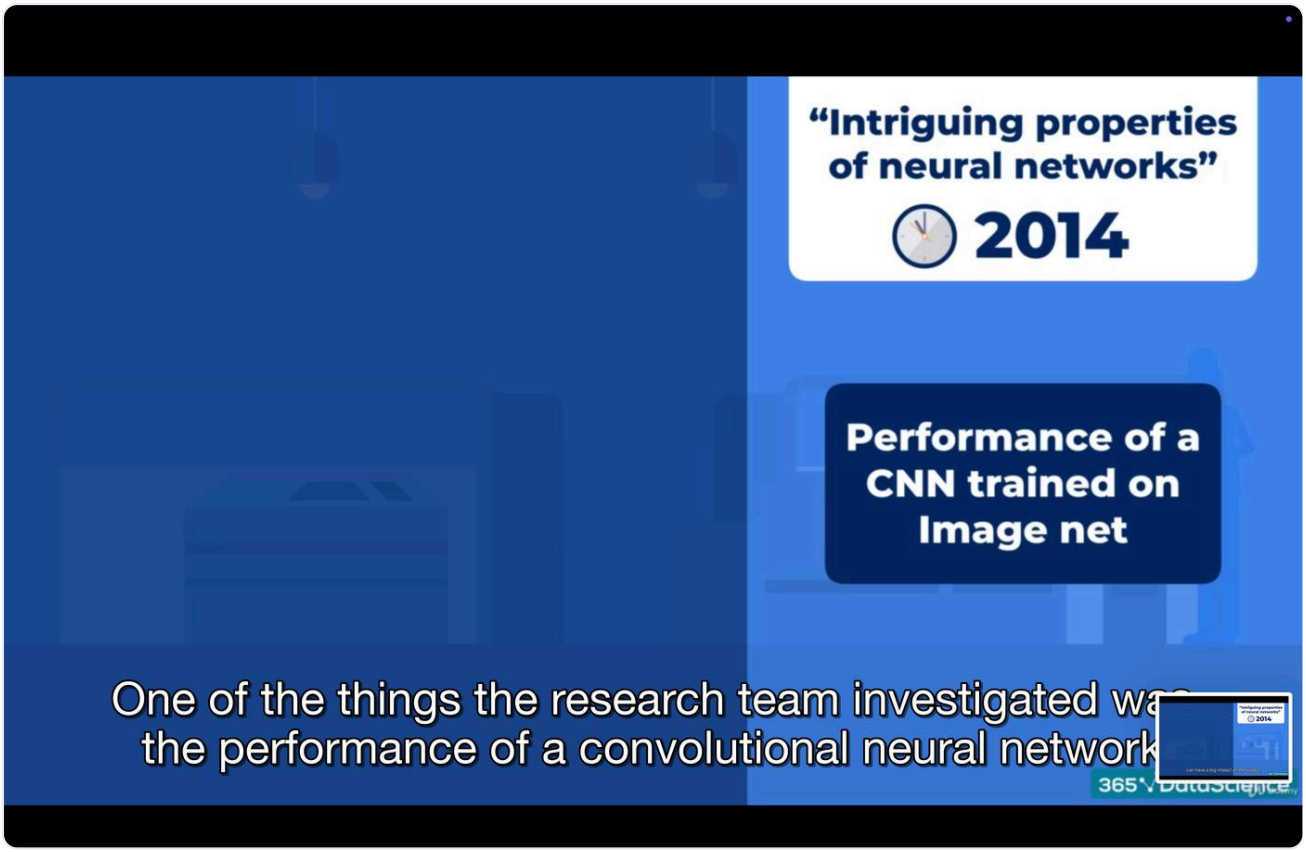


FIGURE 9.22

One of the things the research team investigated was the performance of a convolutional neural network trained on ImageNet ImageNet as a data set for image classification.

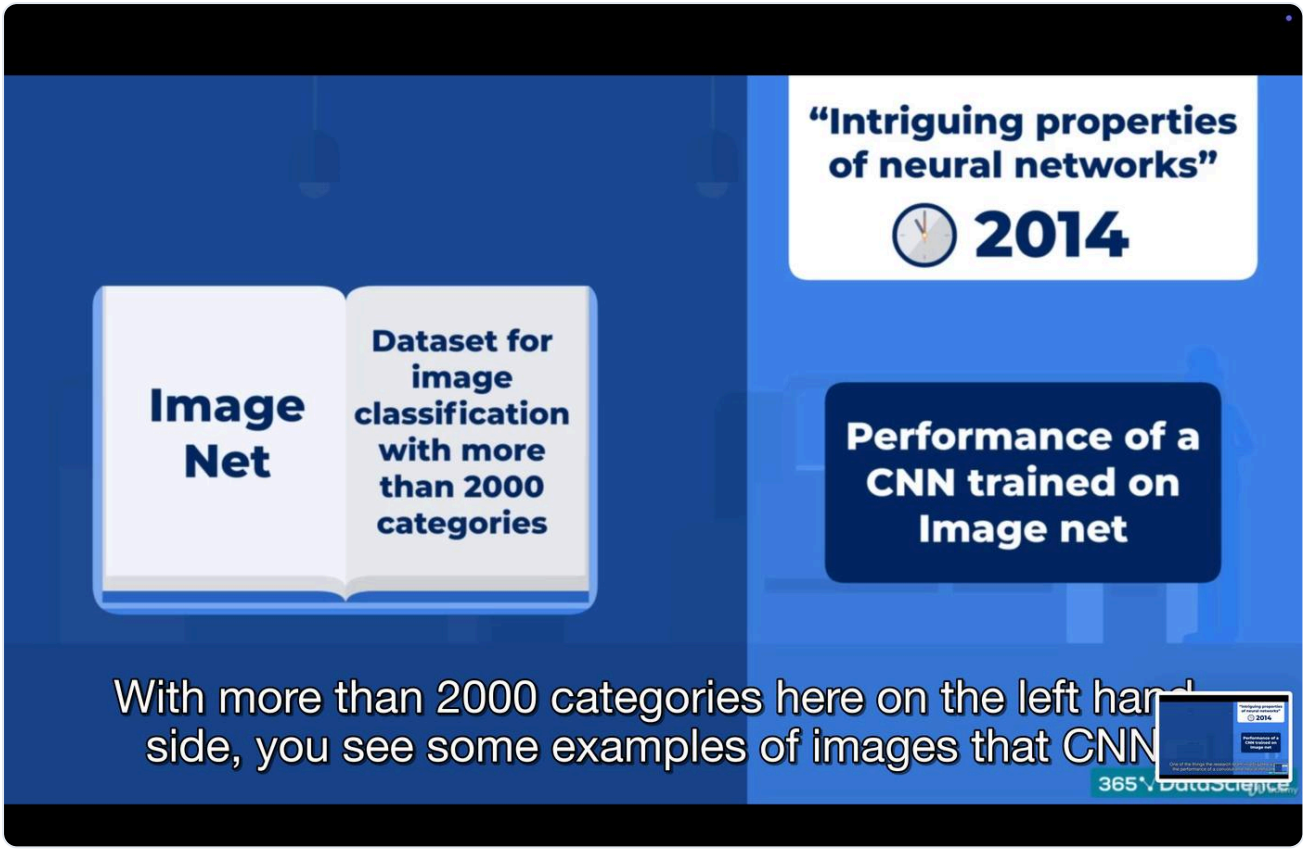


FIGURE 9.23

With more than 2000 categories here on the left hand side, you see some examples of images that CNN initially classified correctly.

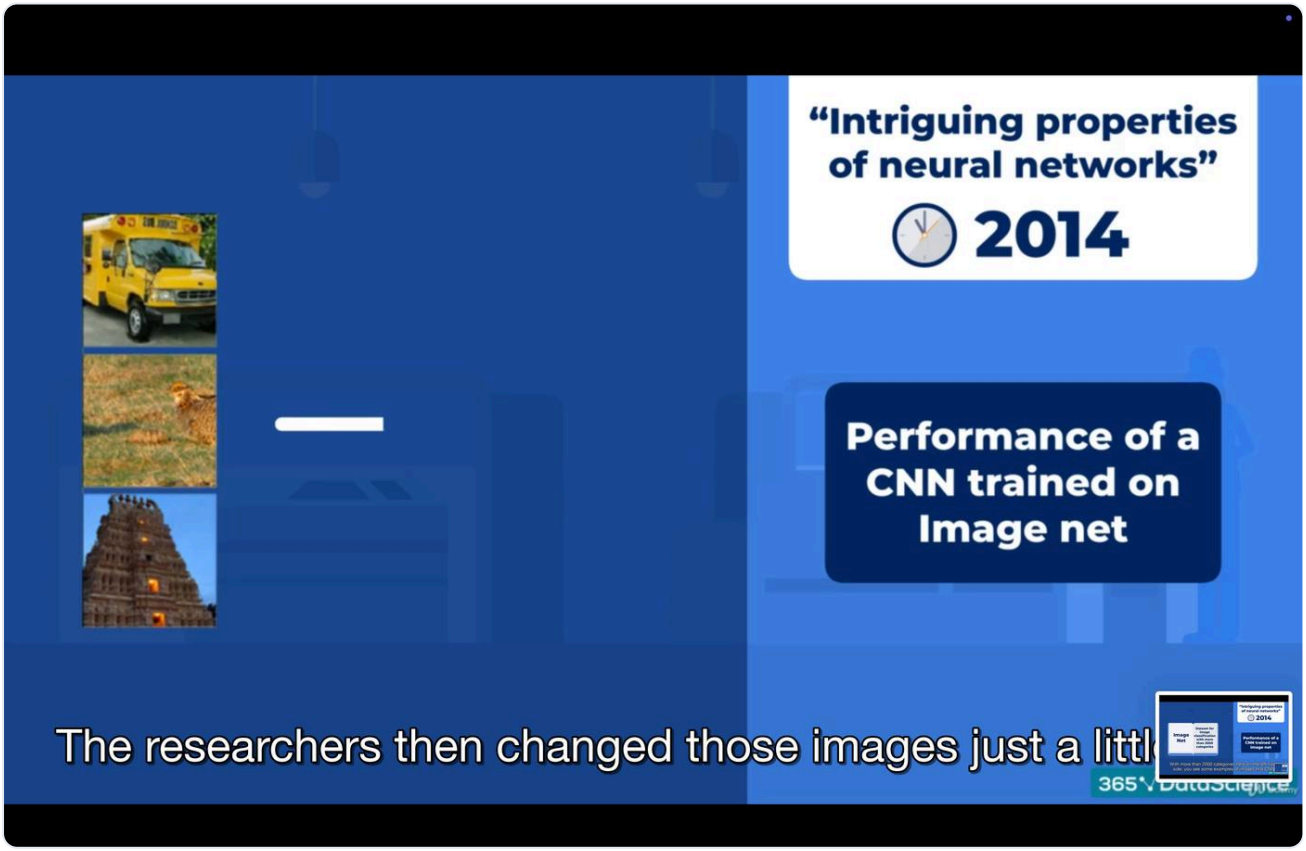


FIGURE 9.24 | Performance of a

The researchers then changed those images just a little bit.

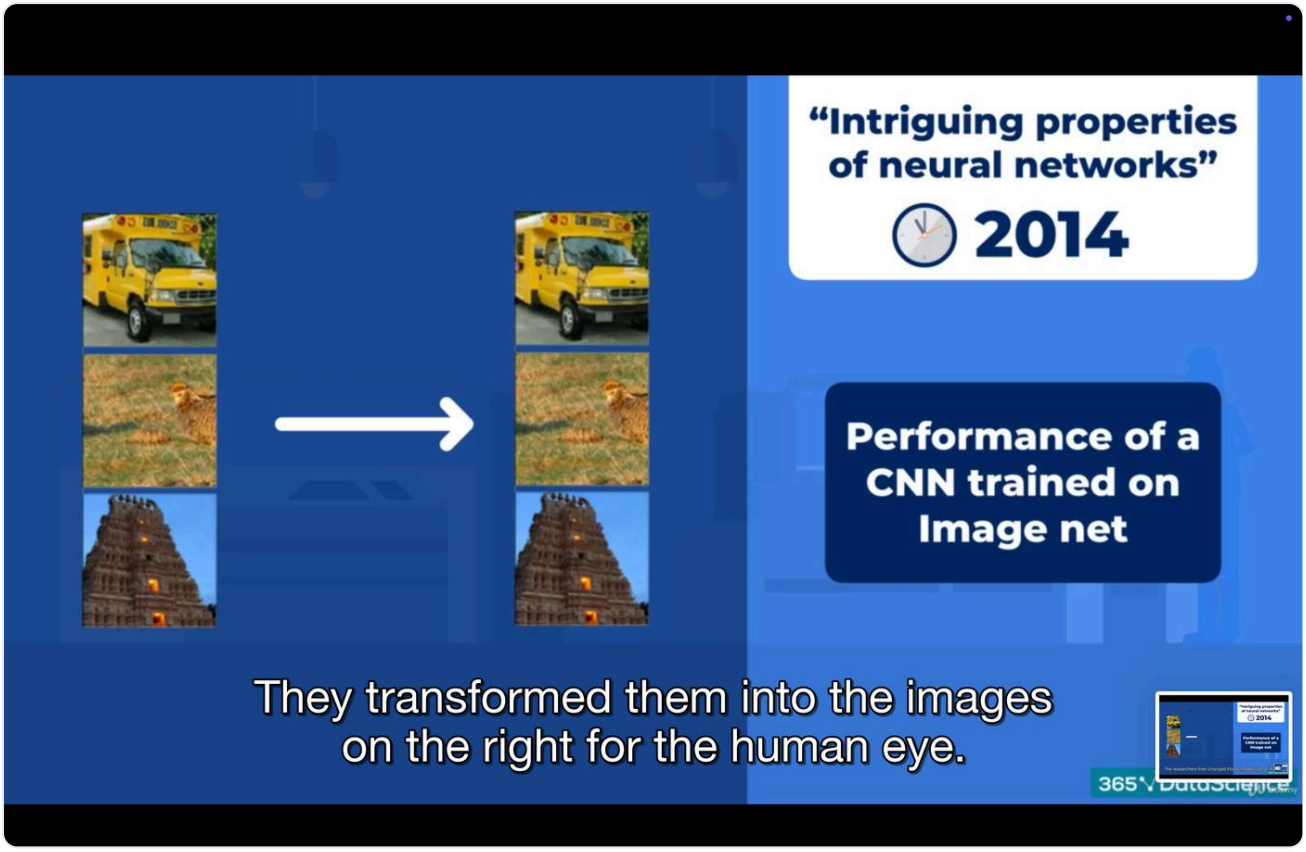


FIGURE 9.25 | Performance of a

They transformed them into the images on the right for the human eye. There is no perceivable difference. They look identical to us.

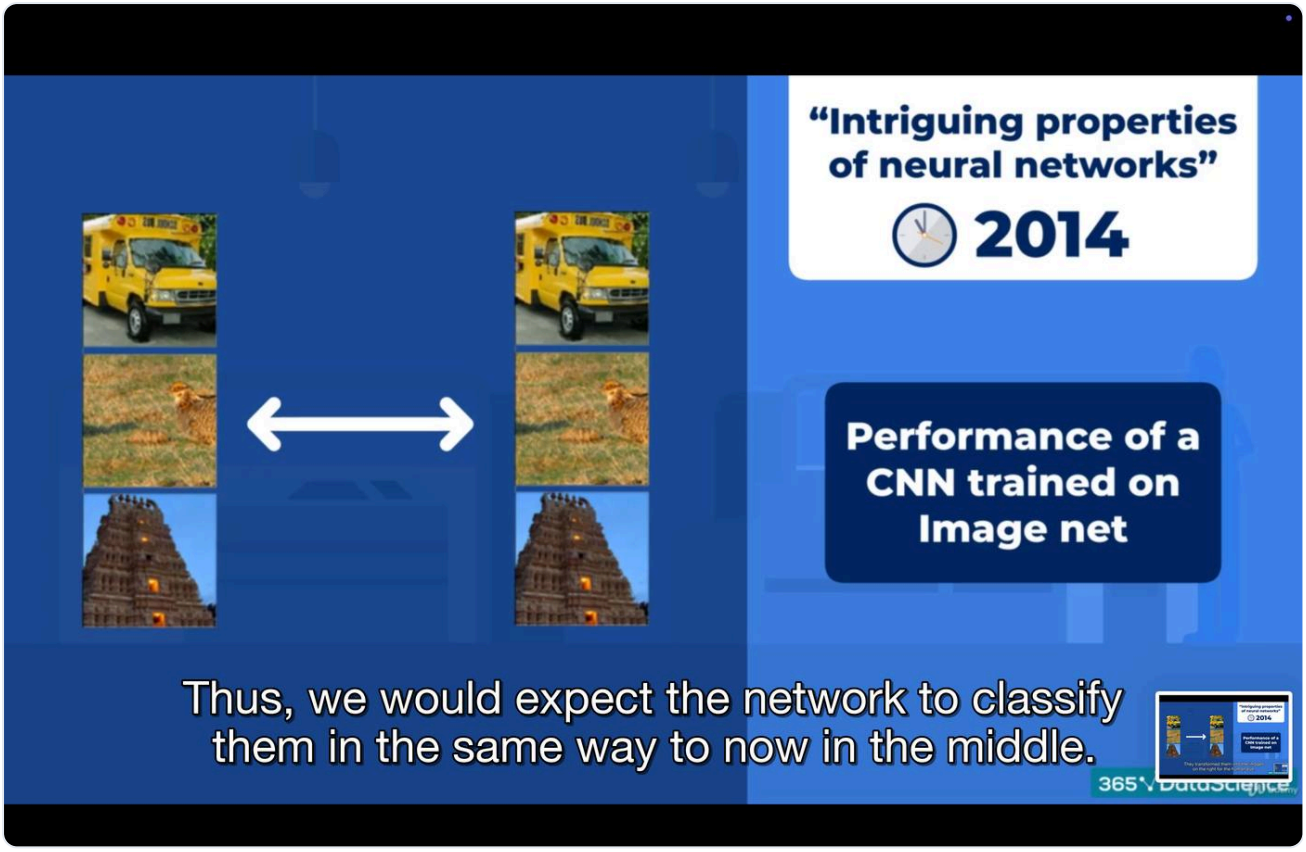


FIGURE 9.26 | Performance of a

Thus, we would expect the network to classify them in the same way to now in the middle.

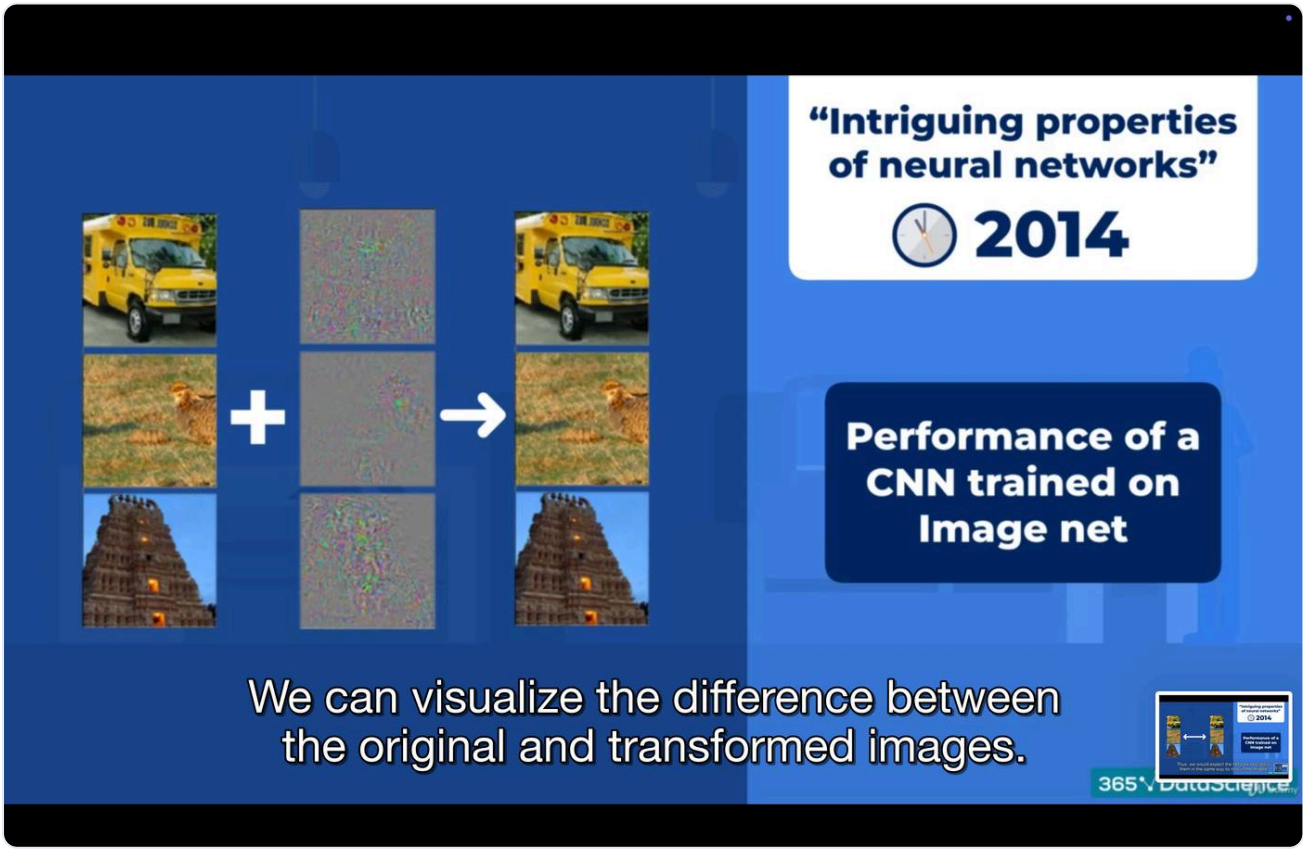


FIGURE 9.27 | Performance of a

We can visualize the difference between the original and transformed images. These represent the pixel value difference magnified 10 times to everyone's surprise, when the network tried to classify the transformed images on the right.

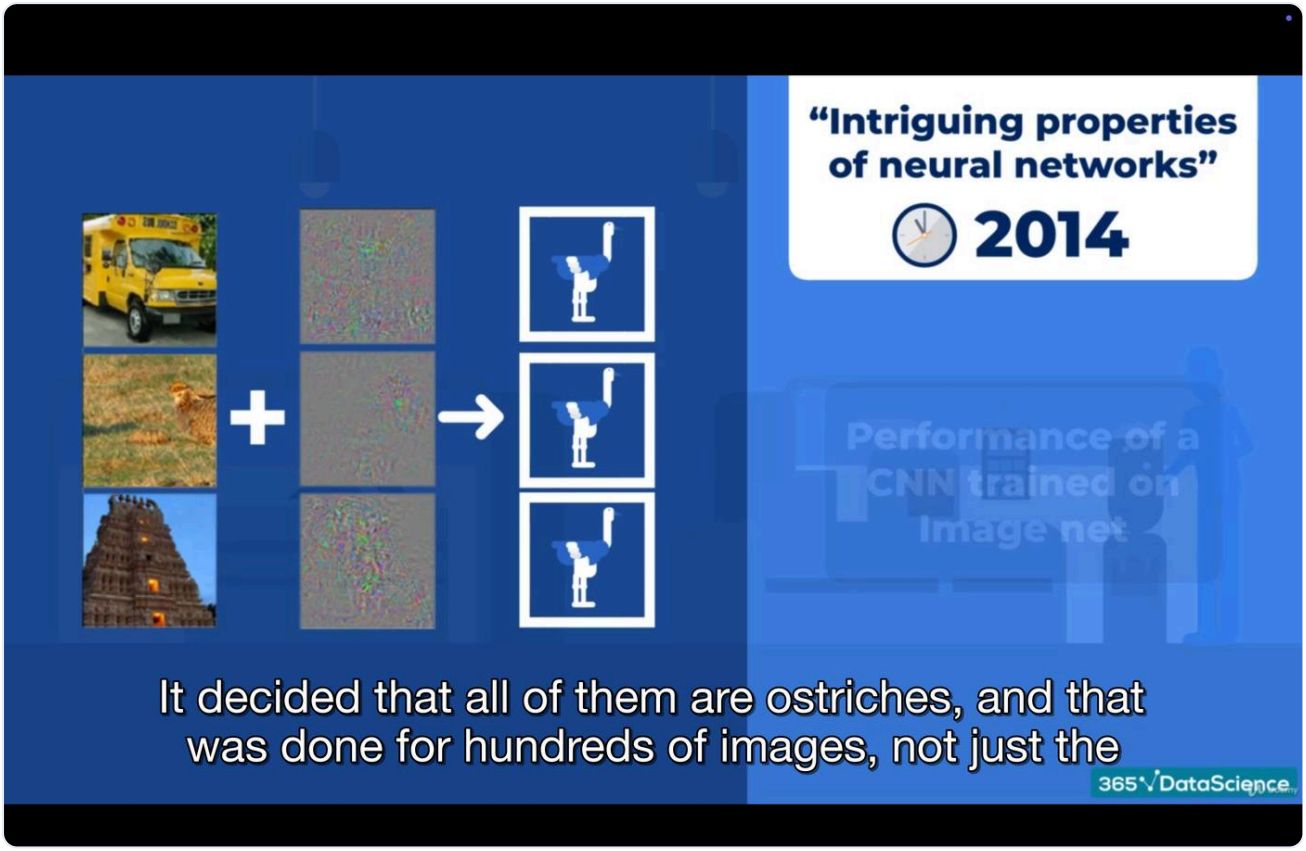


FIGURE 9.28

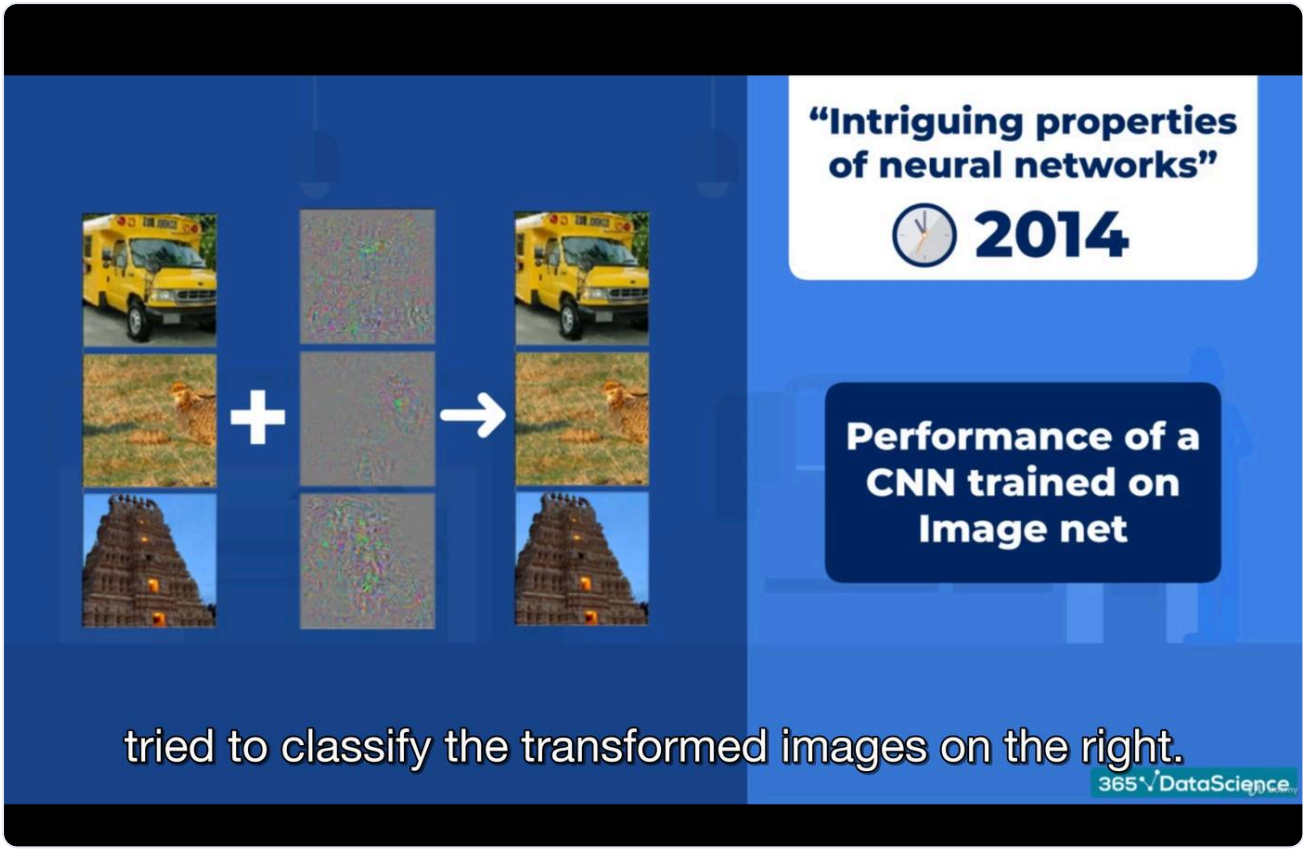


FIGURE 9.29 | Performance of a

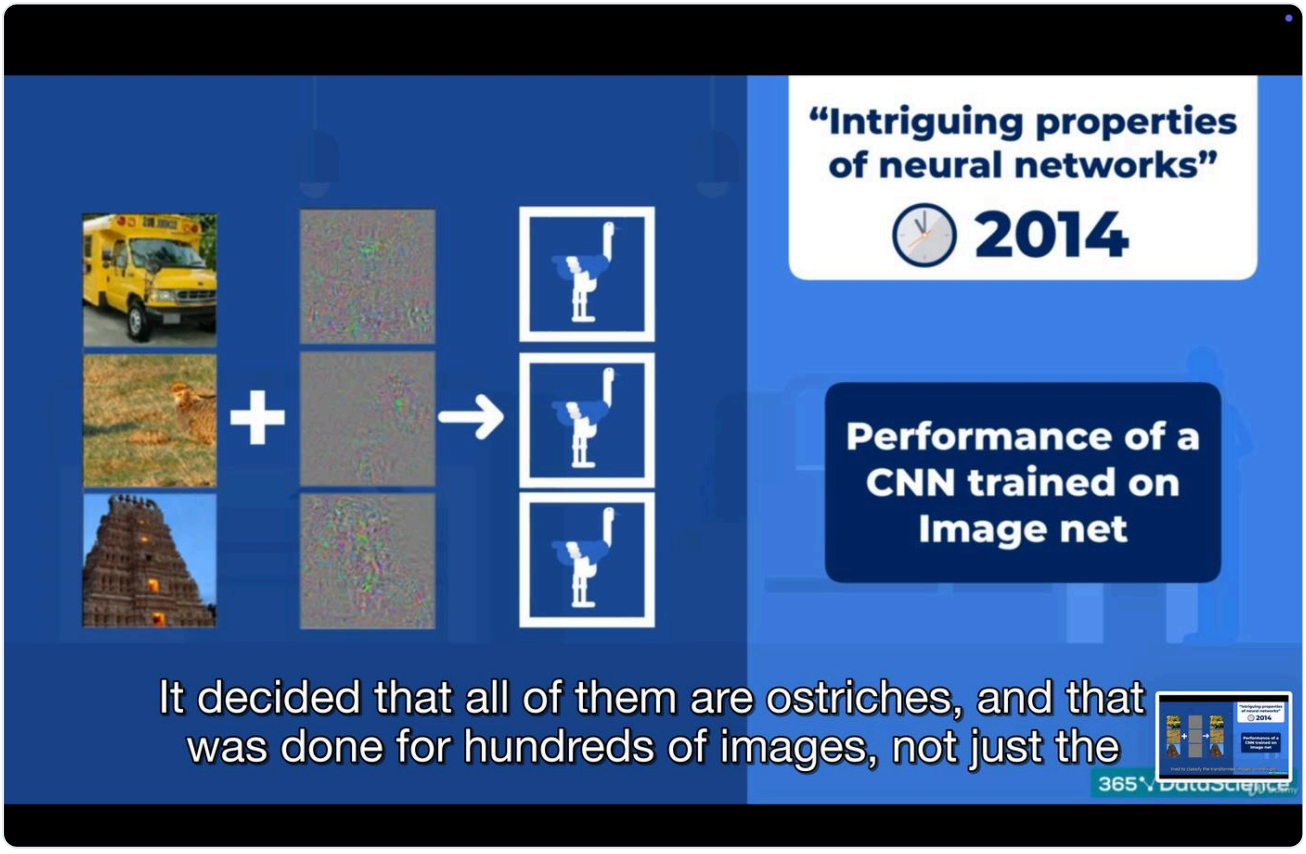


FIGURE 9.30 | Performance of a

It decided that all of them are ostriches, and that was done for hundreds of images, not just the three shown here. Furthermore, the team formalized their findings and documented an approach to constructing these small variations in order to confuse the CNN. This is a good illustration of how fragile machine learning models can be.

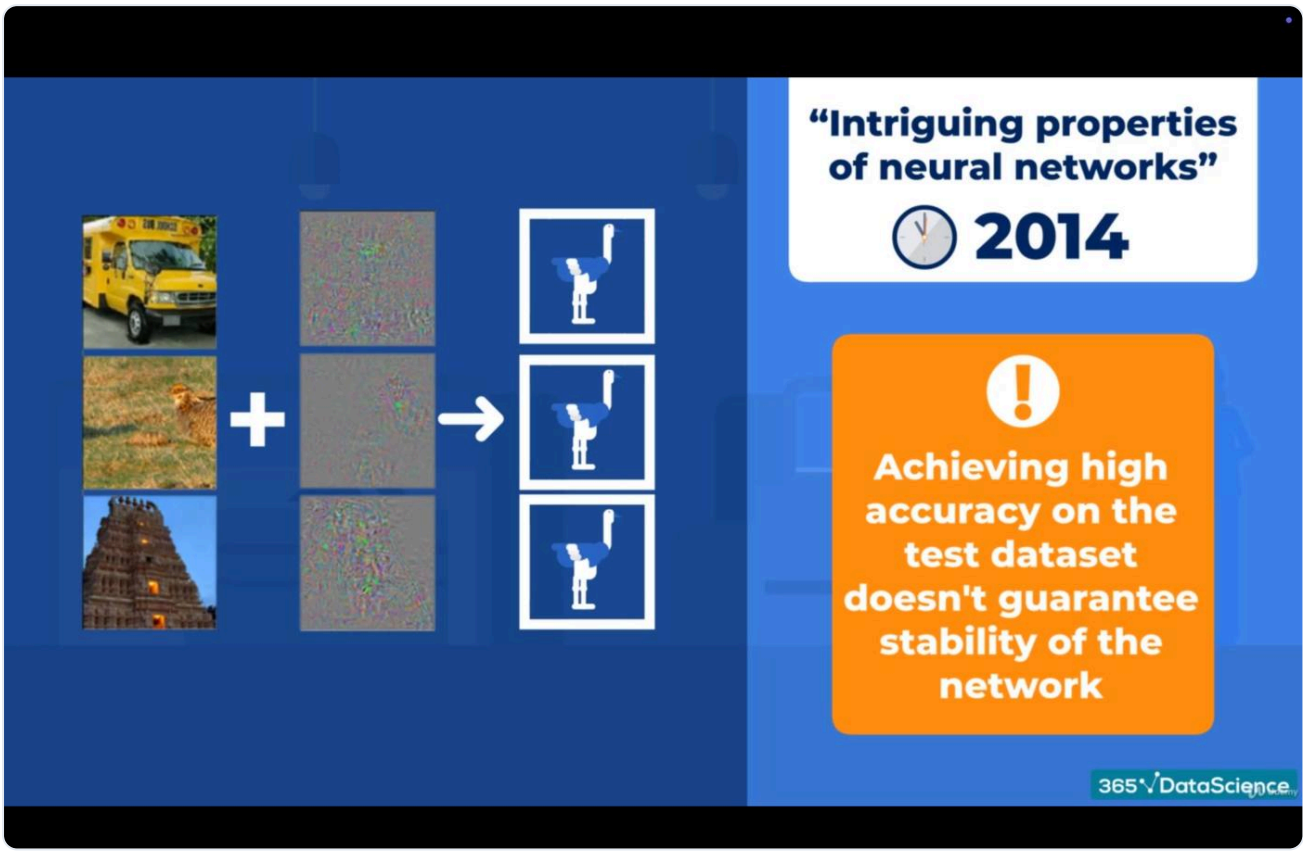


FIGURE 9.31

You may achieve high accuracy even on the test data set, but that does not guarantee your network is stable. While these experiments were made by well-intentioned academics, misusing this knowledge could have serious implications. Imagine you were writing a face recognition software for security purposes.

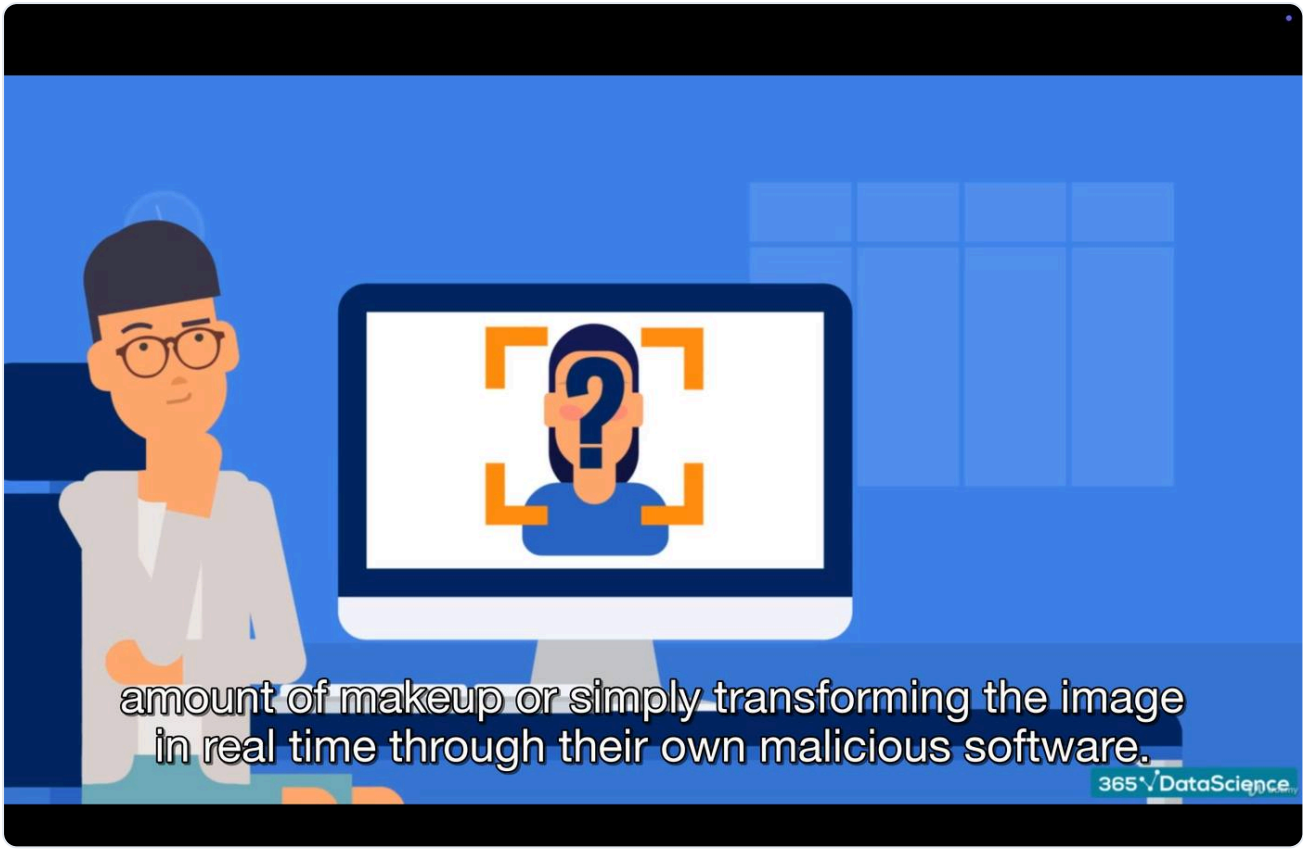


FIGURE 9.32

It would be a real disaster if people were able to fool your model by putting on an indistinguishable amount of makeup or simply transforming the image in real time through their own malicious software.

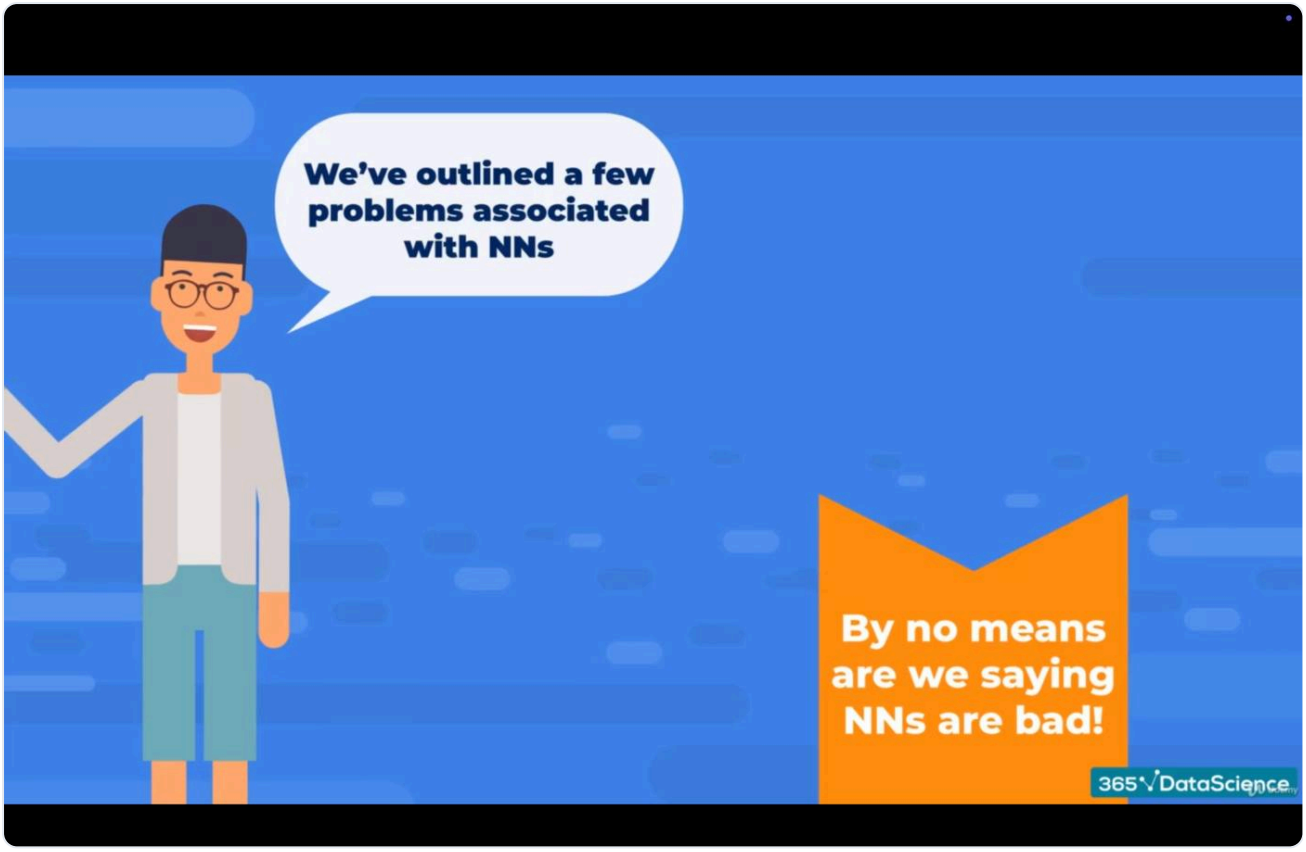


FIGURE 9.33 | We've Outlined a Few

All right, in this video, we've outlined a few problems associated with neural networks, by no means are we saying and ends are bad. We just want to illustrate that understanding the behavior of your network is particularly important. OK, in the next section, we will discuss some of the popular CNN architectures that have been developed. Thanks for watching.

LECTURE 48

Networks

So far, we have examined a couple of convolutional networks whose architectures were our own creation without in-depth research. It is now time to take a look at what the pros have been doing on this front. In this section, we will delve into some of the more popular and influential CNN architectures.

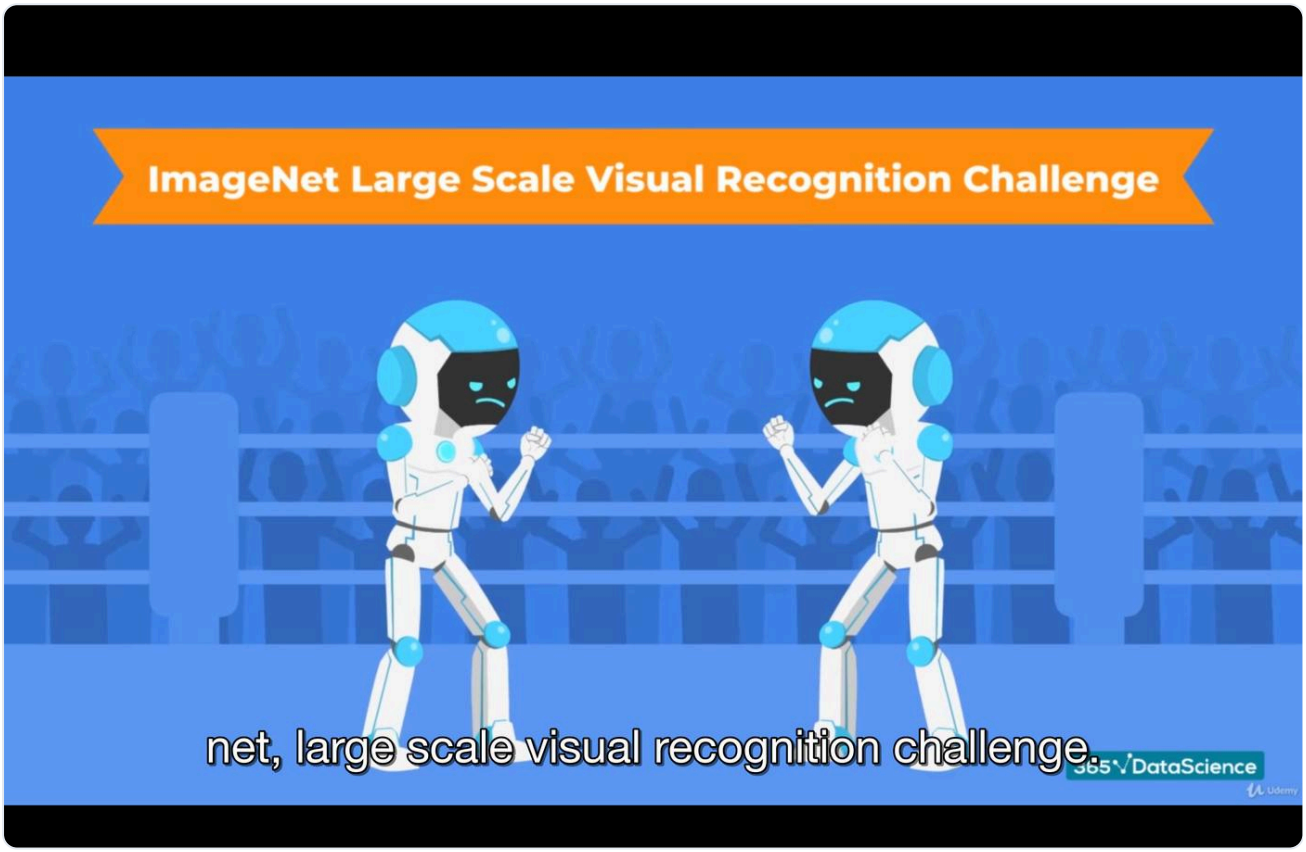


FIGURE 10.1

All right. One of the fronts on which different networks were tested against each other was the so-called ImageNet, large scale visual recognition challenge.

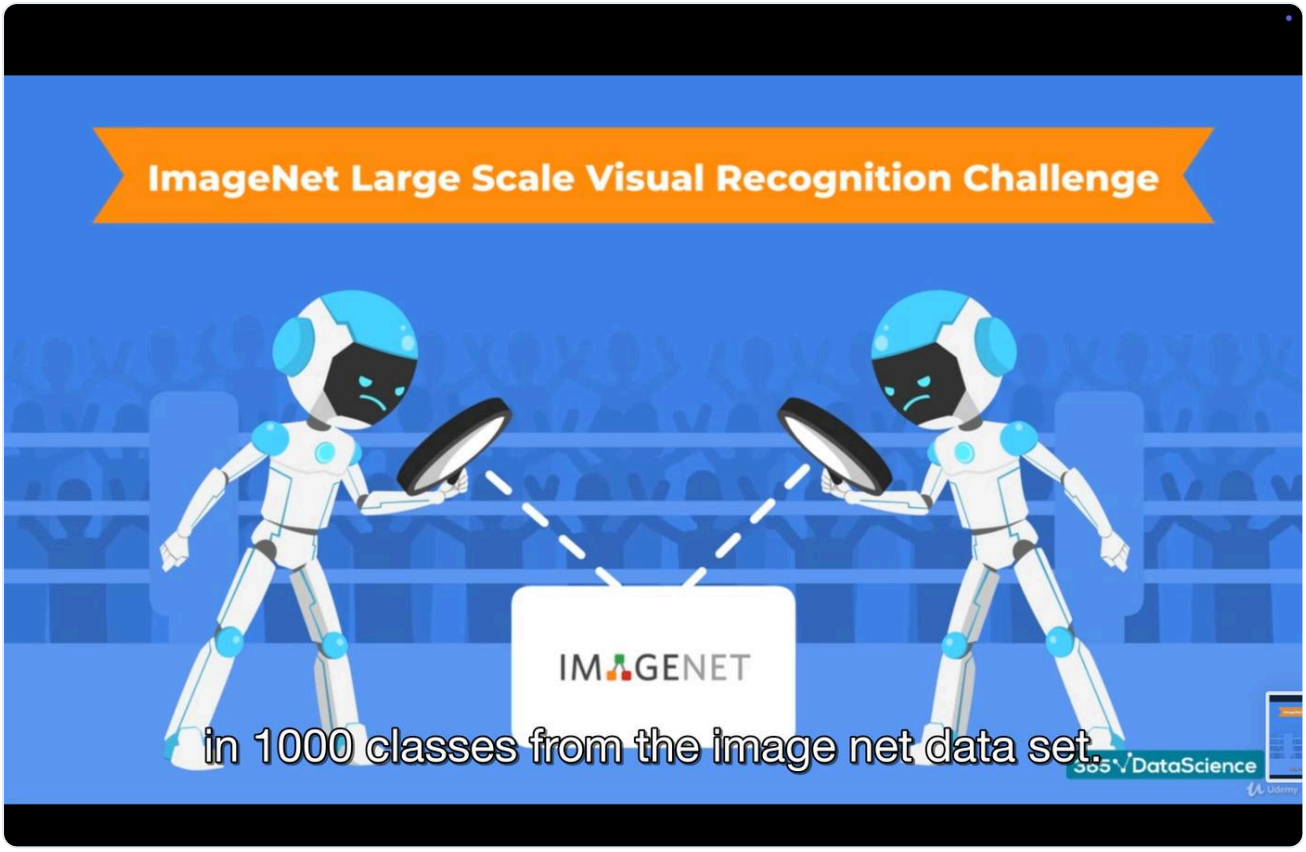


FIGURE 10.2

This was basically a challenge for algorithms to see which one could best classify or detect objects in 1000 classes from the ImageNet data set. The challenge was not explicitly meant for neural networks, and in fact, the first winners were not deep networks. Actually, the ILSVRC was a series of competitions.

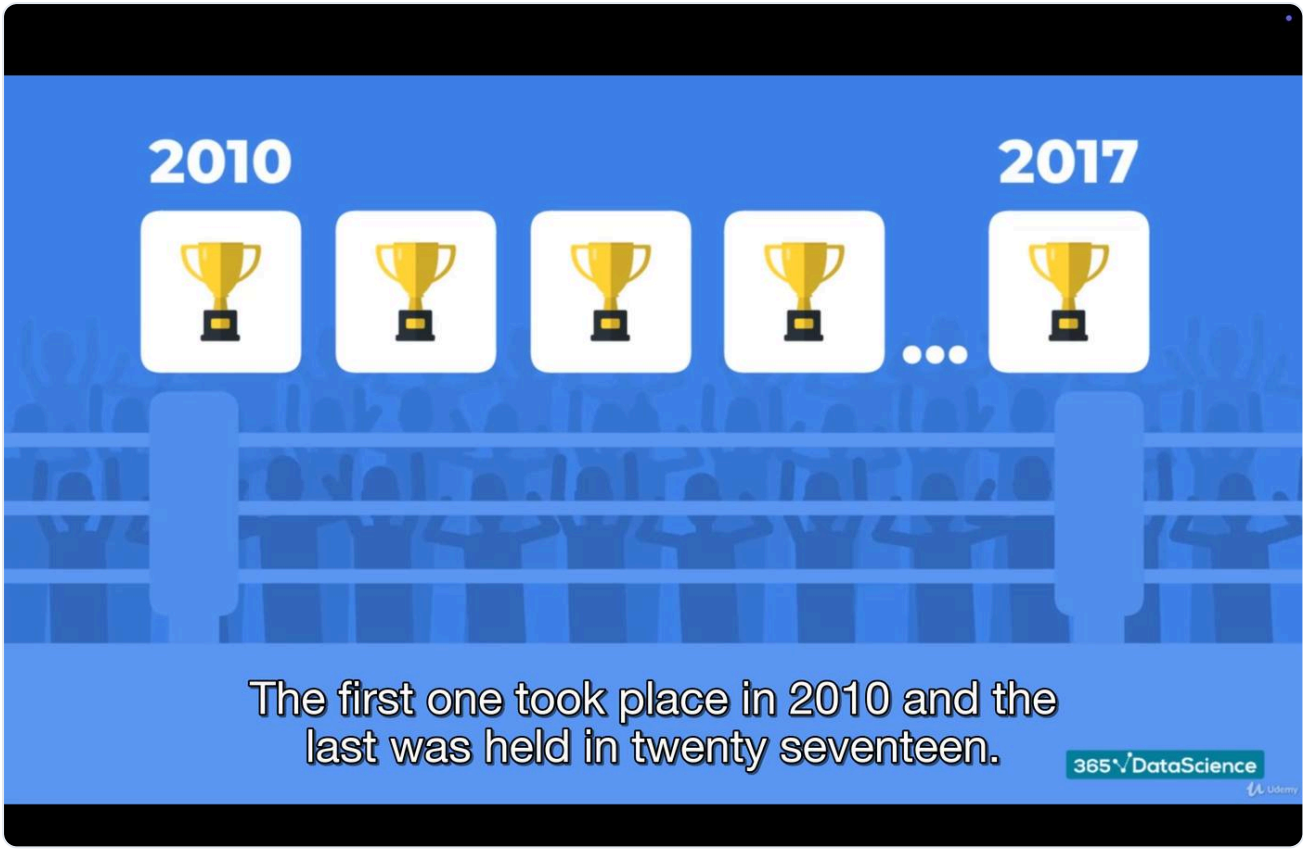


FIGURE 10.3

The first one took place in 2010 and the last was held in twenty seventeen. In the last one.

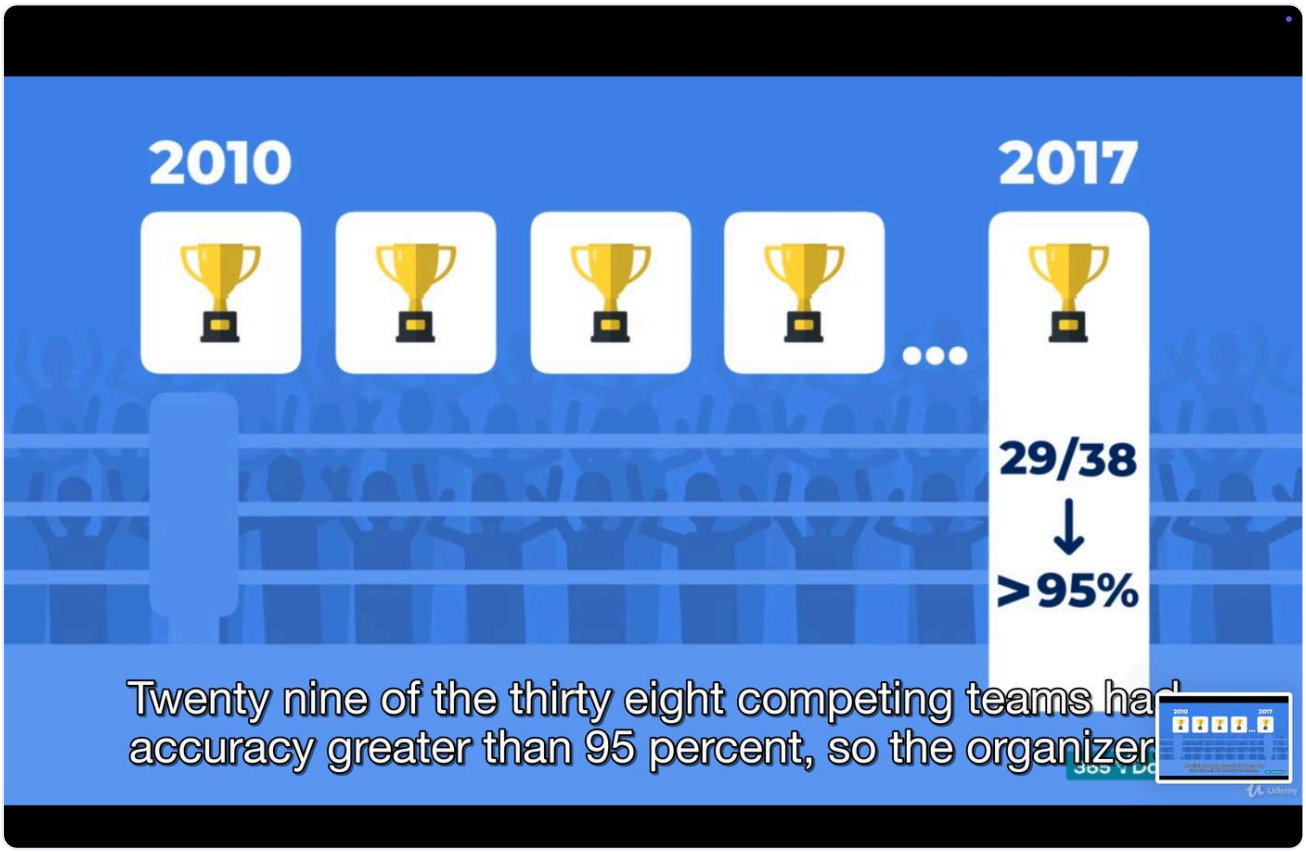


FIGURE 10.4

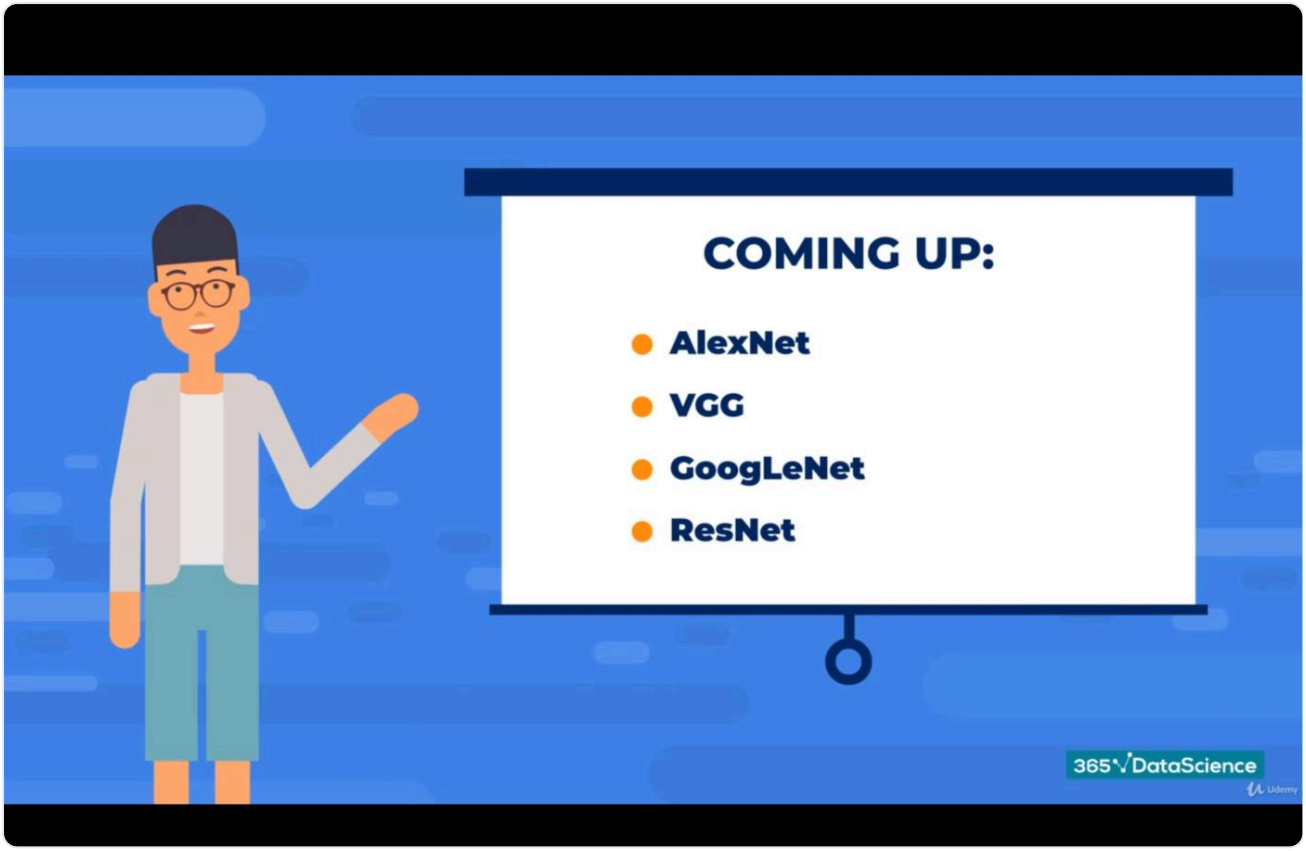


FIGURE 10.5

Twenty nine of the thirty eight competing teams had accuracy greater than 95 percent, so the organizers moved on to more difficult projects. Nevertheless, this competition contributed to a lot of advancements in computer vision. In the next lectures, we will take a look at four different winners of the competition AlexNet, VG, GoogLeNet and Resonate. We will gain an understanding of the underlying concepts behind their design as we go forward. See you there and thanks for watching.

LECTURE 49

Max Pool



FIGURE 11.1

In this video, we will take a closer look at AlexNet, AlexNet was the winning entry of the last ILSVRC contest in 2012. The fact that a deep learning algorithm had won the competition for the first time made it even more special.



FIGURE 11.2

For the most part, it was this convolutional network that started the spree of CNN research and architectures. In the following years, AlexNet was larger than previous seasons used for computer vision tasks.

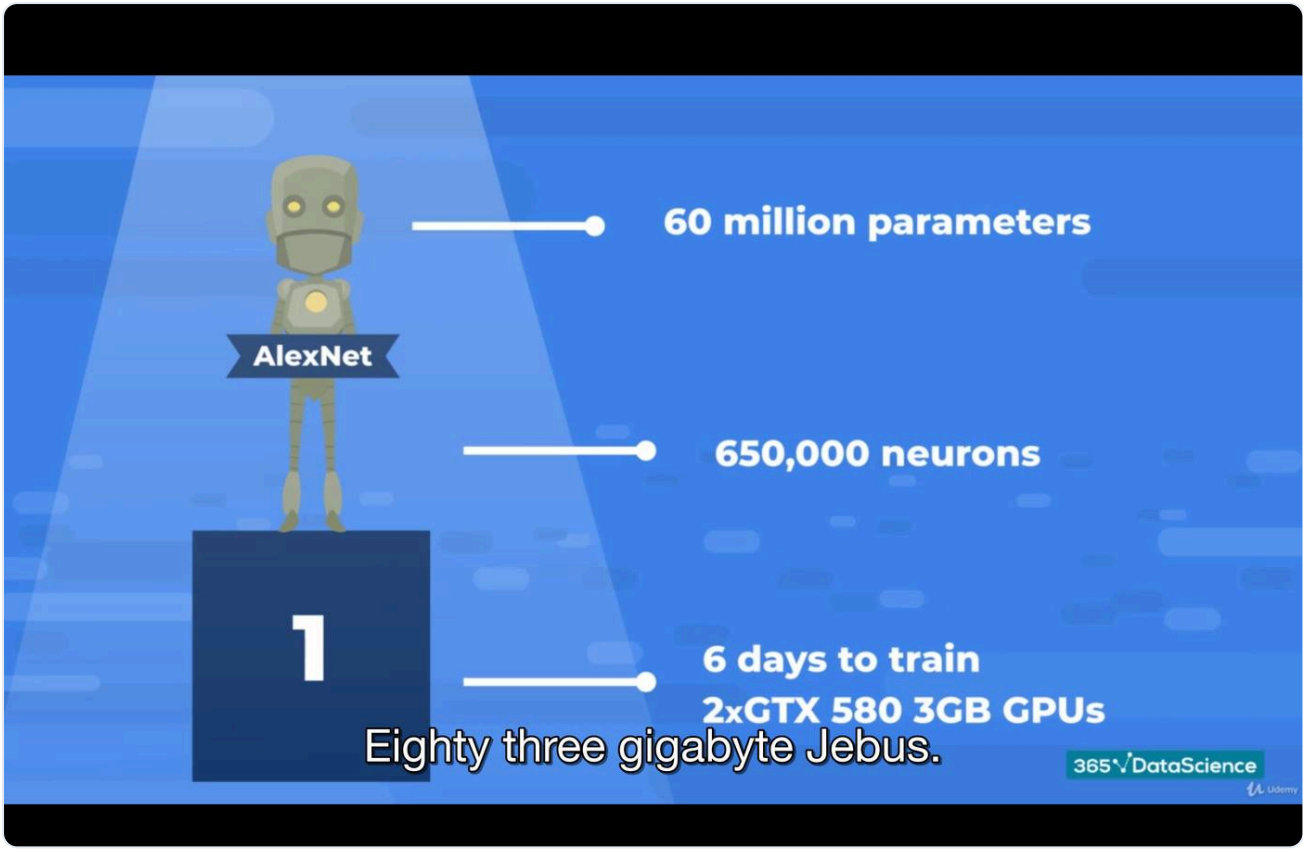


FIGURE 11.3

It had 60 million parameters, 650000 neurons and took six days to train on to five. Eighty three gigabyte GPUs. Nowadays, this may not sound as impressive. You probably have a better video card right now. For its time, however, the 580 was considered a high end GPU.

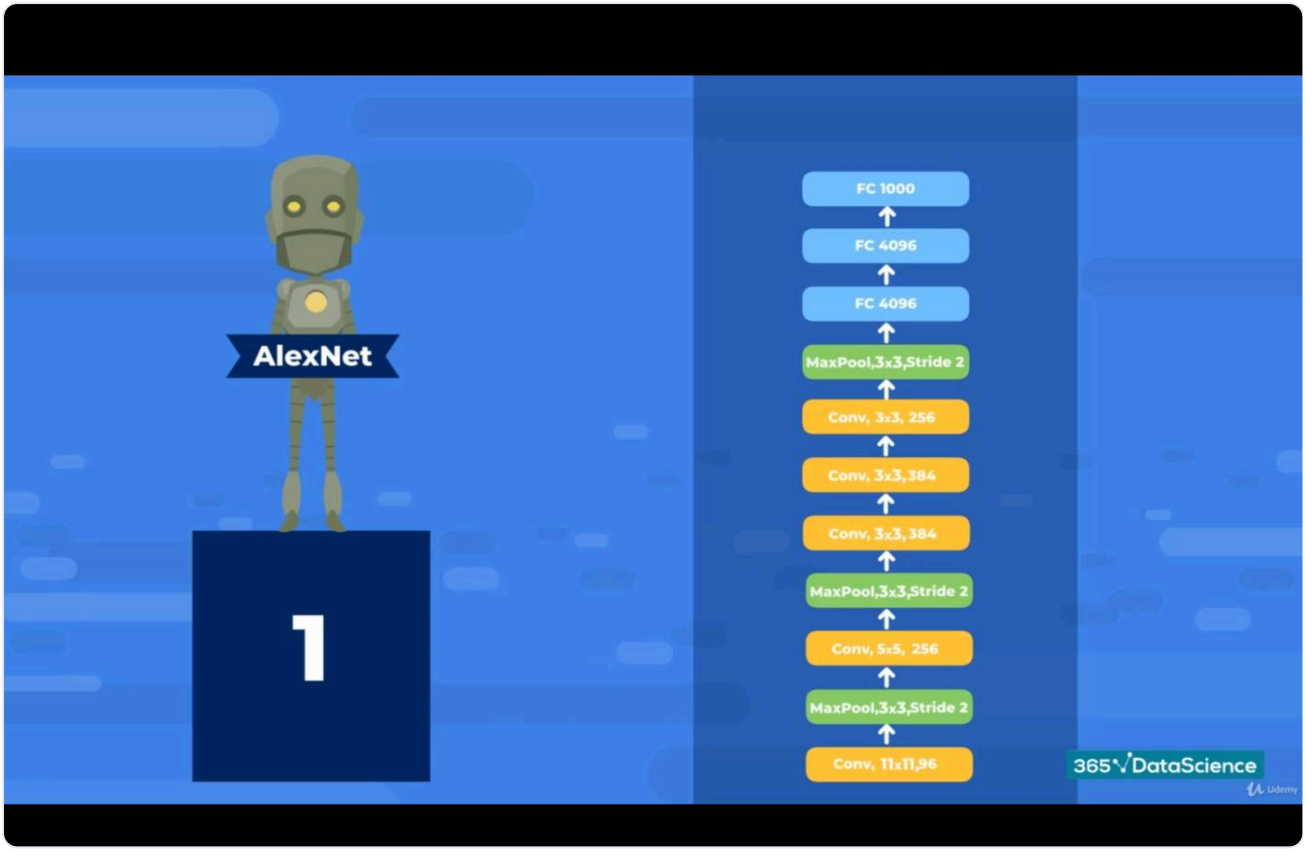


FIGURE 11.4

Today, there are much more complex scenes that can run on faster use very efficiently, even in large datasets. But back in 2012, this was huge. What about the architecture of the network itself, Alex? And it consisted of five convolutional layers, three max pool layers and three fully connected or dense layers. As you can see on the graphic, we first have a convolutional layer followed by a max pool and then repeated again a conveyor and a max pool.

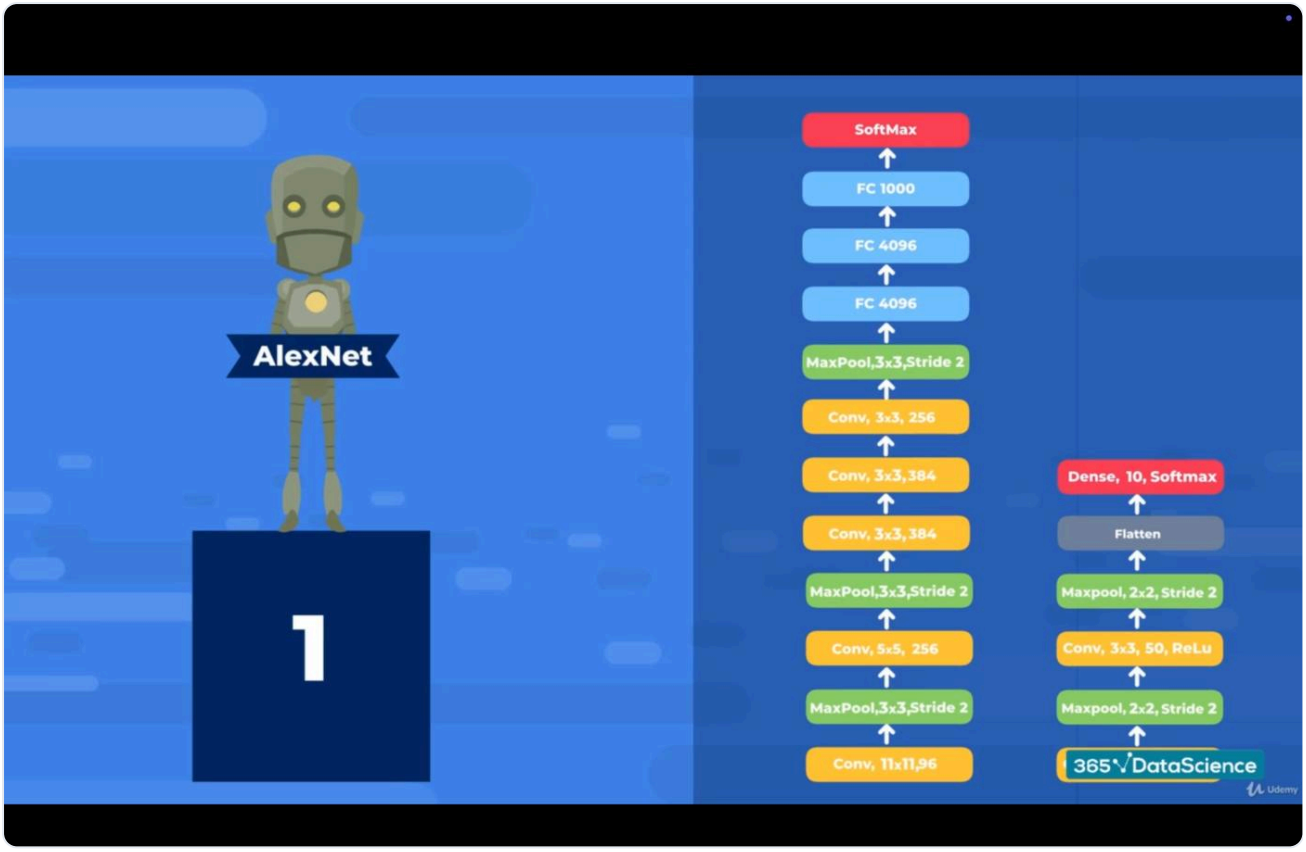


FIGURE 11.5

This is similar to our architecture for the amnesty example. Then there is a section with three convolutional layers connected directly, one after another, followed by the final max pool. After this, two fully connected layers are placed with a final softmax classifier at the end. Overall, nothing too crazy happening here, just a solid CNN configuration. Of course, that was the very first large scale CNN to have good practical results.

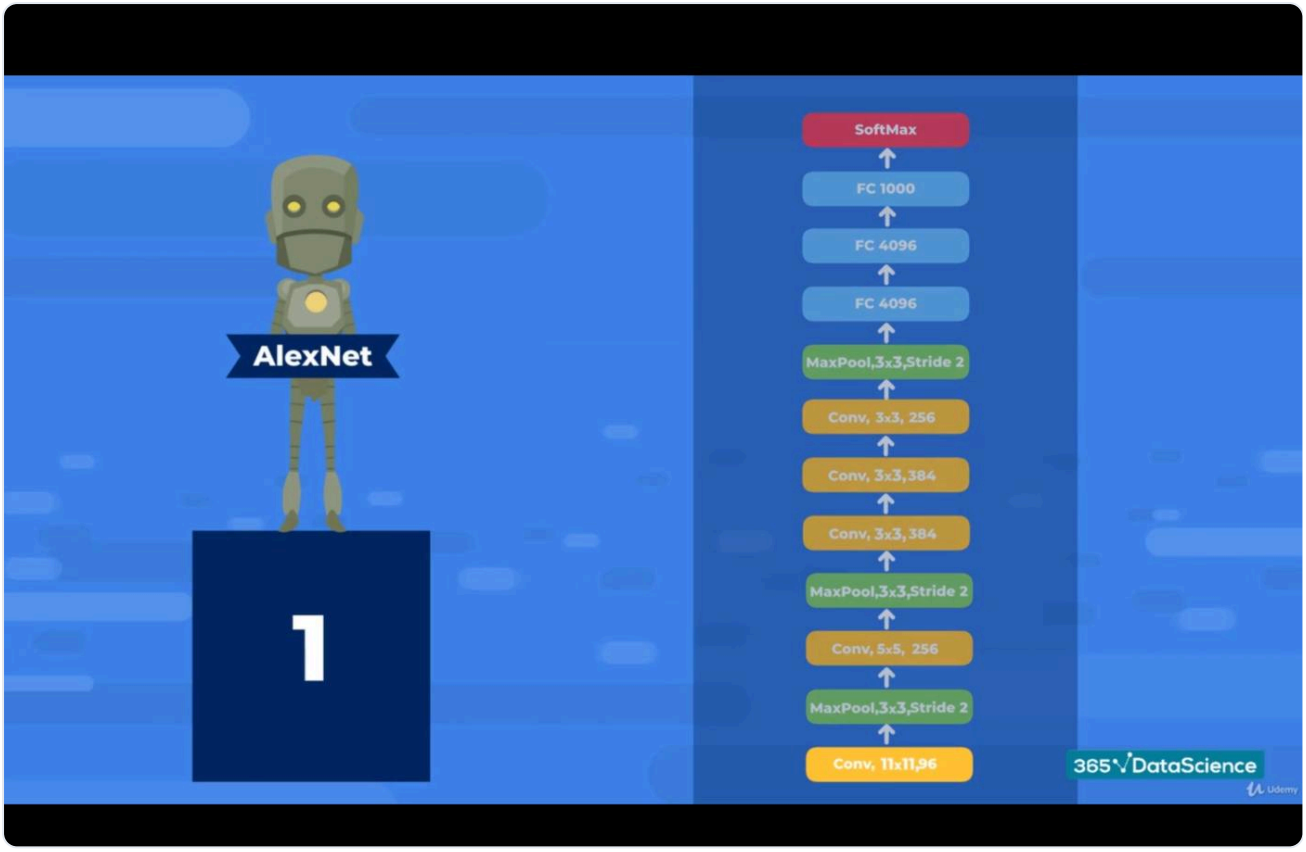


FIGURE 11.6

At present, you can easily build the AlexNet yourself and train it on your own machine. All right. Now let's take a look at the parameters of the different layers. The first convolutional layer consists of 96 kernels of size, eleven by eleven with stride for the second convolution has 256 five by five filters, while the last three all have three by three filters. All the max pool layers have three by three filters at stride to a departure from the usual two by two with stride to note how the filter size decreases as we go deeper into the network from eleven to five to three.

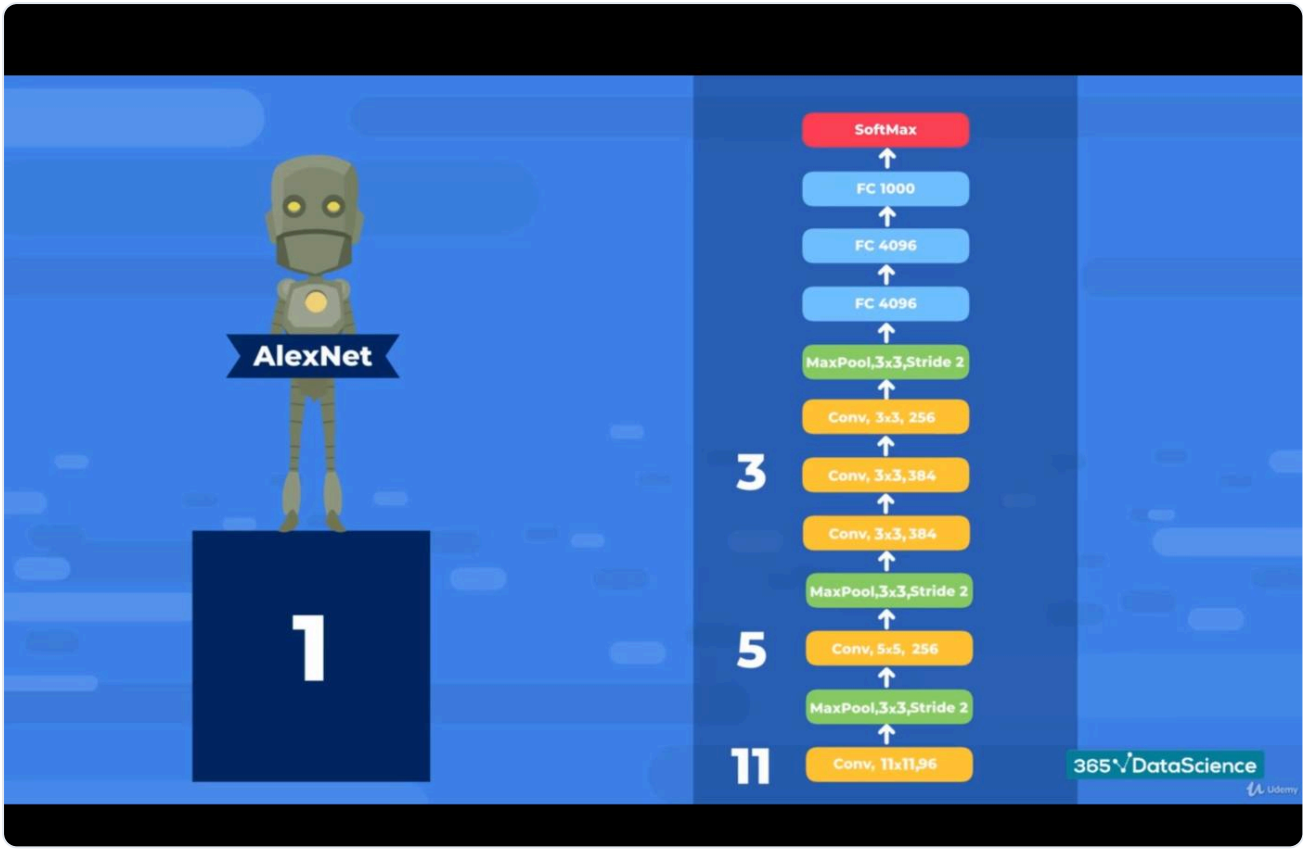


FIGURE 11.7

Finally, the fully connected layers have four thousand ninety six neurons, with the last one having a thousand as it should return the scores for the one thousand classes. OK, so what made AlexNet successful? Besides its architecture, this network was the first one to use Relu as an activation function. All convolutional layers had RELU activation. In addition, the image data wasn't just fed into the network, but heavy use of data augmentation was employed.

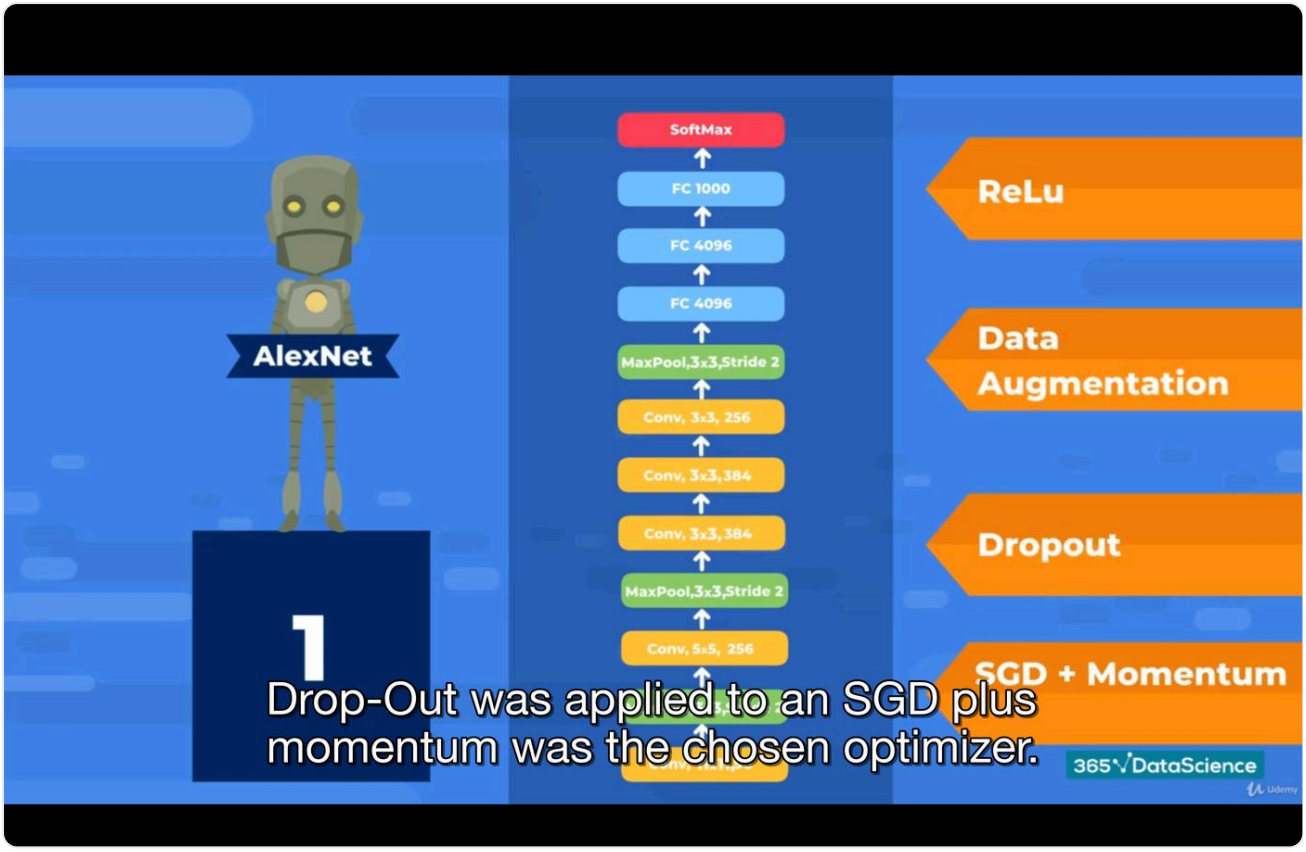


FIGURE 11.8

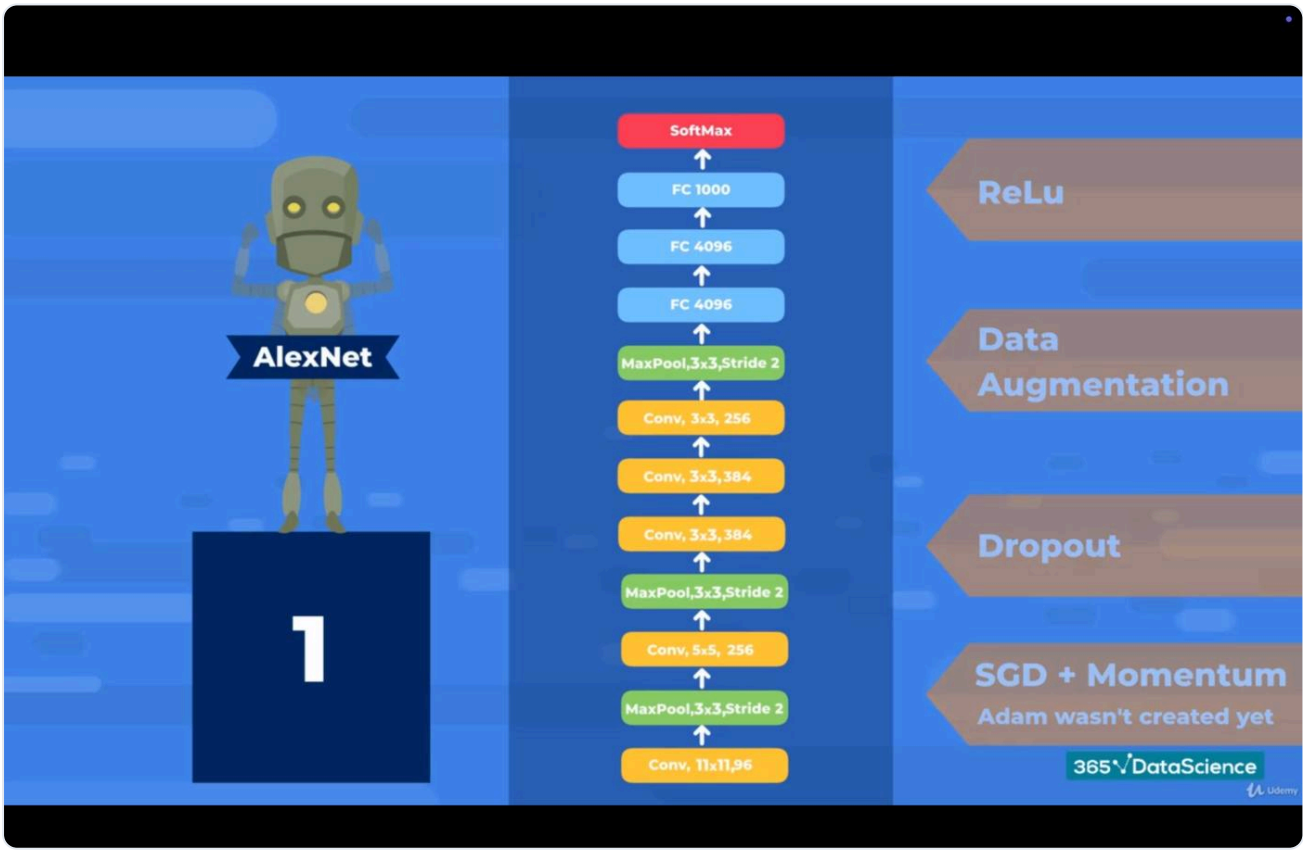


FIGURE 11.9

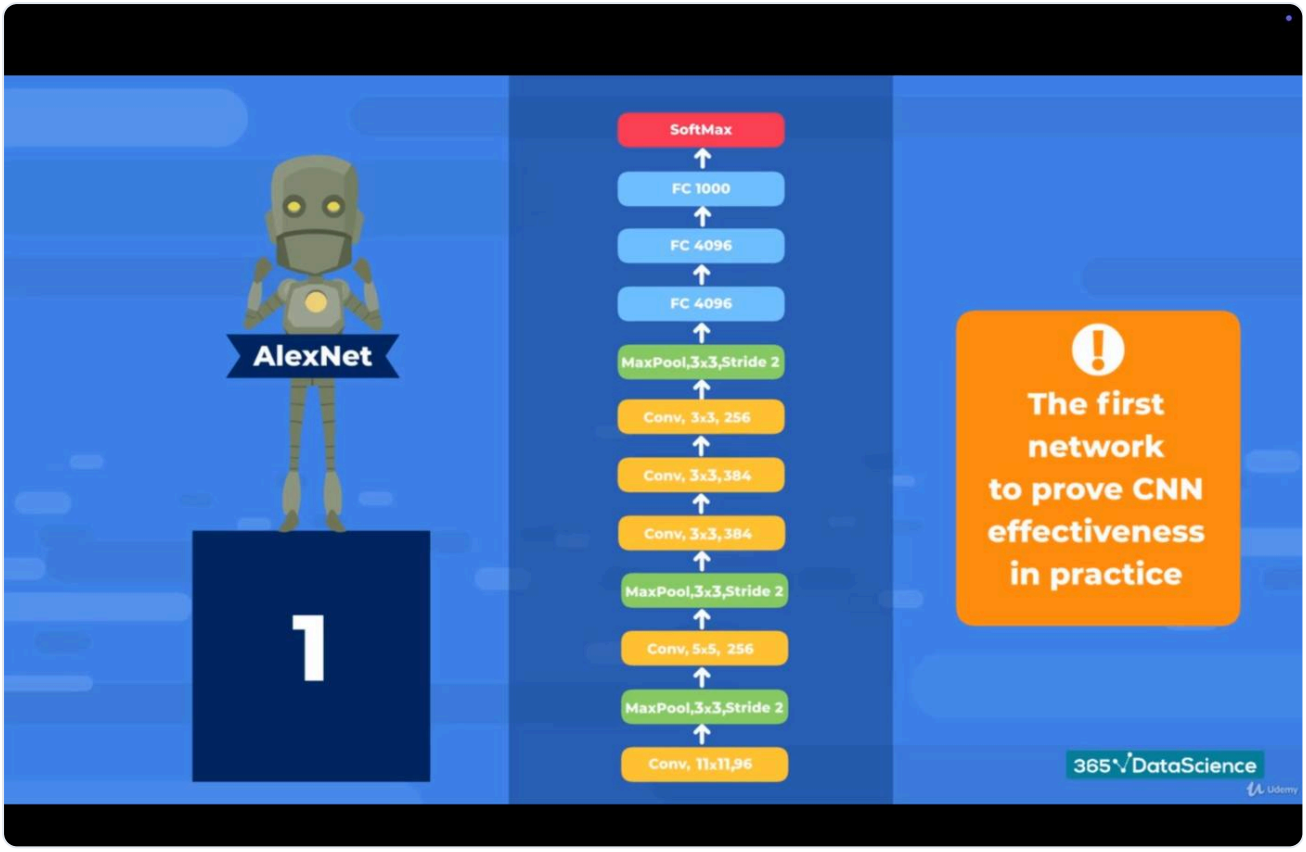


FIGURE 11.10

Drop-Out was applied to an SGD plus momentum was the chosen optimizer. Remember, that was back then when Adam was still not conceived, while Adam Grad had just been released. AlexNet is not as impressive by today's standards, but it is the first network to prove CNN effectiveness in practice. With that in mind, I challenge you to replicate the AlexNet yourself for homework. Next stop VTG. Thanks for watching.

LECTURE 50

Connected Layer

Continuing on with the different architectures today, we will talk about VGG Net following the success of AlexNet in 2012 in ILSVRC 2013.

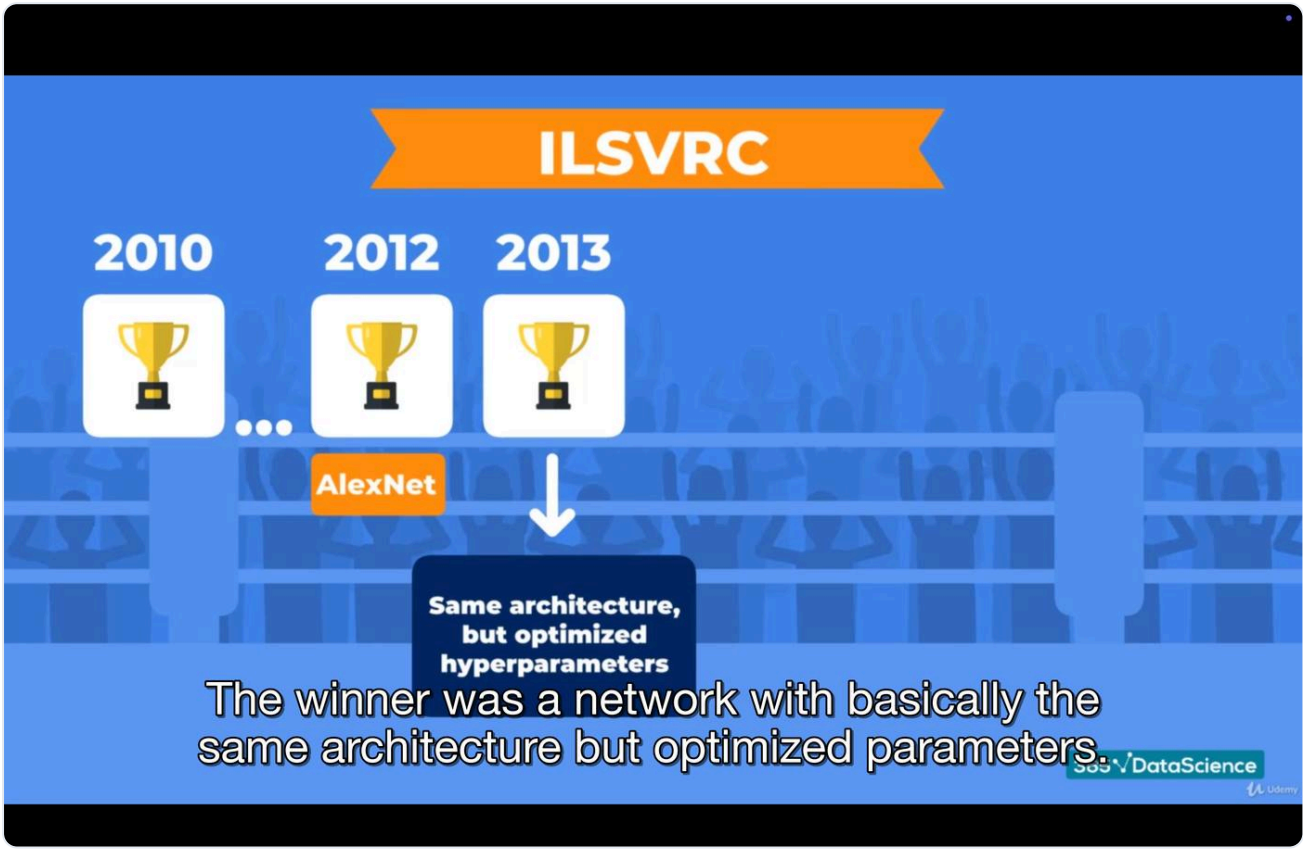


FIGURE 12.1

The winner was a network with basically the same architecture but optimized parameters.



FIGURE 12.2

Interestingly, the competition held in 2014 made way for the next big leap in science. So what happened? There were two main competitors in that year.

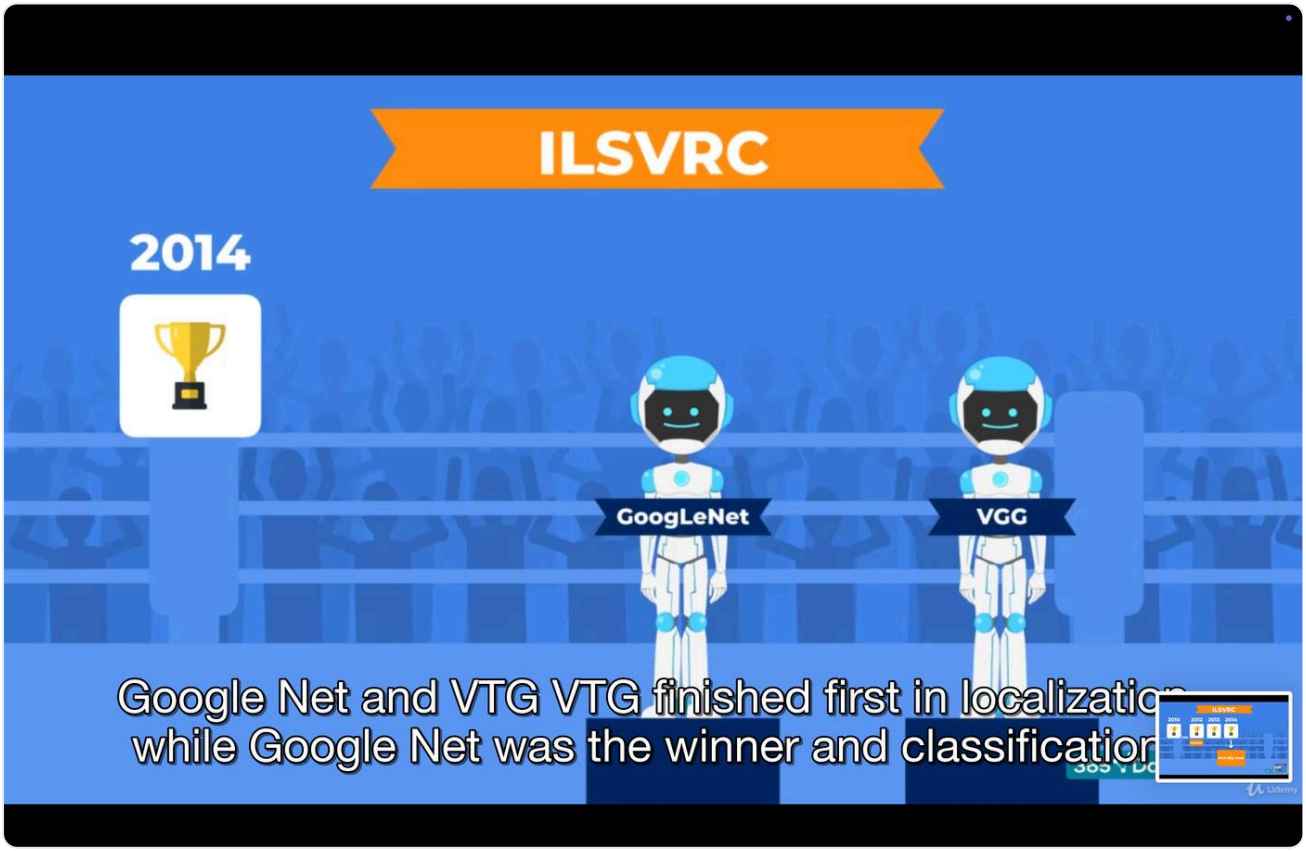


FIGURE 12.3

GoogLeNet and VTG VTG finished first in localization while GoogLeNet was the winner and classification.

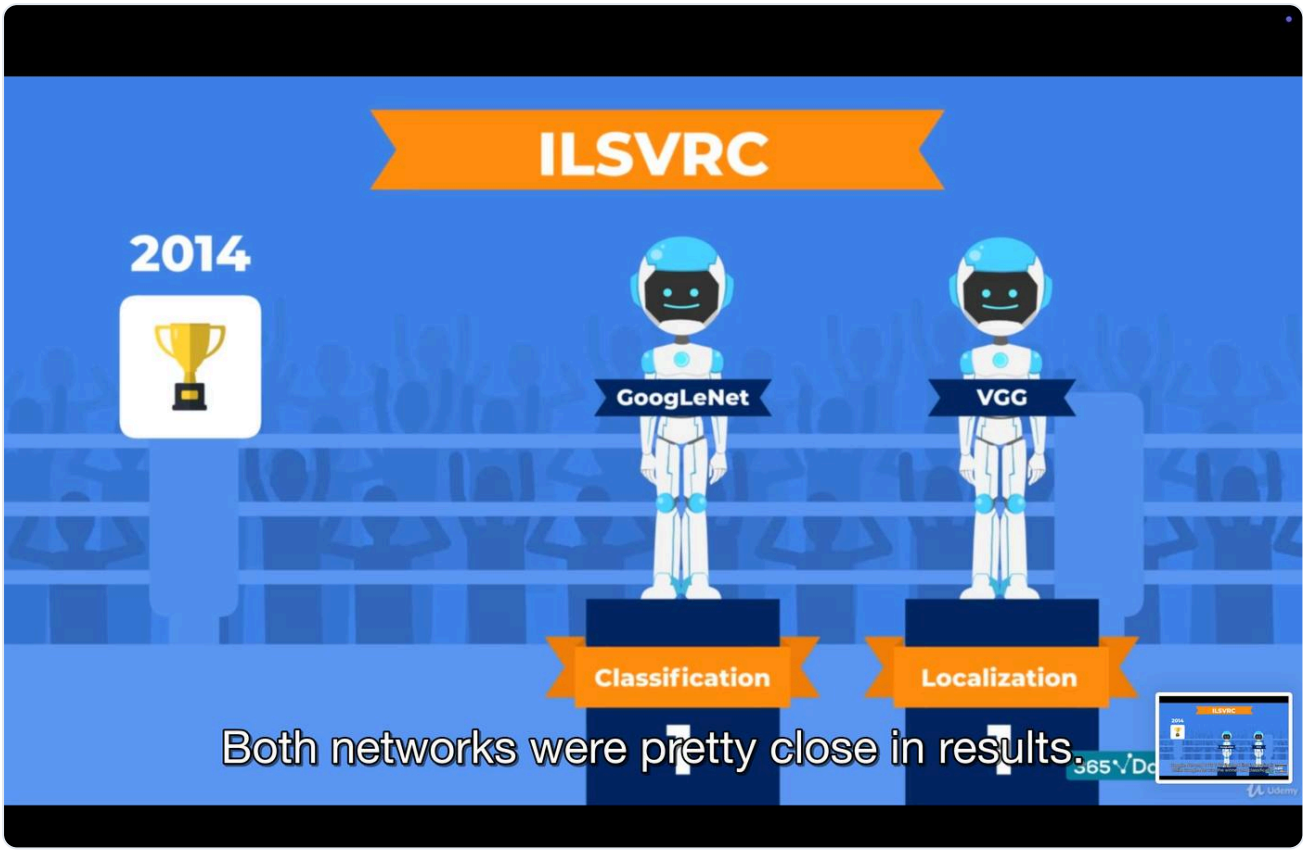


FIGURE 12.4

Both networks were pretty close in results.

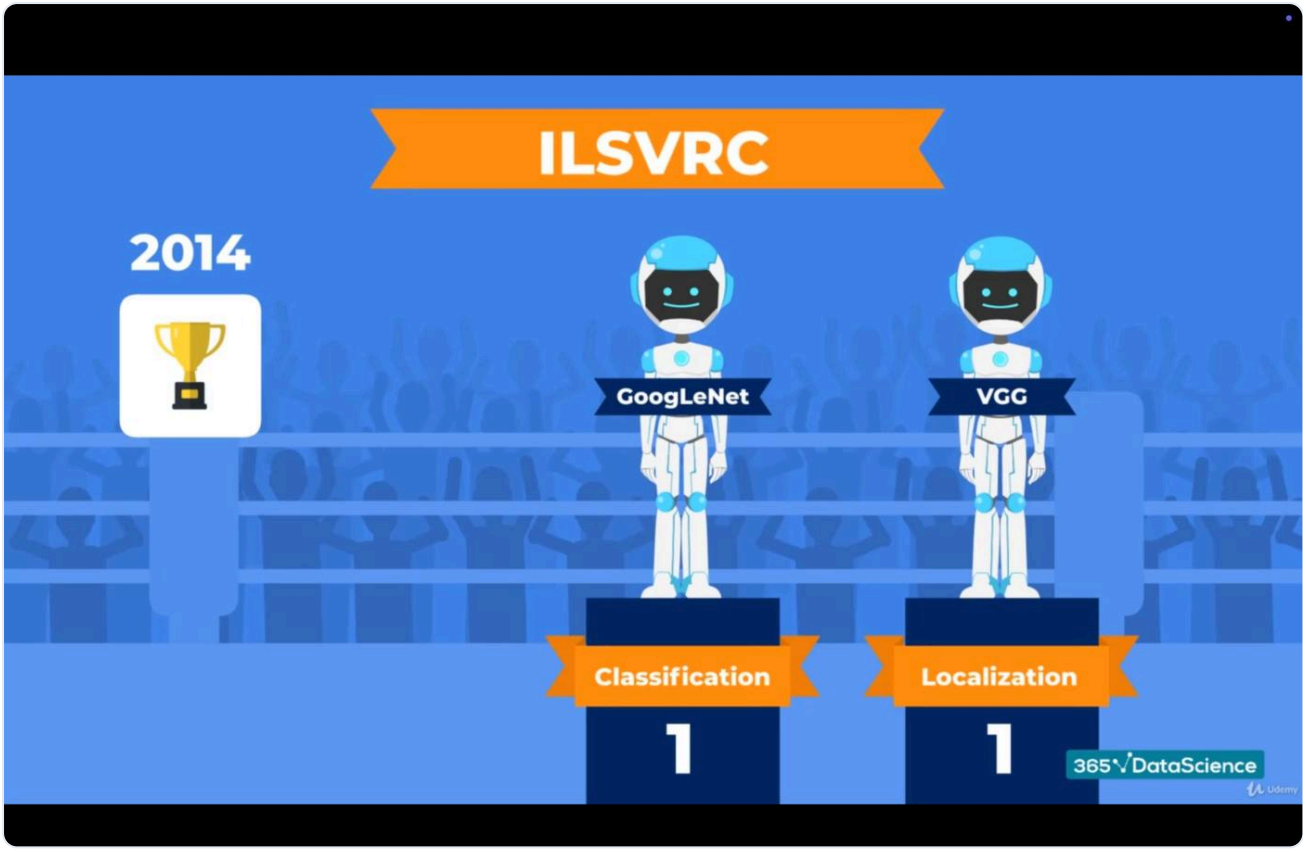


FIGURE 12.5

We will examine them now, starting with the revolution in 2014 had much to do with a number of layers.

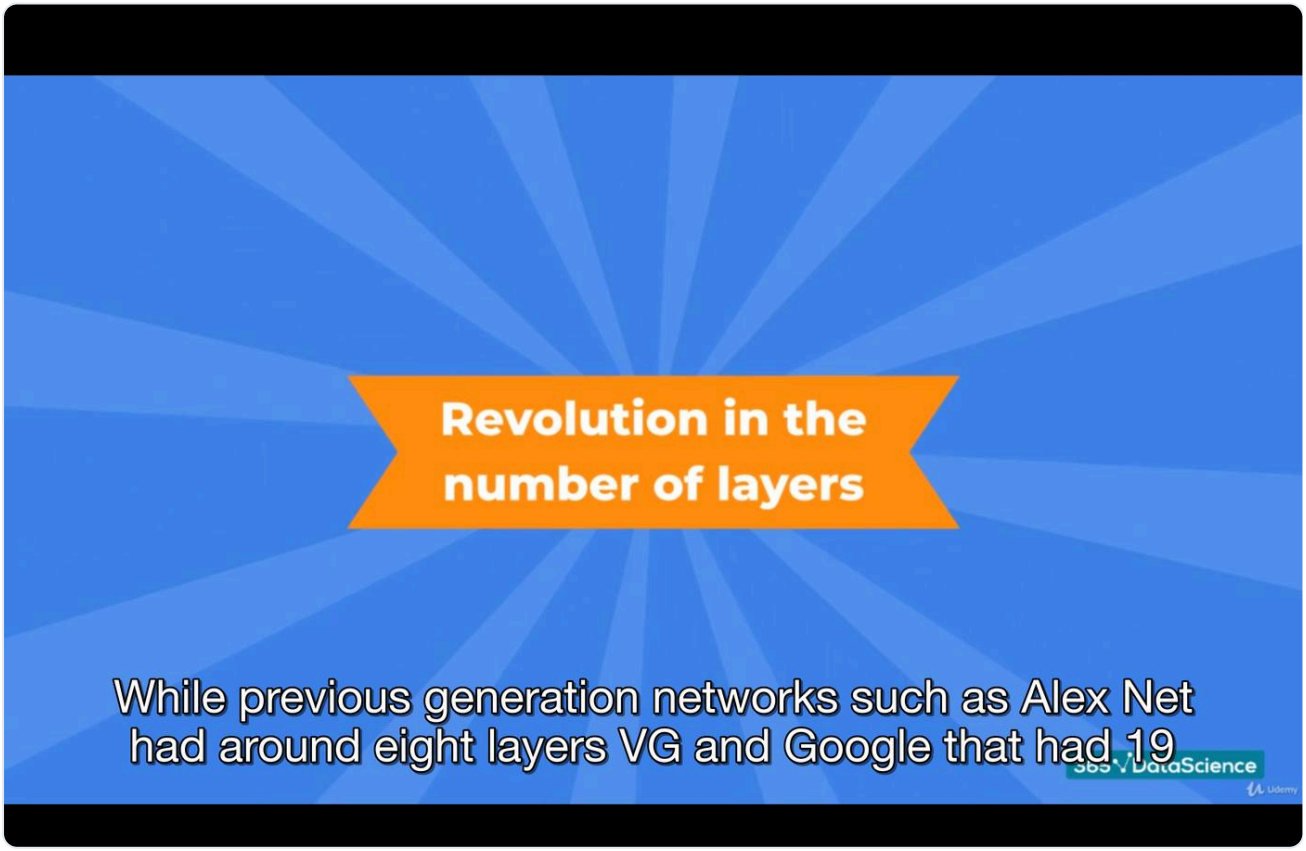


FIGURE 12.6

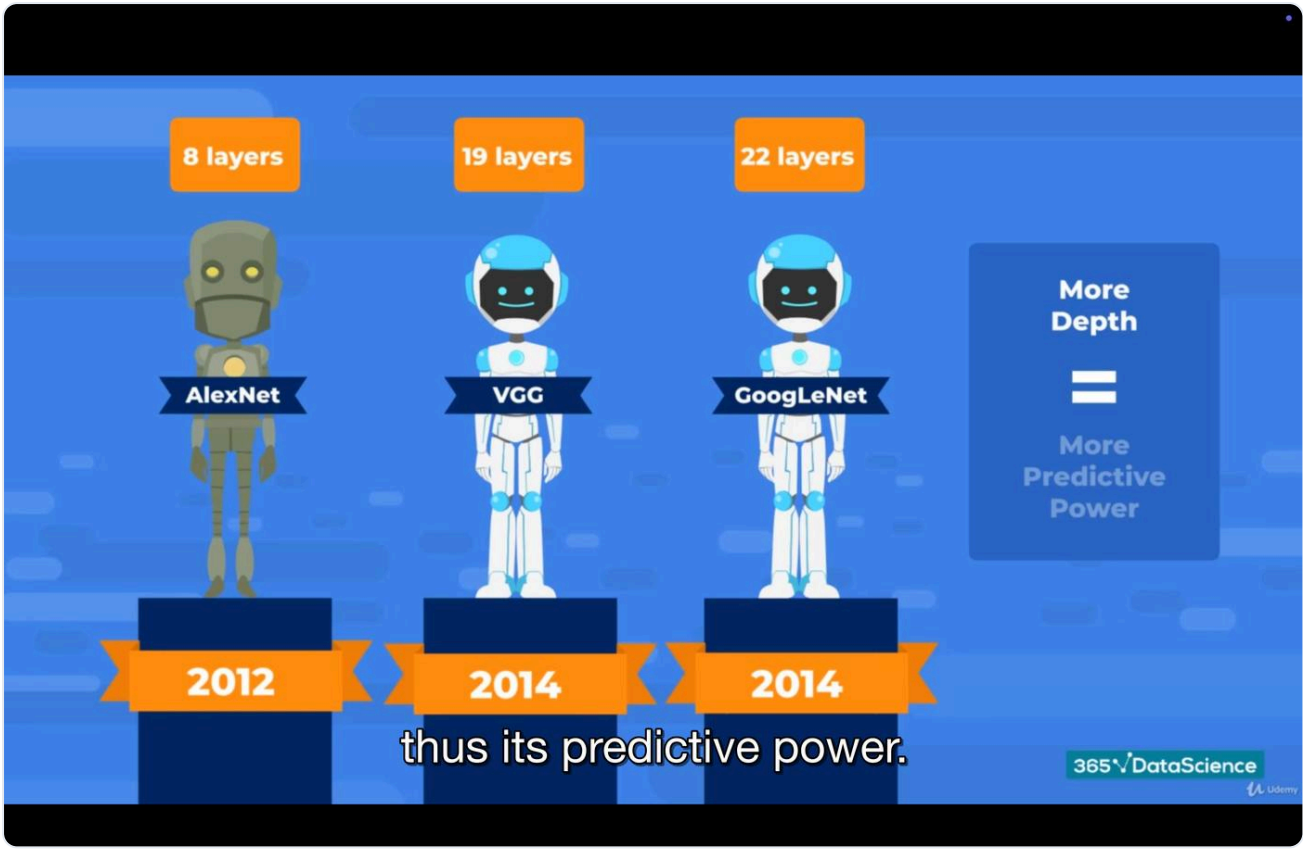


FIGURE 12.7

While previous generation networks such as AlexNet had around eight layers VG and Google that had 19 and 22 respectively, the additional depth of the network increases the non-linearity of our model and thus its predictive power. So let's see how they managed to achieve that. Here is the graph for Vogue.

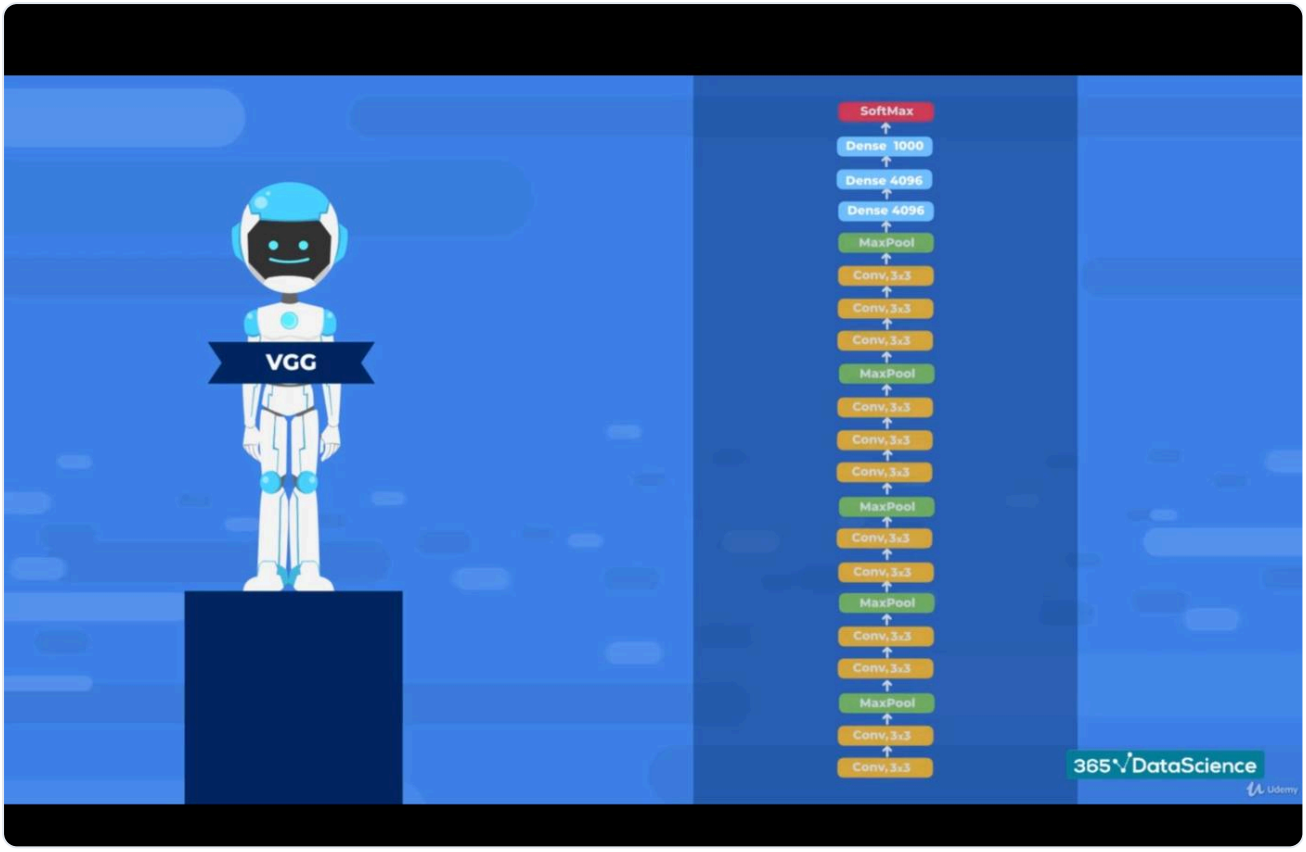


FIGURE 12.8

As expected, there are a lot of convolutional layers stacked together with the occasional max pool. At the end there are the usual three dense layers.

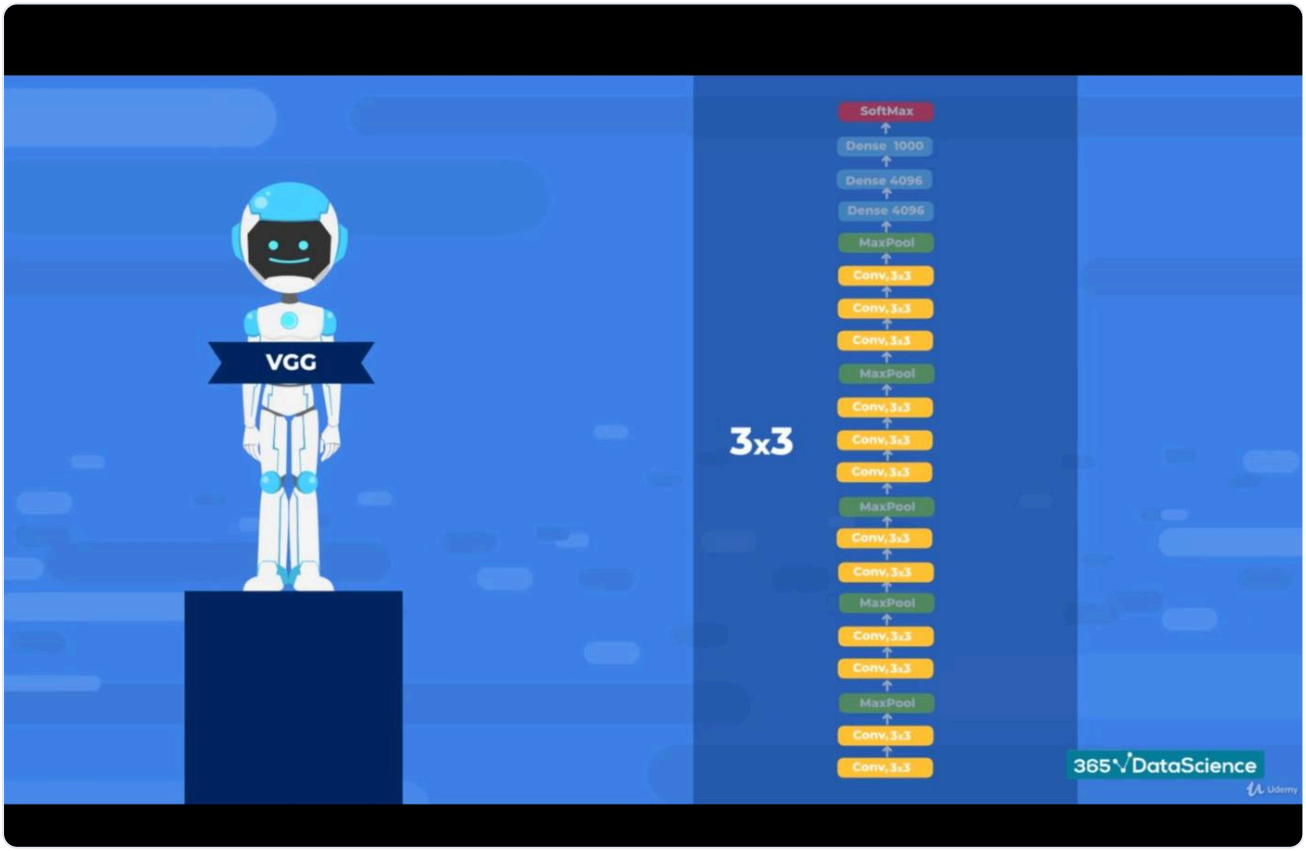


FIGURE 12.9

So what is it that allows for so many layers? Well, the more observant among you may have noticed that all the convolutional layers have kernels of size three by three, not 11 by 11, seven by seven or five by five, just three by three. That drastically reduces the amounts of weights in a single layer.



FIGURE 12.10

For example, a single eleven by 11 kernel has one hundred and twenty one weights, while a three by three kernel has only nine weights. So in theory we can stack 12 three by three filters with the same amount of parameters as a single eleven by eleven.

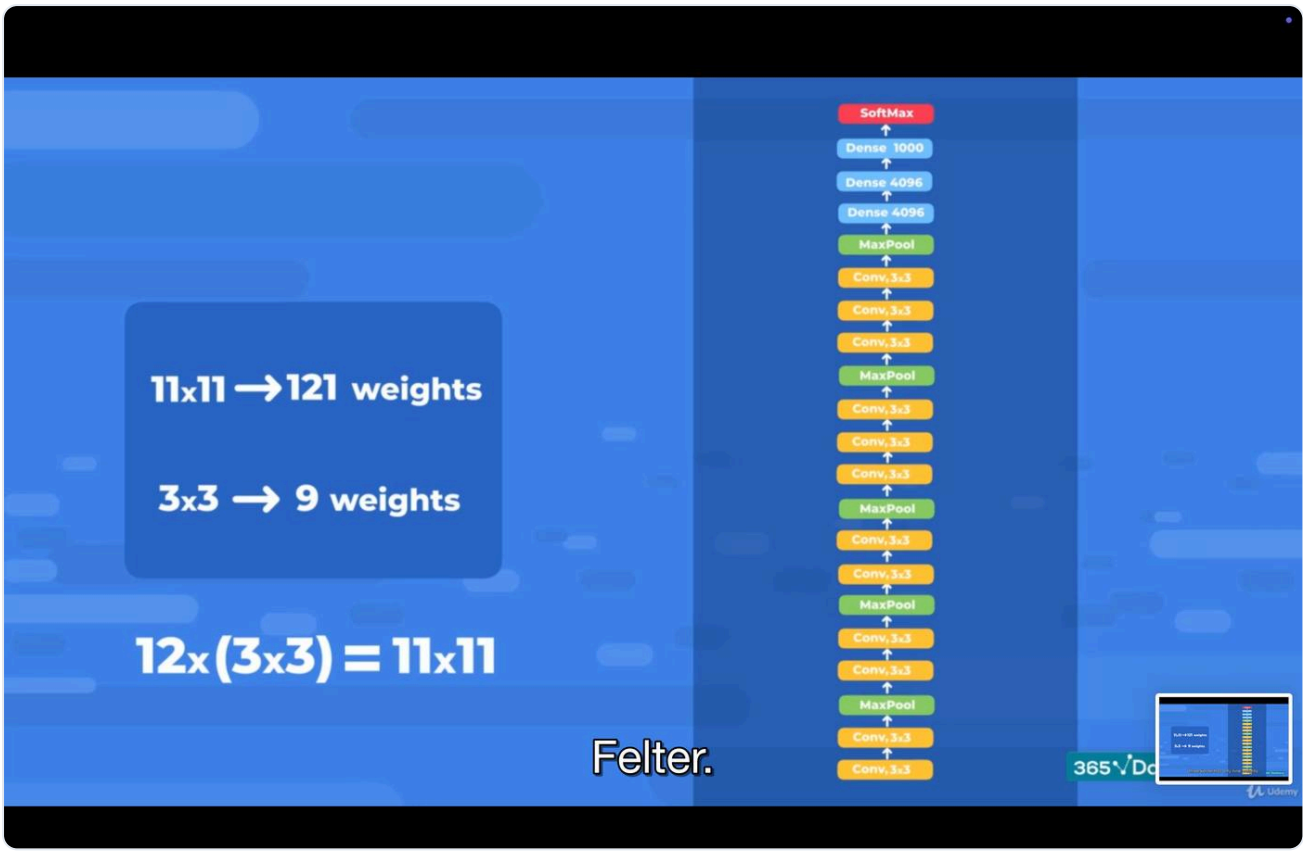


FIGURE 12.11

Filter.

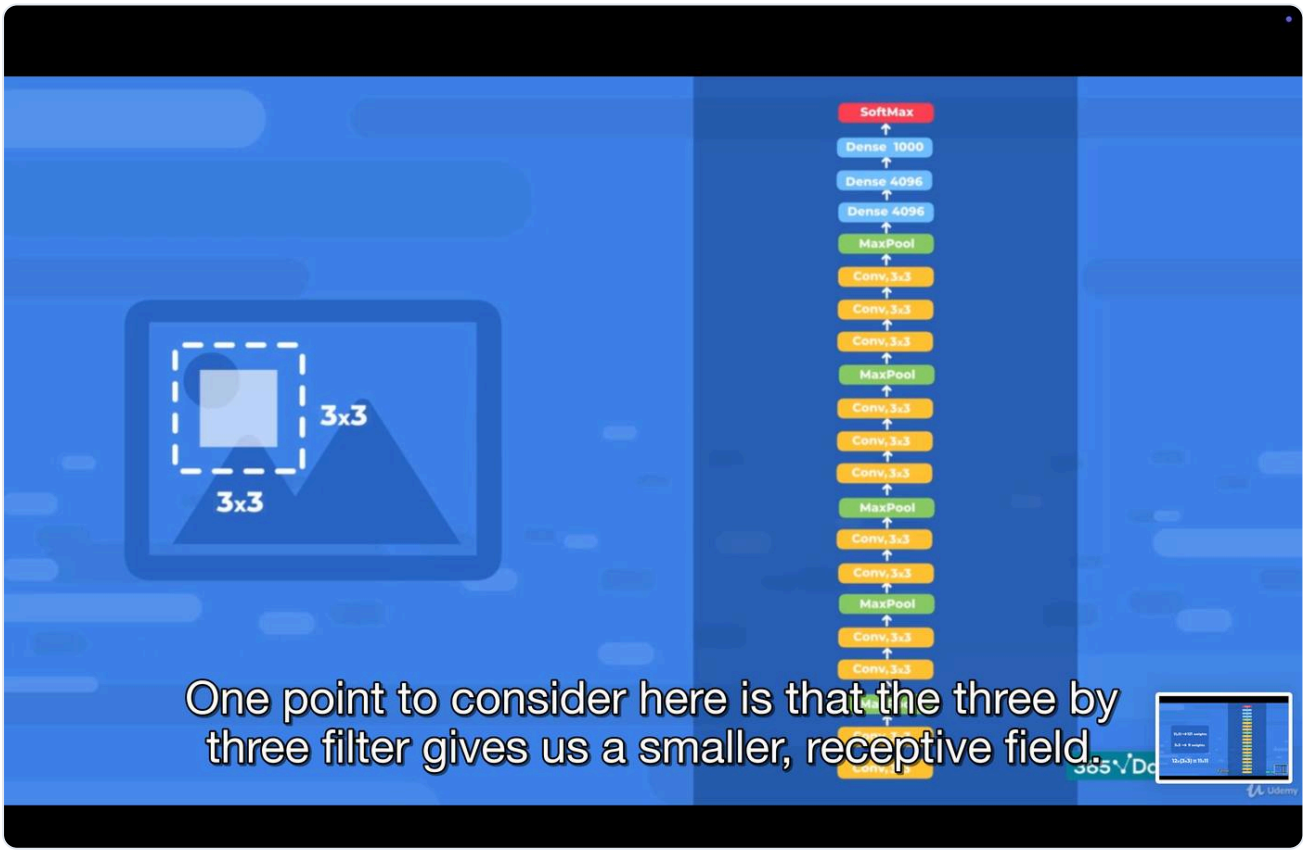


FIGURE 12.12

One point to consider here is that the three by three filter gives us a smaller, receptive field.



FIGURE 12.13

However, that is only true for the first layer. Two such layers actually have a receptive field of five by five, three have seven by seven and so on.

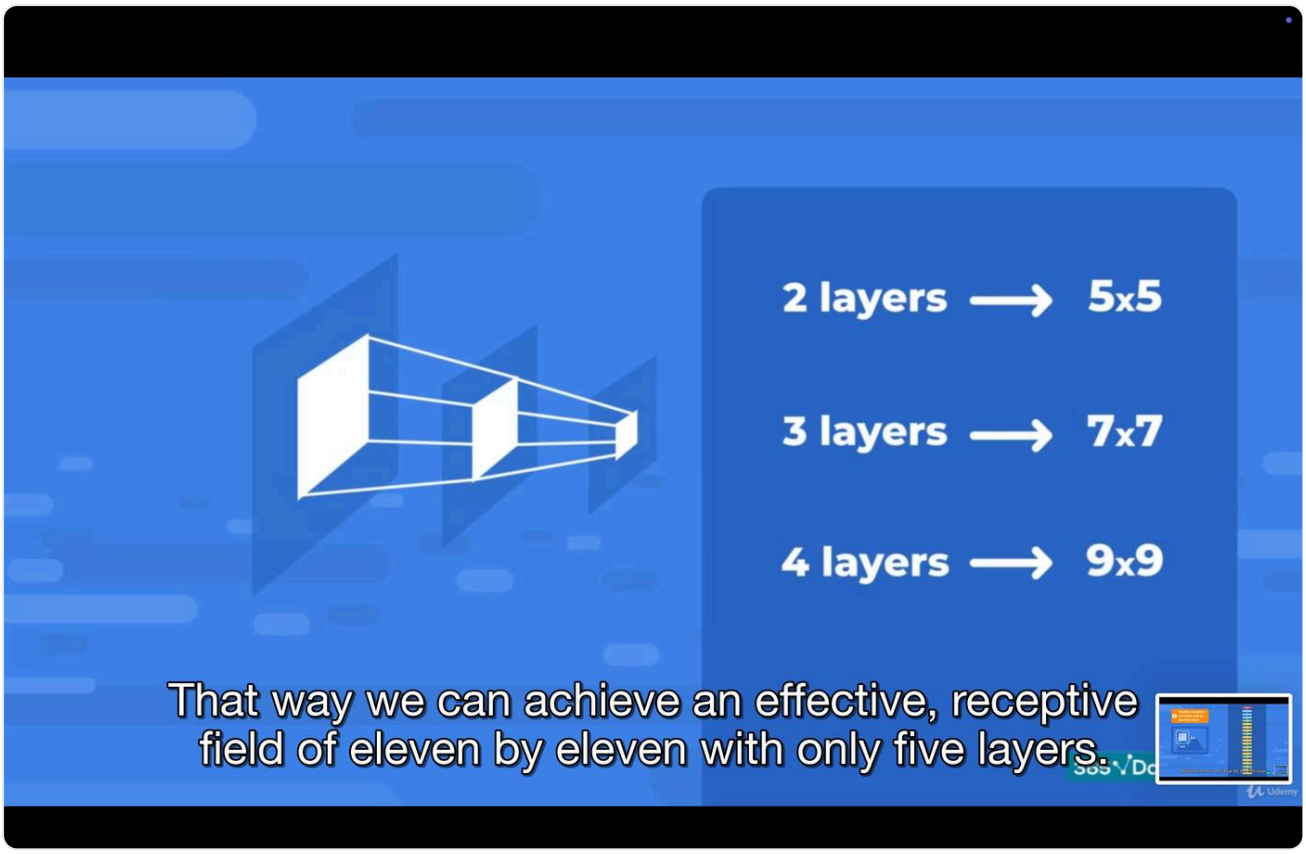


FIGURE 12.14

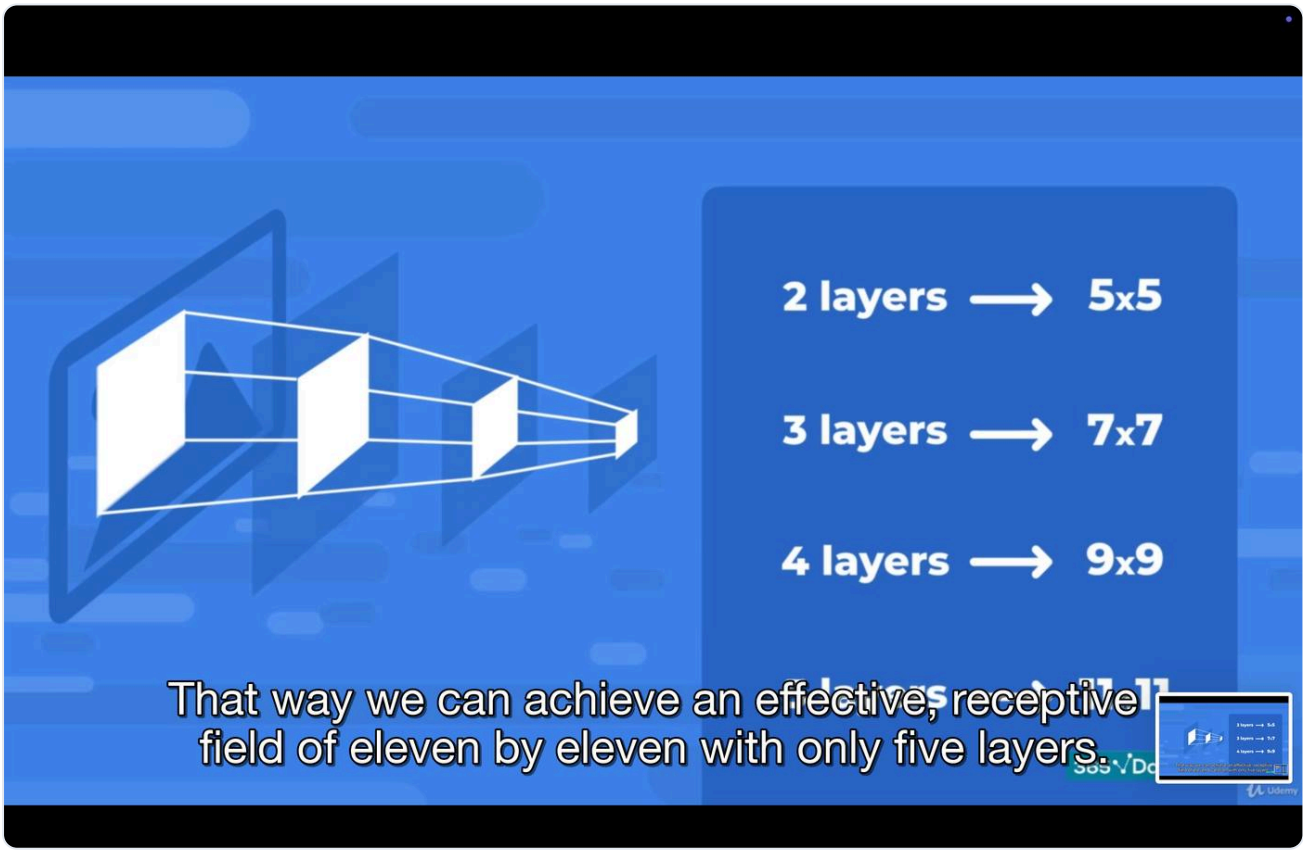


FIGURE 12.15

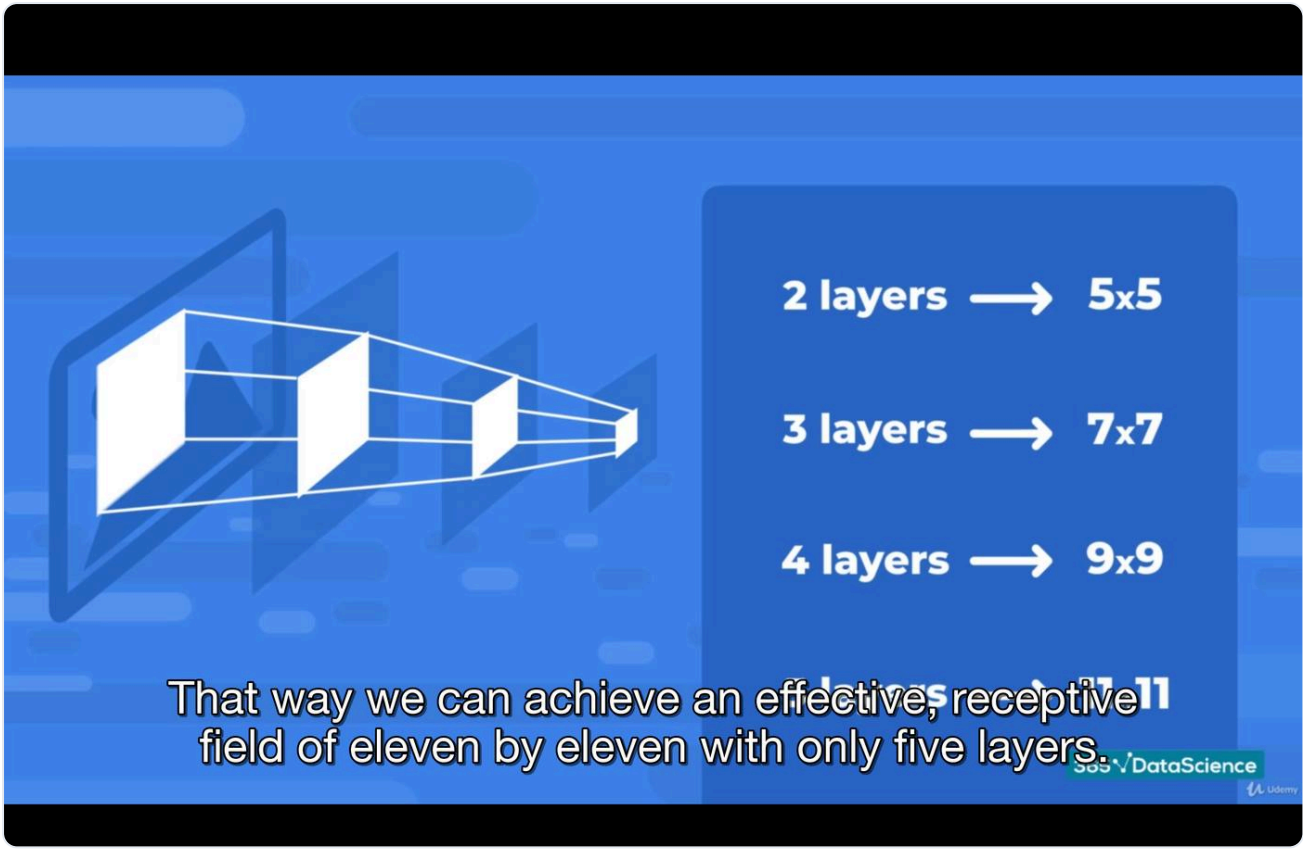


FIGURE 12.16

That way we can achieve an effective, receptive field of eleven by eleven with only five layers. Ultimately, this network could achieve better results with fewer parameters and in a shorter time span.

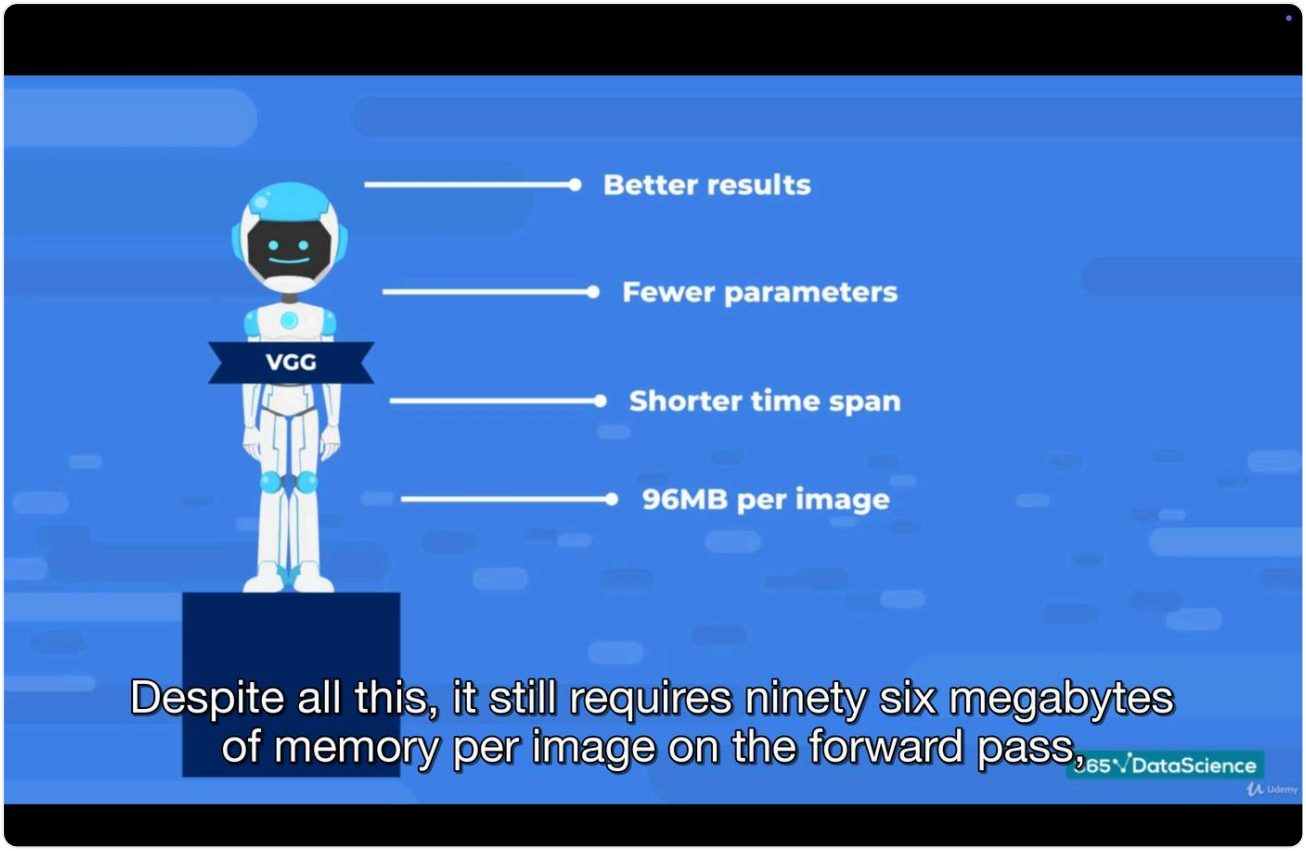


FIGURE 12.17

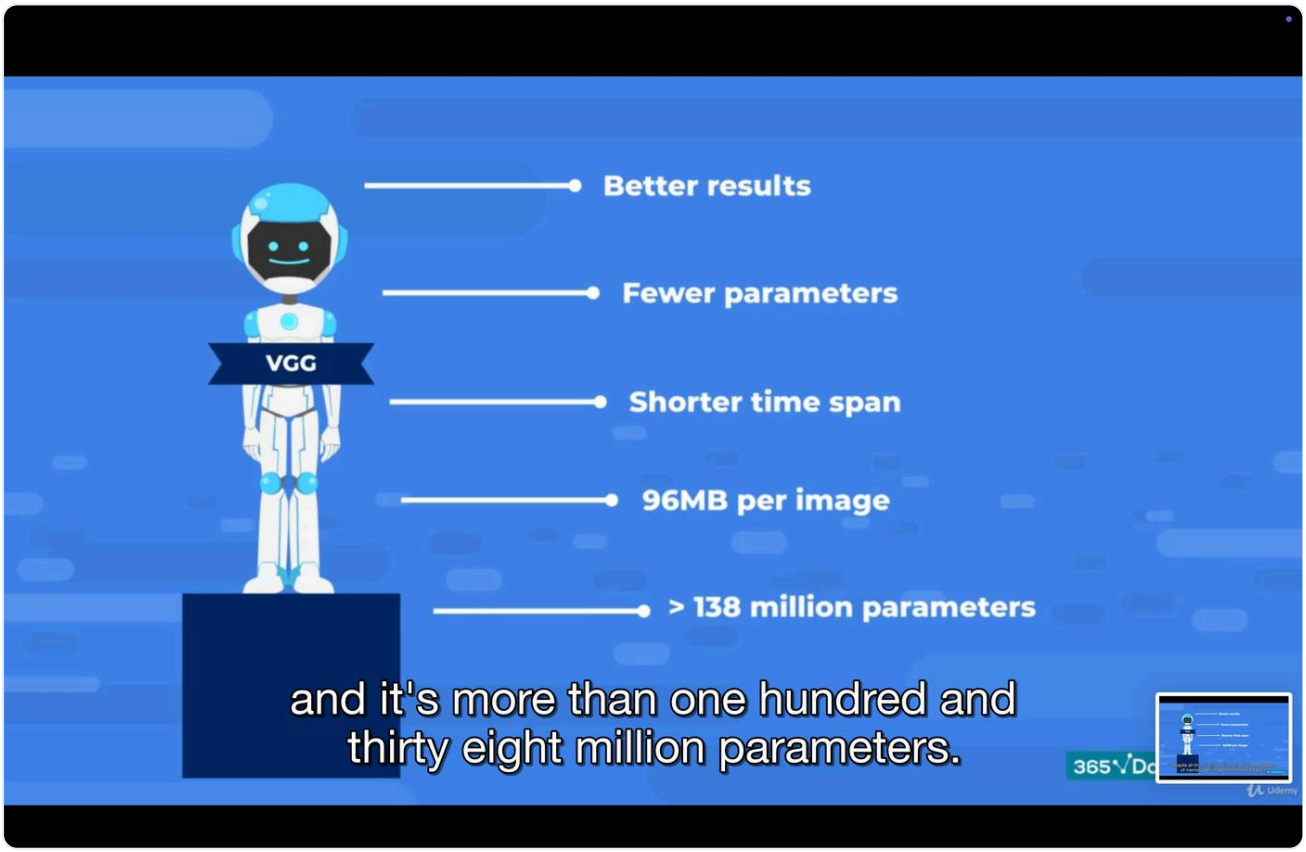


FIGURE 12.18

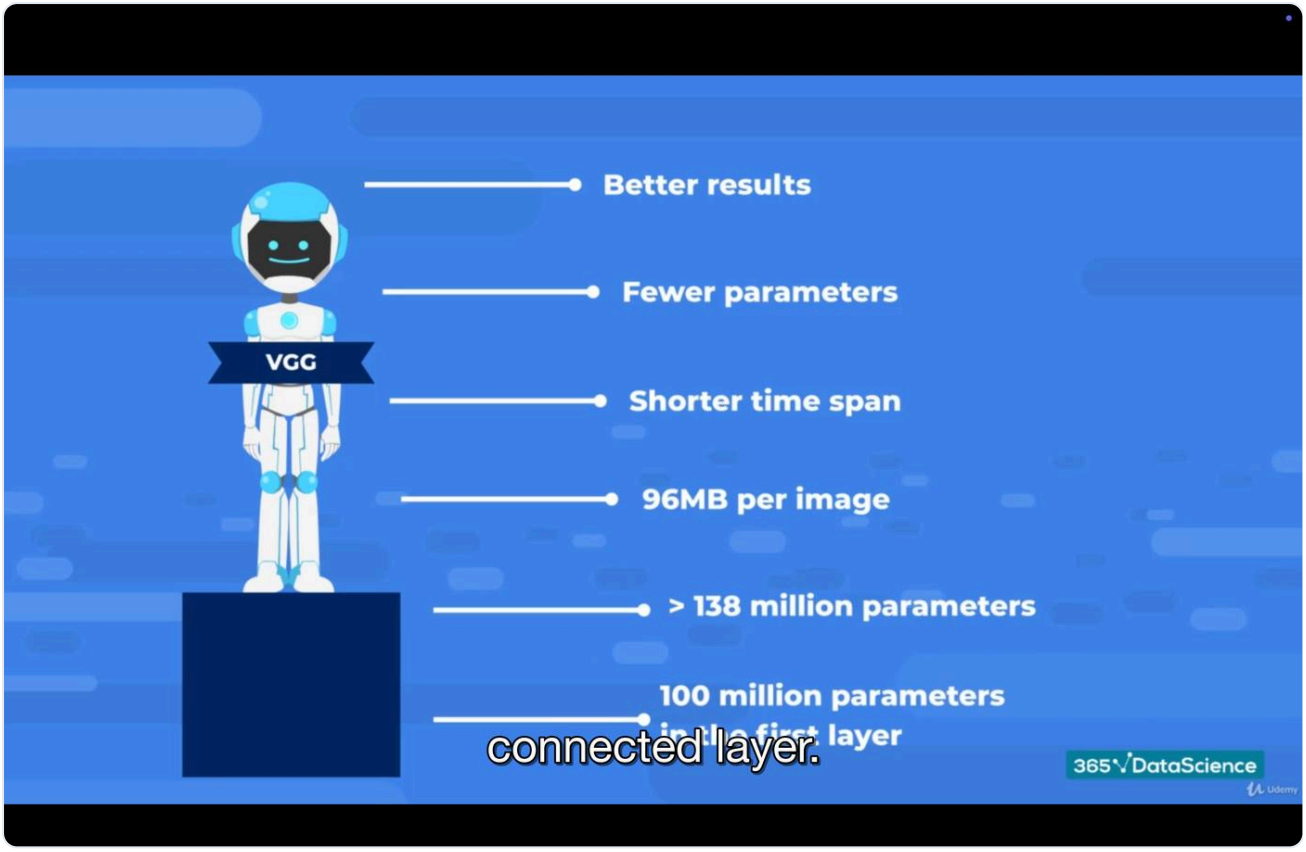


FIGURE 12.19 | Connected Layer



FIGURE 12.20

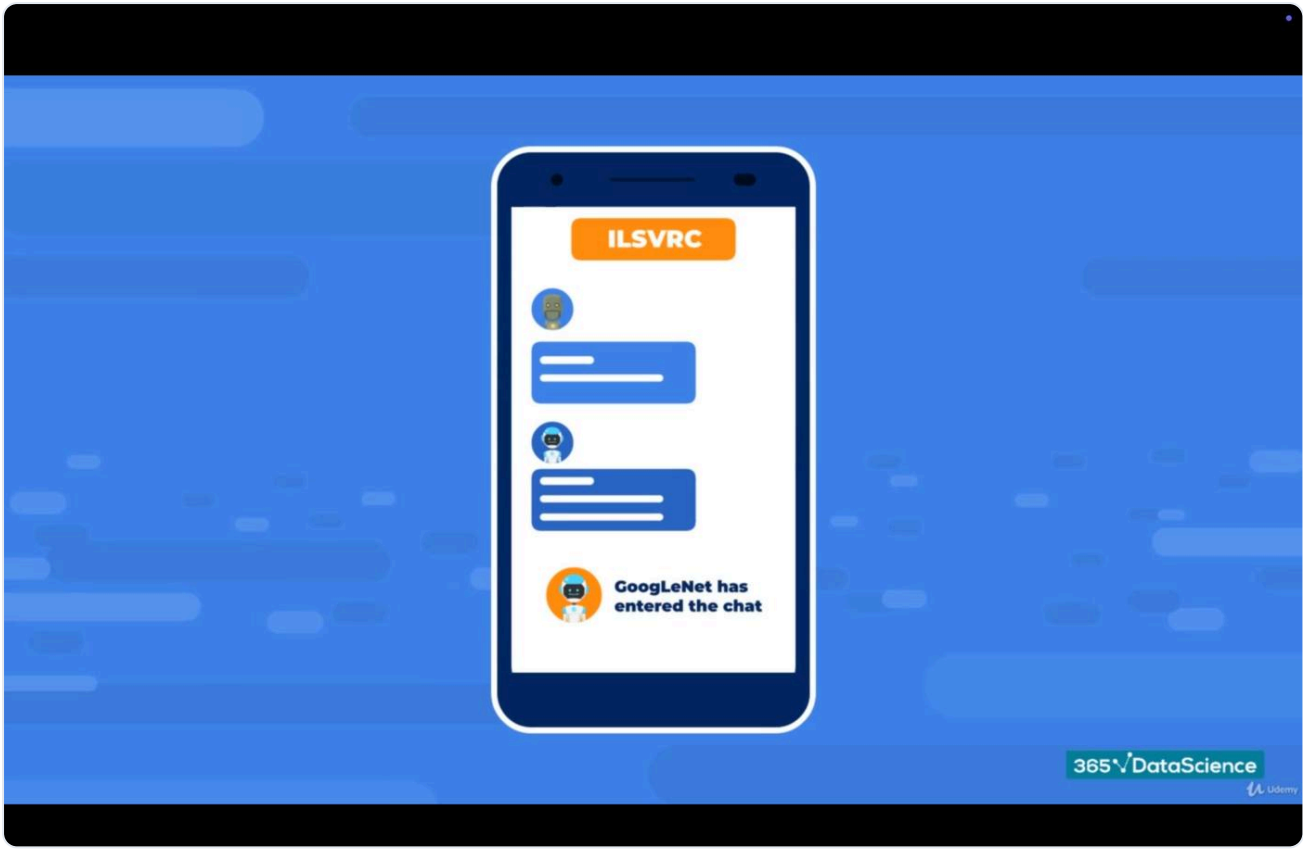


FIGURE 12.21

Despite all this, it still requires ninety six megabytes of memory per image on the forward pass, and it's more than one hundred and thirty eight million parameters. Interestingly, the majority of those parameters, around 100 million, are contained in the first fully connected layer. That is huge. Overall, the VG network is an improvement, but it is still not as computationally efficient as one would assume. And here comes GoogLeNet. More on that in the next lecture. Thanks for watching.

LECTURE 51

Naive Inception

Let's pick up where we left off. In the 2014 edition of the ILSVRC competition, Google had performed the best in the classification task.



FIGURE 13.1

The team behind it achieved this results by having a paradigm shift.

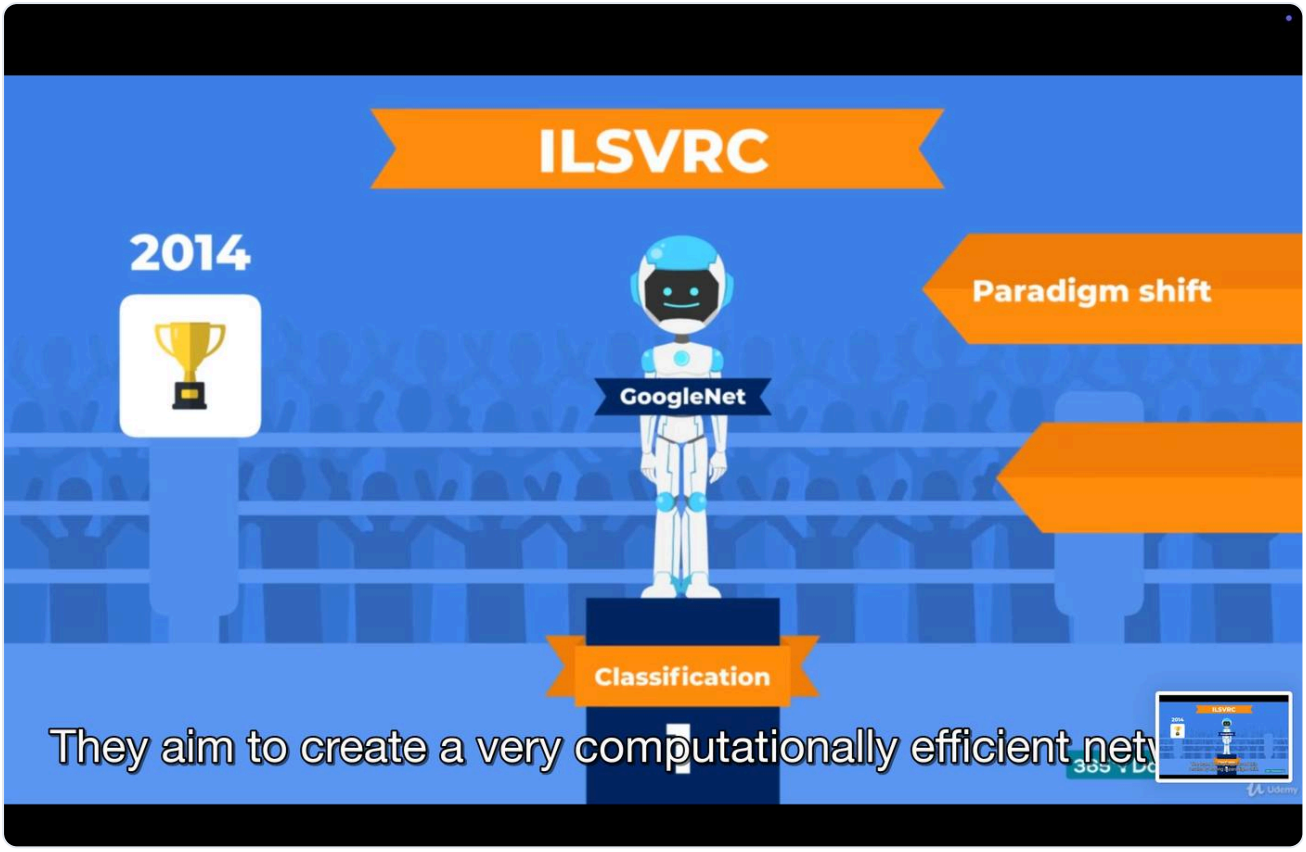


FIGURE 13.2

They aim to create a very computationally efficient network.

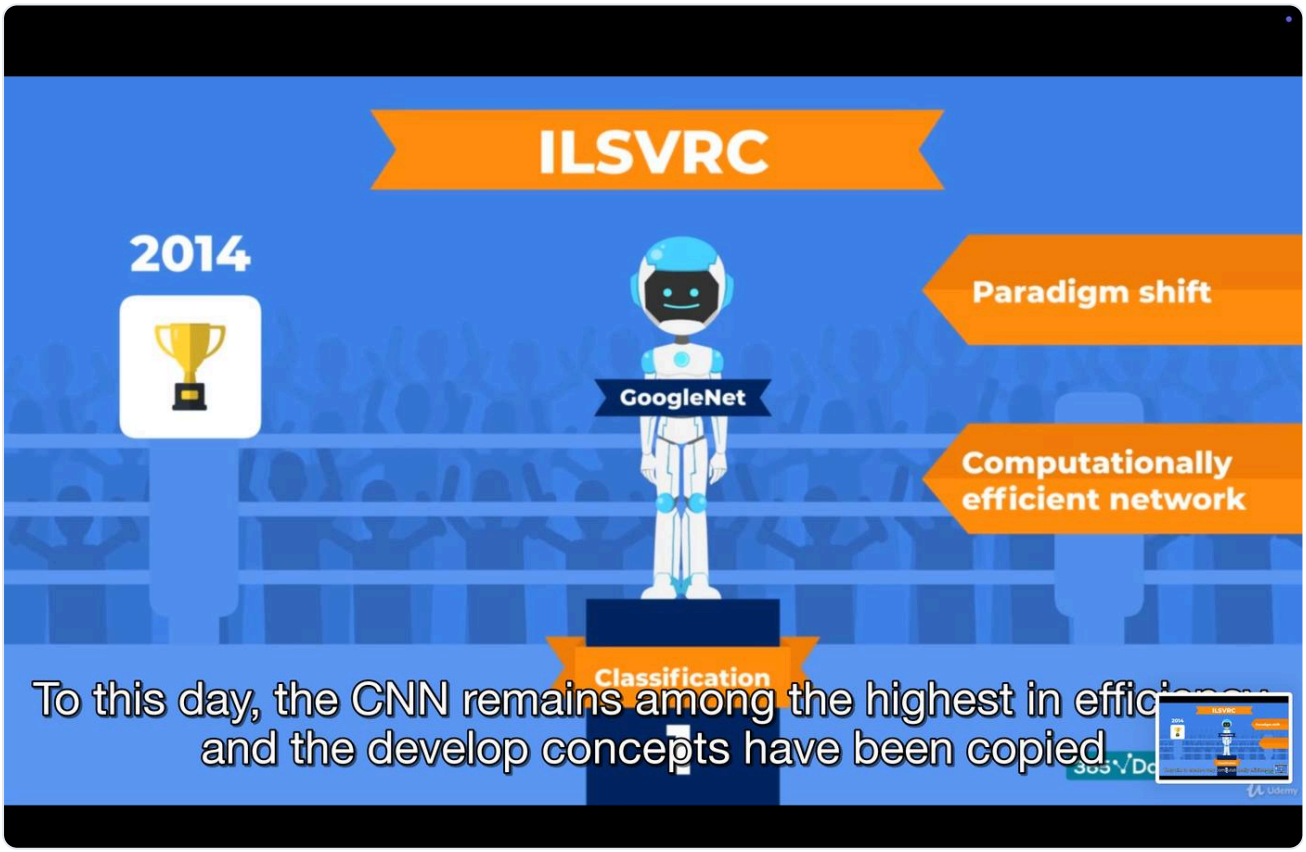


FIGURE 13.3

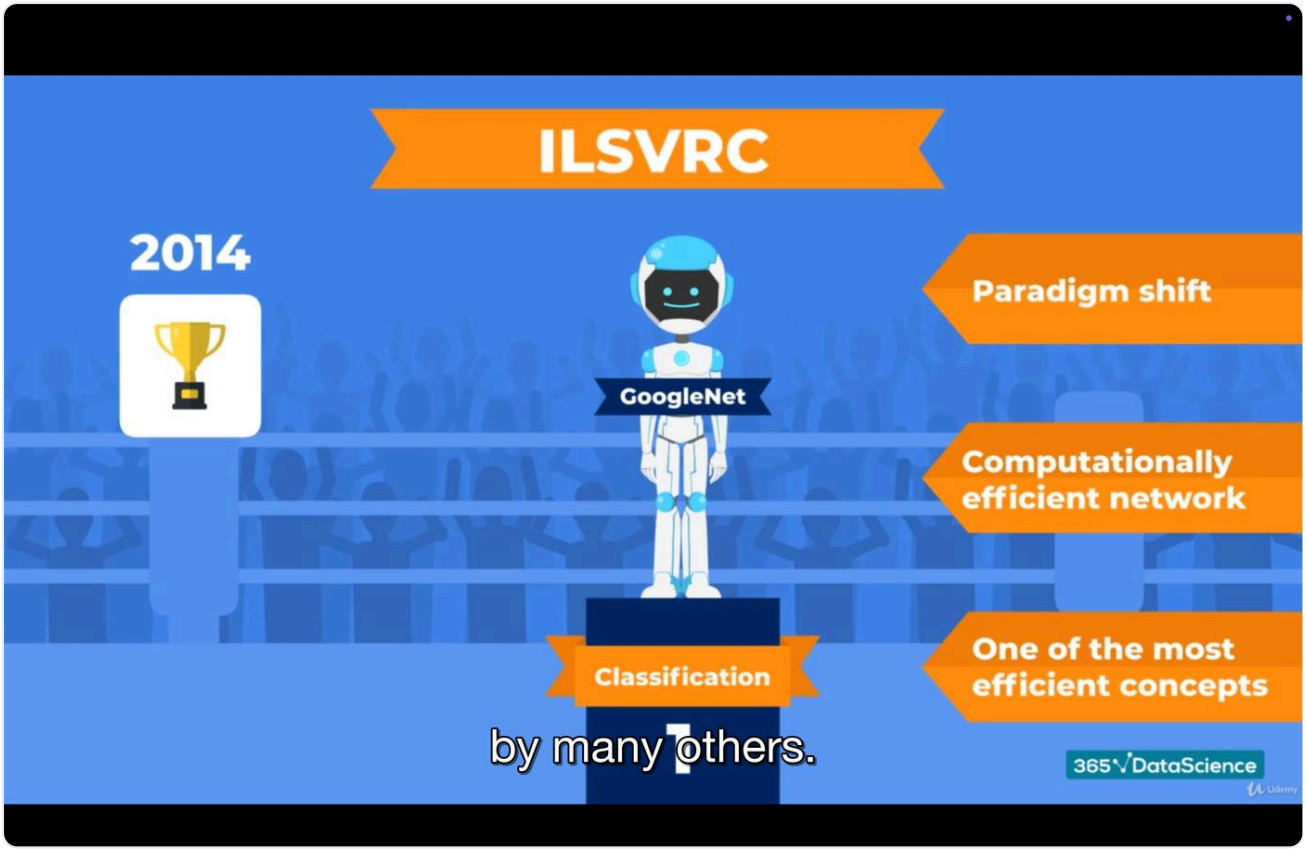


FIGURE 13.4

To this day, the CNN remains among the highest in efficiency, and the develop concepts have been copied by many others. All right. So how did the Google engineers accomplish this? To answer this, we need to study Google. Architecture first.

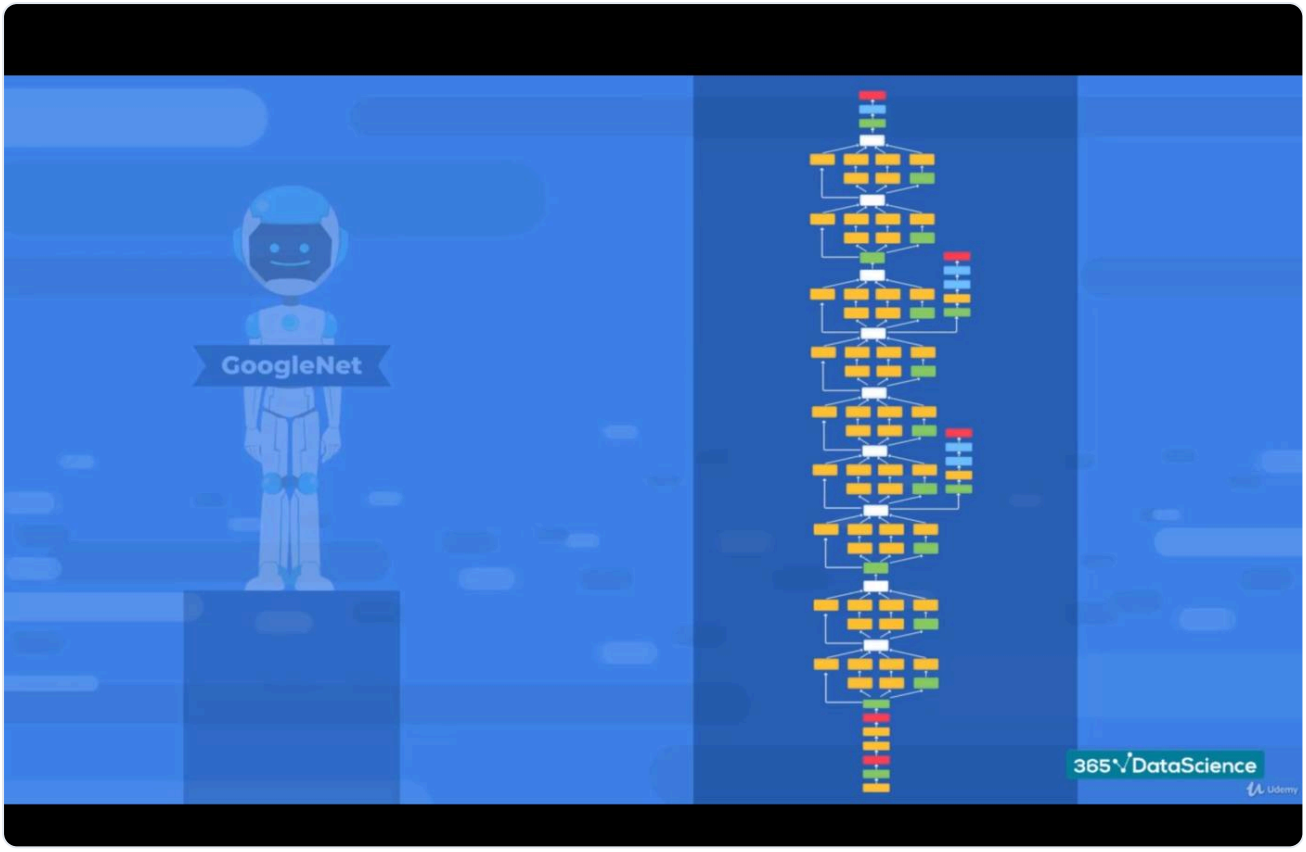


FIGURE 13.5

Wow. Look at all these connections and multiple modules per layer. How can such a convoluted architecture be efficient? Let's try to unravel this beast. This network has 22 layers.

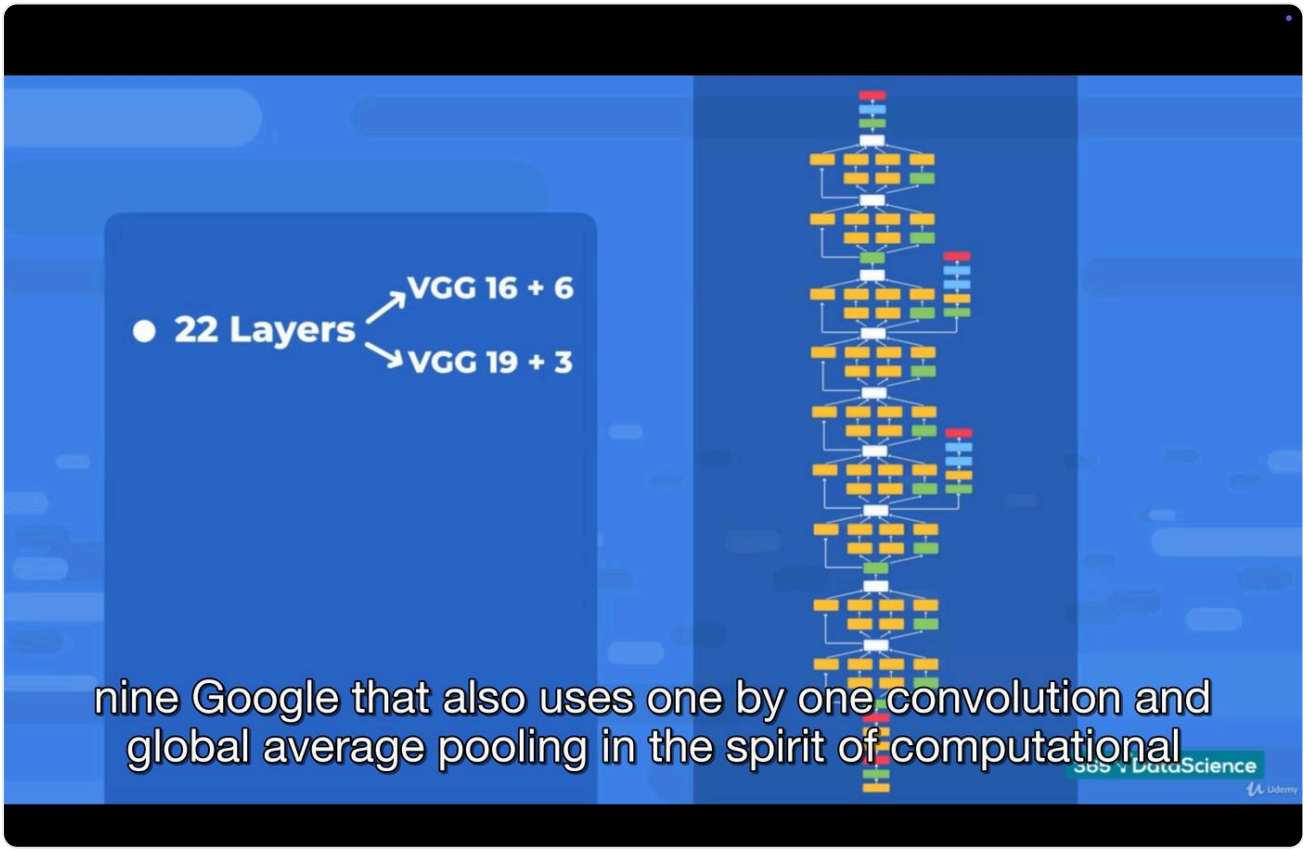


FIGURE 13.6

That's six more than 16 and three more than VTG 19, inspired by a paper called Network in Network or nine Google that also uses one by one convolution and global average pooling in the spirit of computational efficiency.

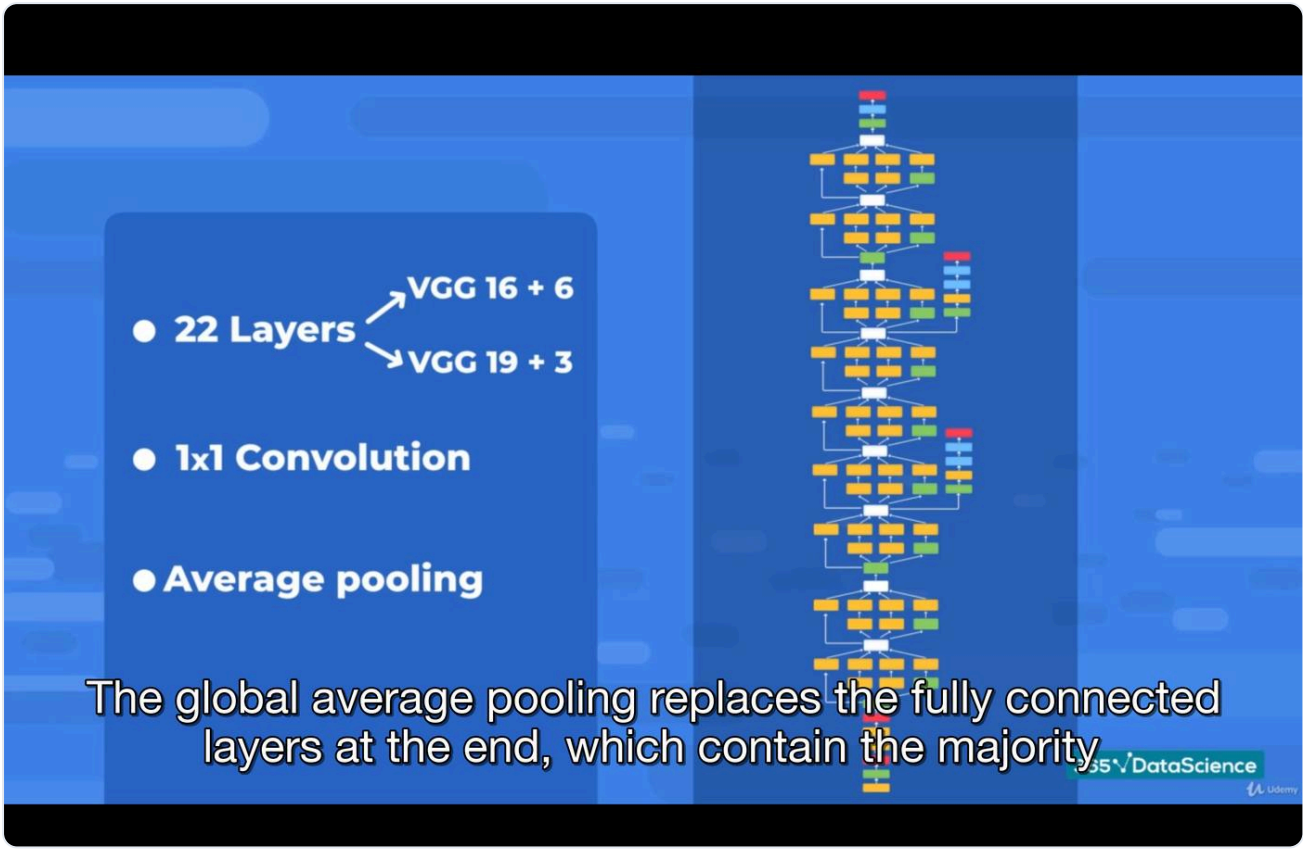


FIGURE 13.7

The global average pooling replaces the fully connected layers at the end, which contain the majority of weights and a network. Thus, the total number of parameters of this system is twelve times less as compared to that of AlexNet.



FIGURE 13.8

But the main feature of this net are these sections here.

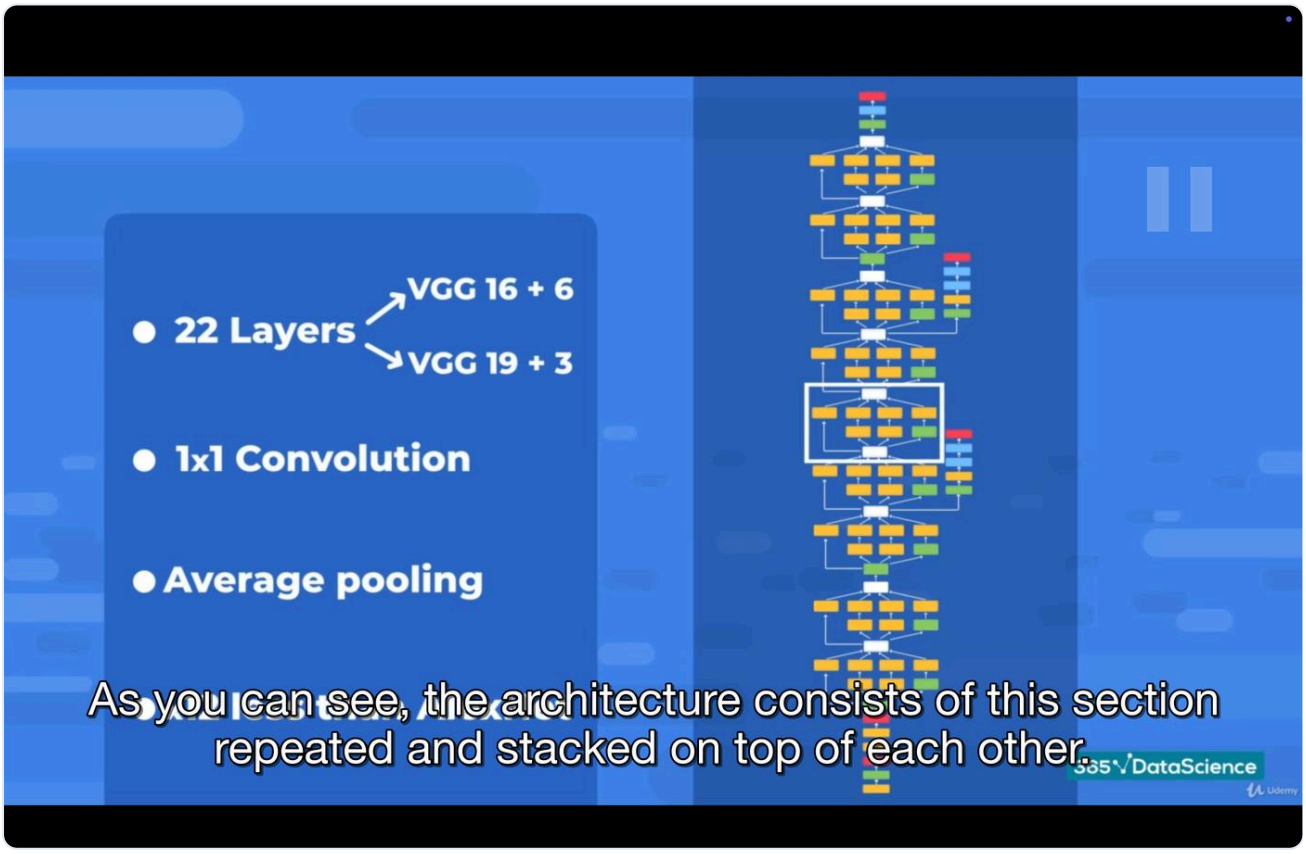


FIGURE 13.9

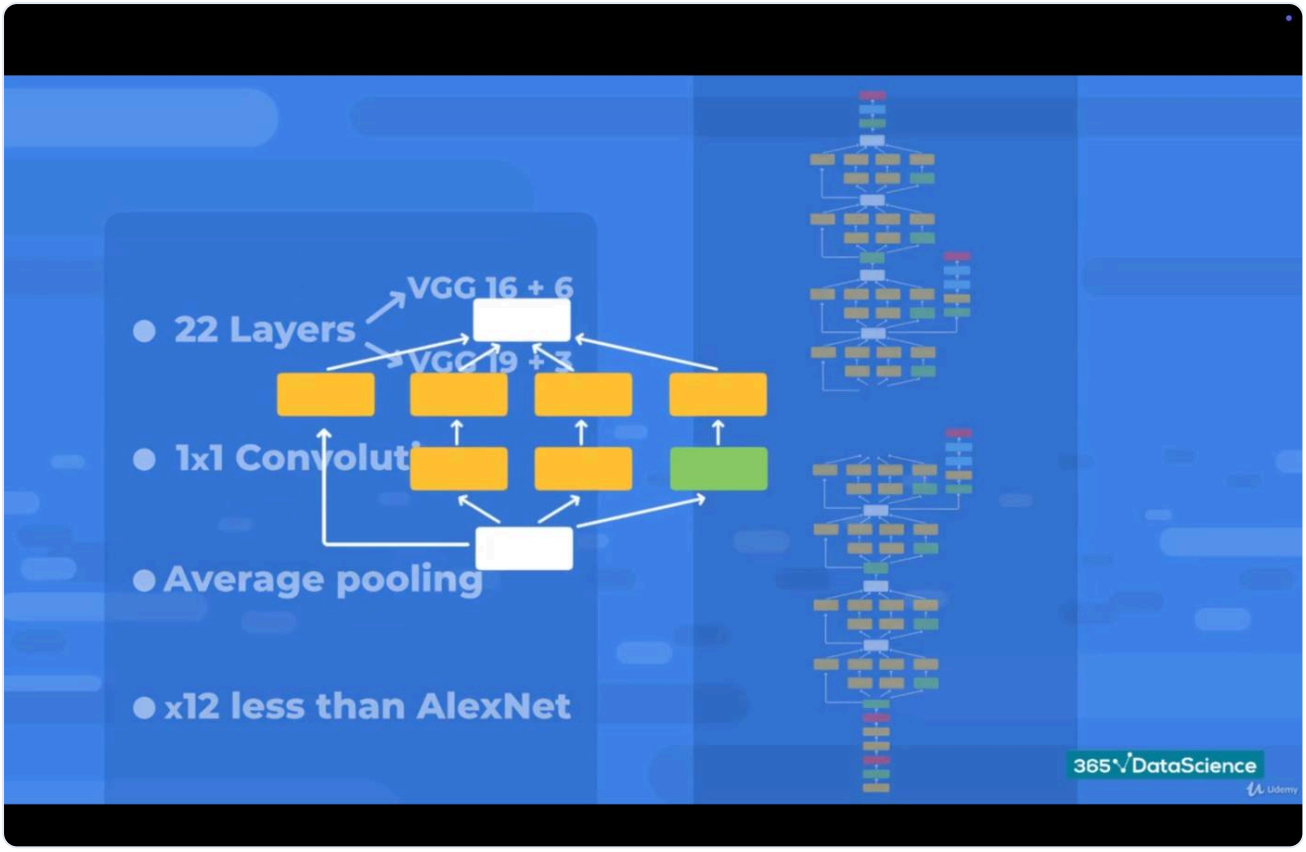


FIGURE 13.10

As you can see, the architecture consists of this section repeated and stacked on top of each other.

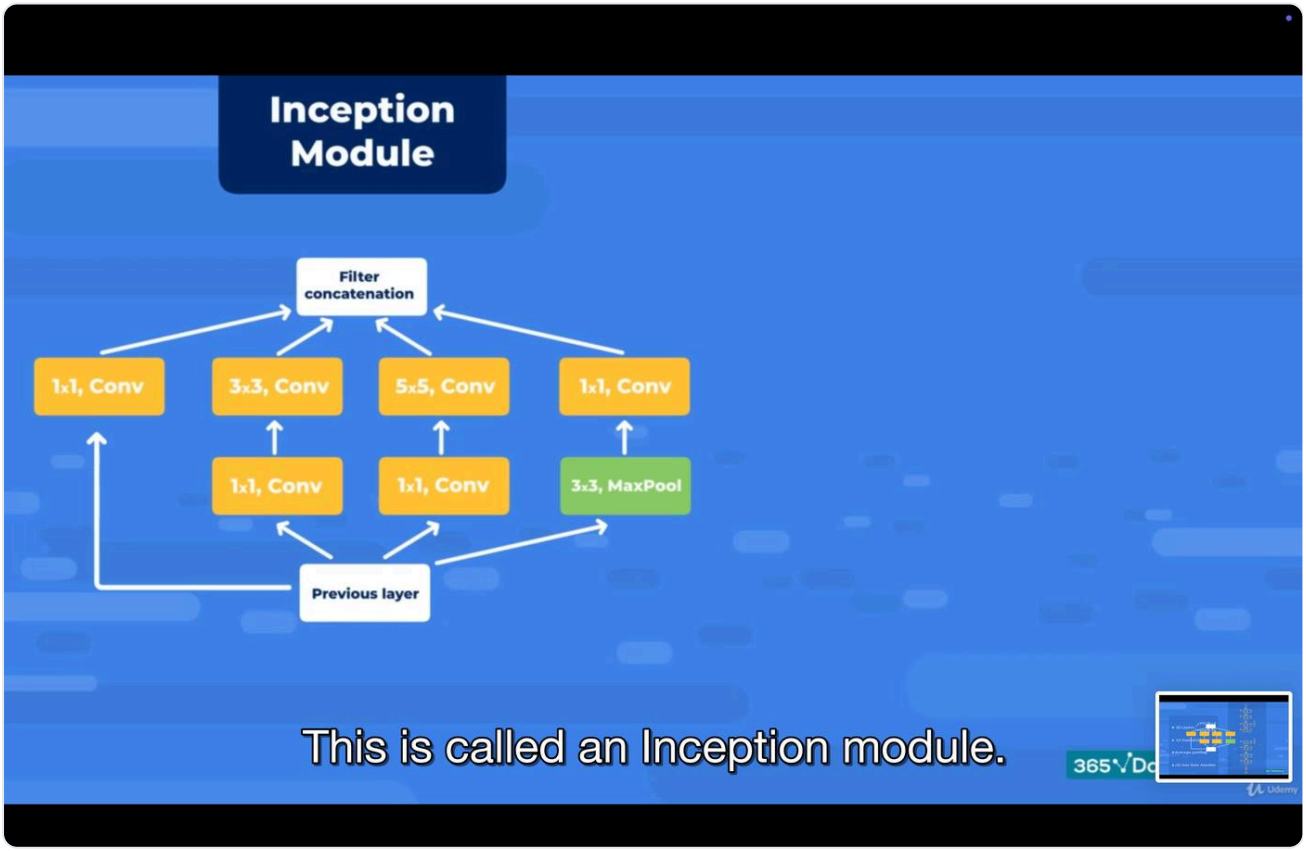


FIGURE 13.11

This is called an Inception module. Here's a fun fact about the origin of its name. The authors have admitted that the name Inception was indeed inspired by the non-paper and the famous Internet meme.

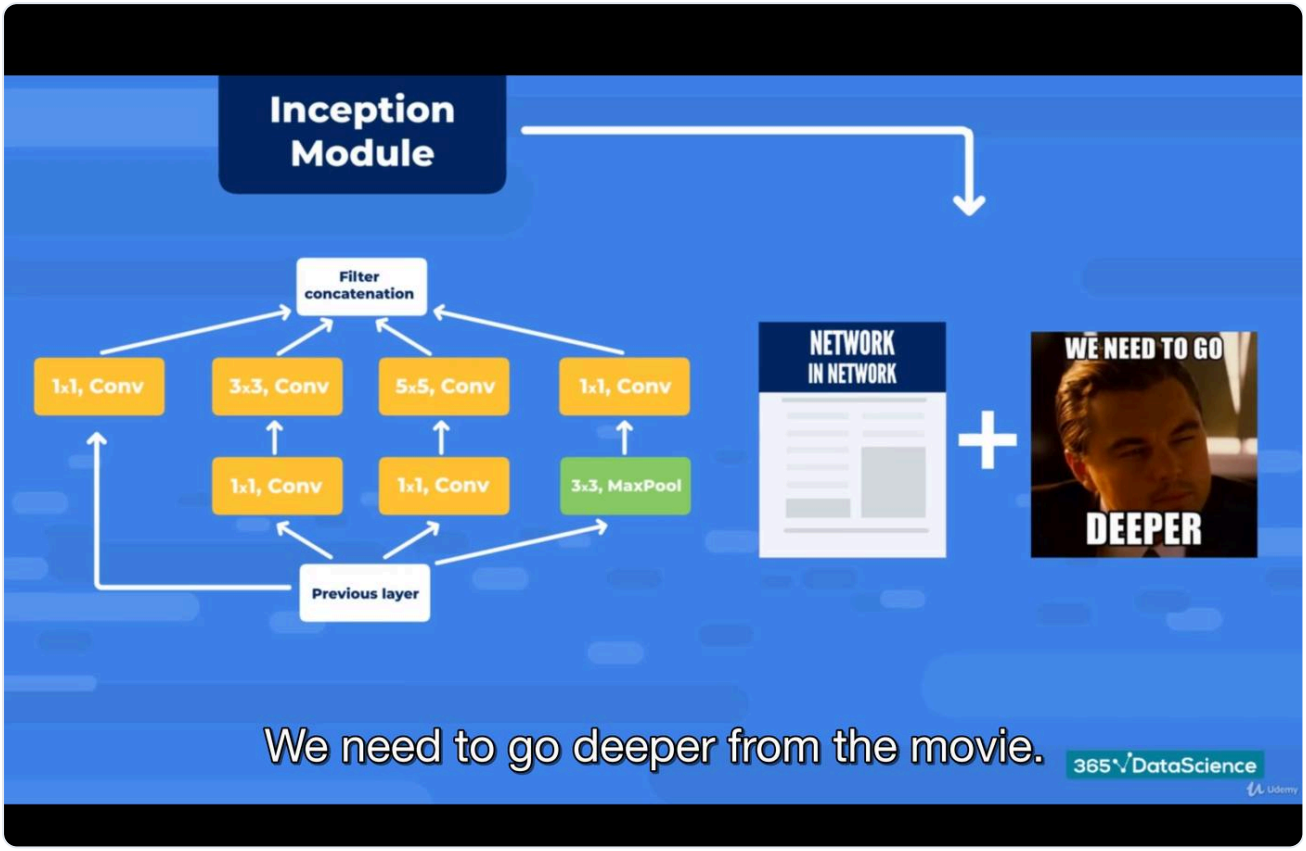


FIGURE 13.12

We need to go deeper from the movie. Inception just goes to show that these high profile engineers are normal people just like us. OK, now back to machine learning. So let's examine this Inception module more closely. The whole concept behind it is to design a good local network topology that is to be a computationally efficient and useful standalone module.

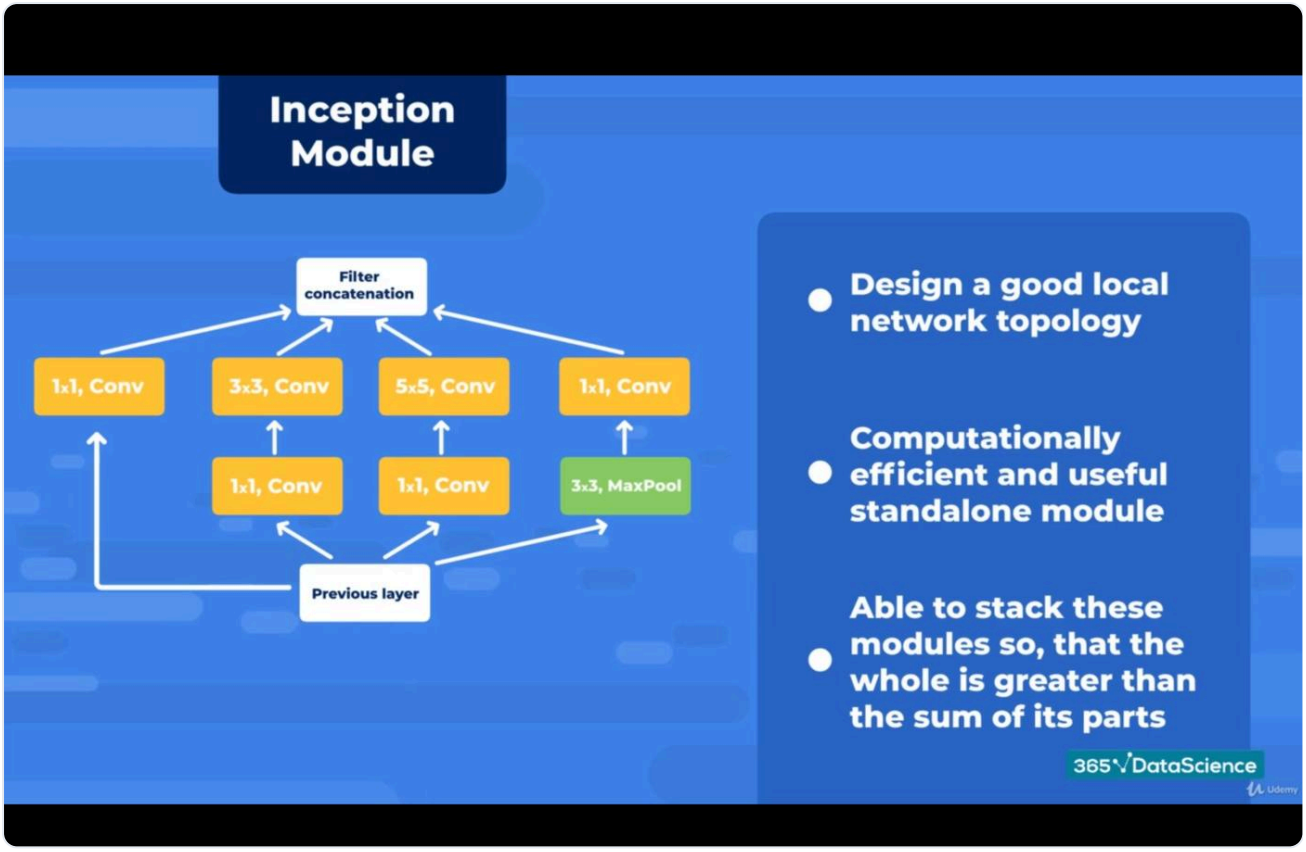


FIGURE 13.13

The main idea is to be able to stack these modules so that the whole is greater than the sum of its parts. How is that achieved?

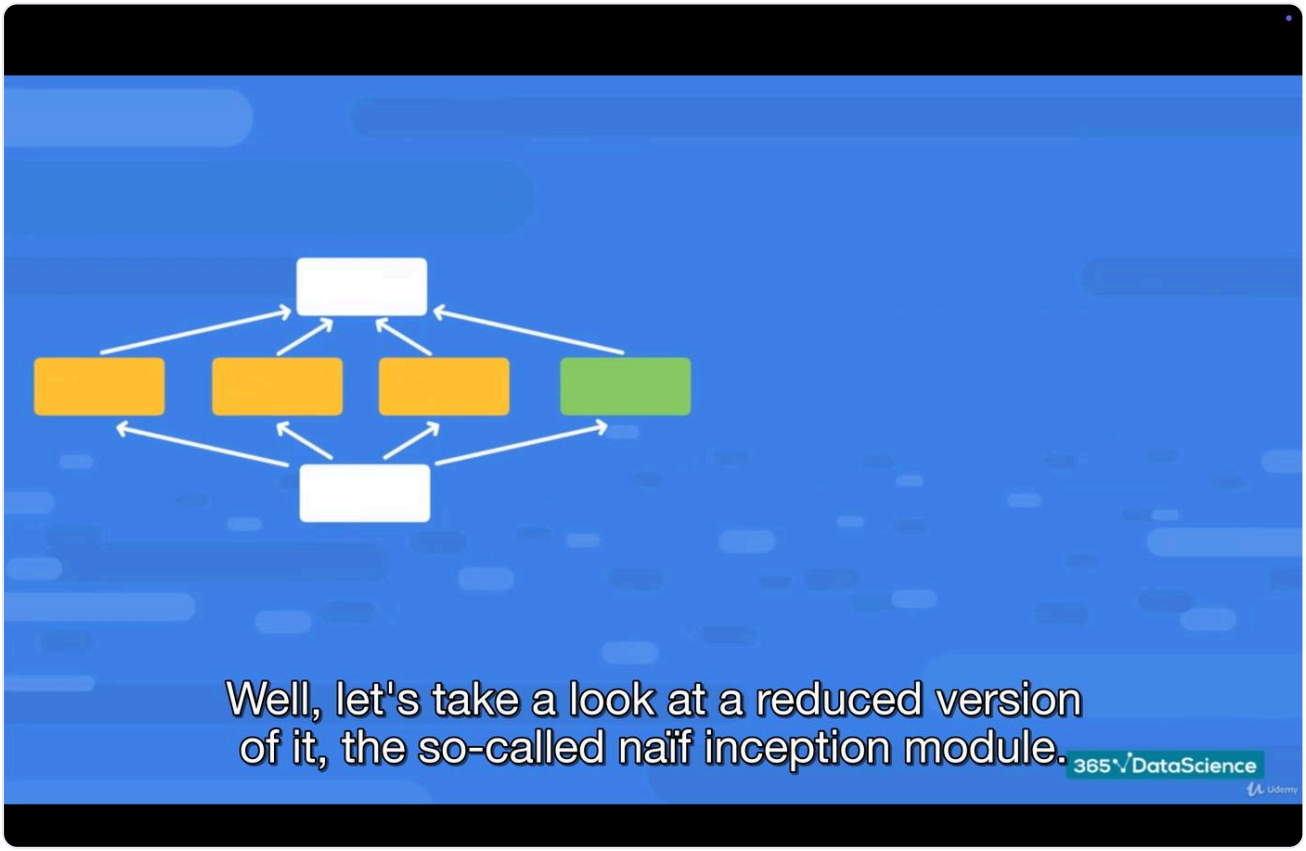


FIGURE 13.14

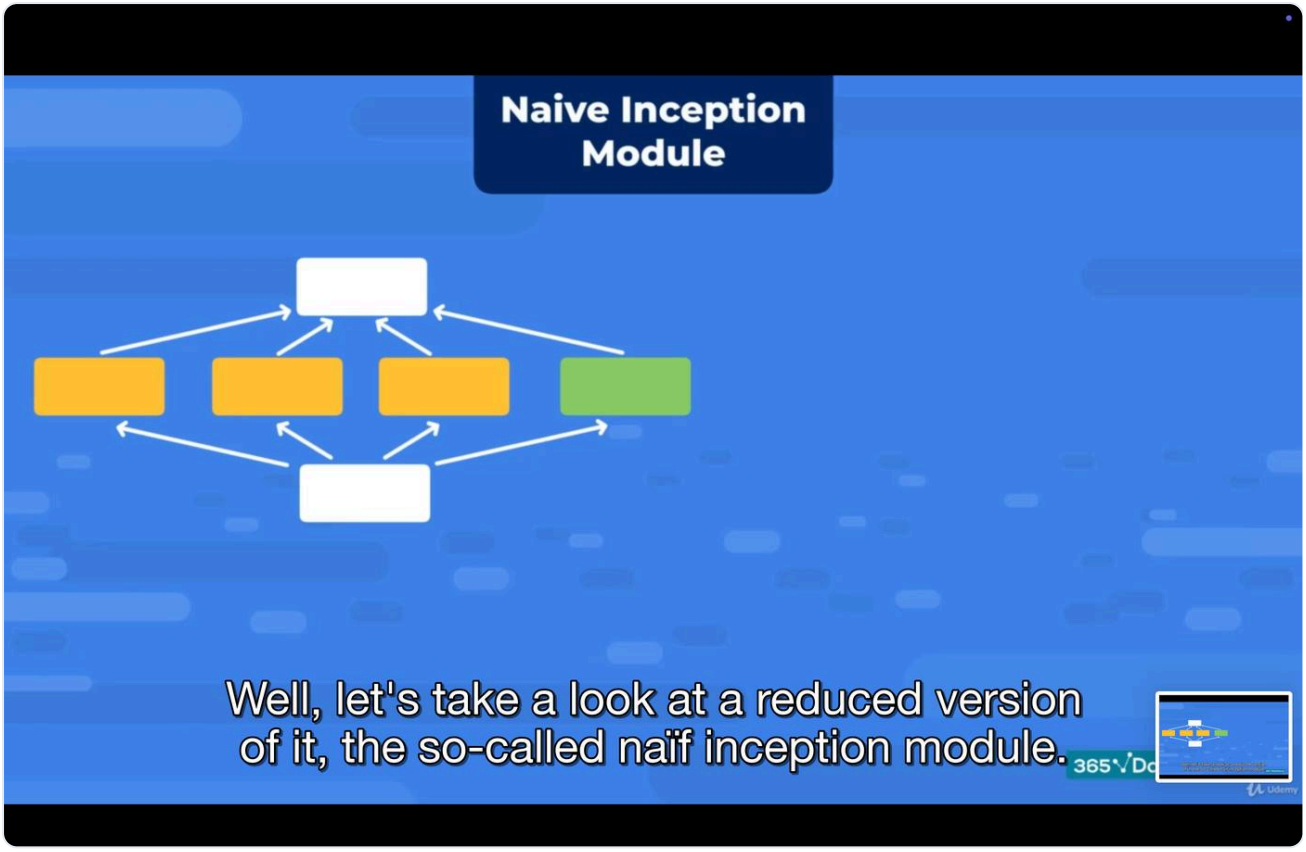


FIGURE 13.15

Well, let's take a look at a reduced version of it, the so-called naïf inception module.

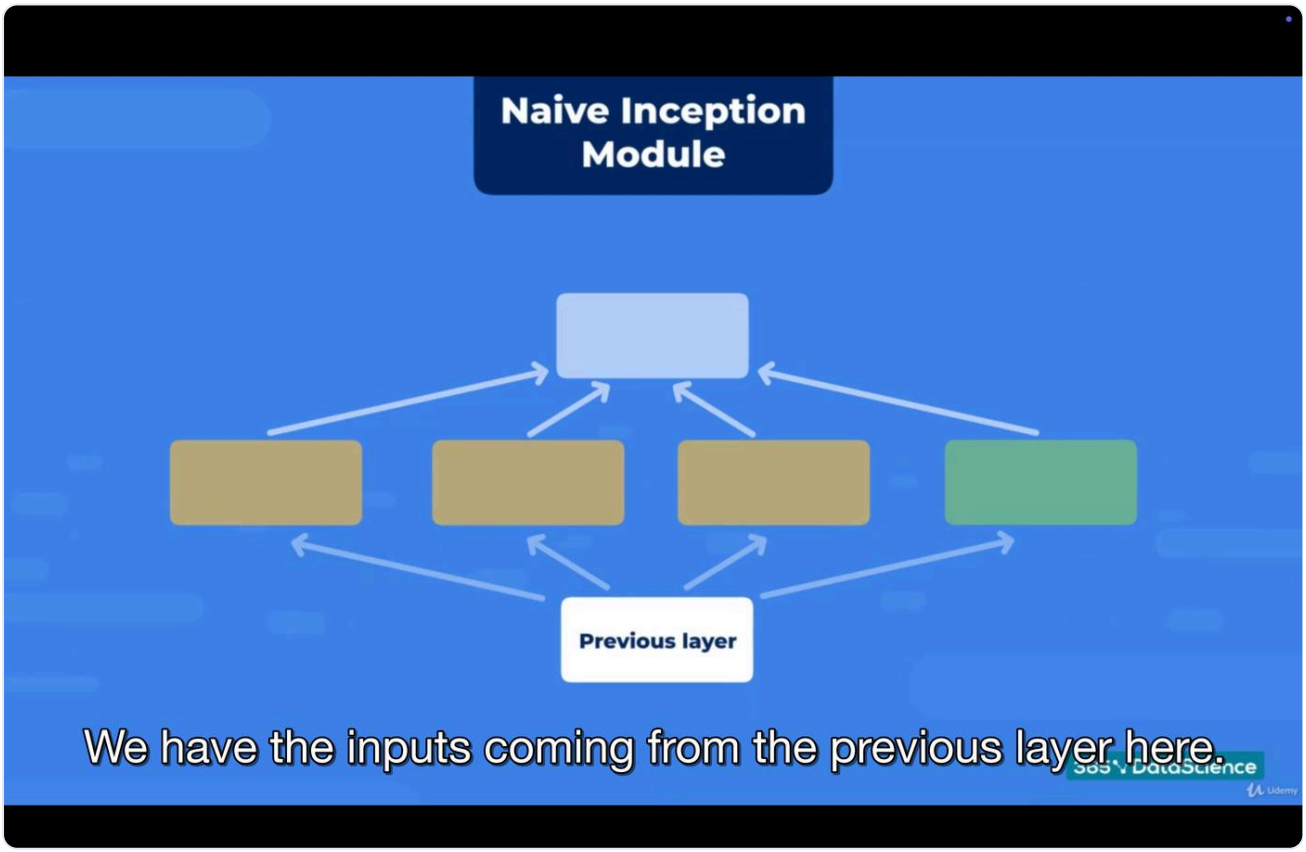


FIGURE 13.16 | Naive Inception

We have the inputs coming from the previous layer here. Now, instead of feeding it into the next filter, as usual, we apply it to numerous parallel filters.

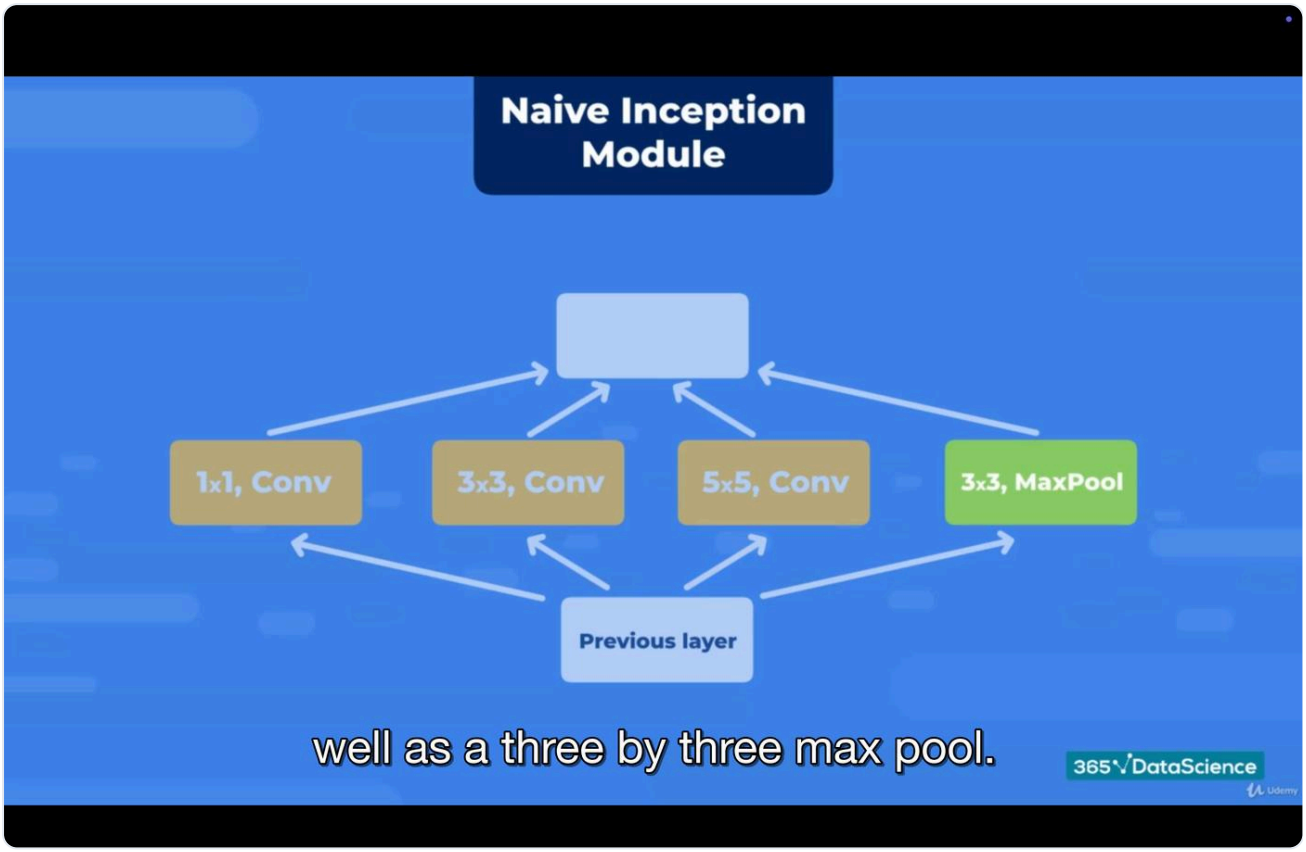


FIGURE 13.17 | Naive Inception

So the same input gets introduced to a one by one, three by three and five by five convolution, as well as a three by three max pool. Thus we have many different kernel sizes all in the same layer.

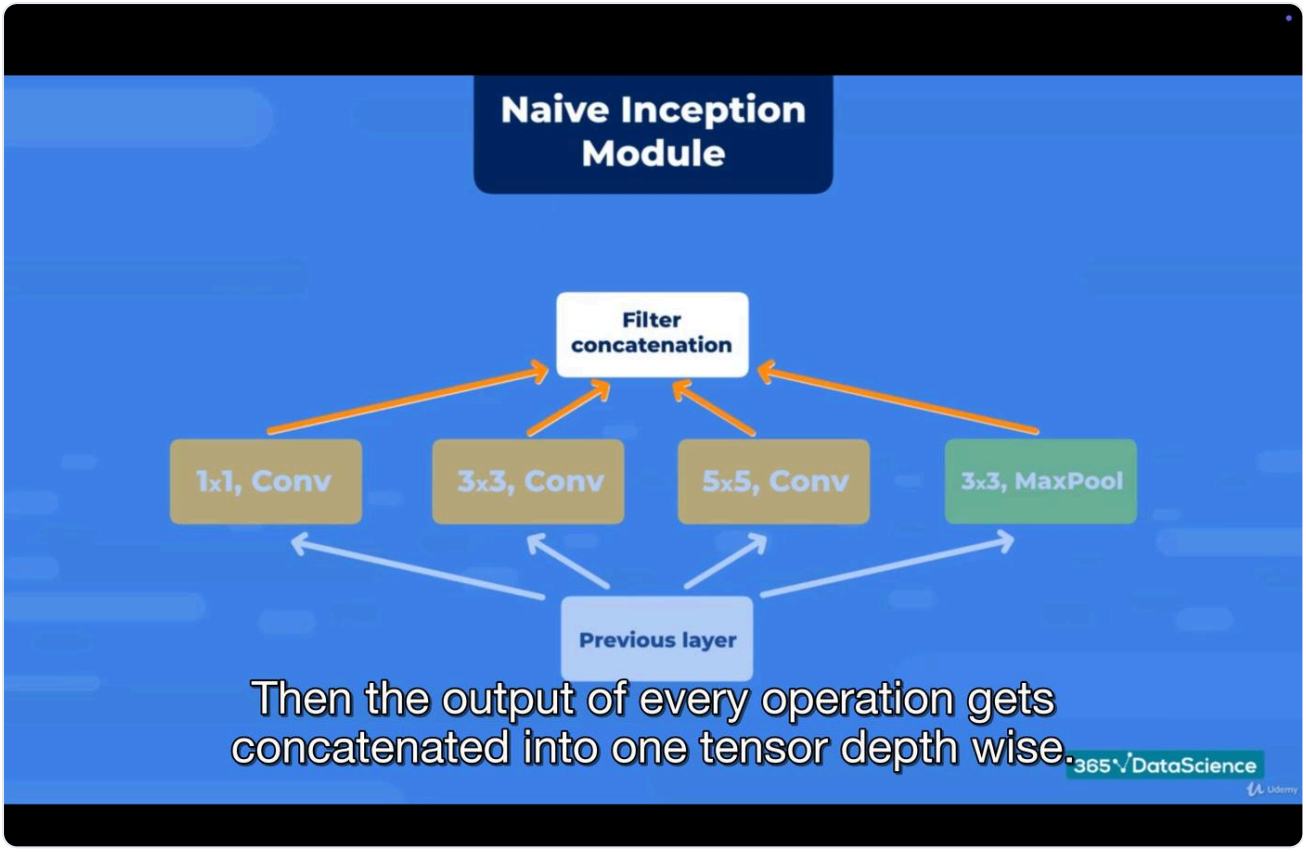


FIGURE 13.18

Then the output of every operation gets concatenated into one tensor depth wise.

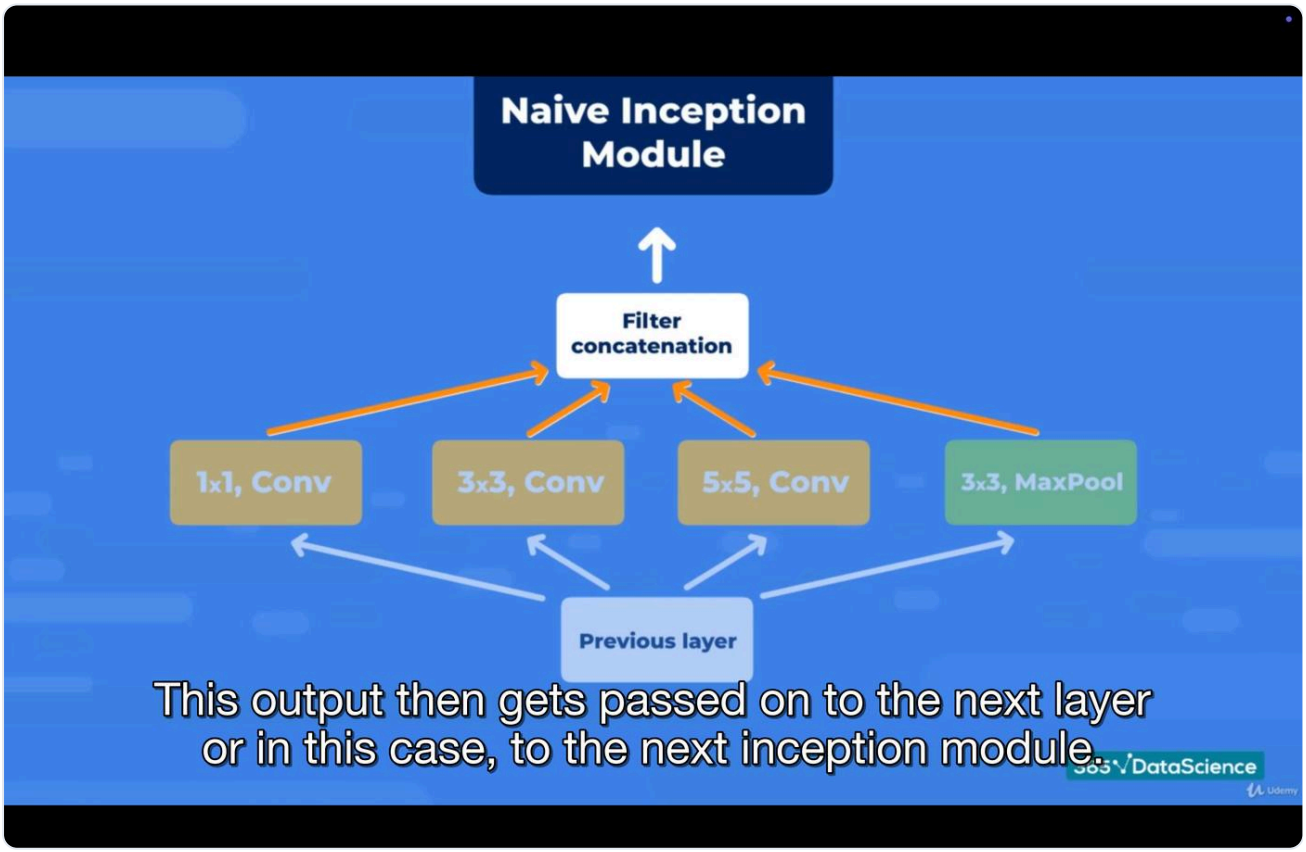


FIGURE 13.19 | Naive Inception

This output then gets passed on to the next layer or in this case, to the next inception module.

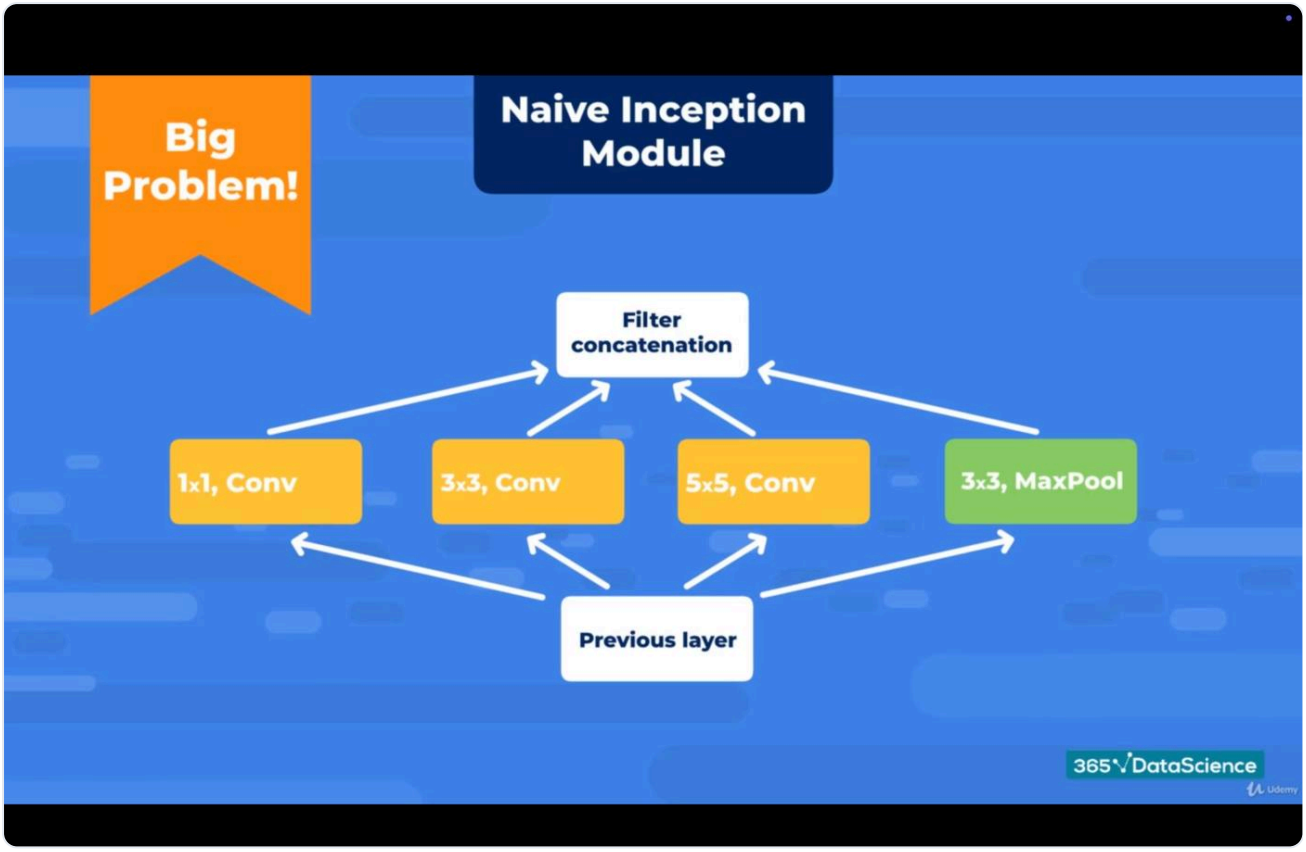


FIGURE 13.20 | Naive Inception

As good as it may all sound, this configuration has one big problem. To illustrate it, we will need to calculate how the dimensions get transformed by it.

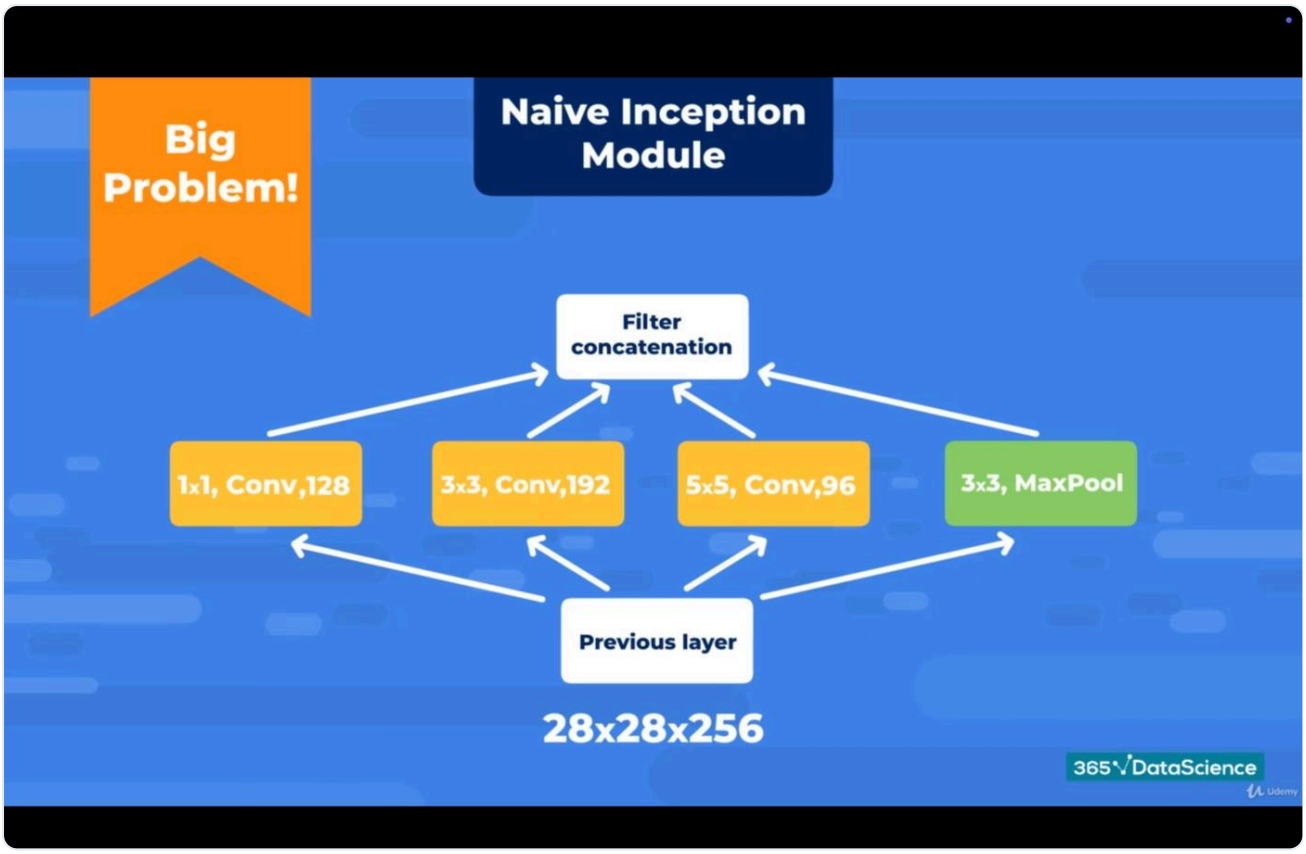


FIGURE 13.21 | Naive Inception

Suppose that our input is twenty eight by twenty eight by 256 and the number of filters in the convolutional layers are one twenty eight one 92 and 96 respectively.

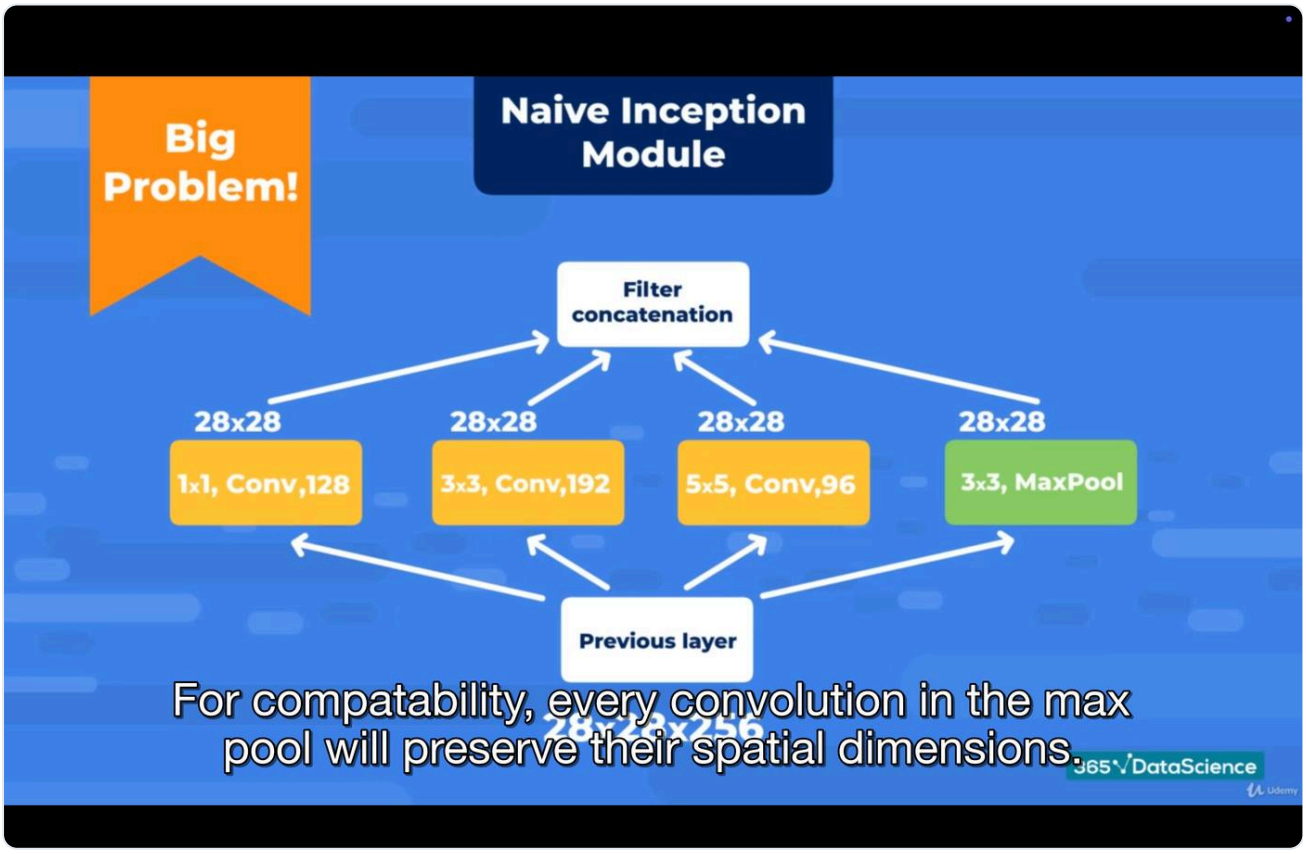


FIGURE 13.22 | Naive Inception

For compatibility, every convolution in the max pool will preserve their spatial dimensions. So we will only look at the depth.

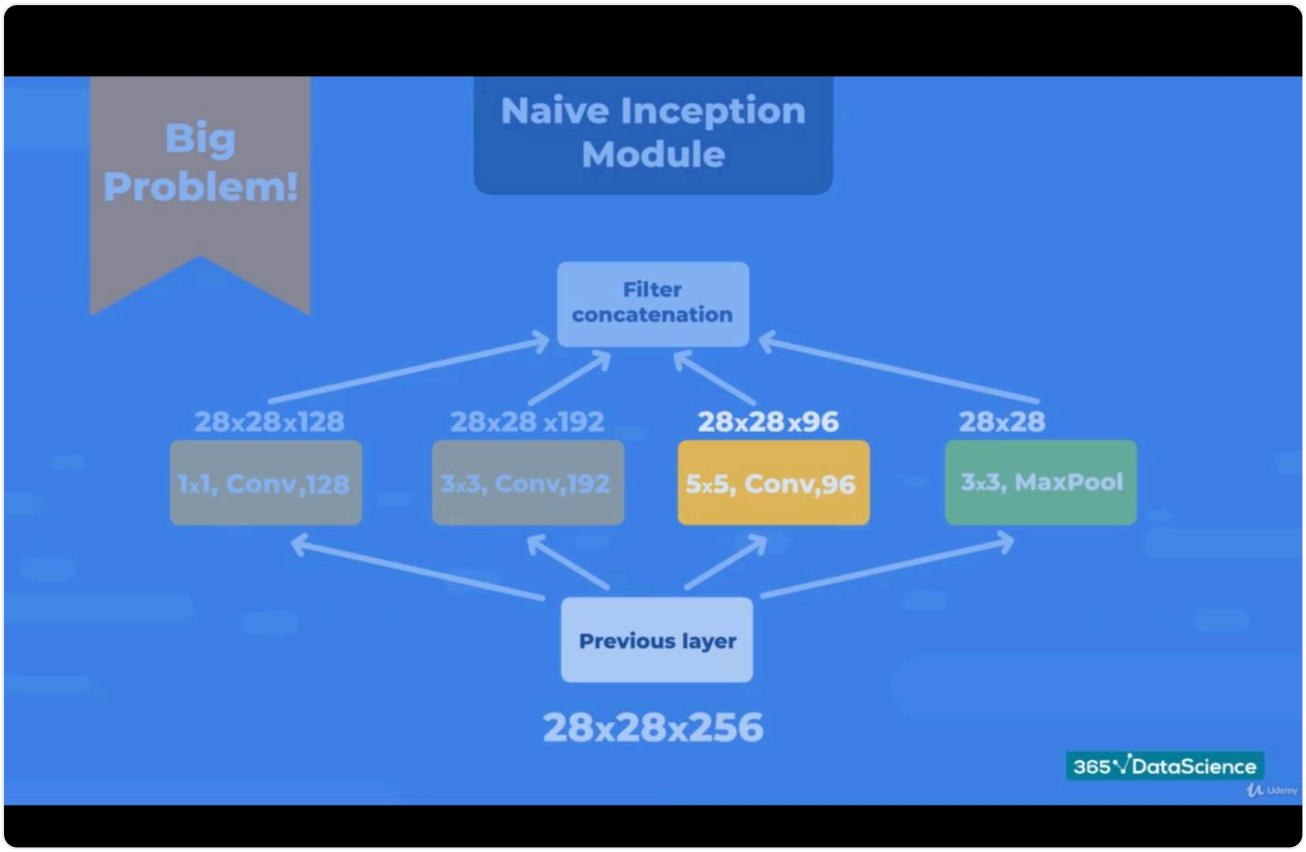


FIGURE 13.23

We already know that the depth of the output from a convolutional layer is the same as the number of filters.

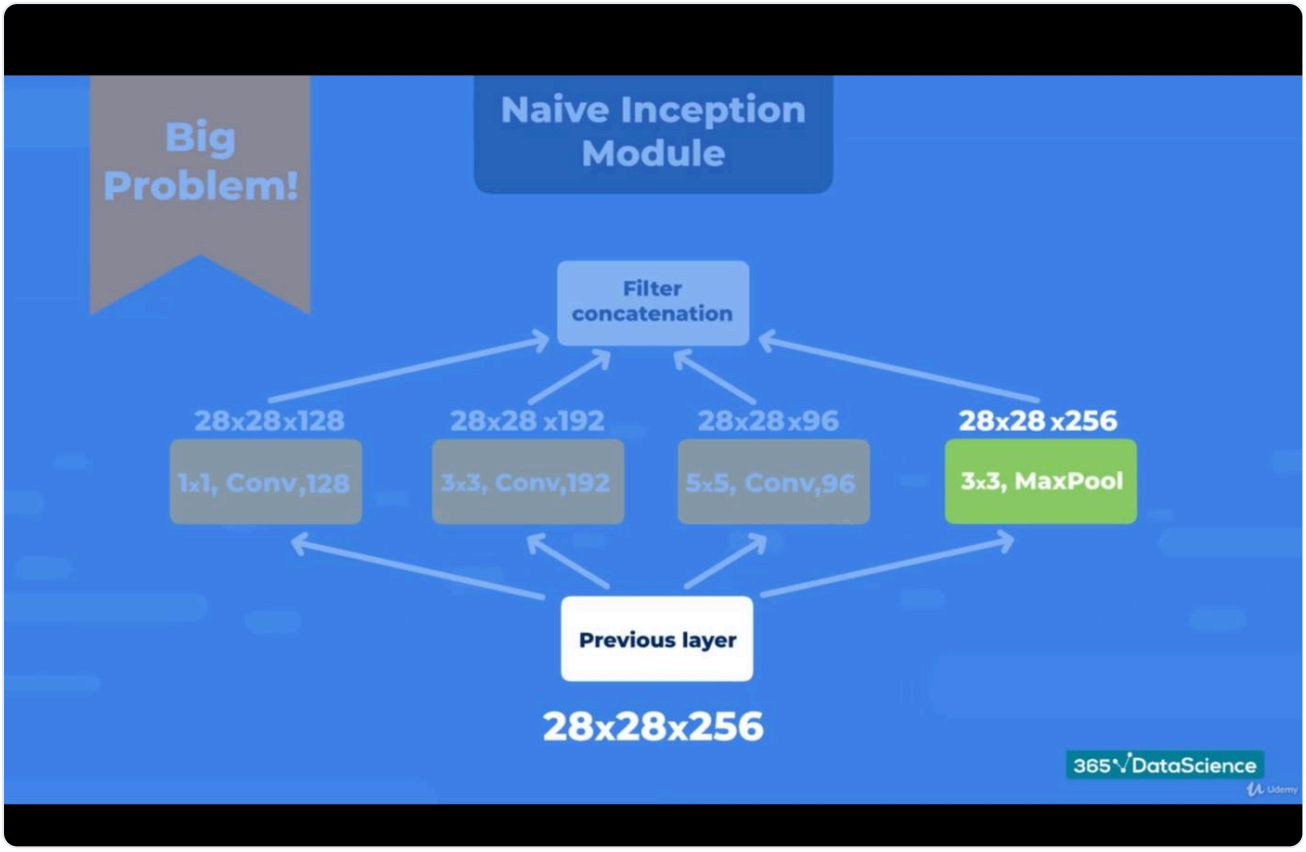


FIGURE 13.24 | Previous Layer

So that's one twenty eight one 92 and 96 for the convolutions. The pooling returns the same depth as the input, making it 256.

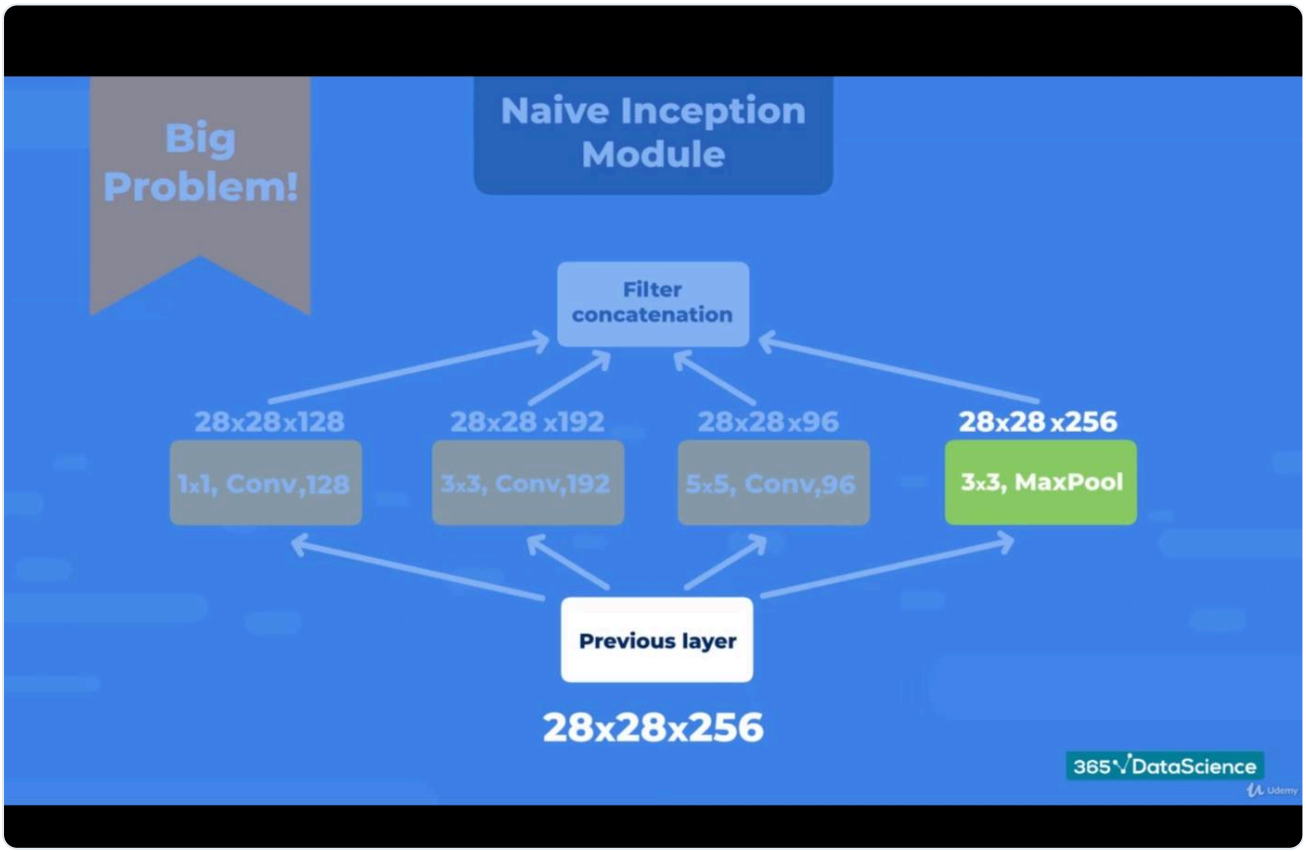


FIGURE 13.25 | Previous Layer

Now all those different outputs are concatenated together in depth. So the final output has the same spatial dimensions as the input that is twenty eight by twenty eight, while its depth is the sum of all the other depths.

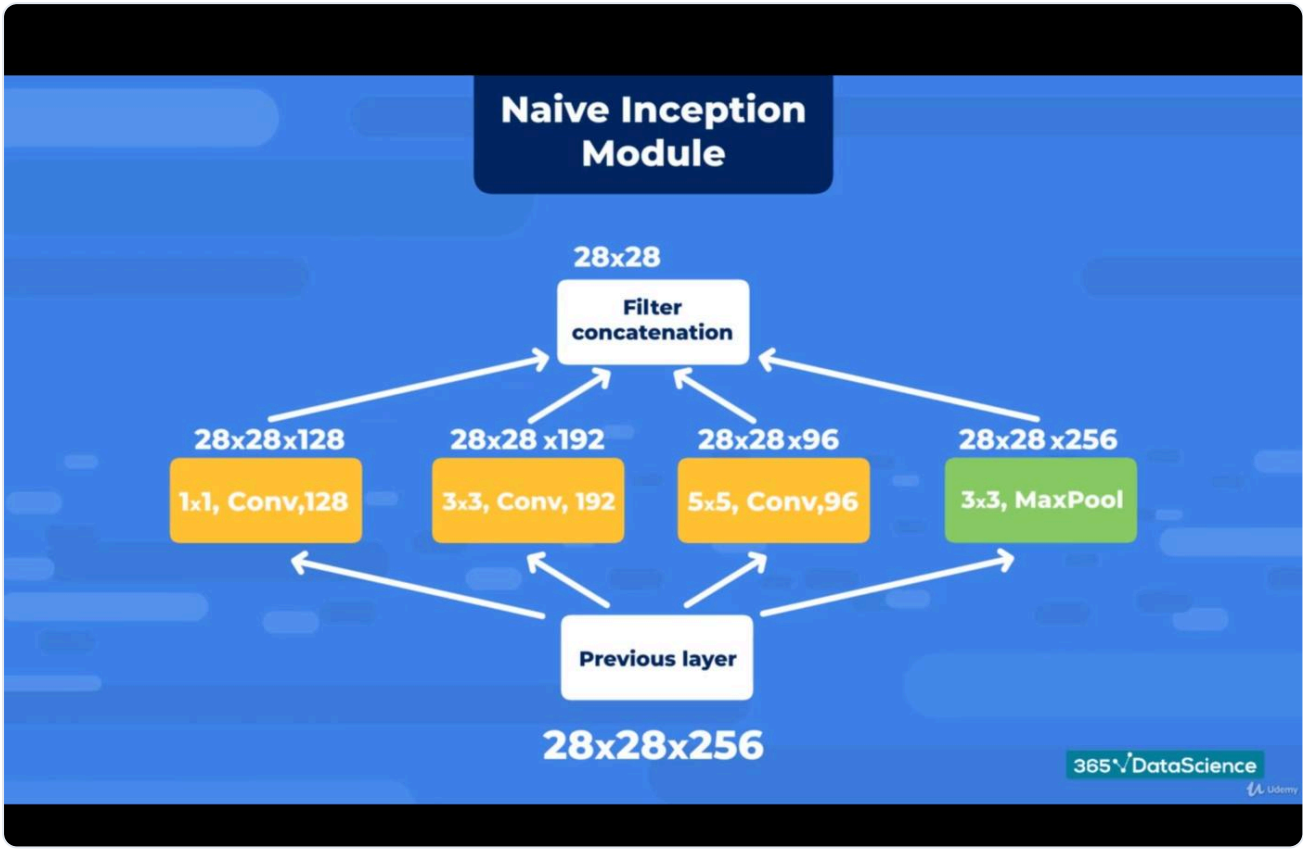


FIGURE 13.26 | Naive Inception

In that case it amounts to six seventy two in the end from an input of twenty eight by twenty eight by 256, we get an output of twenty eight by twenty eight by six seventy two.

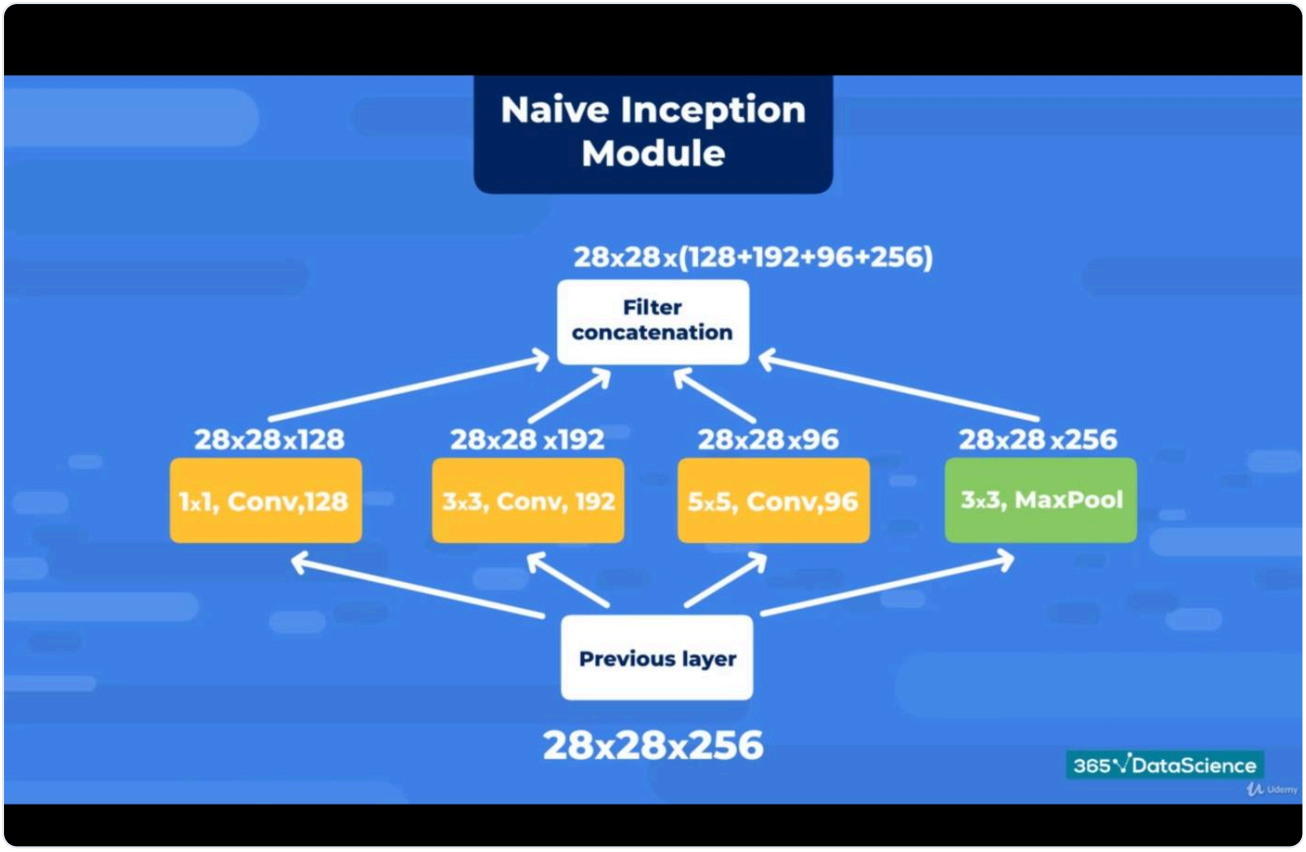


FIGURE 13.27 | Naive Inception

A significant increase. The total number of operations performed in this module is eight hundred and fifty four million.

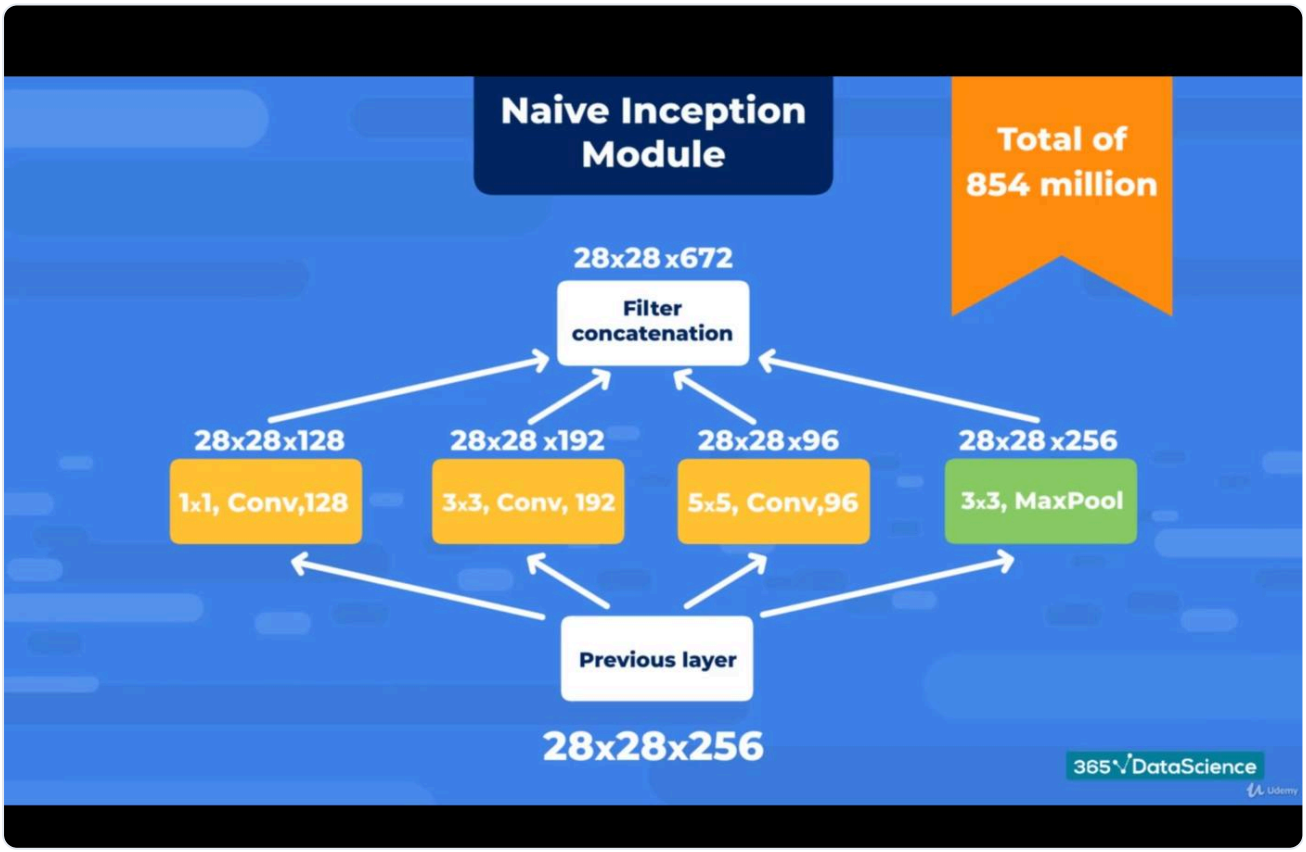


FIGURE 13.28 | Naive Inception

OK. We see that the problem lies in the fact that this naive Inception module is too computationally expensive.

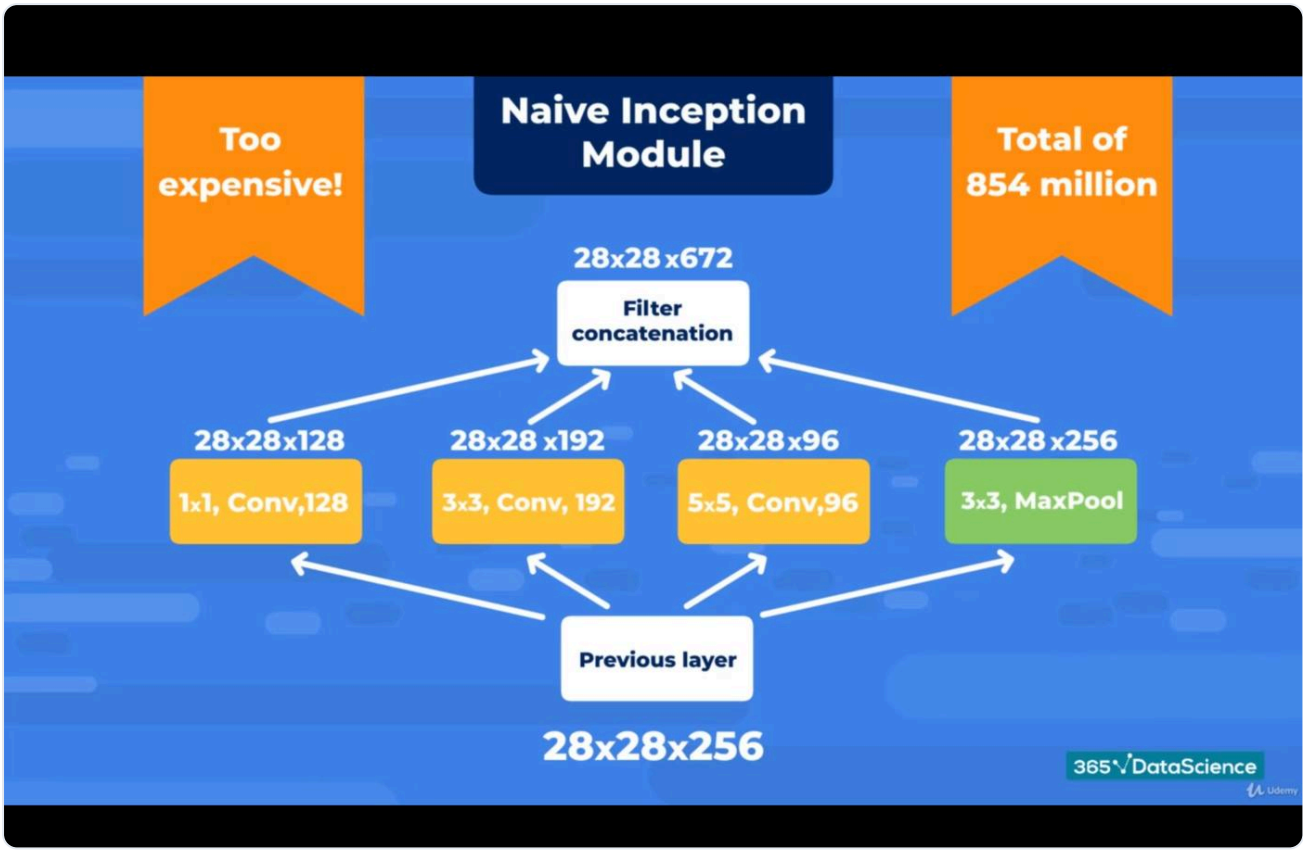


FIGURE 13.29 | Naive Inception

That's exactly the opposite of the goal we are trying to achieve.

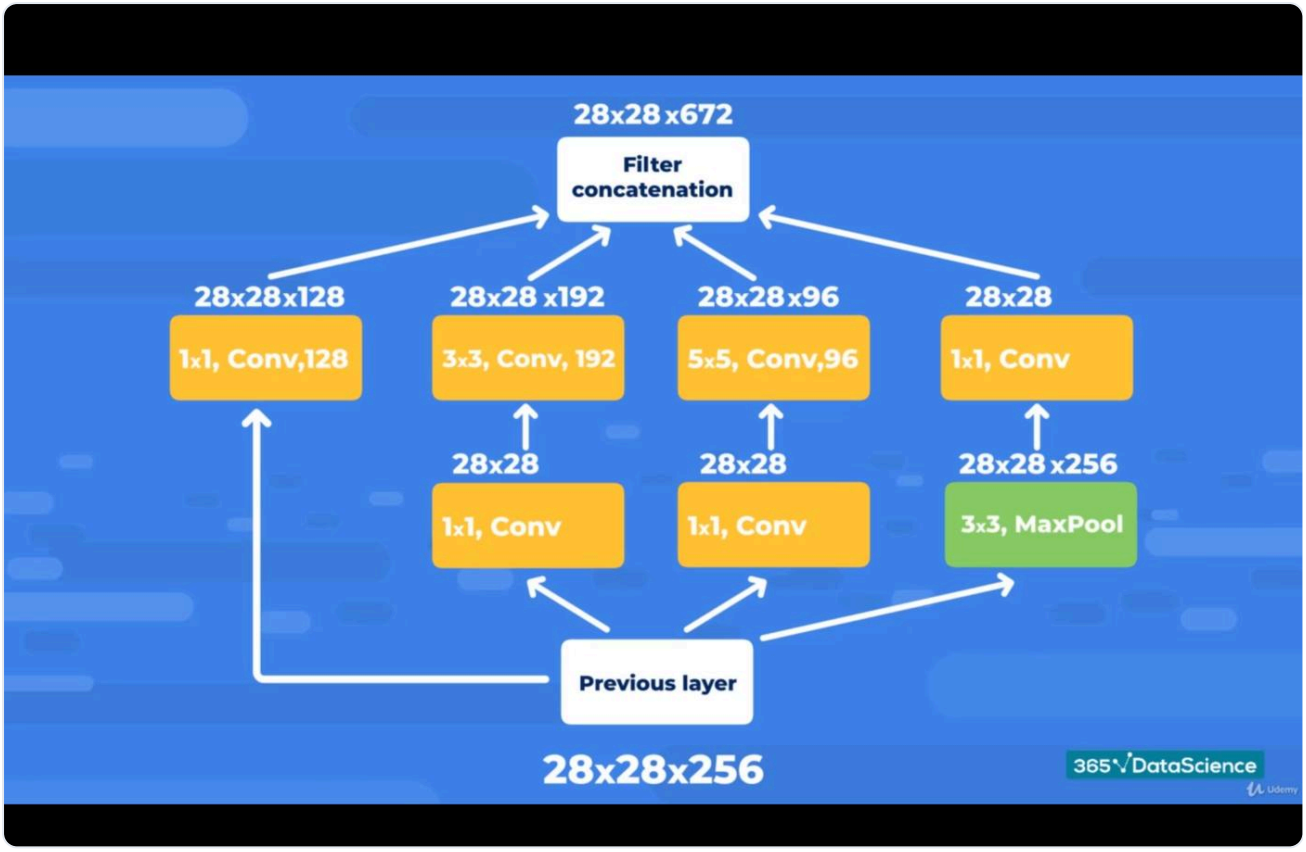


FIGURE 13.30 | Previous Layer

What can we do about that? The solution is to add three additional one by one convolutional layers like that.

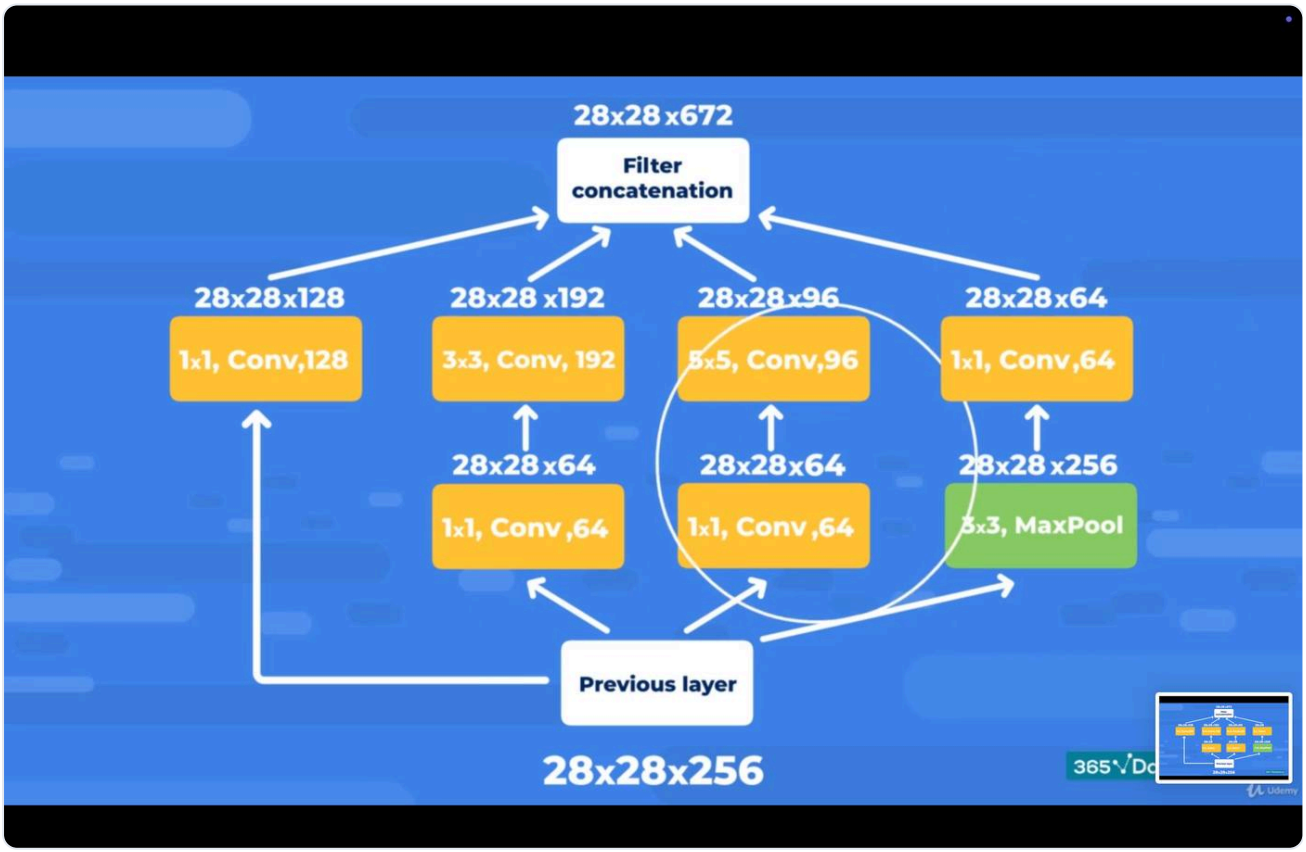


FIGURE 13.31 | Previous Layer

These are called bottleneck layers, and their purpose is to reduce the dimensionality and number of operations. All of those have 64 filters.

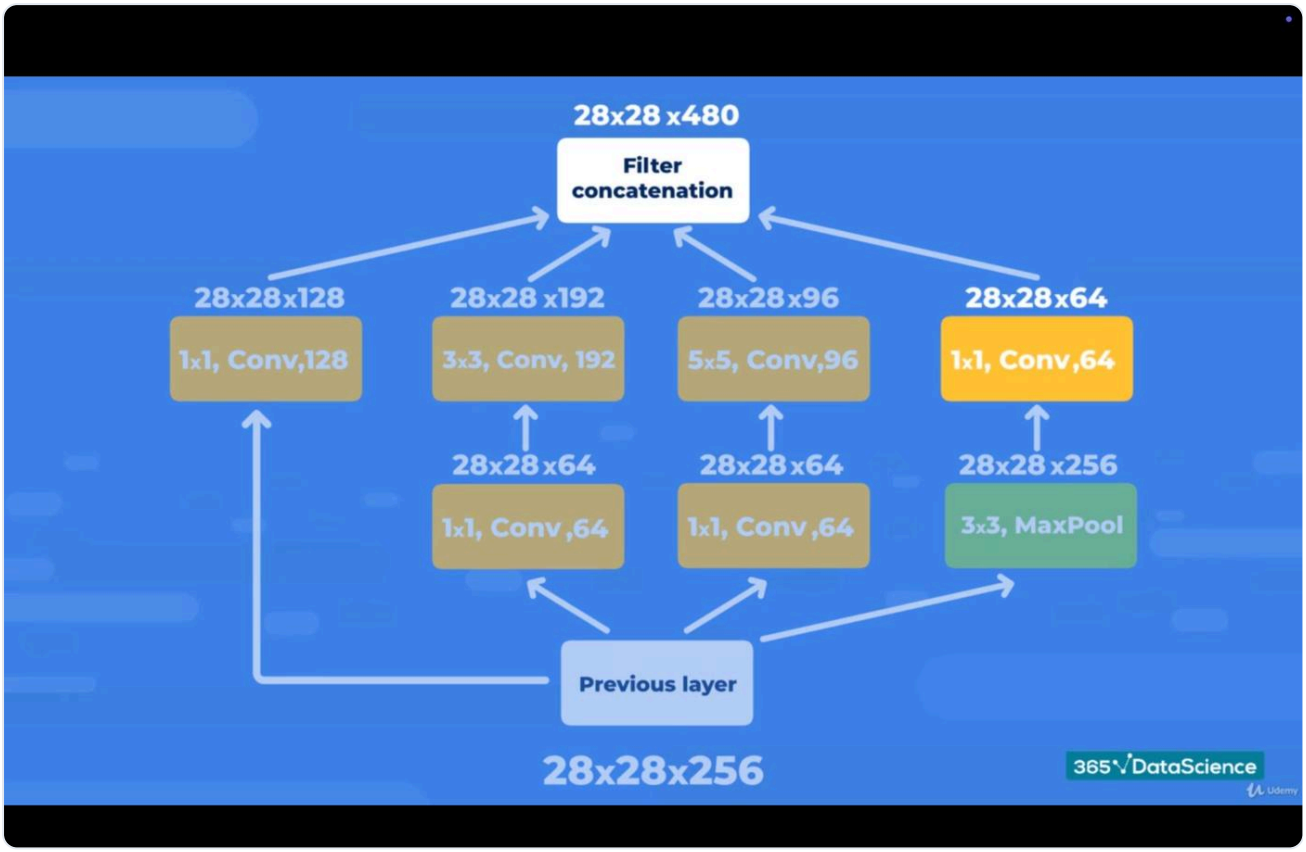


FIGURE 13.32

If we calculate how the tensor dimensions transform in this graph, we will produce an output of four hundred and eighty depth.

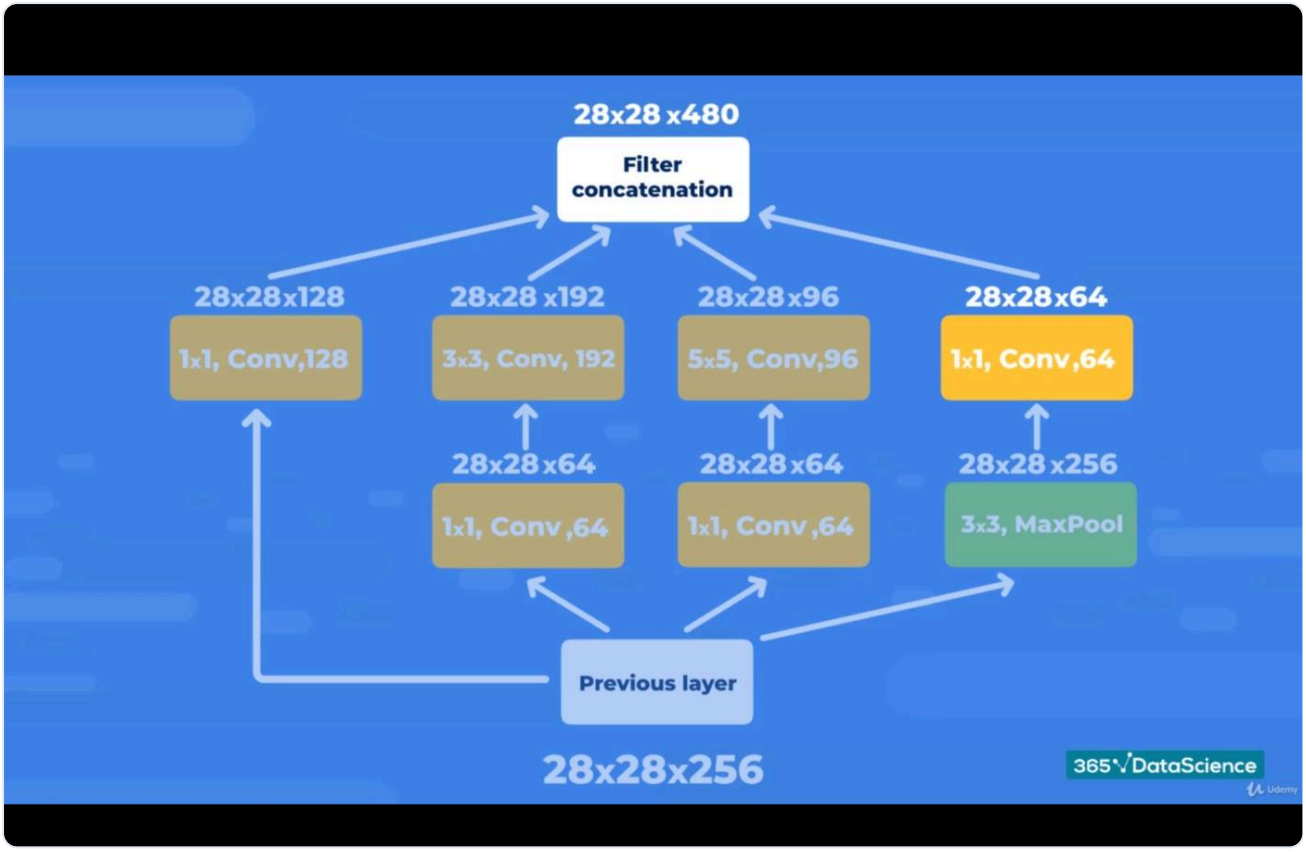


FIGURE 13.33

This reduction comes only from the rightmost bottleneck layer, the one after the max pool, as it produces an output with depth equal to its 64 filters instead of the input depth of 256. So if that is the only bottleneck layer that reduces the depth, what is the purpose of the other ones?

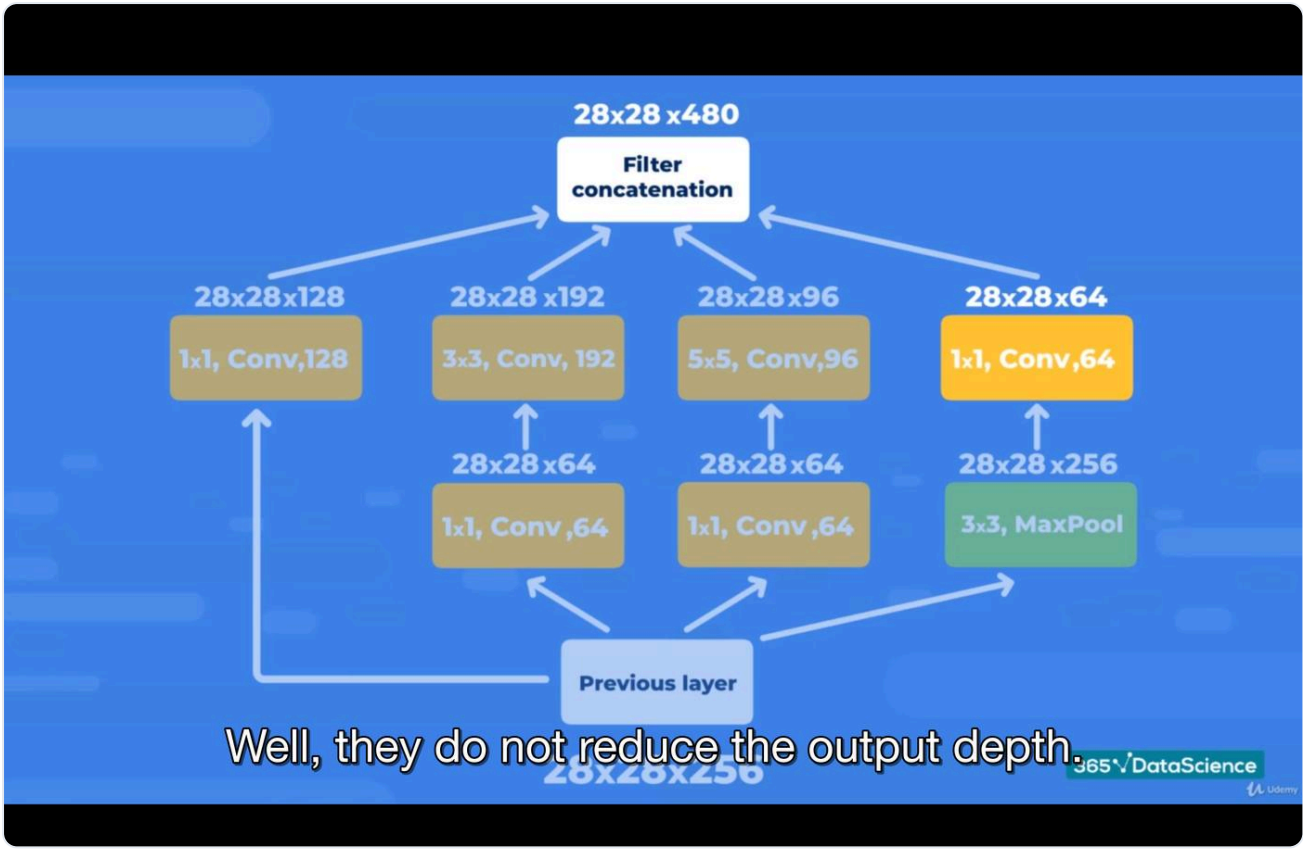


FIGURE 13.34

Well, they do not reduce the output depth. Instead, they reduce the input depth to the convolutions. Thus they are reducing the number of computations needed in this revised Inception module.

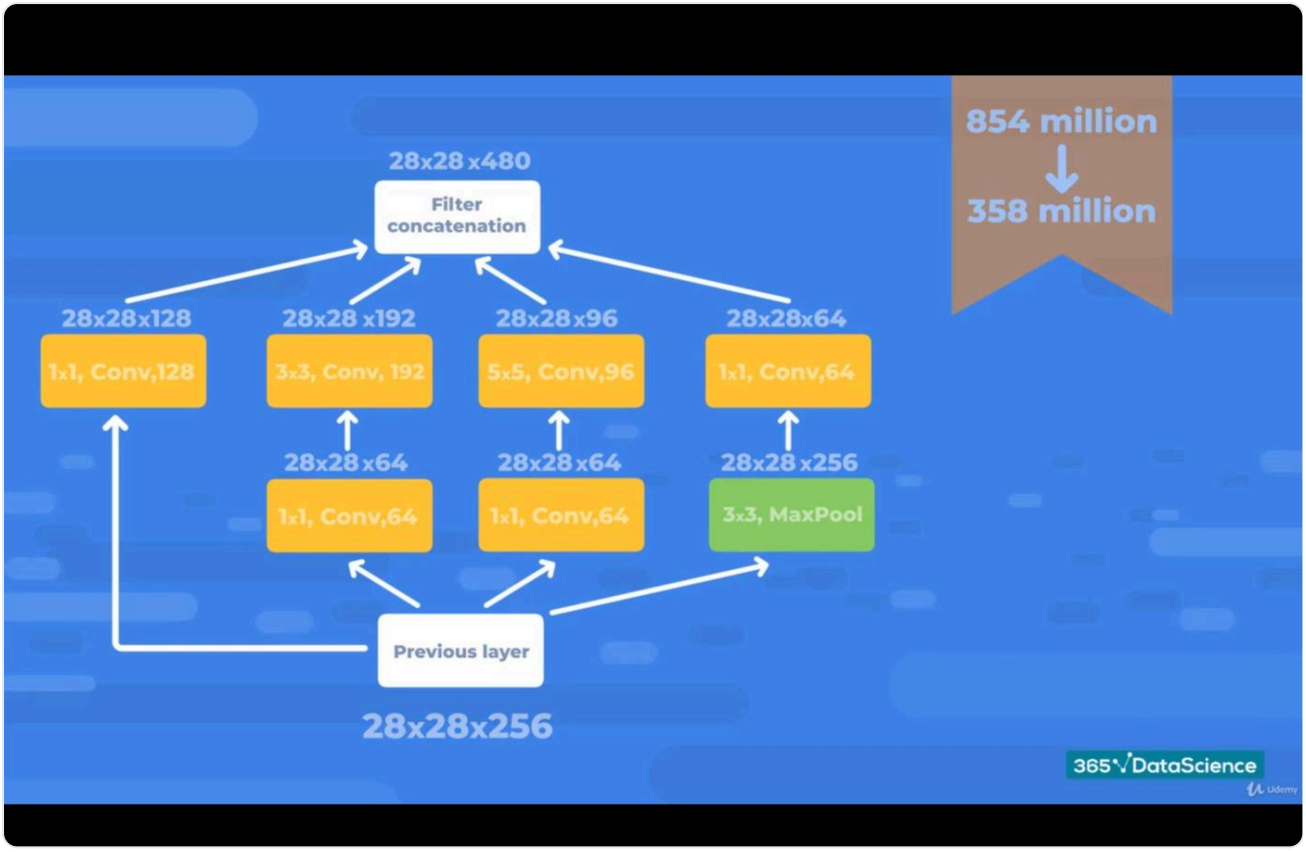


FIGURE 13.35

The total number of operations is three hundred and fifty eight million, down from eight hundred and fifty four million. Great. Now the network is constructed by stacking these mini network modules on top of each other at the beginning of the whole thing.



FIGURE 13.36



FIGURE 13.37

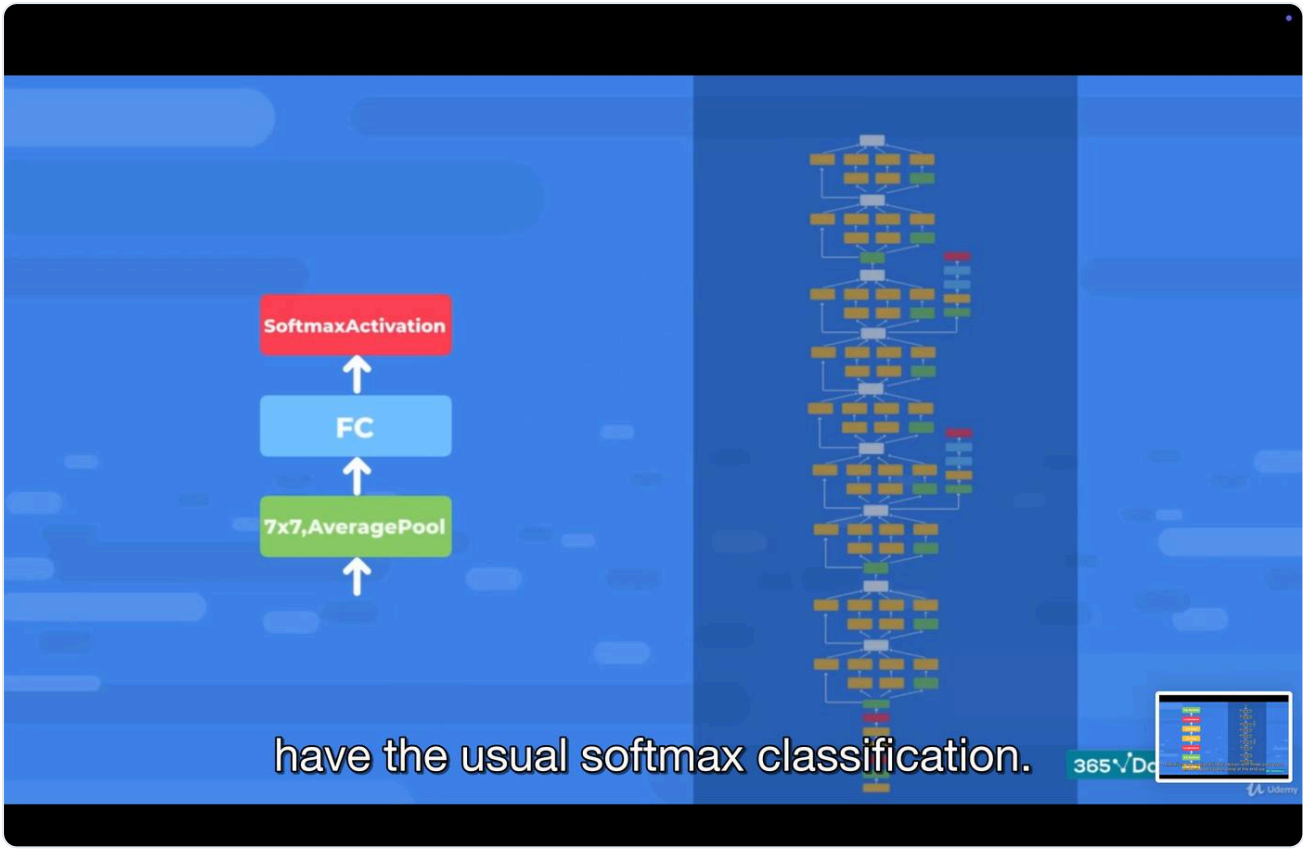


FIGURE 13.38

There is a conventional linear section with three conveyors and two pool layers, while at the end we have the usual softmax classification.

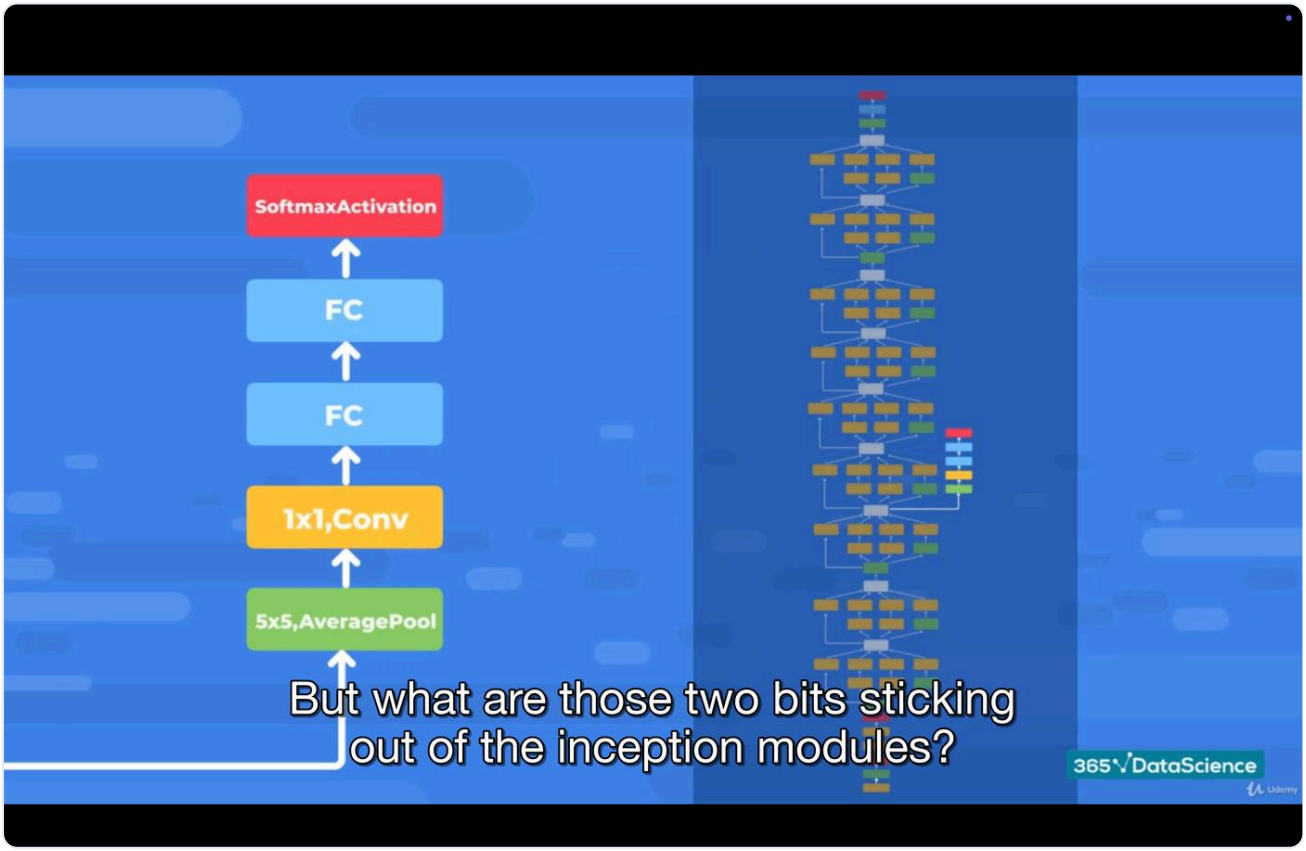


FIGURE 13.39

But what are those two bits sticking out of the inception modules? Those are used only in training.

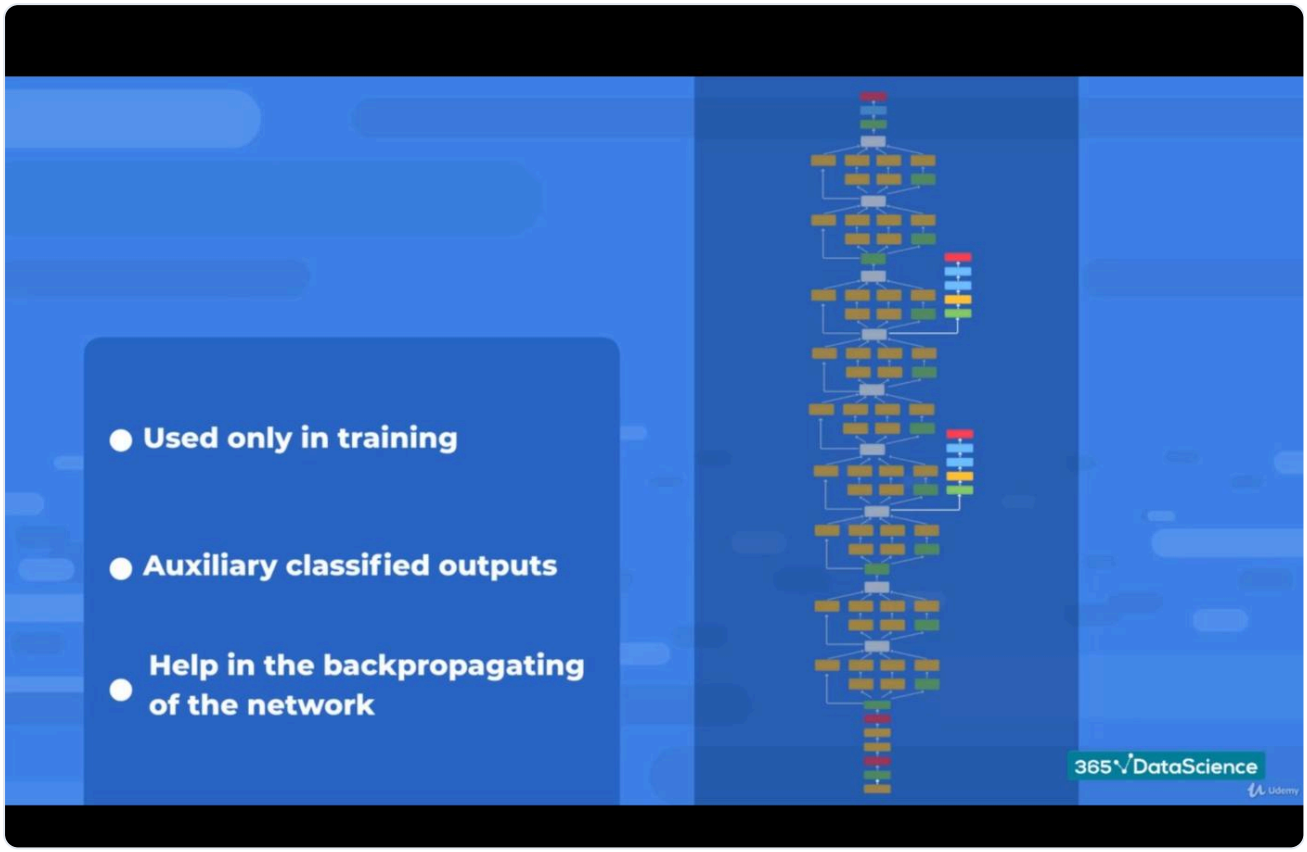


FIGURE 13.40

They are auxiliary classified outputs that help in the back propagating of the network. Remember, in order to train the network, the result compared to the true target is used to adjust all the weights in the net. They are updated by back propagating the gradient from the end to the beginning. Since this net is deeper and larger, the gradient may start to diminish.

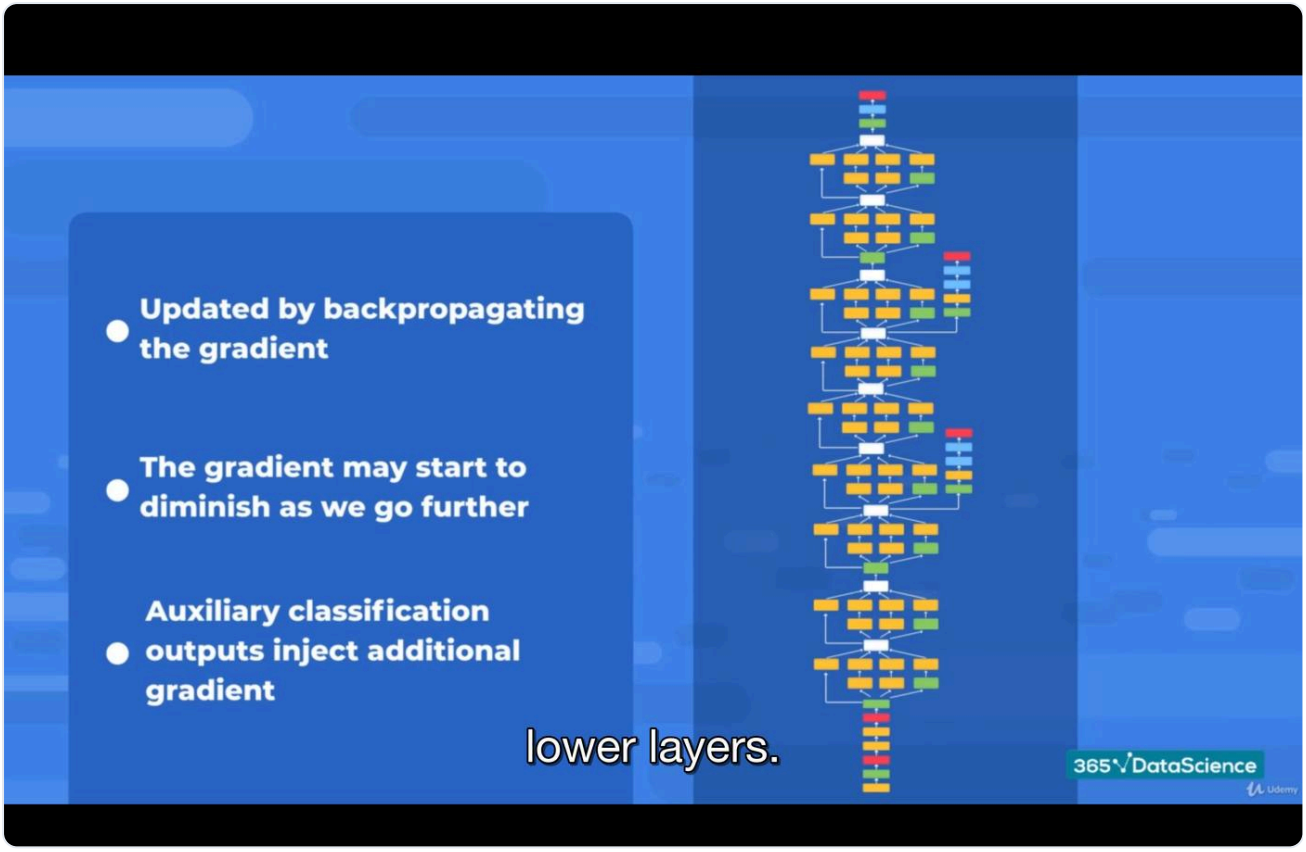


FIGURE 13.41

As we go further, these auxiliary classification outputs are there to inject additional gradient at lower layers. Fantastic. This architecture looks complicated at first, but once we consider it at a basic level, we are able to understand the rationale behind it. Next stop is the winner of the 2015 competition resonate? Thanks for watching.

LECTURE 52

LECTURE 53

Normal Model



FIGURE 15.1

This is the final CNN architecture that we will look into, it is called Resonant and is the winner of the 2015 edition of ILSVRC.



FIGURE 15.2

In fact, it's swept all classification and detection categories in ILSVRC and COCO having 152 layers. This network kicked off the revolution of depth.

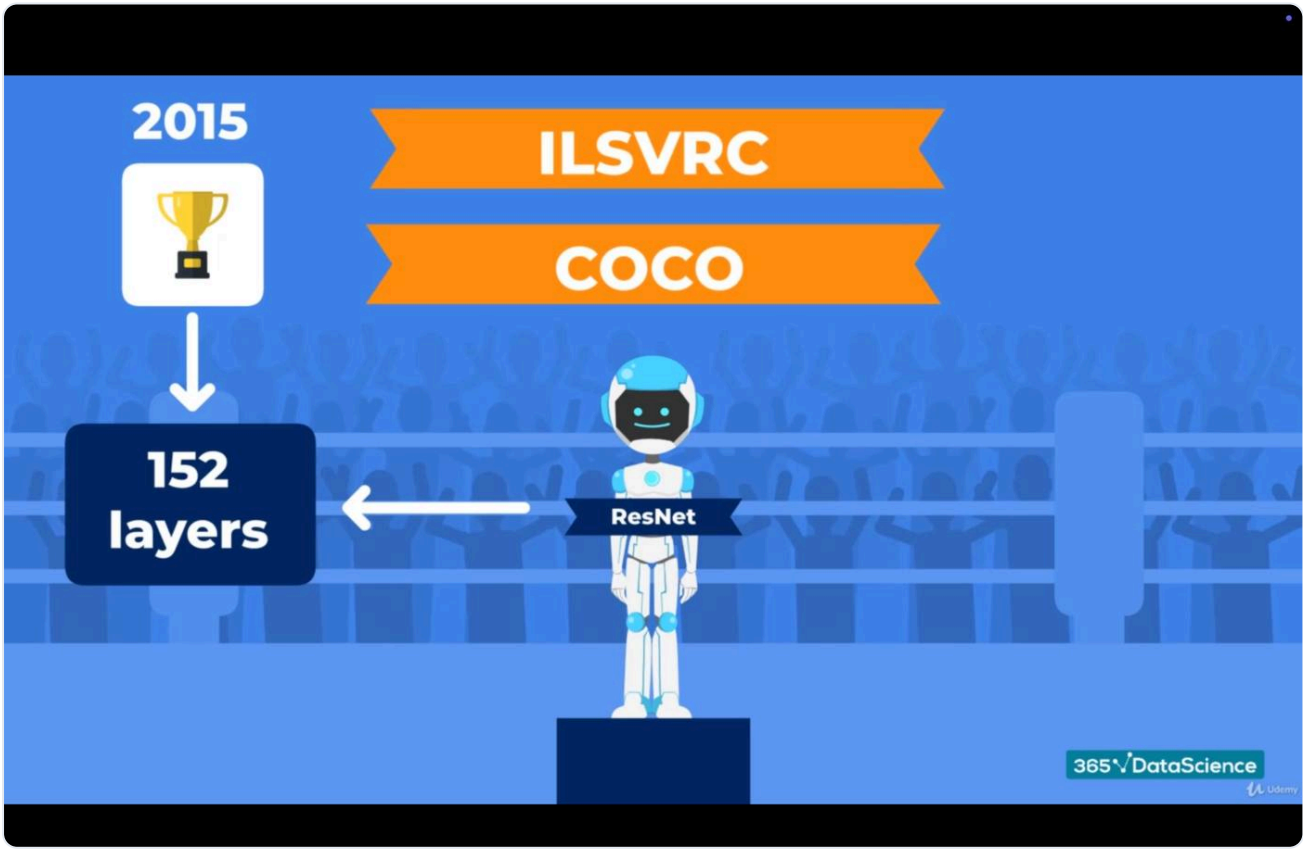


FIGURE 15.3

And you've got Google. That's 22 layers were a lot in general. A model with more layers should always perform at least as good as a model with a lower number of layers.

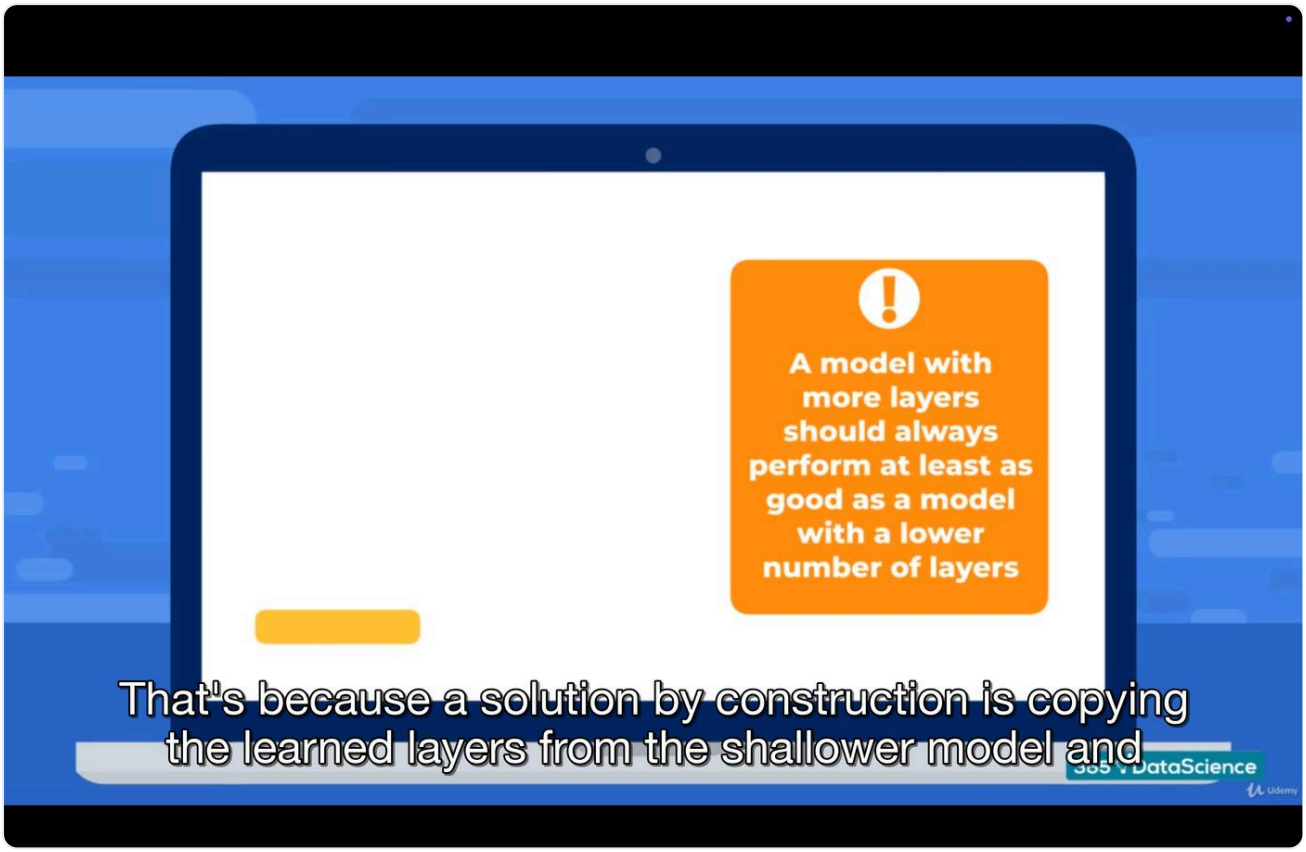


FIGURE 15.4

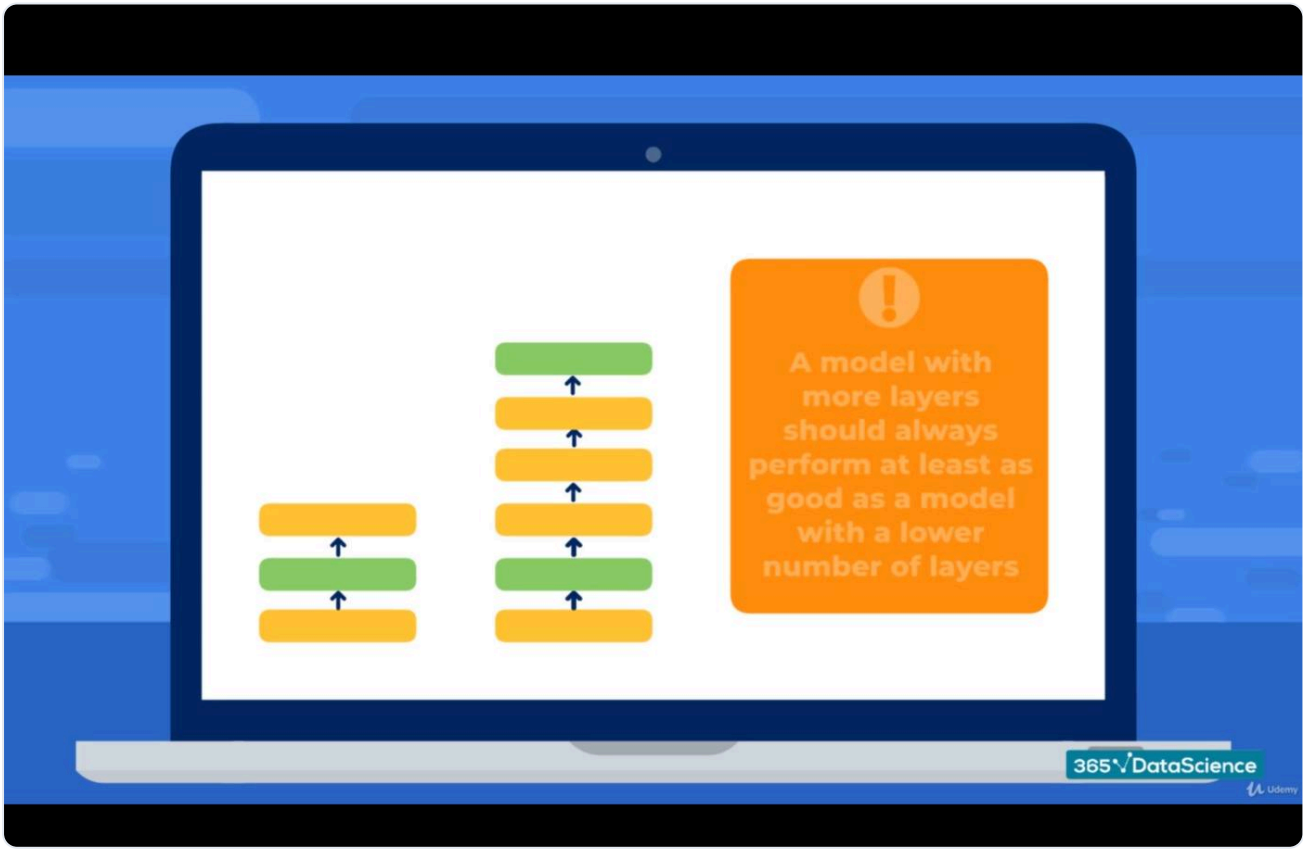


FIGURE 15.5

That's because a solution by construction is copying the learned layers from the shallower model and then setting additional layers to identity mapping.

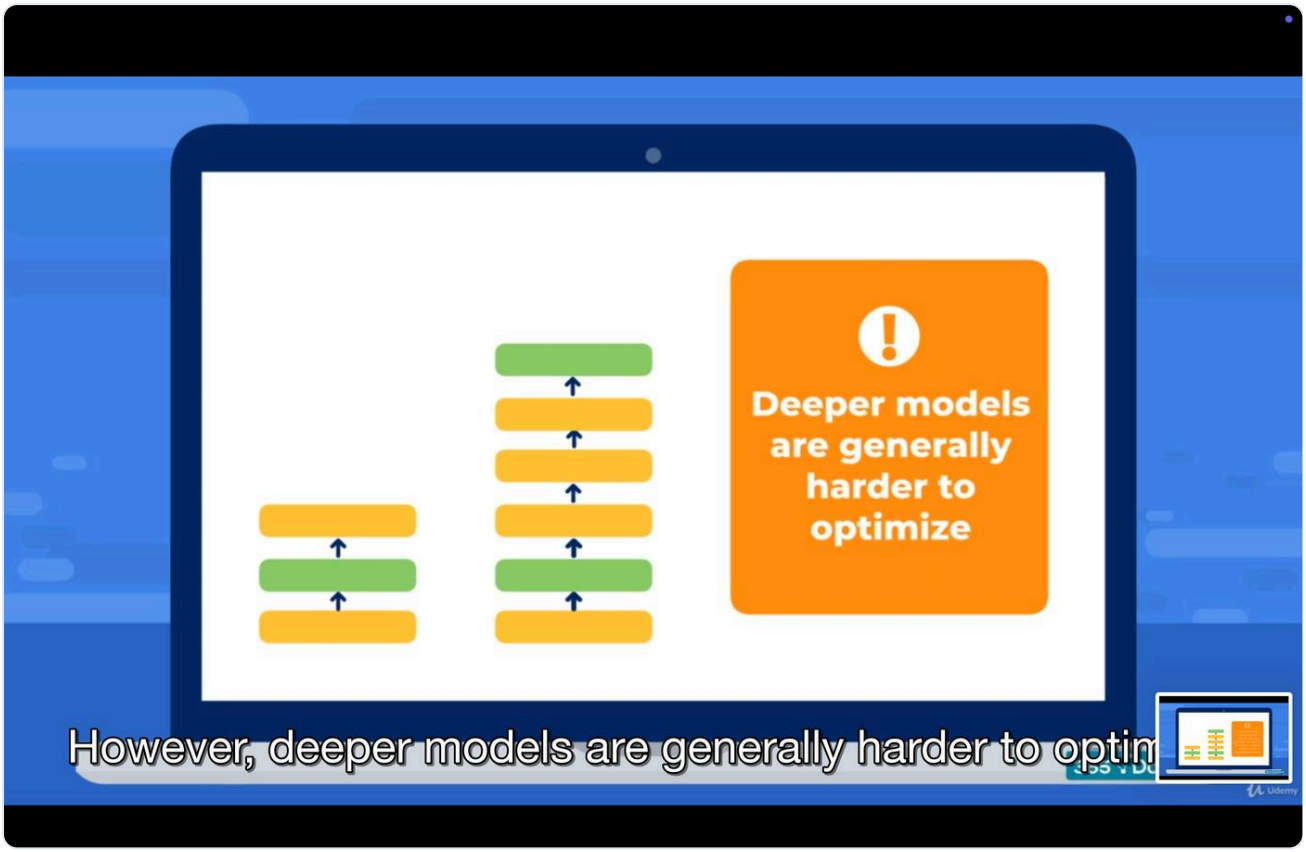


FIGURE 15.6

However, deeper models are generally harder to optimize. So how did the team behind ResNet managed to overcome this issue? Well, they design the network to learn what to change in the input instead of what the output should be.

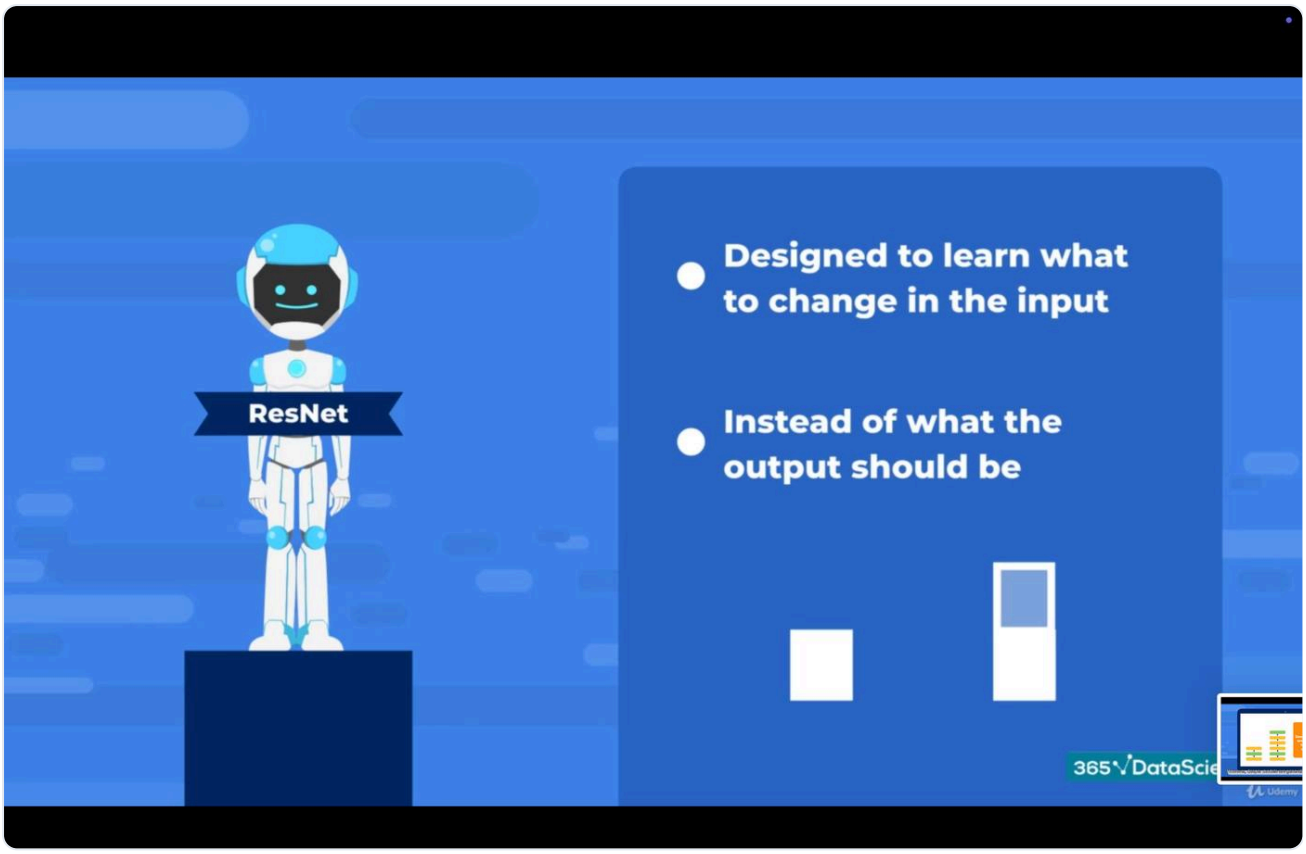


FIGURE 15.7

Let me illustrate this. Suppose we want to stack to convolutional layers in a normal model. We would just append them one after the other.

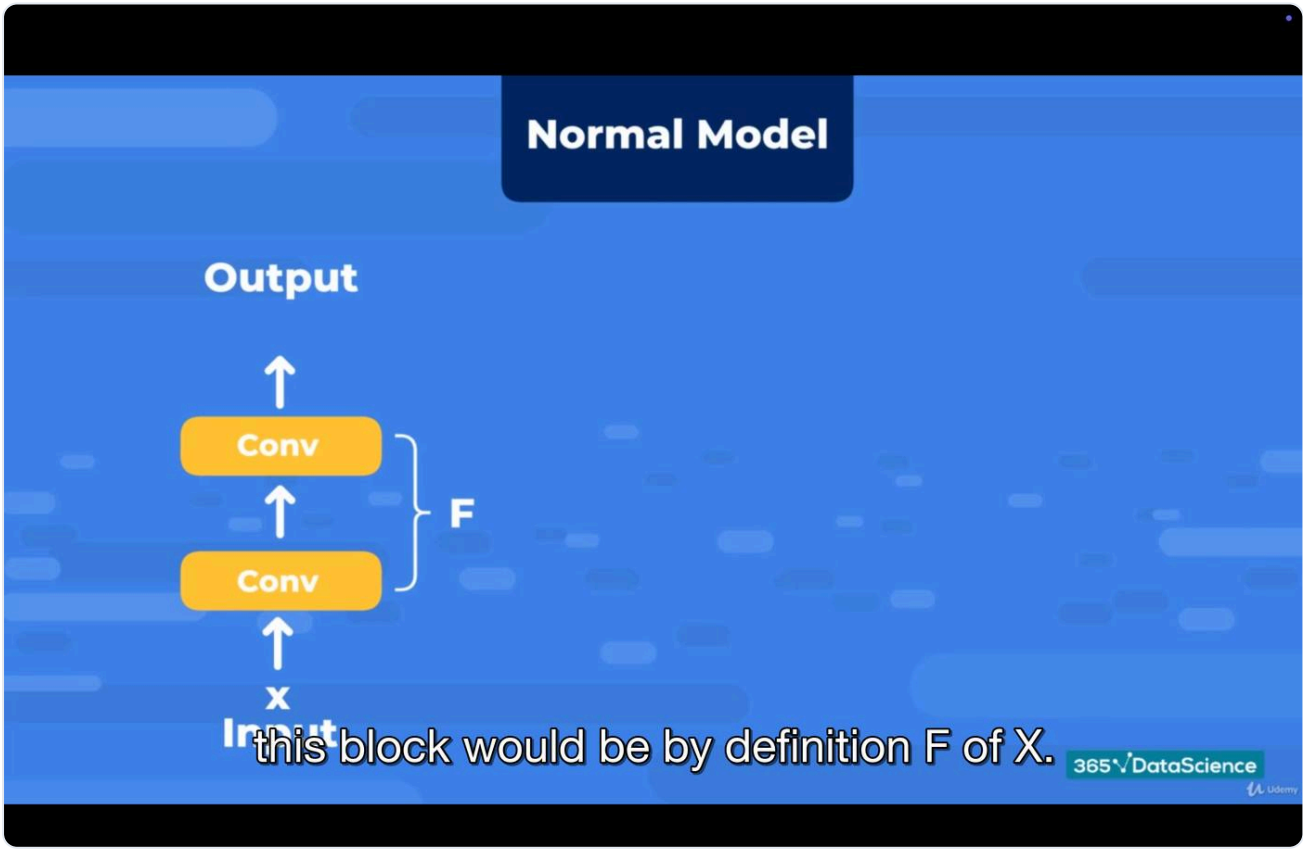


FIGURE 15.8 | Normal Model

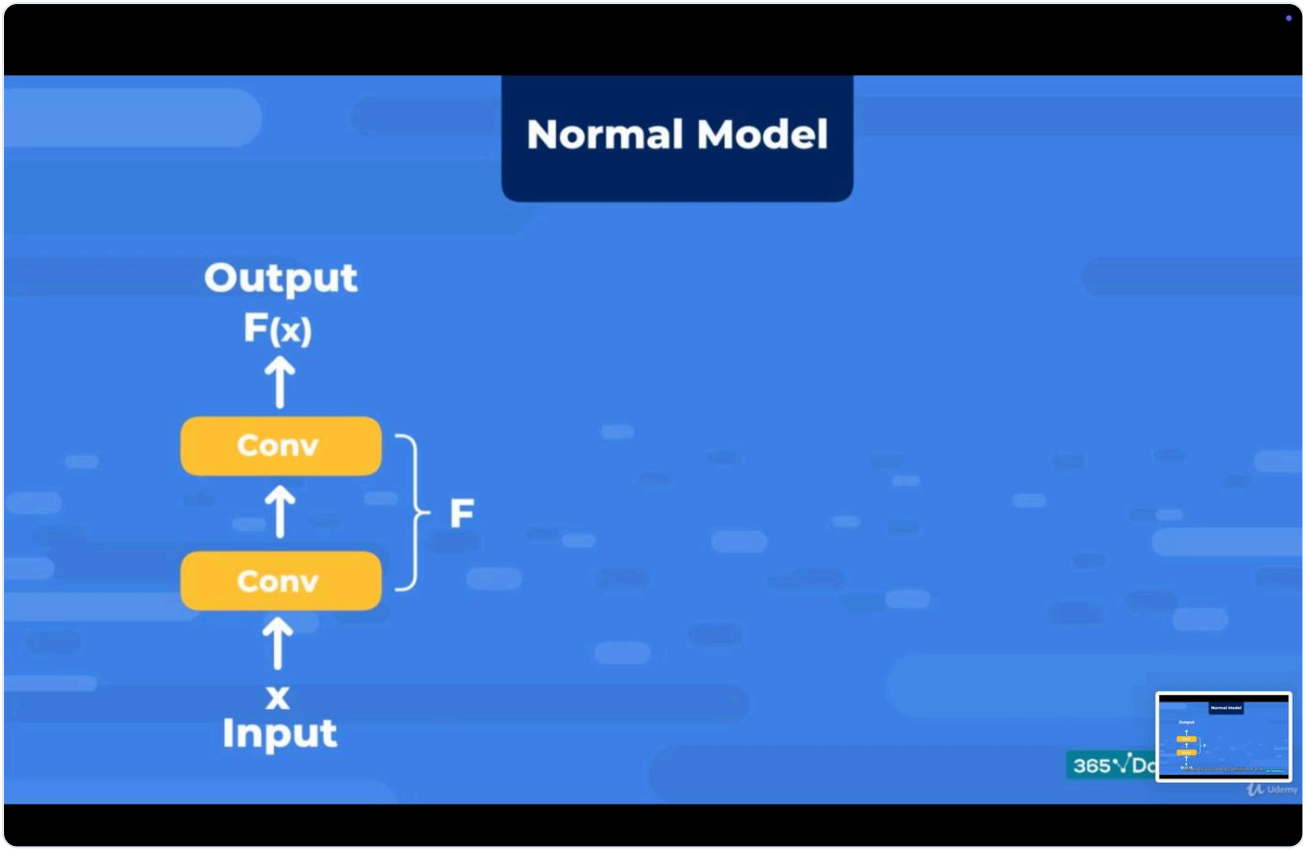


FIGURE 15.9 | Normal Model

So if we have an input X and the two layers transform that input with the function F, the output of this block would be by definition F of X.

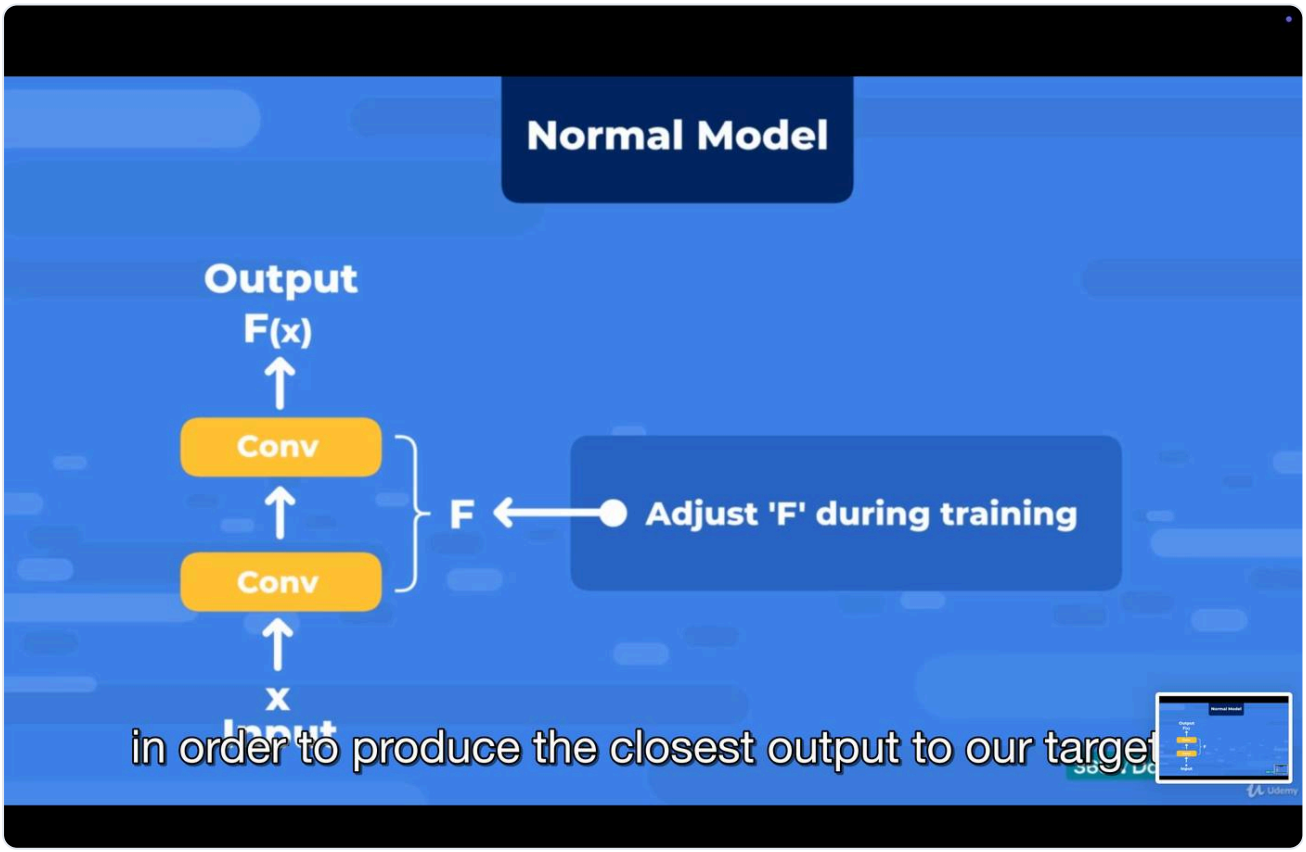


FIGURE 15.10 | Normal Model

The job of the network during training is to adjust this function F through the weights in the layers in order to produce the closest output to our target. Thus the network is trying to learn f, which is the desired output.

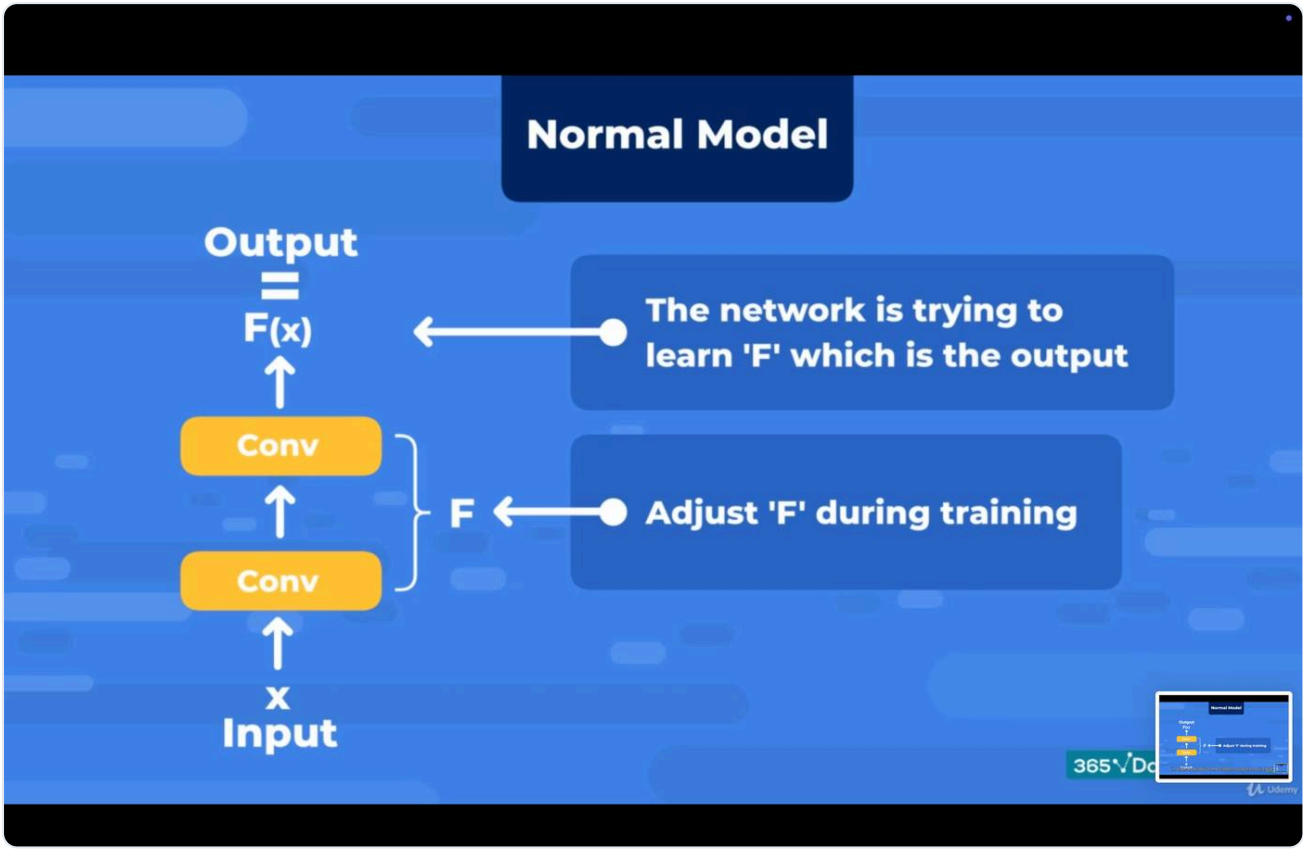


FIGURE 15.11 | Normal Model

Now let's see how this was implemented in ResNet once more. Here are the two convolutional layers, one after the other in Resonant.

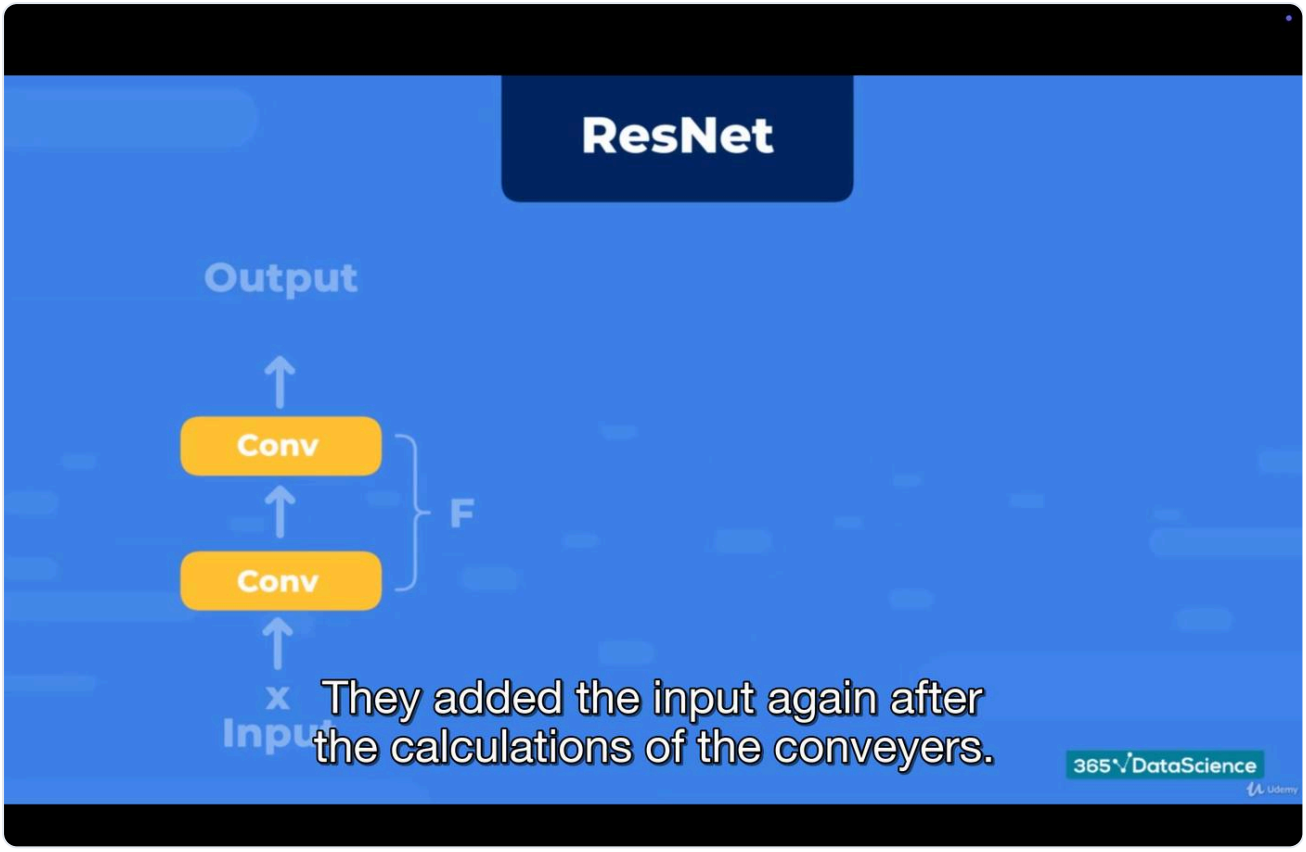


FIGURE 15.12

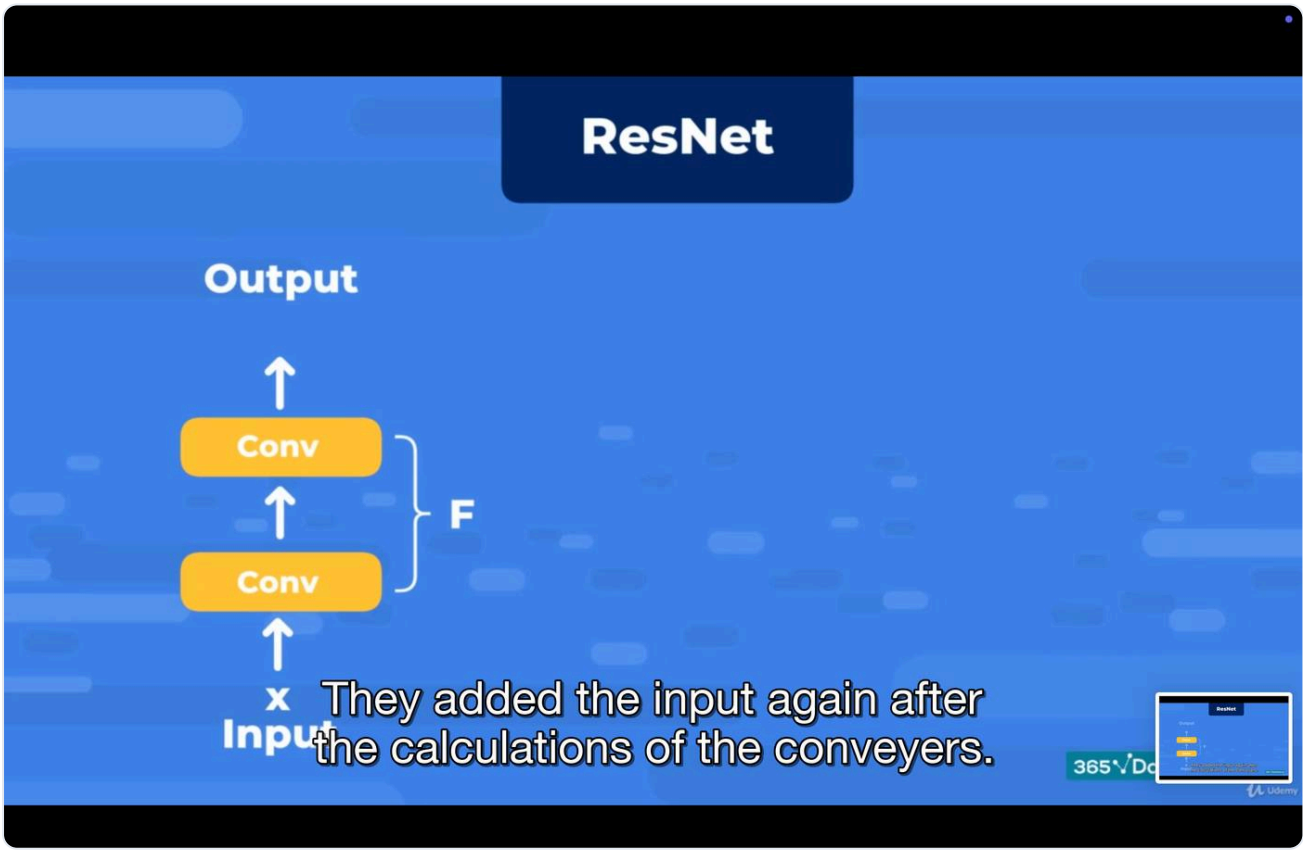


FIGURE 15.13

They added the input again after the calculations of the conv layers.

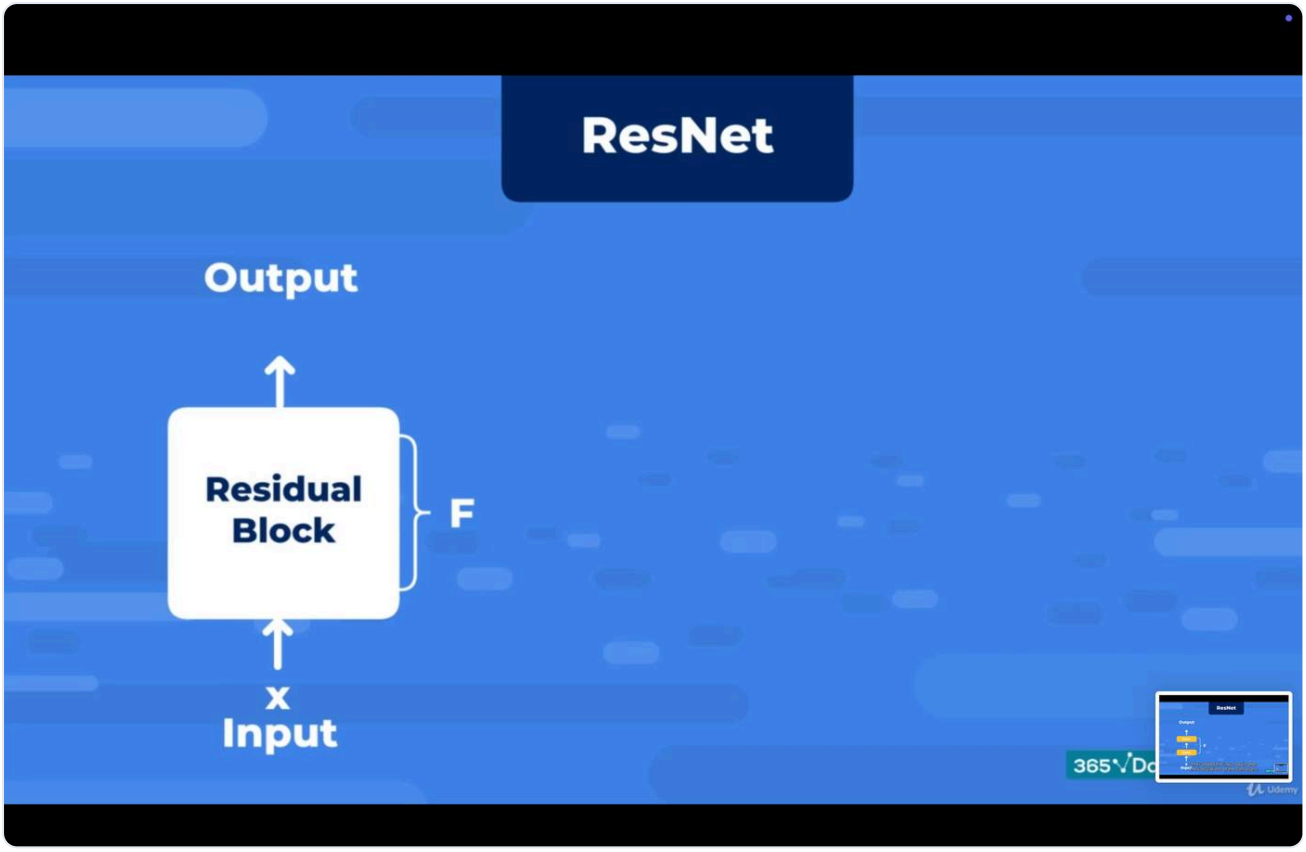


FIGURE 15.14

They called this a residual block.

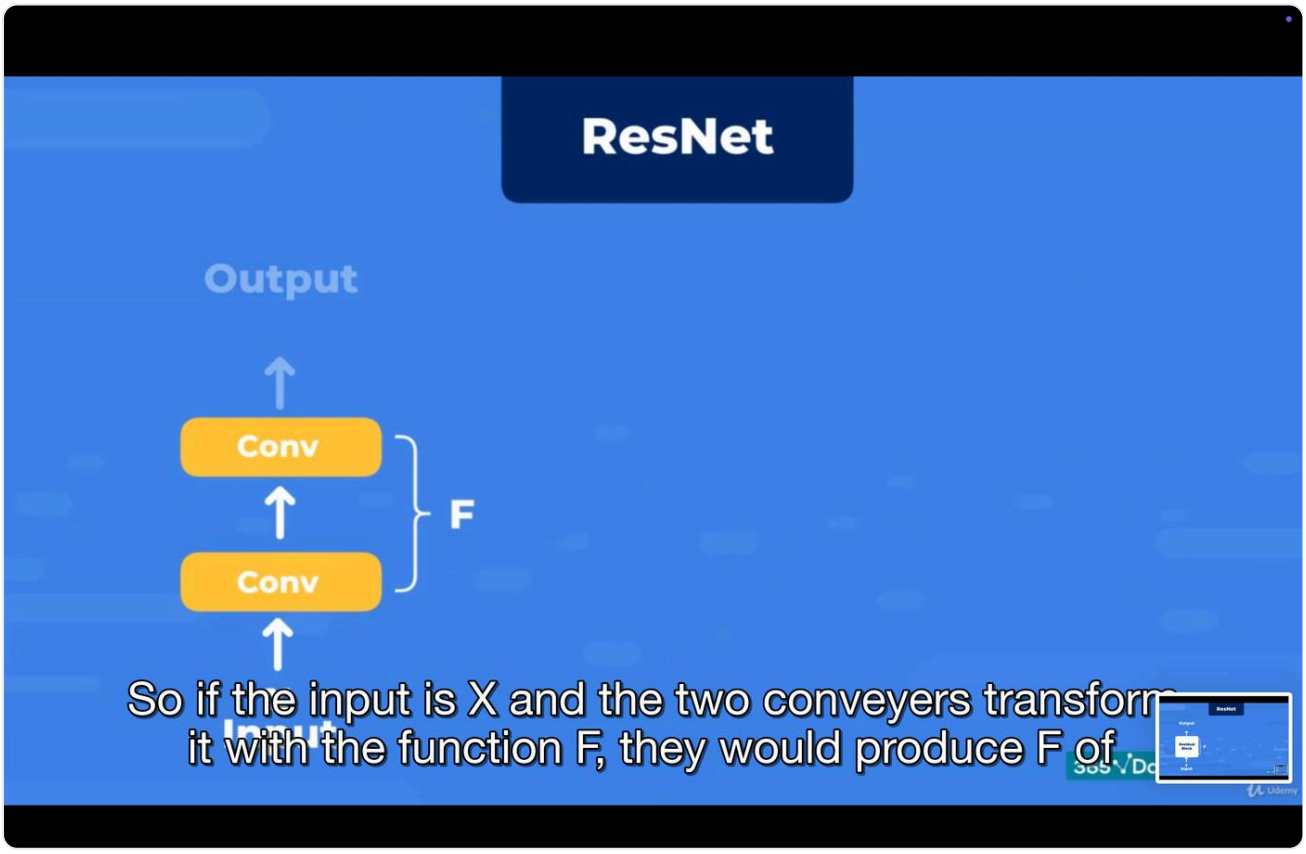


FIGURE 15.15

So if the input is X and the two conv layers transform it with the function F, they would produce F of X.

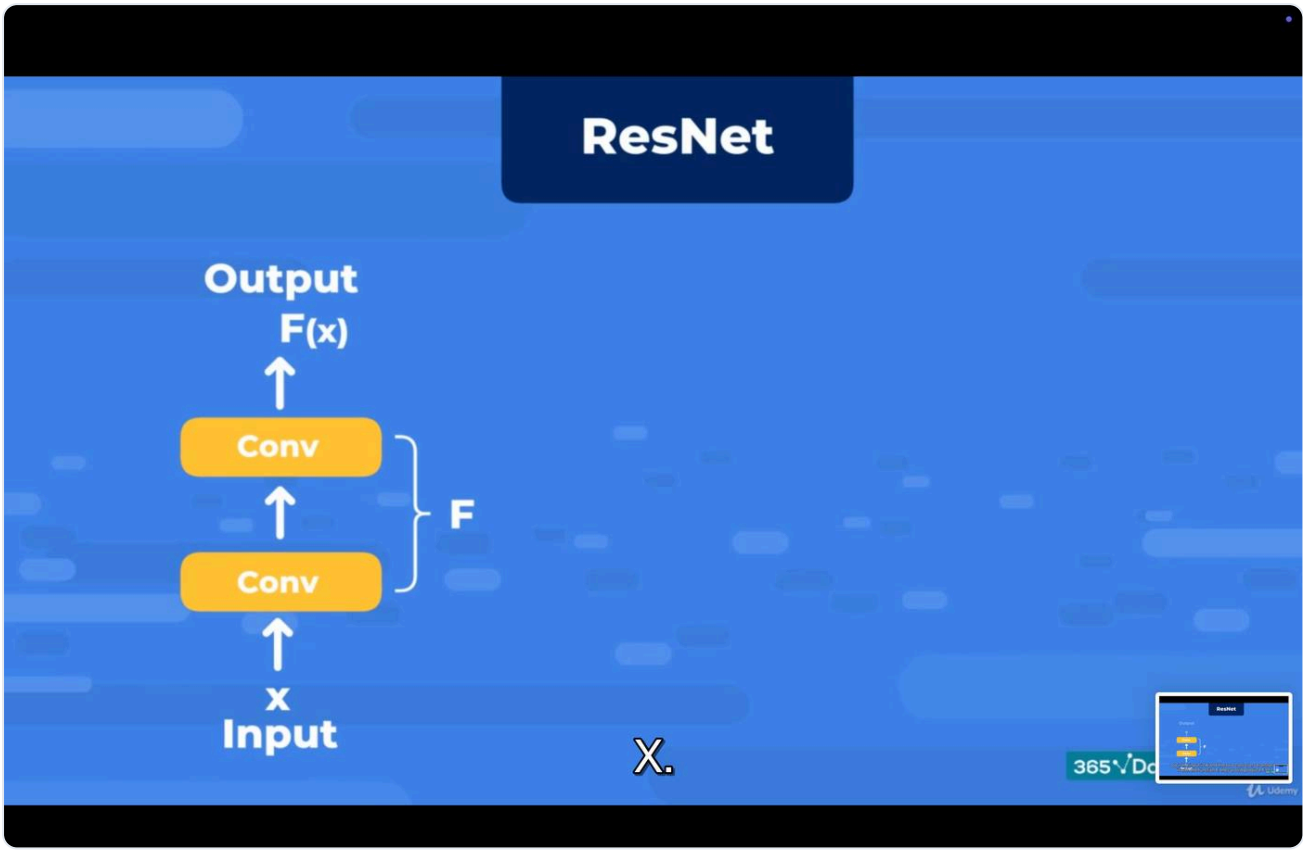


FIGURE 15.16

But that is not the final output of this block to produce the final output.

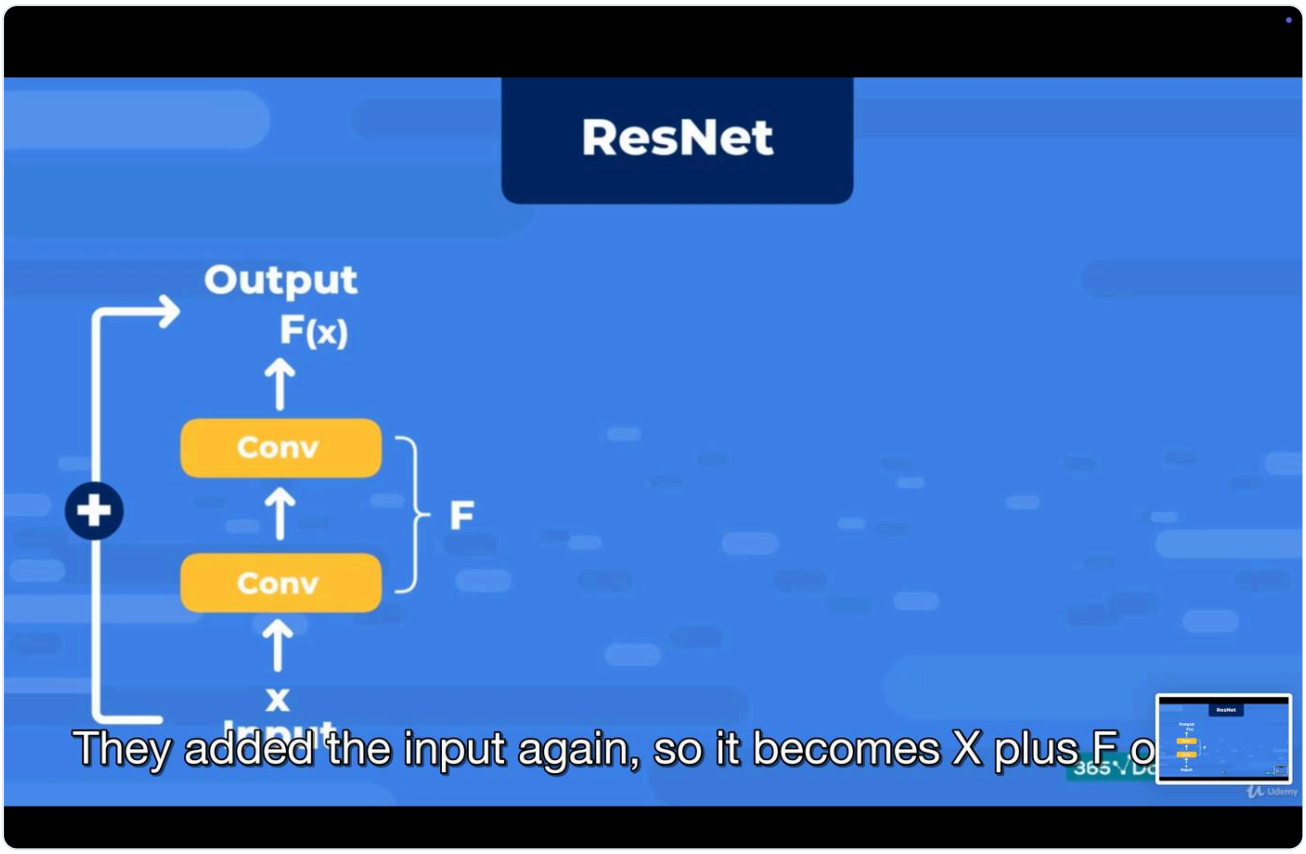


FIGURE 15.17

They added the input again, so it becomes X plus F of X.. So what does this mean?

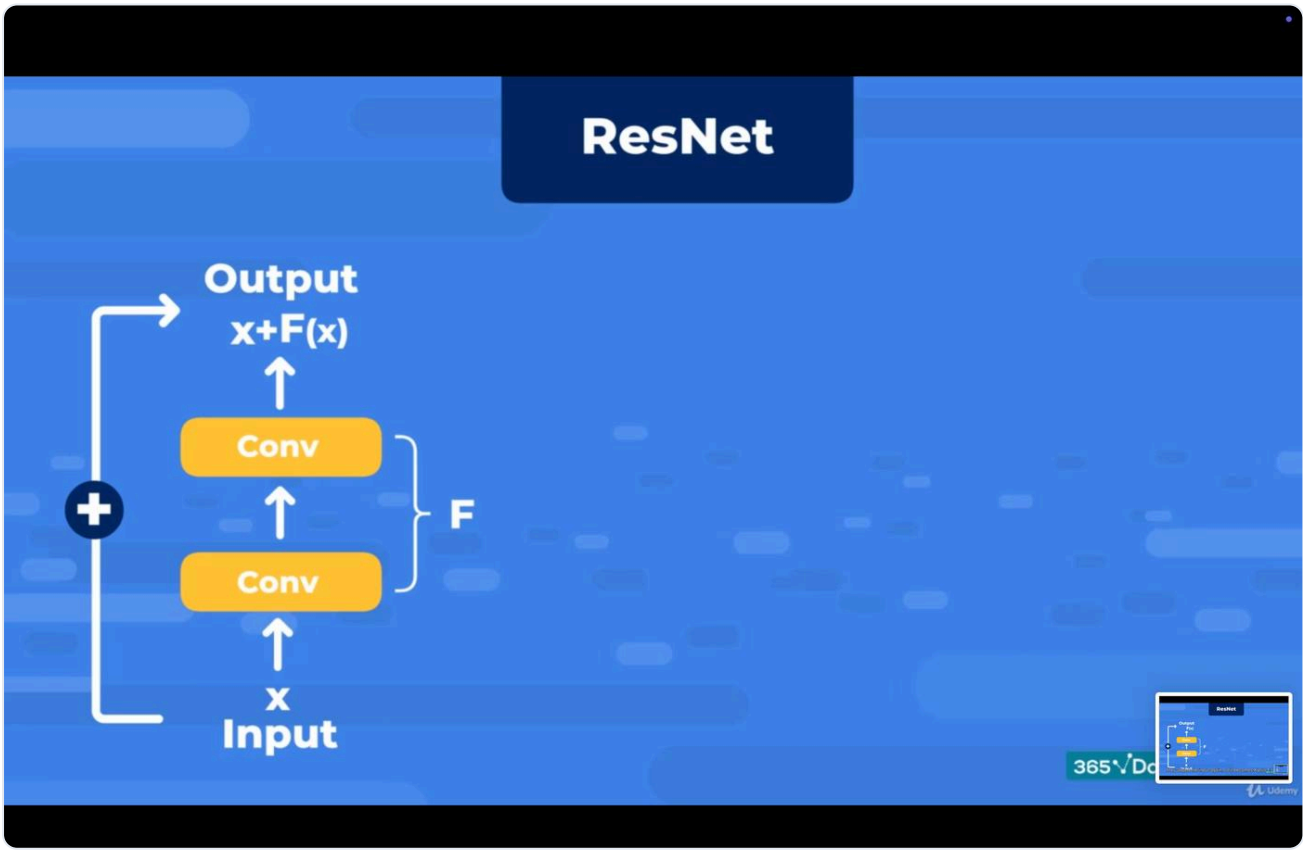


FIGURE 15.18

Well, during the training process, the model is trying to learn what F should be.

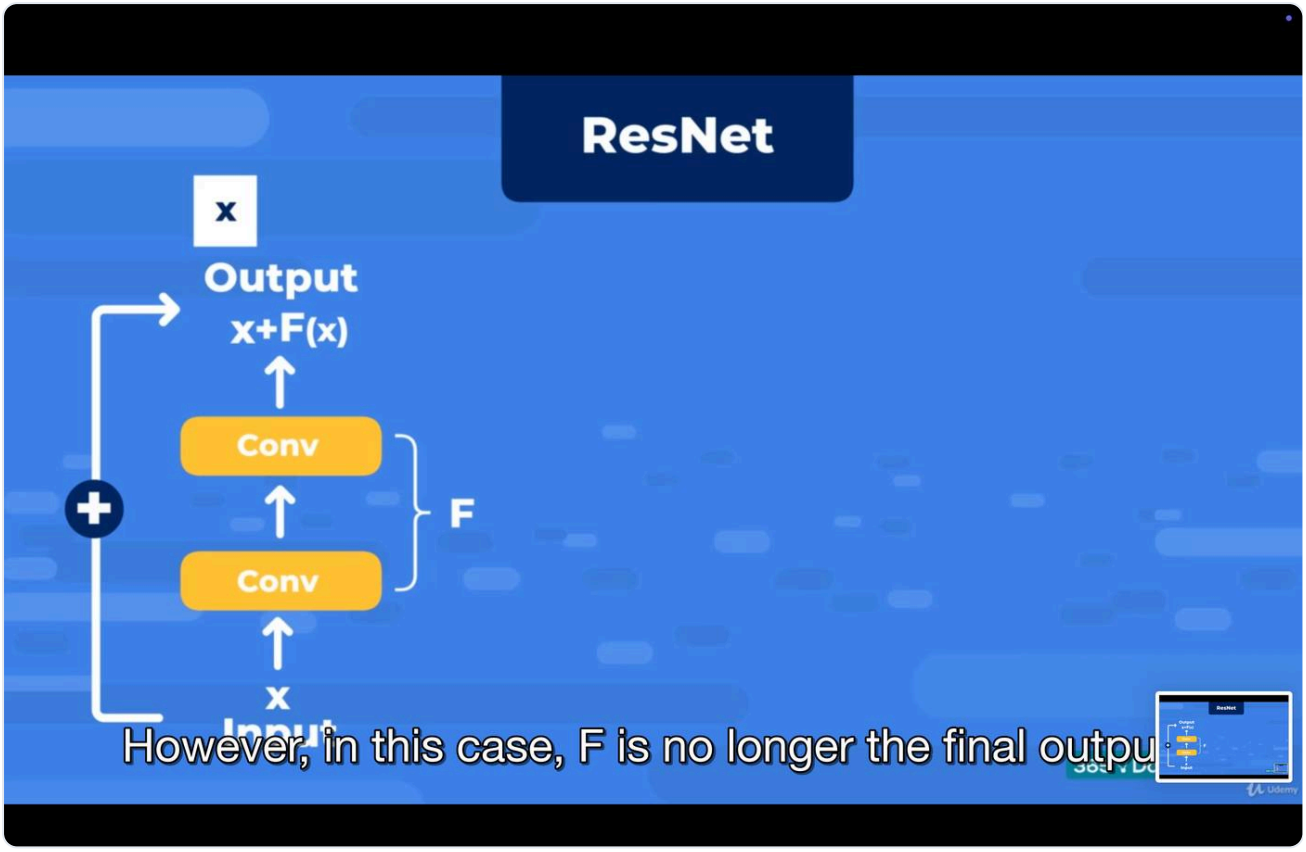


FIGURE 15.19

However, in this case, F is no longer the final output. This is just the change we need to make to our input in order to get the desired output.

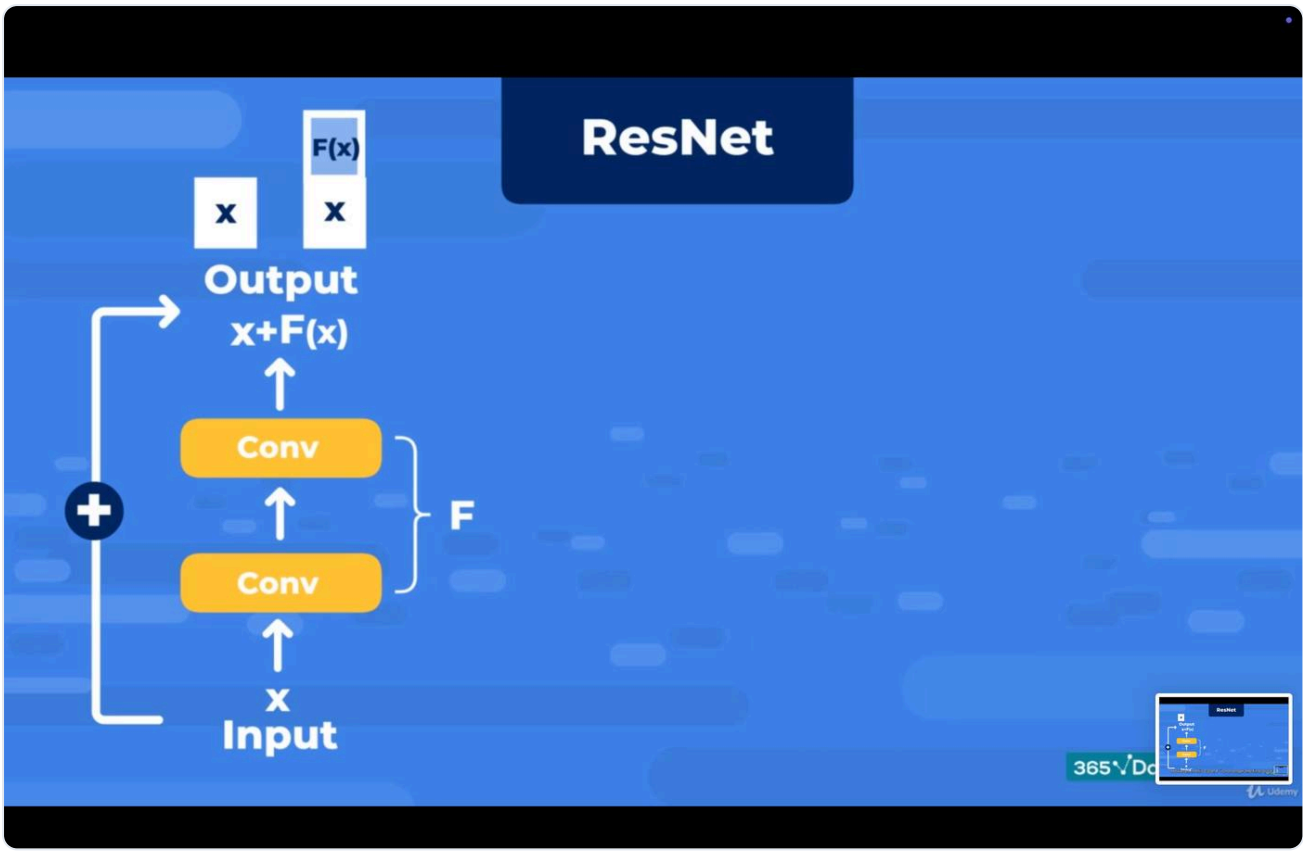


FIGURE 15.20

The difference between the two approaches is subtle to make an analogy here.

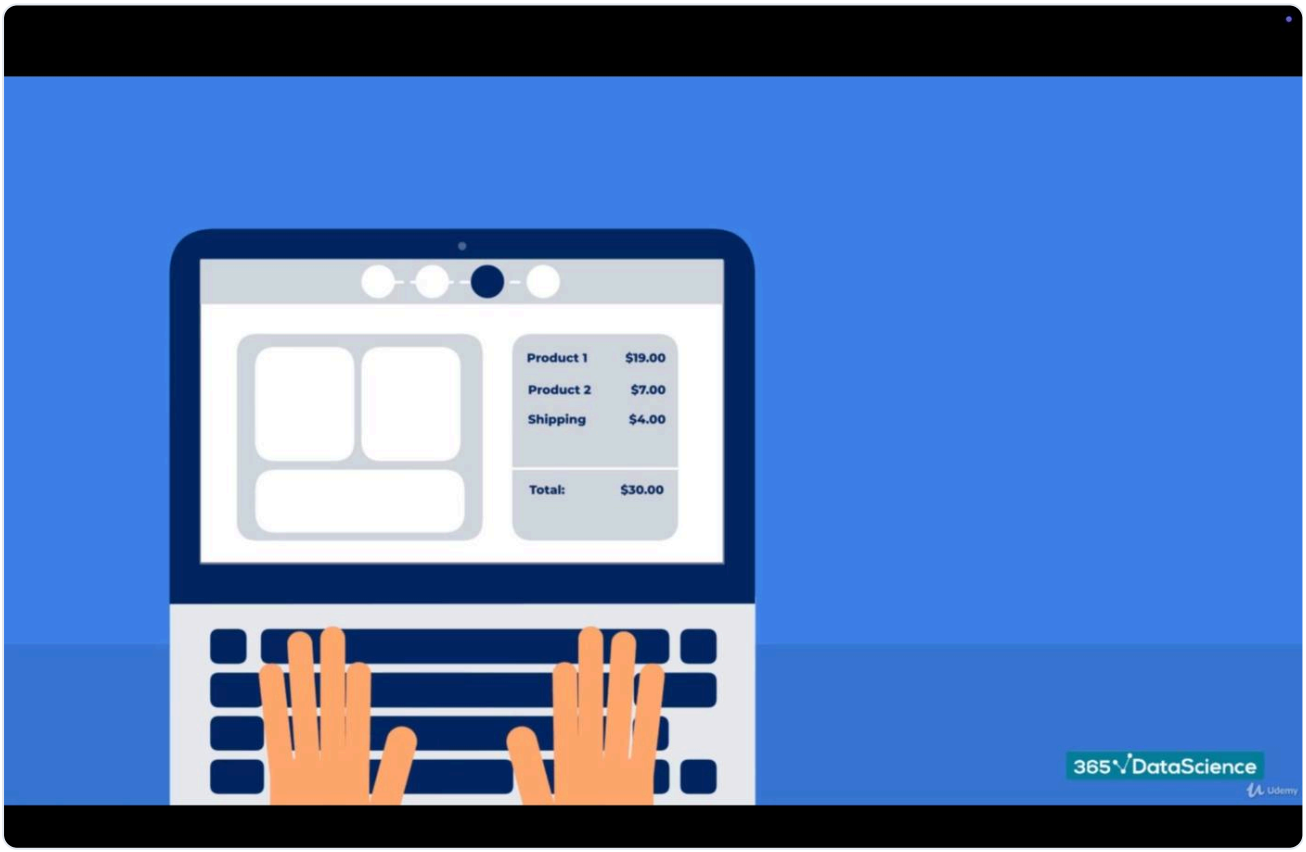


FIGURE 15.21

Imagine you are ordering something online and the subtotal is thirty dollars.

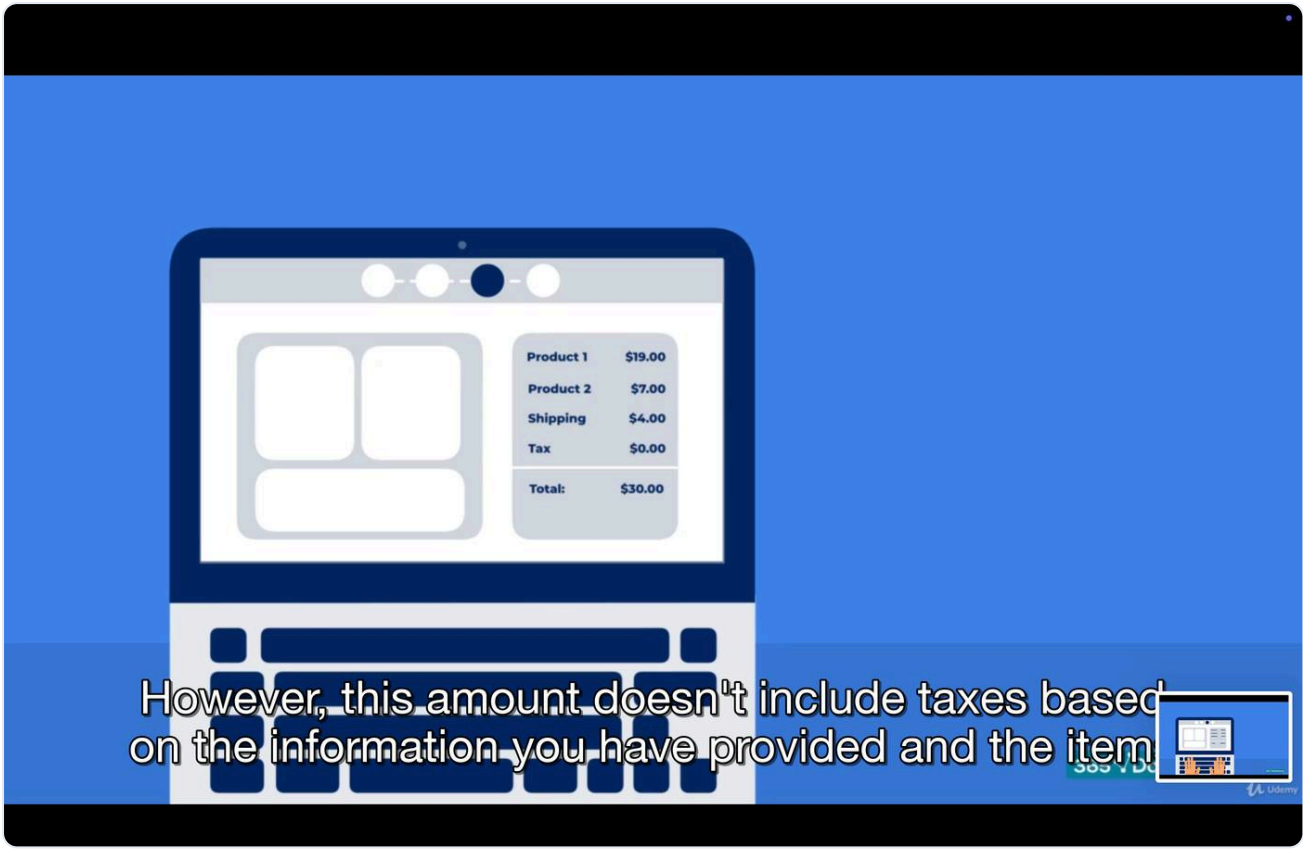


FIGURE 15.22

However, this amount doesn't include taxes based on the information you have provided and the items you've added. The website needs to calculate the respective tax rate and the new price.

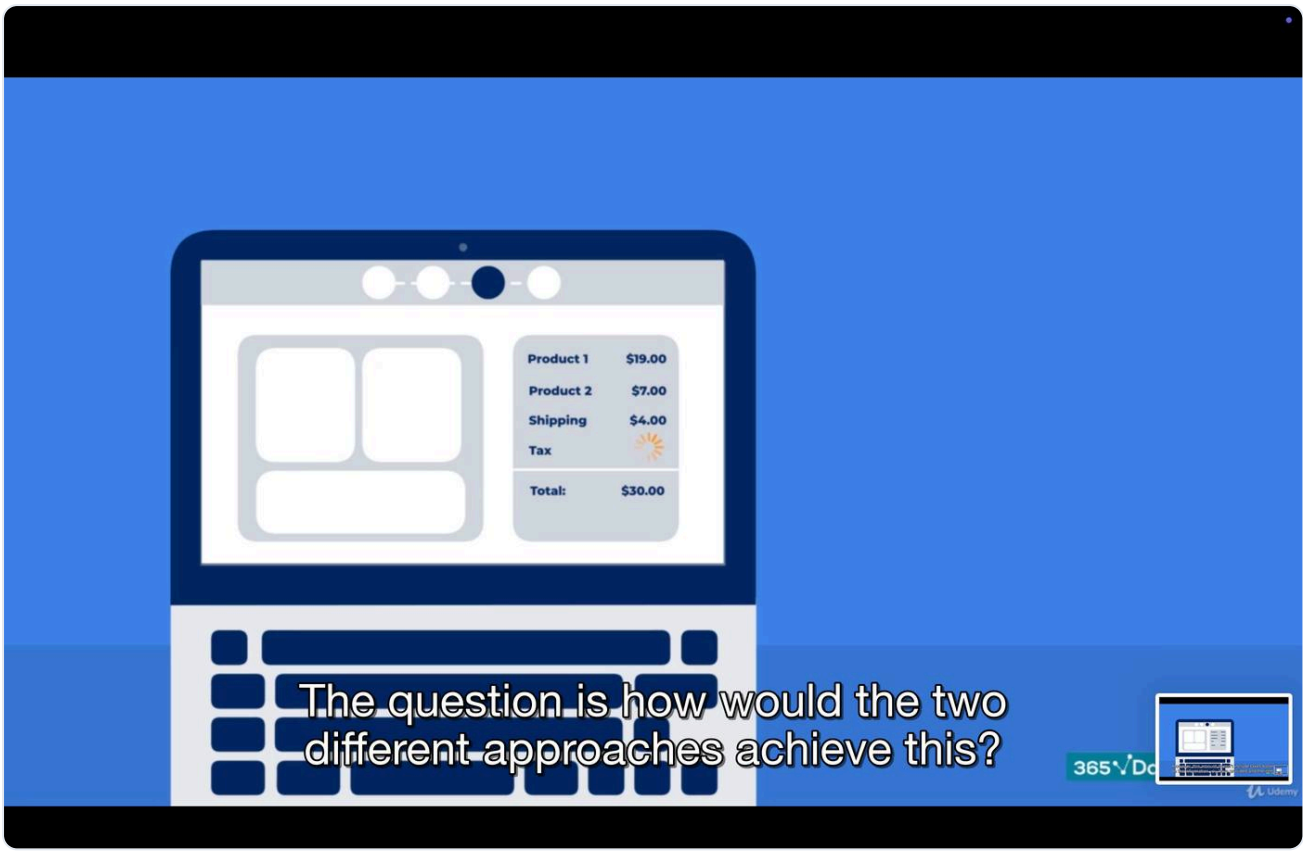


FIGURE 15.23

The question is how would the two different approaches achieve this?

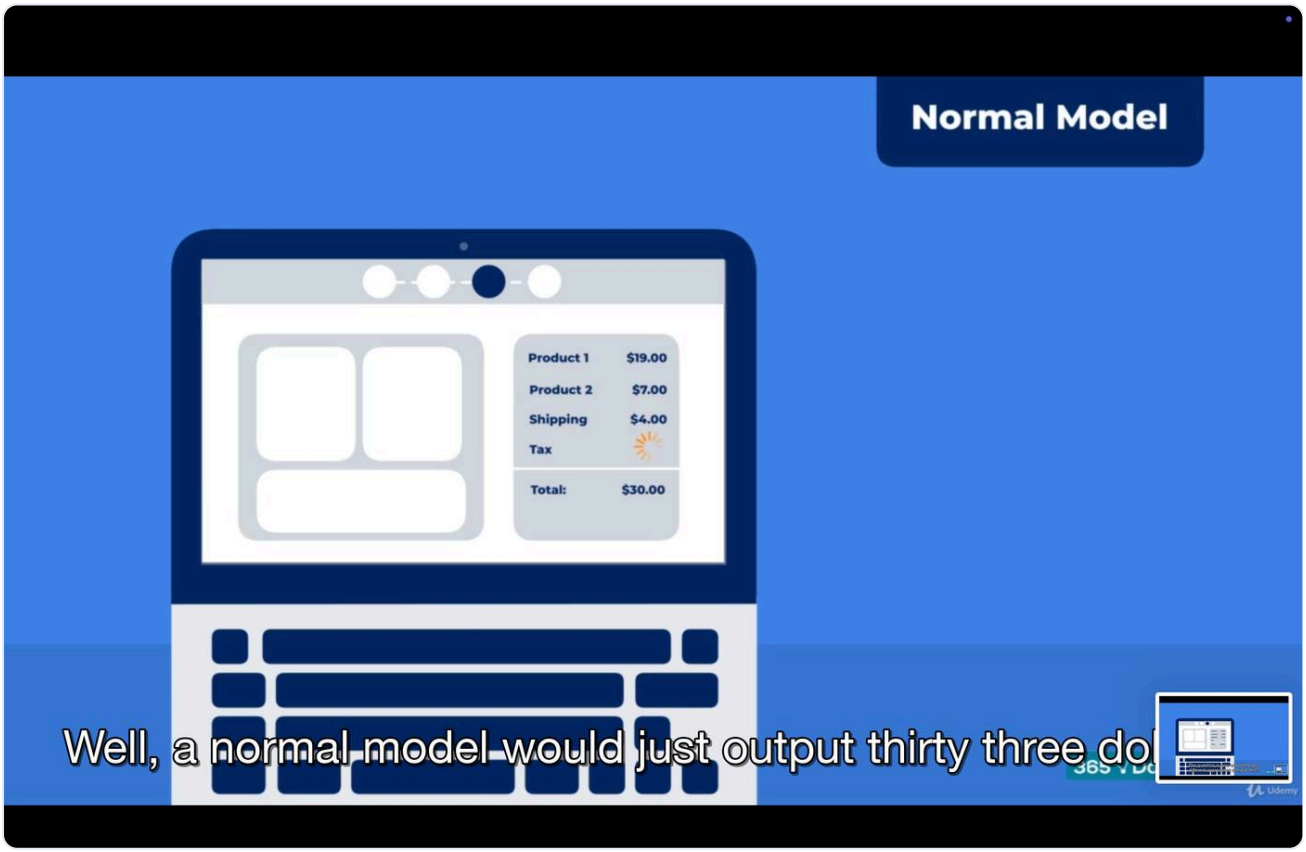


FIGURE 15.24 | Normal Model

Well, a normal model would just output thirty three dollars.

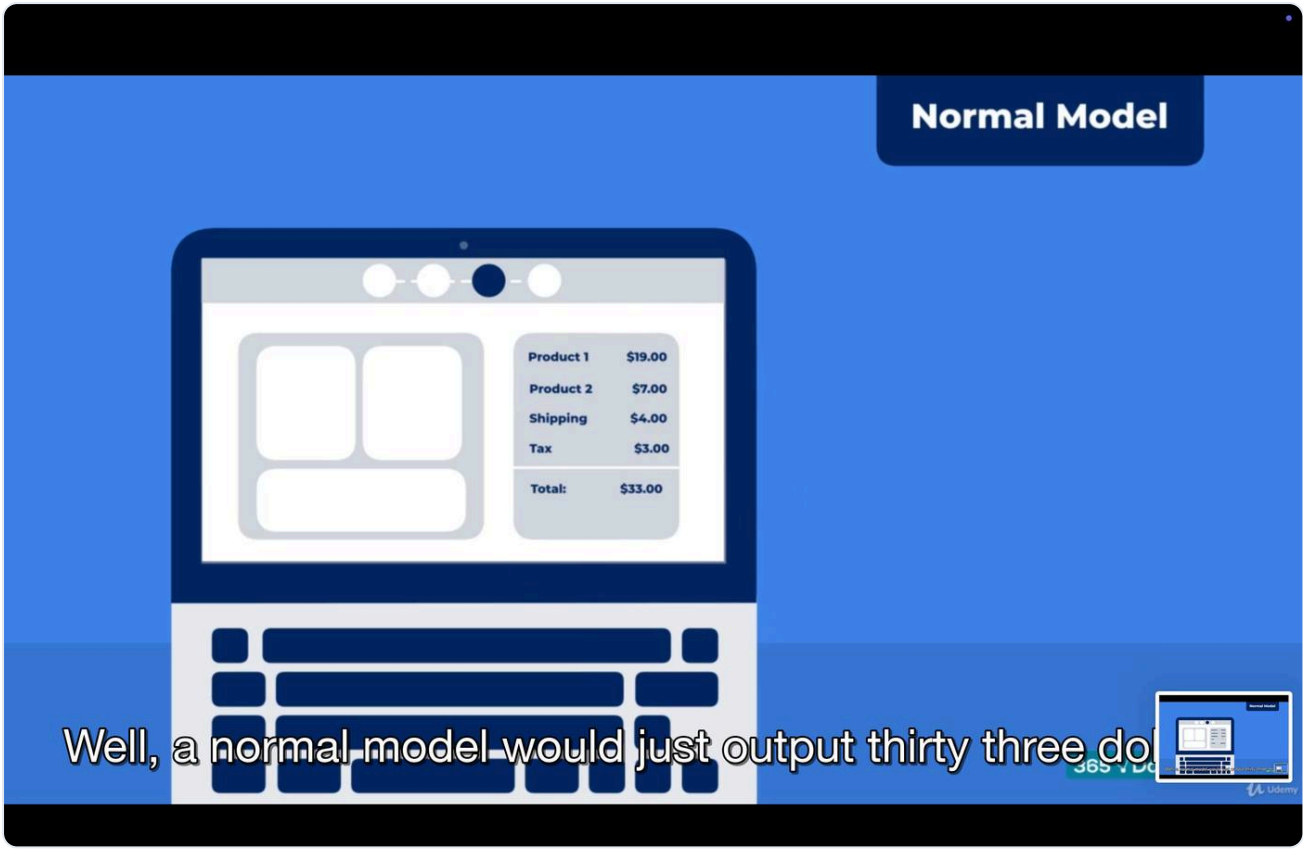


FIGURE 15.25 | Normal Model

That's the final price.

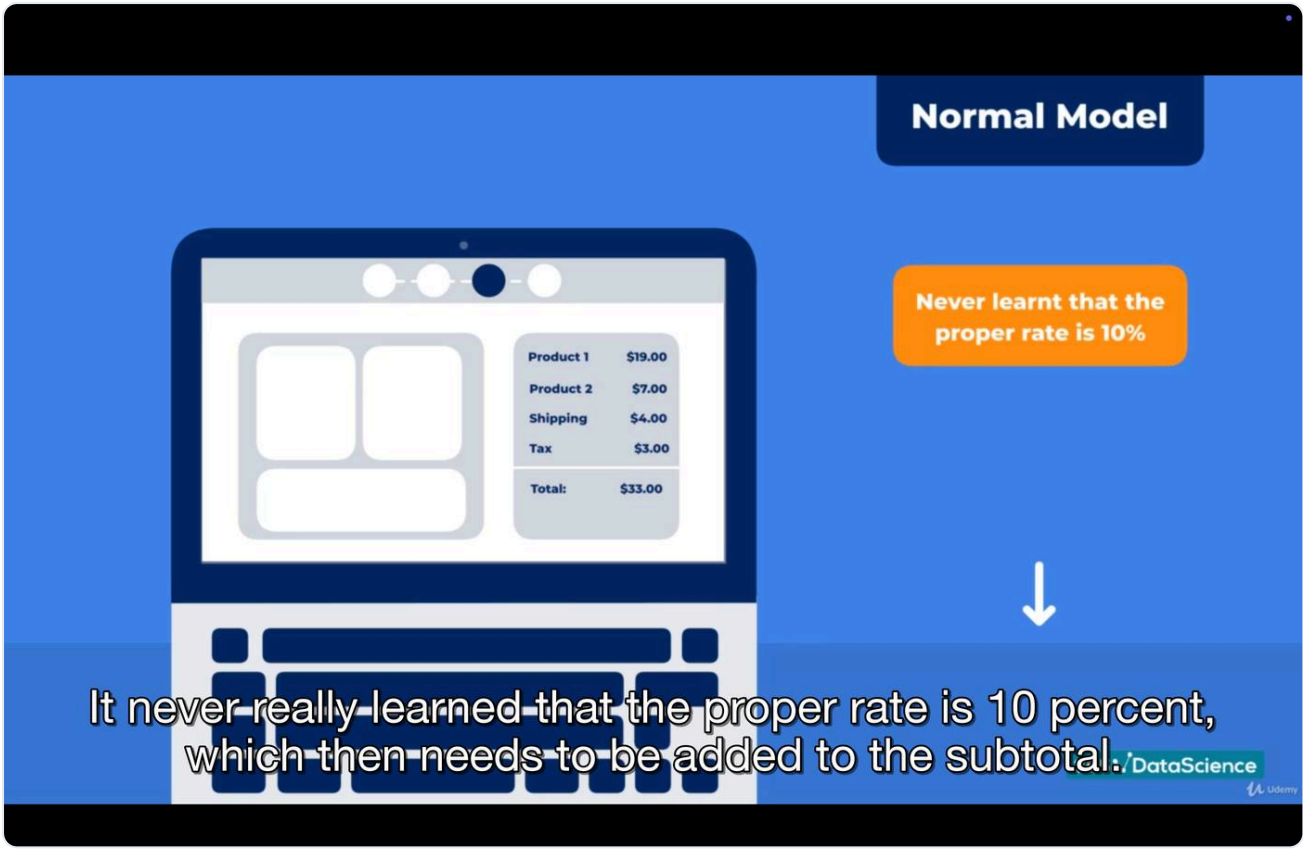


FIGURE 15.26 | Normal Model

It never really learned that the proper rate is 10 percent, which then needs to be added to the subtotal.

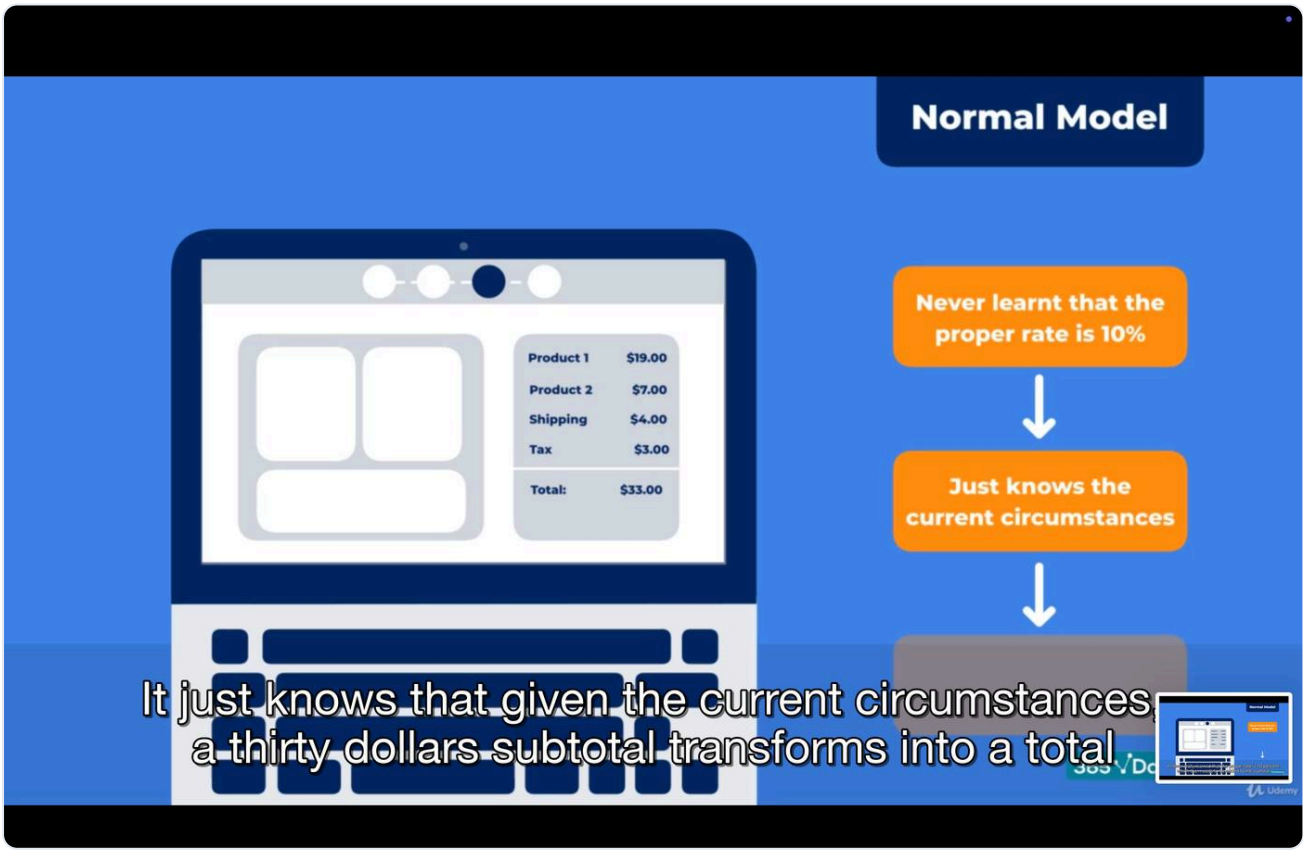


FIGURE 15.27 | Normal Model

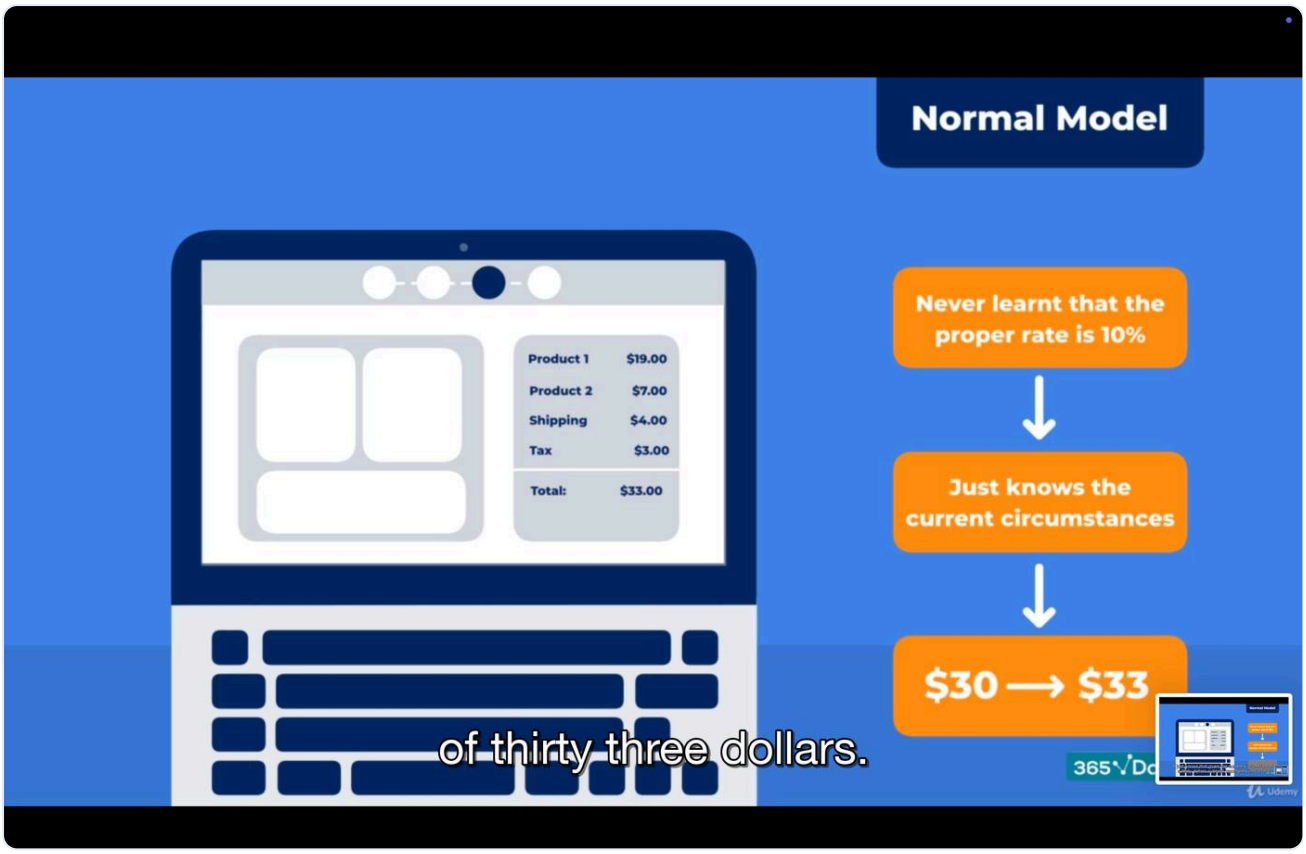


FIGURE 15.28 | Normal Model

It just knows that given the current circumstances, a thirty dollars subtotal transforms into a total of thirty three dollars.

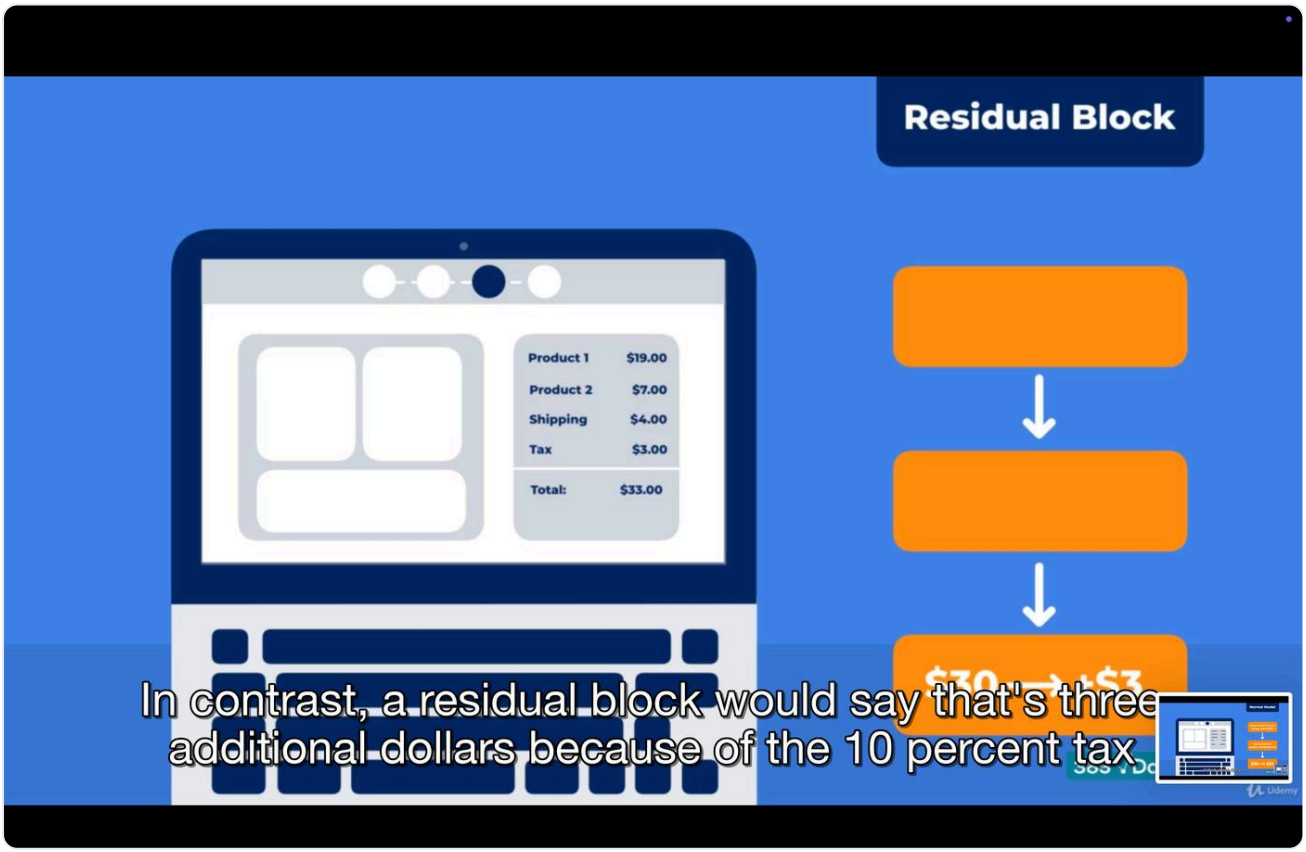


FIGURE 15.29 | Residual Block

In contrast, a residual block would say that's three additional dollars because of the 10 percent tax rate.



FIGURE 15.30 | Residual Block



FIGURE 15.31 | Residual Block

Basically, it cares about how things change, not so much about what they transform into. This works better when generalizing.

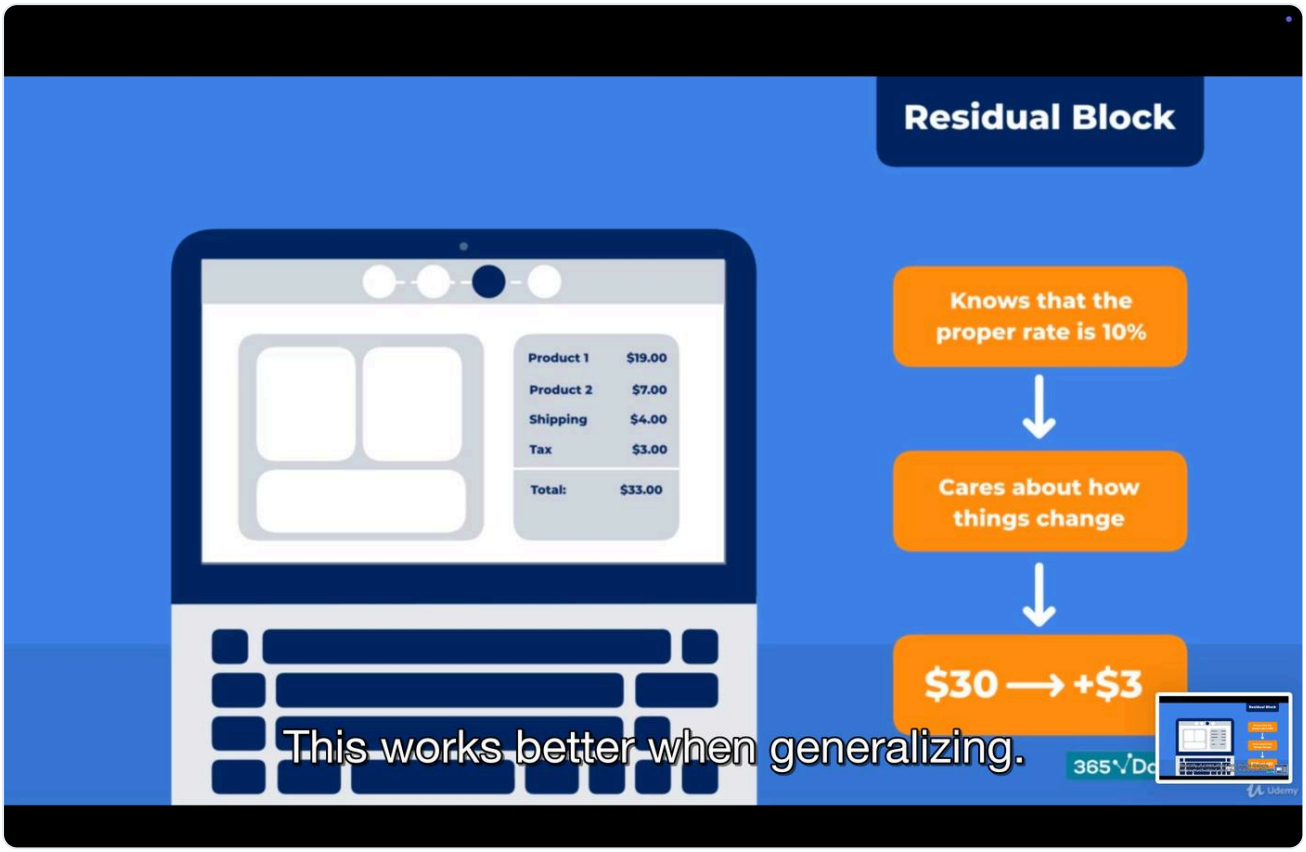


FIGURE 15.32 | Residual Block

Why so? Well, it is easier to learn that a person from a particular country would have 10 percent taxes than it is to learn that if the initial price was thirty, the final one would be thirty three. Makes sense.

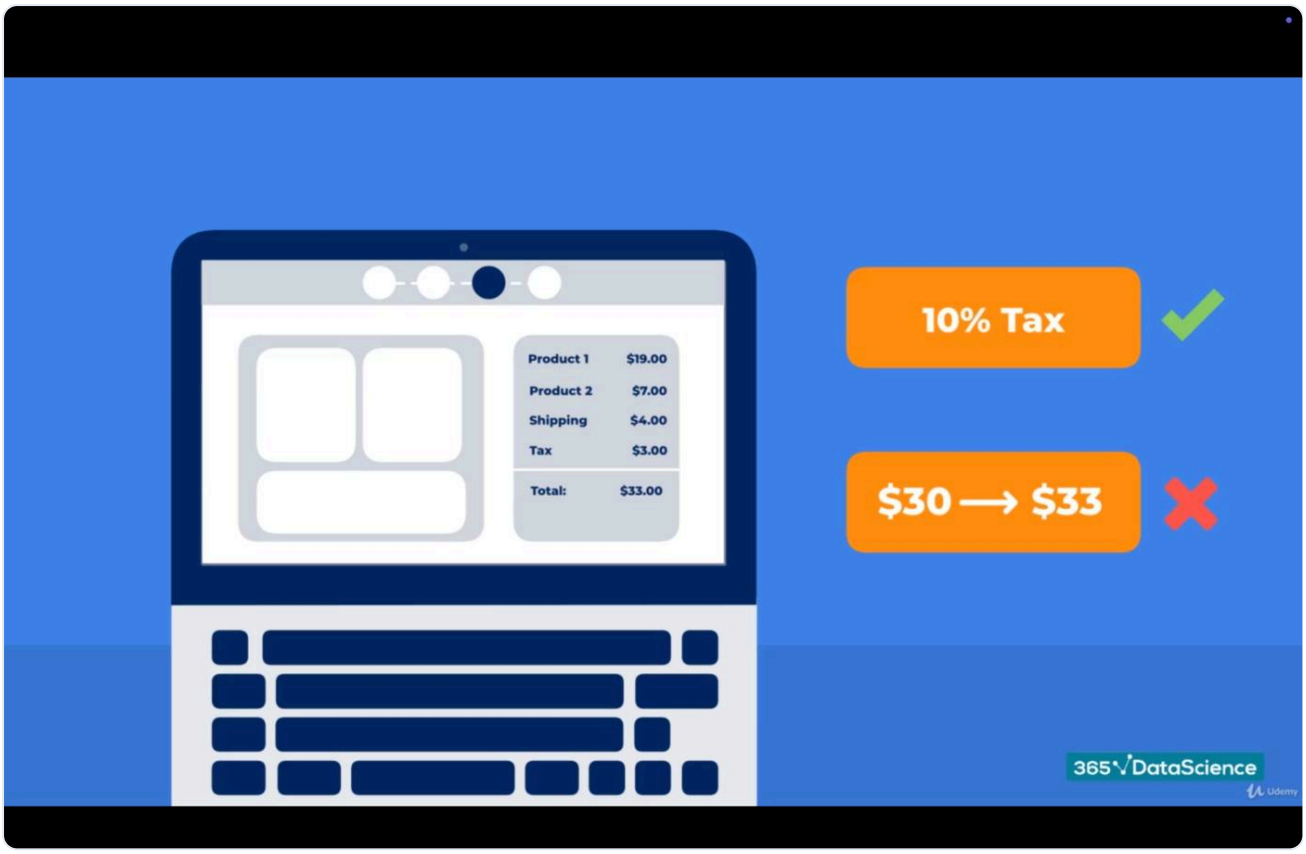


FIGURE 15.33

This is obviously an oversimplification of the whole process, but you get the idea in practice. Normal models don't learn in exactly the same way.

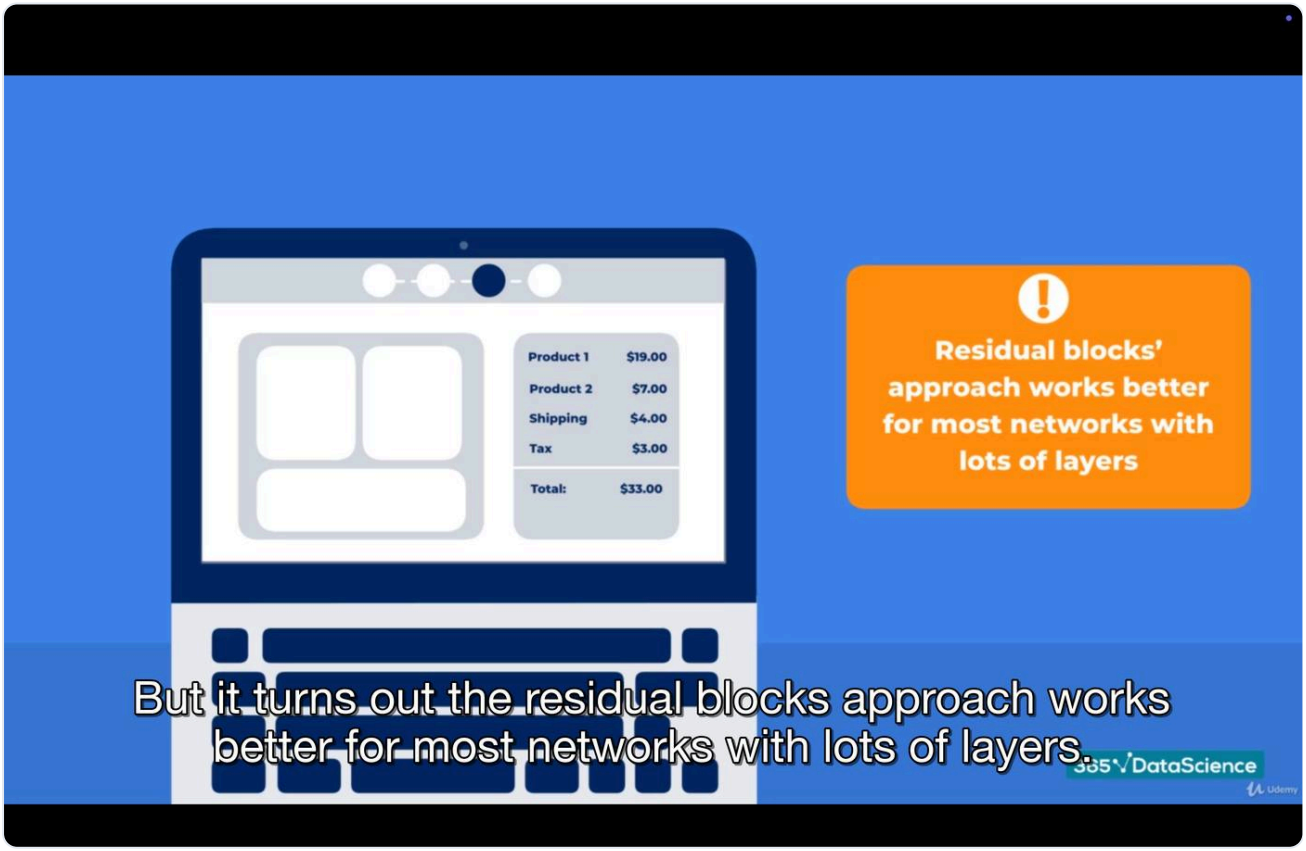


FIGURE 15.34

But it turns out the residual blocks approach works better for most networks with lots of layers. All right. Going back to Resonant, its architecture consisted of many residual blocks stacked on top of each other.

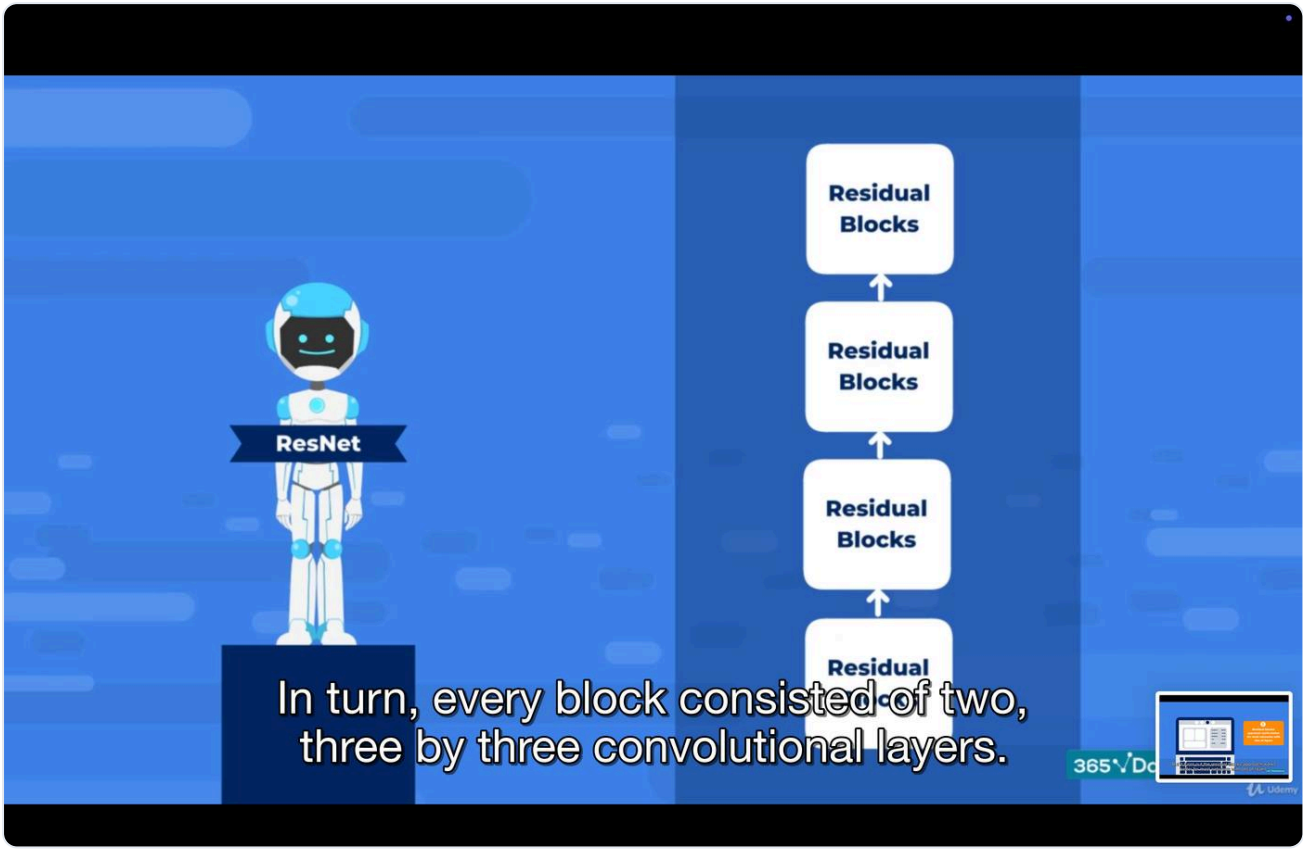


FIGURE 15.35

In turn, every block consisted of two, three by three convolutional layers.

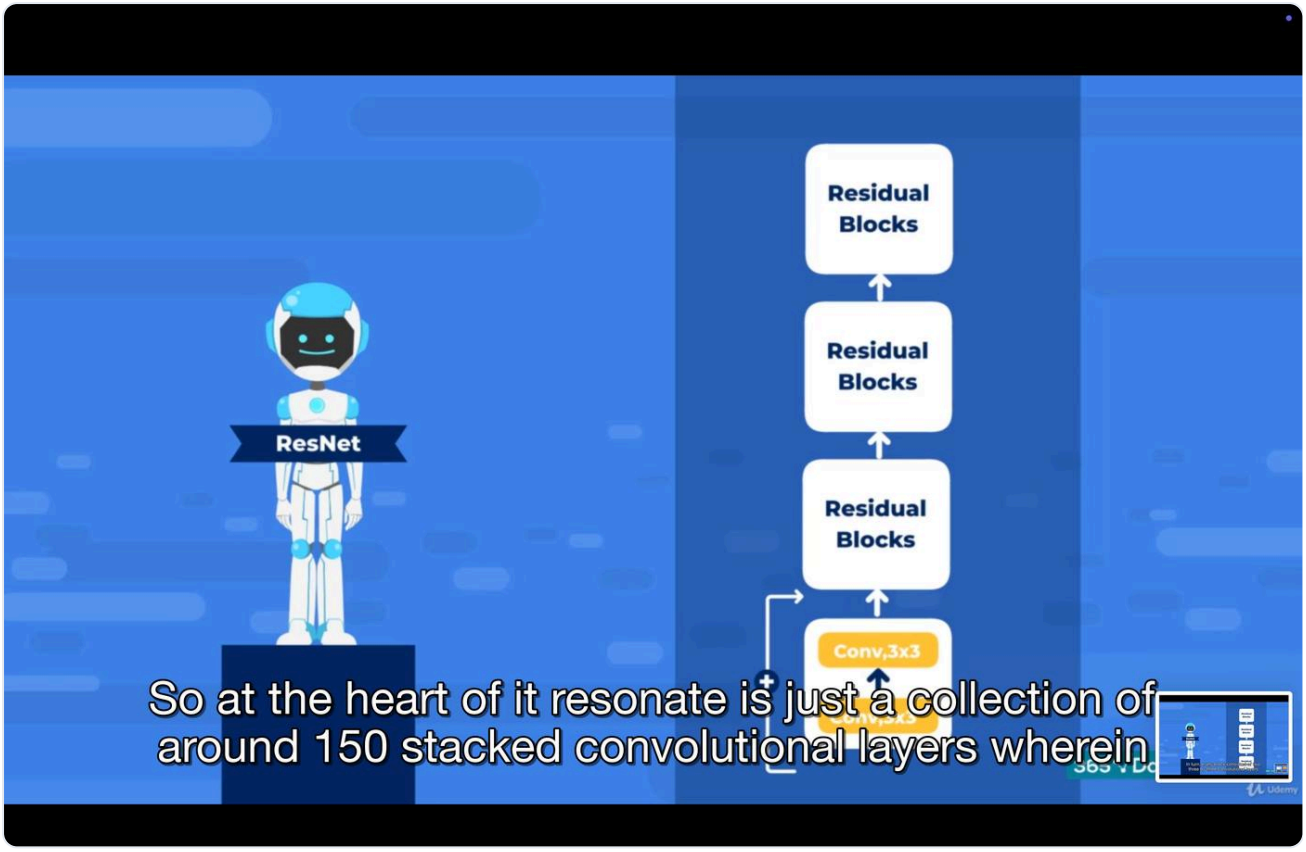


FIGURE 15.36

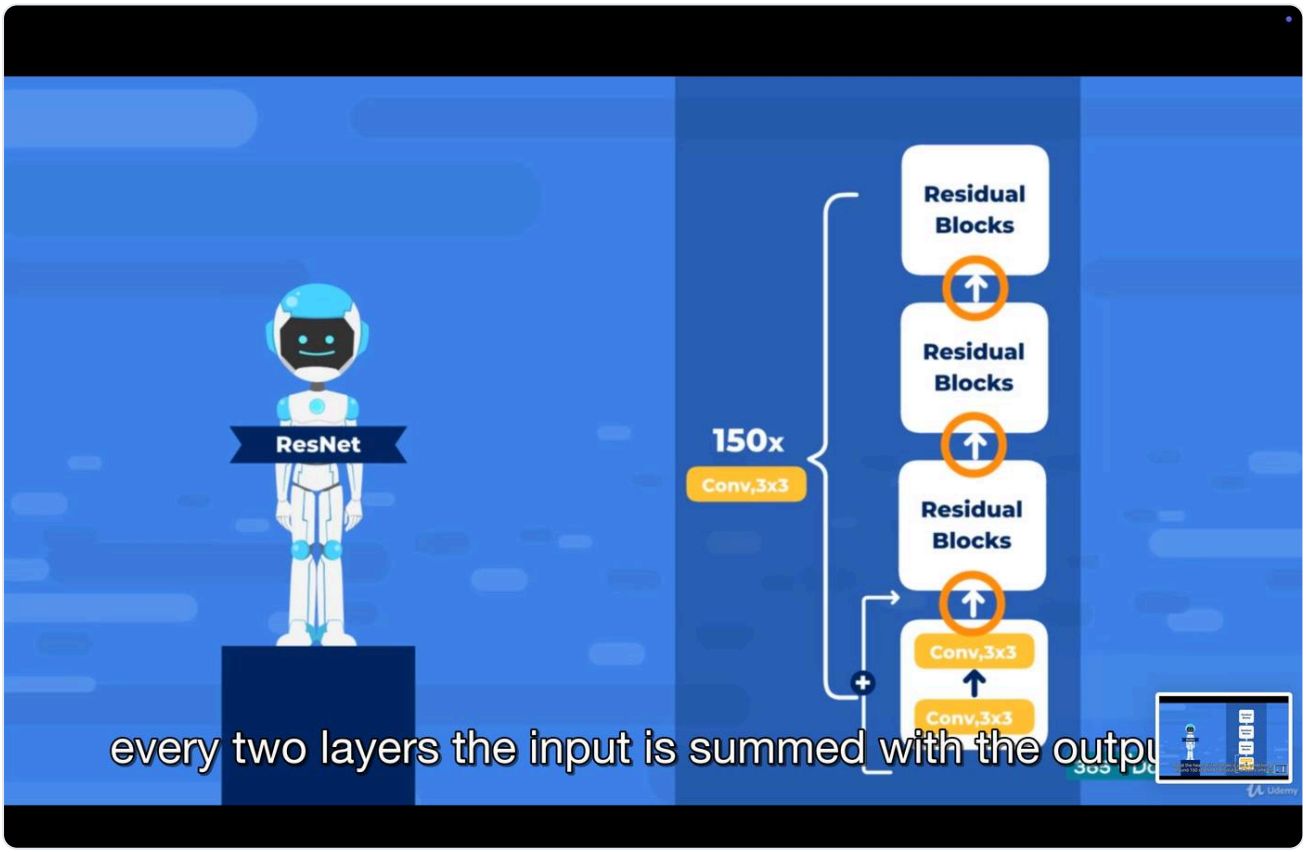


FIGURE 15.37

So at the heart of it resonate is just a collection of around 150 stacked convolutional layers wherein every two layers the input is summed with the output. Great.

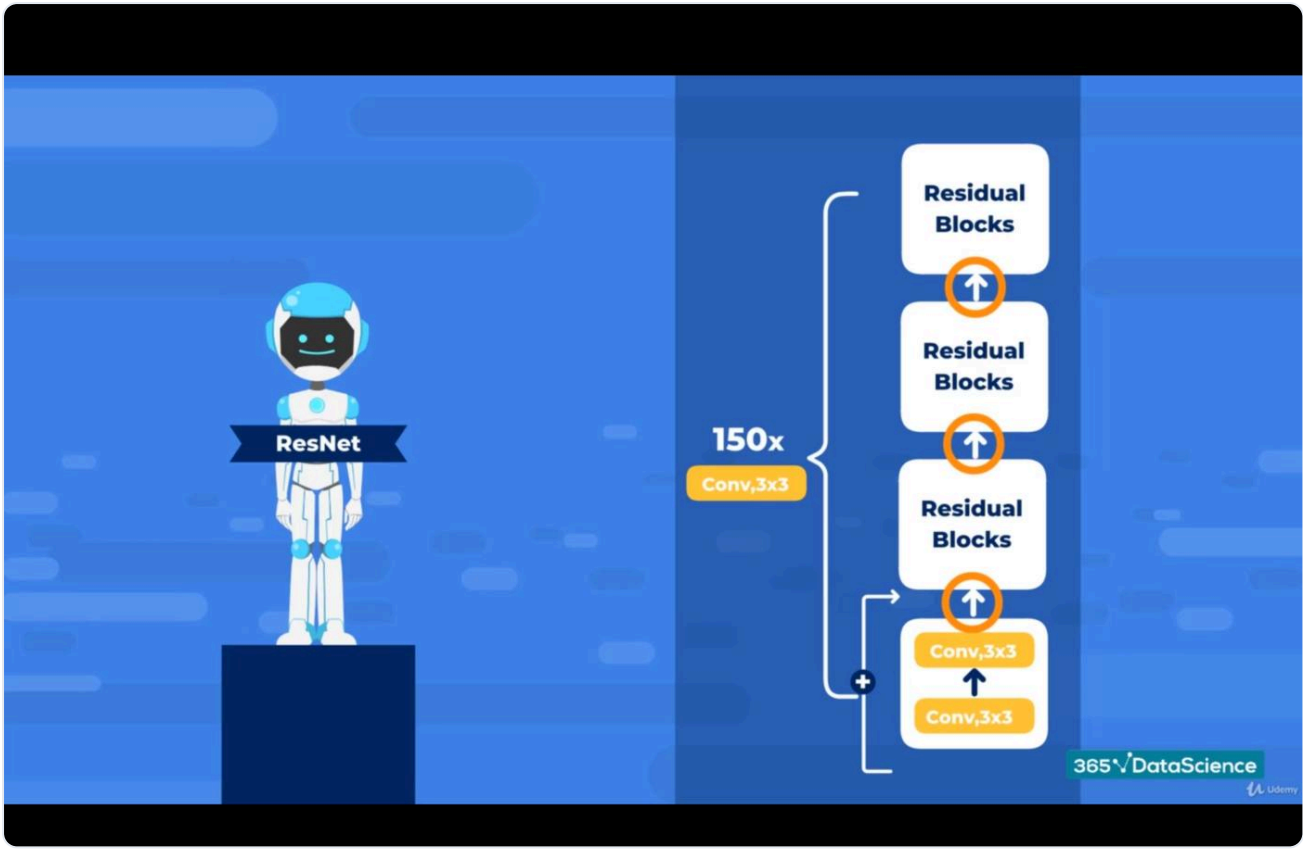


FIGURE 15.38

Some of you may already wonder if we could combine the concepts we've discussed so far with resonance.

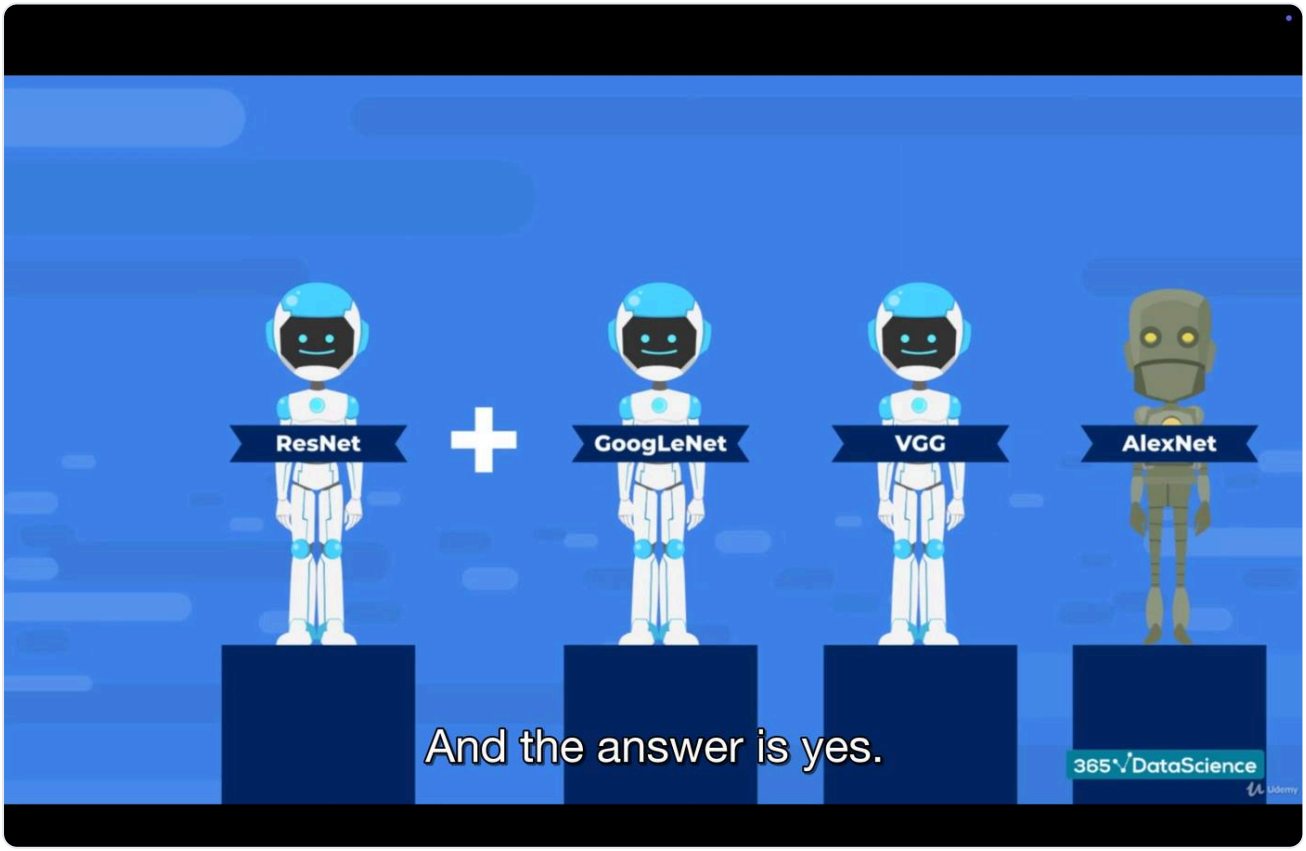


FIGURE 15.39

And the answer is yes. And this has actually been done.

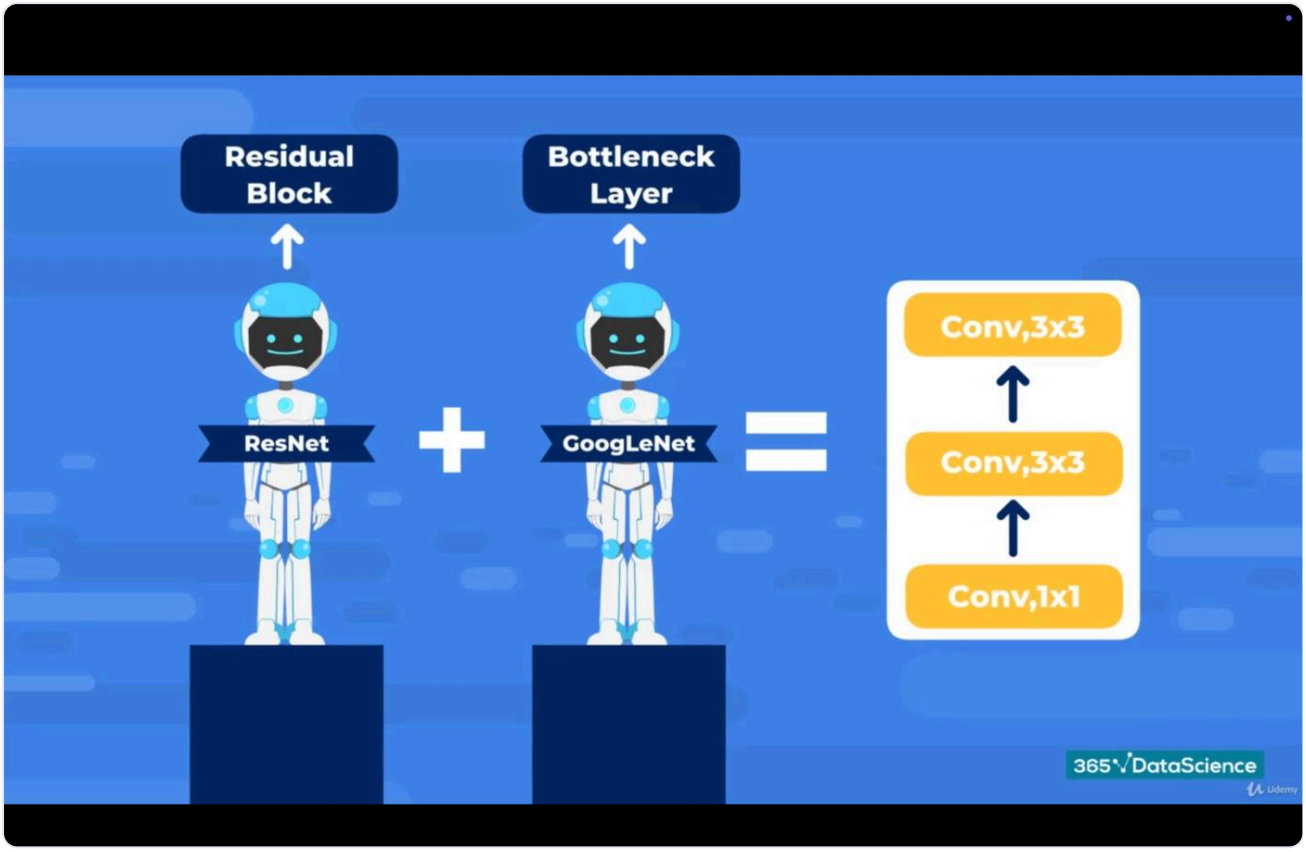


FIGURE 15.40 | Residual Bottleneck

Oftentimes the residual blocks contain the one by one convolutional bottleneck layers.

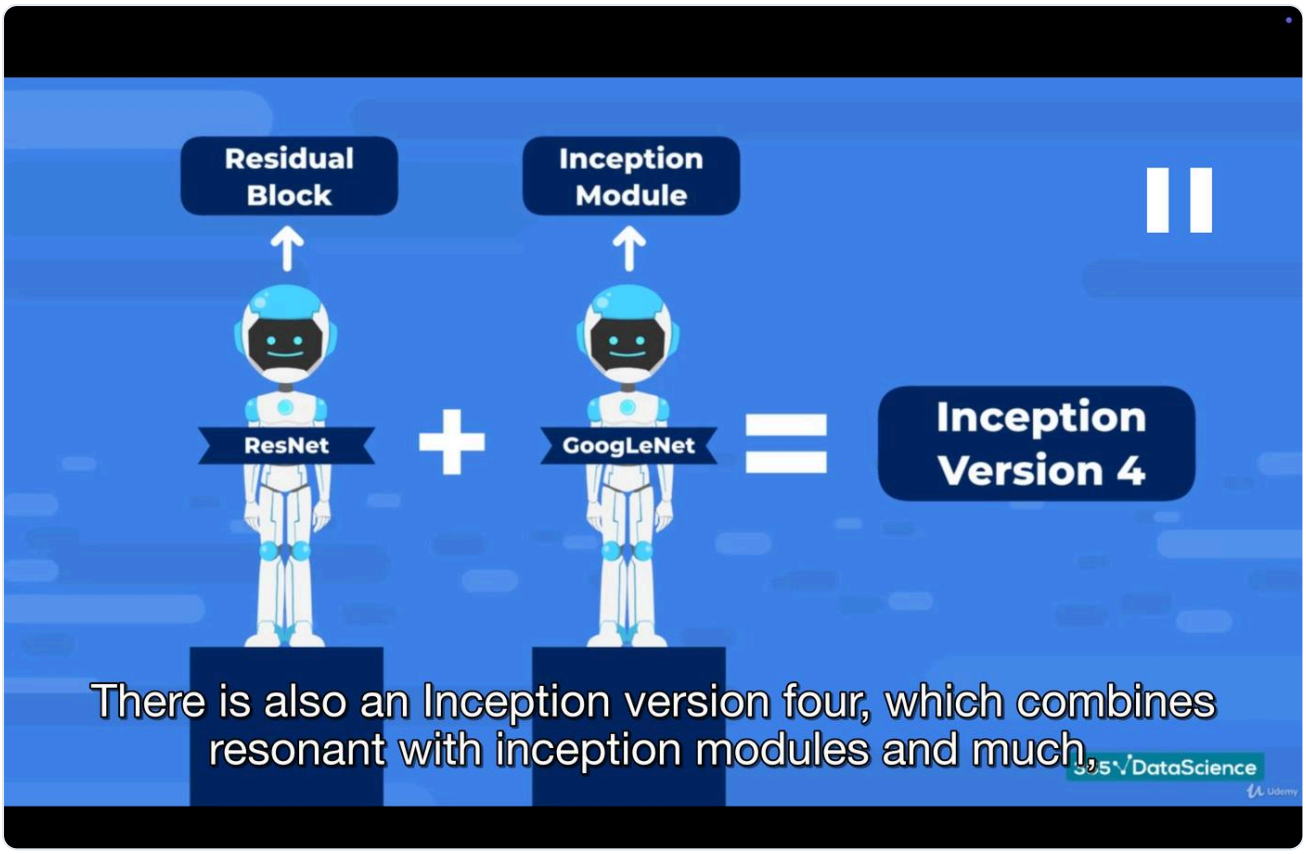


FIGURE 15.41 | Residual Inception

There is also an Inception version four, which combines resonant with inception modules and much, much more well done with this, our journey through convolutional neural networks comes to an end. Throughout this course, we have been shaping up your knowledge and understanding of the principles at work so that the universe of research papers and articles looks less daunting. Now you have the opportunity to venture out, learn about neural networks and why not even contribute to this growing field with your own research? Speaking for myself, I can definitely say I had a great deal of fun creating this course, and I hope it was a great deal of fun for you to stay safe and expand the boundaries of computer vision.

And as always, thanks for watching.